

DBA LIBRARY

The Cloud Database Engineer



RAJESH GUNTUPALLI

DBA LIBRARY

40 chapters · Free edition

© 2026 Rajesh Guntupalli

Chapter 1: Why Cloud Databases Changed Everything

If you have spent years managing SQL Server failover clusters on bare metal, tuning instances on VMware, or babysitting Always On availability groups through a 2 AM failover, this chapter is your orientation briefing. The cloud did not simply move databases to someone else's hardware — it fundamentally restructured what database engineering means, what DBAs are responsible for, and where the real leverage points are in a modern production system. Before diving into AWS RDS parameter groups or Azure SQL Elastic Pools, you need to understand what changed, what stayed the same, and what you have actually traded away in exchange for managed infrastructure.

1.1 The Shift from Infrastructure Owner to Service Consumer

When you managed an on-premises SQL Server 2022 cluster, you owned the entire stack. You decided the Windows Server build, the disk layout (data, log, tempdb, and backup volumes on separate LUNs), the storage controller queue depth, the NIC teaming configuration, and the power plan. You configured startup parameters and trace flags in SQL Server Configuration Manager. You managed logins, server roles, and endpoint permissions line by line. Failure meant your pager went off. Victory meant the database stayed up without anyone noticing.

Cloud managed databases make a different bargain. AWS RDS, Azure SQL Database, and Google Cloud SQL take ownership of the infrastructure layer — hardware provisioning, OS patching, storage volume management, and replication topology — and hand you back a connection string and a set of configuration knobs that are smaller than what you had before, but much faster to operate.

This is not a demotion. It is a role shift. The DBA who spent 40% of their week on storage provisioning tickets and OS-level tuning can redirect that time toward query optimization, schema design, connection pooling strategy, and multi-region availability architecture. The work that was invisible to the business becomes visible. The work that was visible to the business — keeping the database running — gets absorbed into a service-level agreement with a cloud provider.

The practical consequence is that you need to internalize the boundary between what you control and what the managed service controls. That boundary is different on every platform, and it shifts with every major release. On RDS for SQL Server, you cannot set startup

parameters or trace flags the way you would in SQL Server Configuration Manager — only the options and parameters AWS exposes through parameter groups and option groups are available, and SQL Server Agent is not offered at all. On Azure SQL Database, you cannot access the underlying SQL Server instance with a sysadmin login the way you would on a self-managed instance. On Cloud SQL for SQL Server, you cannot use features that require OS-level access, and job scheduling has to move off SQL Server Agent to Cloud Scheduler.

Knowing where the wall is before you hit it at 2 AM is the entire point of Chapter 1.

1.2 What Managed Services Actually Manage

The phrase "managed database" gets used loosely. Here is a concrete breakdown of what each major cloud platform actually takes off your plate, and where the line is drawn.

Infrastructure management is fully handled across all three platforms: AWS, Azure, and GCP provision the underlying compute, attach storage, configure the network interfaces, and run the hypervisor or bare-metal allocation. You never SSH into the host running your RDS instance. You never run `fdisk` on an Azure SQL storage volume. This is non-negotiable and non-configurable.

OS patching is managed by the provider on a schedule you can influence but not fully control. RDS maintenance windows let you specify a preferred day and time for minor version upgrades and OS patches, but AWS reserves the right to apply security patches outside that window when the vulnerability is severe enough. Azure SQL uses a similar maintenance window configuration. If your compliance posture requires knowing every patch that touches a database host, cloud managed services require additional audit tooling — the provider's patch logs are available through CloudTrail on AWS and Azure Monitor on Azure, but they are not in the same format as a traditional change management ticket.

High availability and replication is where managed services deliver their most significant operational value. A Multi-AZ RDS deployment synchronously replicates your data to a standby in a different Availability Zone. Failover is automatic and AWS states it typically completes within one to two minutes for standard RDS Multi-AZ. Compare that to a manual Pacemaker/Corosync cluster failover on-premises, where timing depends heavily on how well-rehearsed the runbook is — a well-tested setup might land in a similar window, while an untested one can turn into an hour on the phone with a vendor.

Backups are automated, with point-in-time recovery available up to the retention period you configure (1–35 days on RDS, 1–35 days on Azure SQL, 1–35 days on Cloud SQL). The backup mechanism itself is managed — on RDS for SQL Server, automated backups combine

storage-level snapshots with continuous transaction log uploads to S3. You do not run `BACKUP DATABASE` manually. You do not schedule your own log backups or manage log shipping. You configure a retention period and trust the service.

What managed services do **not** manage: your schema design, your index strategy, your query plans, your connection pool sizing, your application-level caching logic, your data lifecycle policies, and your disaster recovery architecture above the single-region level. These remain entirely your responsibility, and they represent the highest-leverage work a cloud DBA performs.

The Cloud Shift: Who Manages What

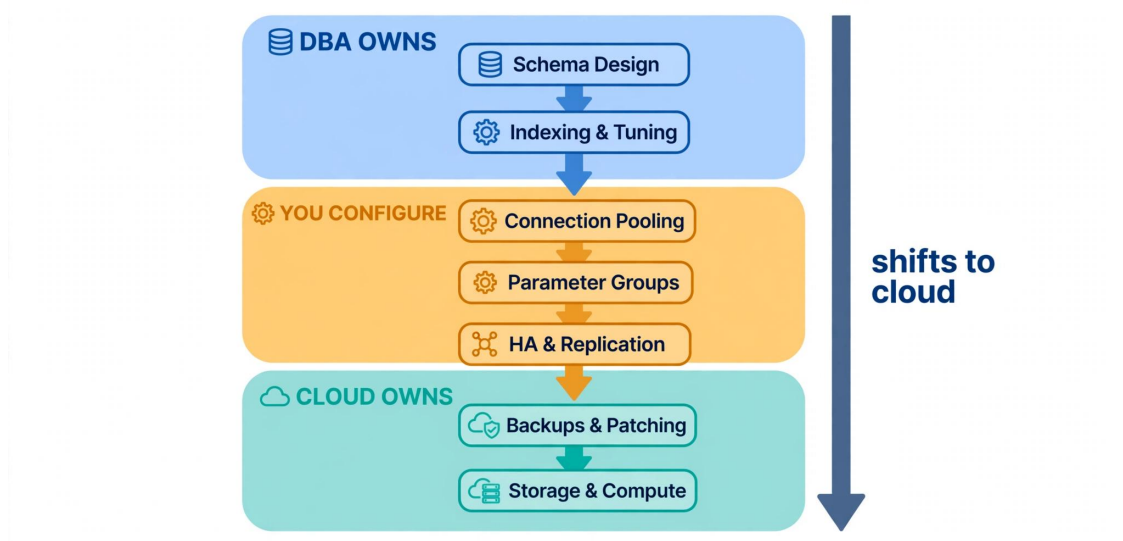


Figure 1.1 — The Cloud Shift: Who Manages What.

Read it top to bottom: the blue zone is work that never leaves your desk — schema design, indexing, and query tuning are yours no matter where the database lives. The amber zone is the shared middle ground where you configure but the platform executes: connection pooling, parameter groups, and HA topology. Everything in the teal zone — backups, patching, storage, and compute — is fully owned by the cloud provider, which is exactly why your job shifts from operator to architect.

The layers a cloud DBA still owns (top) versus the layers the provider takes over (bottom). Parameter configuration sits in between: you set the values, but the provider enforces which ones are exposed.

1.3 The Configuration Model: Parameter Groups, Server Parameters, and Database Flags

On-premises SQL Server is configured through `sp_configure`, trace flags, startup parameters, and the SQL Server Configuration Manager. When you move to managed databases, these familiar mechanisms are replaced by provider-specific configuration abstractions — parameter groups, option groups, database flags, and database-scoped configurations.

AWS RDS Parameter Groups are collections of engine parameters that you apply to one or more DB instances. For RDS for SQL Server they map to the instance-level settings you would normally change with `sp_configure` — max server memory, cost threshold for parallelism, max degree of parallelism, fill factor, and the like. Some parameters are static (requiring a reboot to apply), and some are dynamic (applied immediately). The critical difference from on-premises is that some capabilities are locked entirely — you cannot add arbitrary trace flags or startup parameters, and several features (SQL Server Agent, replication publisher roles, native linked-server providers beyond the supported set) are simply unavailable on the managed surface.

Here is how you create and apply a custom parameter group for RDS for SQL Server using the AWS CLI:

```
# Create a custom parameter group for SQL Server 2022 (Standard Edition)
aws rds create-db-parameter-group \
  --db-parameter-group-name prod-sql2022-tuned \
  --db-parameter-group-family sqlserver-se-16.0 \
  --description "Production SQL Server 2022 tuned parameters" \
  --region us-east-1

# Tune the settings you would normally change with sp_configure.
# RDS supports a formula language ({DBInstanceClassMemory/...}) so the
# parameter group stays portable across instance classes.
aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-sql2022-tuned \
  --parameters \
    "ParameterName=max server memory (MB),ParameterValue={DBInstanceClassMemory/1048576}*3/4,ApplyMethod=pending-reboot" \
    "ParameterName=cost threshold for parallelism,ParameterValue=50,ApplyMethod=immediate" \
    "ParameterName=max degree of parallelism,ParameterValue=8,ApplyMethod=immediate" \
    "ParameterName=fill factor (%),ParameterValue=90,ApplyMethod=pending-reboot" \
    "ParameterName=optimize for ad hoc workloads,ParameterValue=1,ApplyMethod=immediate" \
  --region us-east-1

# Apply the parameter group to an existing RDS for SQL Server instance
aws rds modify-db-instance \
  --db-instance-identifier prod-sql-primary \
  --db-parameter-group-name prod-sql2022-tuned \
  --apply-immediately \
  --region us-east-1
```

Notice the `{DBInstanceClassMemory/1048576}` formula. RDS resolves these formulas against the actual memory of whatever instance class the parameter group is applied to. This is actually a feature — the parameter group becomes portable across instance sizes without manual recalculation — but it is a mental model shift if you are used to running `sp_configure 'max server memory (MB)', 98304; RECONFIGURE;` with a hard-coded number on each host.

Azure SQL configuration splits into two layers. For Azure SQL Managed Instance, server-level settings are managed through the portal or the `az sql mi` CLI; for Azure SQL Database, most tuning happens through database-scoped configurations run as T-SQL. The CLI and T-SQL equivalents:

```
# Azure CLI: list server configuration options on a Managed Instance
az sql mi show \
  --resource-group prod-rg \
  --name prod-sqlmi \
  --query "{vCores: vCores, storageSizeInGB: storageSizeInGB, licenseType:
licenseType}"

# For Azure SQL Database, use database-scoped configurations (via sqlcmd or SSMS)
# -- set parallelism and cardinality estimator behavior per database:
# ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 4;
# ALTER DATABASE SCOPED CONFIGURATION SET LEGACY_CARDINALITY_ESTIMATION = OFF;
# ALTER DATABASE SCOPED CONFIGURATION SET QUERY_OPTIMIZER_HOTFIXES = ON;

# Query Store is on by default on Azure SQL Database – configure retention:
# ALTER DATABASE [production] SET QUERY_STORE
#   (OPERATION_MODE = READ_WRITE, CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS =
30));
```

Google Cloud SQL uses database flags, which are passed at instance creation or modification time. Cloud SQL for SQL Server exposes a focused set of SQL Server flags — most SQL Server tuning on GCP happens through the console or `sp_configure` inside the instance:

```
# GCP CLI: Set database flags on a Cloud SQL for SQL Server instance
gcloud sql instances patch prod-sql-instance \
  --database-flags \
    max degree of parallelism=4 \
  --project my-prod-project

# Some flags require an instance restart – gcloud will warn you
# Check which flags are supported and whether restart is needed
gcloud sql instances describe prod-sql-instance \
  --project my-prod-project \
  --format="json(settings.databaseFlags)"

# Everything else is T-SQL inside the instance, e.g.:
# EXEC sp_configure 'cost threshold for parallelism', 50; RECONFIGURE;
```

A production gotcha shared across all three platforms: you will encounter settings that exist on-premises but are either locked, unavailable, or behave differently in managed environments. On RDS for SQL Server, only a curated subset of `sp_configure` options are exposed, and features like SQL Server Agent and replication publisher roles are unavailable entirely. On Cloud SQL for SQL Server, cross-database queries and linked servers have restrictions you would never hit on a self-managed instance. On Azure SQL Database, you cannot use `DBCC FREEPROCCACHE` without specific permissions, and `DBCC DROPCLEANBUFFERS` is not available at all in the serverless tier.

1.4 Networking and Security: The New Perimeter

The most jarring transition for on-premises DBAs is often not the configuration model — it is the networking and authentication model. On-premises, your security perimeter is relatively straightforward: SQL Server logins and Windows authentication control who can connect, Windows Firewall or network ACLs control which IPs can reach port 1433, and TLS is configured in SQL Server Configuration Manager with certificates you manage yourself.

Cloud managed databases replace this with layered abstractions: VPCs, security groups, private endpoints, IAM authentication, and certificate authorities managed by the cloud provider. Each layer is more powerful and more auditable than what you had before — but only if you understand how they compose.

VPC placement is the first decision. Every RDS instance, Azure SQL server, or Cloud SQL instance should live inside a private subnet with no public accessibility. Public accessibility should be off by default. On RDS, this means creating the DB subnet group from private subnets only:

```
# Create an RDS subnet group using private subnets only
aws rds create-db-subnet-group \
  --db-subnet-group-name prod-db-subnet-group \
  --db-subnet-group-description "Private subnets for production RDS" \
  --subnet-ids subnet-0a1b2c3d4e5f subnet-0f5e4d3c2b1a \
  --region us-east-1

# Create the RDS instance with no public access
aws rds create-db-instance \
  --db-instance-identifier prod-sql-primary \
  --db-instance-class db.r6i.2xlarge \
  --engine sqlserver-se \
  --engine-version 16.00 \
  --master-username dbadmin \
  --master-user-password "$(aws secretsmanager get-secret-value \
    --secret-id prod/sqlserver/master-password \
    --query SecretString --output text)" \
  --db-subnet-group-name prod-db-subnet-group \
  --vpc-security-group-ids sg-0123456789abcdef0 \
  --no-publicly-accessible \
  --multi-az \
  --allocated-storage 500 \
  --storage-type gp3 \
  --storage-encrypted \
  --region us-east-1
```


IAM authentication replaces the password-in-config-file pattern that was acceptable on-premises and is a liability in cloud environments. With RDS IAM authentication, database users are mapped to IAM roles, and the authentication token is generated on the fly with a 15-minute validity window. No long-lived passwords stored in application config files. What you used to manage with SQL Server logins, contained database users, and Windows authentication groups on-premises, you now manage with IAM policies and the `rds_iam_roles` mapping.

```
# Python: Generate an IAM auth token and connect to RDS for SQL Server
import boto3
import pyodbc
from botocore.exceptions import ClientError

def get_rds_iam_token(hostname: str, port: int, username: str, region: str) -> str:
    """Generate a short-lived IAM authentication token for RDS."""
    client = boto3.client('rds', region_name=region)
    token = client.generate_db_auth_token(
        DBHostname=hostname,
        Port=port,
        DBUsername=username,
        Region=region
    )
    return token

def connect_with_iam_auth():
    hostname = "prod-sql-primary.xxxxxxxxxxxxxx.us-east-1.rds.amazonaws.com"
    port = 1433
    username = "app_readonly"
    region = "us-east-1"
    database = "production"

    # Token is valid for 15 minutes – generate fresh per connection
    token = get_rds_iam_token(hostname, port, username, region)

    conn_str = (
        f"DRIVER={{ODBC Driver 18 for SQL Server}};"
        f"SERVER={hostname},{port};DATABASE={database};"
        f"UID={username};PWD={token};Encrypt=yes;"
        f"TrustServerCertificate=no"
    )
    return pyodbc.connect(conn_str)

# The IAM role attached to the EC2/Lambda/ECS task must have:
# rds-db:connect permission on the specific DB user resource ARN
# arn:aws:rds-db:us-east-1:123456789012:dbuser:prod-sql-primary/app_readonly
#
# NOTE: the SQL Server login must already exist on the instance and be
# mapped to the IAM role – IAM auth does not create logins for you.
```

Private Link and Private Endpoints eliminate the exposure of database traffic to the public internet entirely, even within a VPC. Azure SQL Private Link creates a private IP in your VNet that maps to the Azure SQL server — traffic never leaves Microsoft's backbone. AWS PrivateLink for RDS works similarly. On GCP, Cloud SQL's Private IP option keeps all traffic within the VPC.

The mental model shift: you are no longer thinking about firewall rules as a flat list of IP-to-port mappings. You are thinking about IAM principals, VPC routing, and service endpoints as a layered security architecture. The good news is that this architecture is auditable in ways that a text file on a server never was.

1.5 Operational Observability: From DMVs to CloudWatch and Beyond

On a self-managed SQL Server instance, your observability toolkit is the dynamic management views (`sys.dm_exec_requests` , `sys.dm_exec_query_stats` , `sys.dm_os_wait_stats`), Extended Events, Query Store, the SQL Server error log, and whatever you have wired up with Prometheus and Grafana. You own the data collection pipeline end to end.

Cloud managed databases provide observability through their native monitoring platforms — Amazon CloudWatch for RDS, Azure Monitor for Azure SQL, and Cloud Monitoring for Cloud SQL — and these platforms are significantly more powerful at the infrastructure level than anything you ran on-premises. They are also more limited at the database engine internals level, because you do not own the host.

CloudWatch for RDS exposes metrics at 1-minute granularity by default (10-second granularity with Enhanced Monitoring enabled, which adds a small cost). Key metrics you should always alarm on in production:

```
# Create a CloudWatch alarm for RDS CPU exceeding 80% for 10 minutes
aws cloudwatch put-metric-alarm \
  --alarm-name "prod-rds-cpu-high" \
  --alarm-description "RDS CPU above 80% for 10 minutes" \
  --metric-name CPUUtilization \
  --namespace AWS/RDS \
  --statistic Average \
  --period 300 \
  --evaluation-periods 2 \
  --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --dimensions Name=DBInstanceIdentifier,Value=prod-sql-primary \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:prod-dba-alerts \
  --ok-actions arn:aws:sns:us-east-1:123456789012:prod-dba-alerts \
  --region us-east-1

# Create alarm for availability-group replica lag on a self-managed secondary
# (RDS for SQL Server has no read replicas, so lag monitoring for read
# scale-out lives on your AG secondaries or Azure Hyperscale replicas)
aws cloudwatch put-metric-alarm \
  --alarm-name "prod-ag-secondary-lag-high" \
  --alarm-description "AG secondary redo lag above 30 seconds" \
  --metric-name AGRedoLagSeconds \
  --namespace SQLServer/Custom \
  --statistic Maximum \
  --period 60 \
  --evaluation-periods 3 \
  --threshold 30 \
  --comparison-operator GreaterThanThreshold \
  --dimensions Name=ReplicaName,Value=prod-sql-ag-secondary-1 \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:prod-dba-alerts \
  --region us-east-1
```

The metrics that matter most in production cloud databases — `FreeStorageSpace`, `DatabaseConnections`, `ReadLatency`, `WriteLatency`, `ReadIOPS`, `FreeableMemory` — map directly to the concerns you had on-premises, but the collection and alerting infrastructure is provided rather than self-built.

What you lose is raw access to the operating system metrics underneath. Enhanced Monitoring on RDS gives you per-process CPU and memory breakdowns through an agent that runs on the DB host and ships to CloudWatch Logs, but it is read-only telemetry — you cannot act on it by tuning kernel parameters. You observe what the managed service gives you, and you tune within the parameters it exposes.

Performance Insights, available on RDS, is the closest cloud equivalent to querying `sys.dm_exec_query_stats` joined to a wait-statistics analysis. It shows you top SQL by load, broken down by wait events (CPU, I/O, lock, etc.), across a 7-day free window or up to 2 years with the paid tier. For most production workload analysis, it replaces the combination of periodic DMV snapshots and manual wait-stat diffing you ran on-premises.

1.6 The Economics of Cloud Databases: What You Pay For and What It Costs You

No chapter on why cloud databases changed everything would be complete without an honest discussion of cost, because cost is where cloud database decisions most often go wrong. On-premises infrastructure is a capital expense (CapEx) that depreciates over a hardware lifecycle. Cloud databases are an operating expense (OpEx) that runs indefinitely and scales directly with usage. This changes how you make architectural decisions.

An `db.r6i.4xlarge` RDS instance (16 vCPU, 128 GB RAM) carries a meaningful on-demand hourly rate — check current AWS pricing for your region and engine rather than relying on a number printed in a book, since rates change over time. A Multi-AZ deployment roughly doubles the compute cost compared to single-AZ, because you are paying for a synchronous standby that stays warm and ready. Adding several terabytes of `gp3` storage adds a proportional monthly storage cost on top of compute. Adding a readable secondary for read scale-out (an Always On secondary on EC2, or a Hyperscale named replica on Azure) adds its own compute line item on top.

Before you reach for the largest instance class, understand what you are buying. An `r6i.4xlarge` with 128 GB RAM is appropriate when your working set — the data actively accessed by your queries — genuinely approaches that size. If your total database is 200 GB but your hot working set is 20 GB, a smaller instance such as an `r6i.xlarge` (32 GB RAM, a fraction of the 4xlarge's cost) with appropriate indexing will outperform an `r6i.4xlarge` with poor query patterns.

Reserved Instances can meaningfully reduce RDS costs compared to on-demand pricing, with the exact discount depending on the term (1-year vs. 3-year) and payment option — check current AWS pricing rather than budgeting off a fixed percentage. For production databases with stable workloads, Reserved Instances (or Savings Plans) are not optional — they are the responsible operational choice.

The cost comparison with on-premises is more complex than the cloud vendors' calculators suggest. You need to include fully loaded infrastructure costs: hardware amortization, data center space and power, network equipment, OS licensing (for Windows-based databases),

DBA time spent on patching and hardware maintenance, and the cost of unplanned downtime. When all of these are included honestly, cloud managed databases are frequently cost-competitive or cheaper for mid-sized organizations that lack economies of scale in hardware procurement. For very large deployments — hundreds of nodes, petabytes of data — the calculus often favors hybrid or private cloud approaches, which is why this book exists rather than one that simply tells you to migrate everything.

The Terraform pattern below represents the production baseline for an RDS deployment — the minimum configuration a production DBA should require before any database goes live:

```

# Terraform: Production-grade RDS for SQL Server baseline
resource "aws_db_instance" "prod_sqlserver" {
  identifier      = "prod-sql-primary"
  engine          = "sqlserver-se"
  engine_version  = "16.00"
  instance_class  = "db.r6i.2xlarge"

  allocated_storage      = 500
  max_allocated_storage  = 2000 # Enable storage autoscaling up to 2TB
  storage_type            = "gp3"
  storage_encrypted       = true
  kms_key_id              = aws_kms_key.rds_encryption_key.arn

  db_name  = "production"
  username = "dbadmin"
  password = data.aws_secretsmanager_secret_version.db_master_password.secret_string

  multi_az      = true
  db_subnet_group_name = aws_db_subnet_group.prod.name
  vpc_security_group_ids = [aws_security_group.rds_sg.id]
  publicly_accessible = false

  parameter_group_name = aws_db_parameter_group.prod_sql2022.name
  option_group_name     = aws_db_option_group.prod_sqlserver.name # e.g.
  # SQLSERVER_BACKUP_RESTORE for native backup to S3

  backup_retention_period = 14
  backup_window           = "03:00-04:00"
  maintenance_window      = "sun:04:00-sun:05:00"
  auto_minor_version_upgrade = false # Control upgrades explicitly in production

  performance_insights_enabled      = true
  performance_insights_retention_period = 731 # 2 years
  monitoring_interval                = 60    # Enhanced Monitoring every 60
seconds
  monitoring_role_arn                = aws_iam_role.rds_enhanced_monitoring.arn

  enabled_cloudwatch_logs_exports = ["error", "agent"]

  deletion_protection = true
  skip_final_snapshot = false
  final_snapshot_identifier = "prod-sqlserver-final-snapshot"

  tags = {
    Environment = "production"
    Team        = "platform-data"
    CostCenter  = "engineering"
  }
}

```

Every parameter in that Terraform block represents a decision that a production DBA needs to make consciously and own. `auto_minor_version_upgrade = false` means you control when minor version upgrades happen — the default is true, which lets AWS apply them during your maintenance window without further approval. In a regulated environment or one with strict change management, that default will cause you problems. `deletion_protection = true` means you cannot accidentally destroy a production database with a `terraform destroy` or a misclick in the console. `performance_insights_retention_period = 731` keeps two years of query performance history, which matters when you are debugging a regression that appeared after a deployment six weeks ago.

These are not configuration details — they are operational commitments. Infrastructure as code forces you to make them explicit.

1.7 What Stayed the Same: The Fundamentals Don't Move

After four sections on what changed, it is worth being direct about what did not change, because this is where many cloud migration projects go wrong. Engineers assume that because the infrastructure layer is managed, the database layer is somehow simpler. It is not.

Query optimization is unchanged. A table scan on 500 million rows costs the same on RDS for SQL Server as it does on a bare-metal server running the same SQL Server version with the same data. The query optimizer has the same logic, the same cost model, the same sensitivity to stale statistics. Actual execution plans look identical. The fix is the same: appropriate indexing, updated statistics, rewritten queries. The cloud provider does not optimize your queries.


```
-- SQL Server: Top resource-consuming queries (works on RDS SQL Server, Azure SQL
identically)
SELECT TOP 20
    qs.execution_count,
    qs.total_logical_reads,
    qs.total_logical_reads / NULLIF(qs.execution_count, 0) AS avg_logical_reads,
    qs.total_worker_time / 1000 AS total_cpu_ms,
    qs.total_elapsed_time / 1000 AS total_elapsed_ms,
    SUBSTRING(st.text, (qs.statement_start_offset / 2) + 1,
        ((CASE qs.statement_end_offset WHEN -1 THEN DATALENGTH(st.text)
            ELSE qs.statement_end_offset END - qs.statement_start_offset) / 2) + 1) AS
statement_text,
    qp.query_plan
FROM sys.dm_exec_query_stats AS qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) AS st
OUTER APPLY sys.dm_exec_query_plan(qs.plan_handle) AS qp
ORDER BY qs.total_logical_reads DESC;
```

```
-- SQL Server: Missing index DMV analysis
SELECT
    DB_NAME(mid.database_id) AS database_name,
    OBJECT_NAME(mid.object_id, mid.database_id) AS table_name,
    migs.avg_total_user_cost * migs.avg_user_impact * (migs.user_seeks +
migs.user_scans) AS improvement_measure,
    'CREATE INDEX [IX_' + OBJECT_NAME(mid.object_id, mid.database_id)
      + '_' + REPLACE(REPLACE(ISNULL(mid.equality_columns, ''), '['), ')', '')
      + CASE WHEN mid.inequality_columns IS NOT NULL THEN '_' +
REPLACE(REPLACE(mid.inequality_columns, '['), ')', '') ELSE '' END
      + ']'
      + ' ON ' + mid.statement
      + ' (' + ISNULL(mid.equality_columns, '')
      + CASE WHEN mid.equality_columns IS NOT NULL AND mid.inequality_columns IS
NOT NULL THEN ',' ELSE '' END
      + ISNULL(mid.inequality_columns, '') + ')'
      + ISNULL(' INCLUDE (' + mid.included_columns + ')', '') AS
create_index_statement,
    migs.user_seeks,
    migs.user_scans,
    migs.last_user_seek,
    mid.equality_columns,
    mid.inequality_columns,
    mid.included_columns
FROM
    sys.dm_db_missing_index_groups mig
    INNER JOIN sys.dm_db_missing_index_group_stats migs
      ON migs.group_handle = mig.index_group_handle
    INNER JOIN sys.dm_db_missing_index_details mid
      ON mig.index_handle = mid.index_handle
WHERE
    migs.avg_total_user_cost * migs.avg_user_impact * (migs.user_seeks +
migs.user_scans) > 10
ORDER BY
    improvement_measure DESC;
```

Both queries work without modification against the equivalent managed service. The DMV and catalog views are the same engine, running in the same way, just on infrastructure you do not own.

Transaction isolation and locking behavior is unchanged. Deadlocks happen on RDS for SQL Server for exactly the same reasons they happen on-premises. Lock escalation, blocking chains, and tempdb contention under snapshot isolation work identically. Row-versioning behavior is the same whether the database runs in your data center or in a managed service — enabling `READ_COMMITTED_SNAPSHOT` still moves version-store pressure into tempdb. If your application fragments indexes with heavy random inserts and you never rebuild or reorganize, the managed service will not do index maintenance for you either.

Connection management is unchanged — and arguably more important. SQL Server's worker-thread and session model means that thousands of concurrent connections consume substantial memory and scheduler overhead regardless of whether the instance runs on RDS, Azure SQL, or bare metal. On-premises, the instinct was often to raise the connection limit and hope for the best. In cloud environments, especially when running many Lambda functions or microservices that each hold database connections, this problem becomes acute. Proper connection pooling in the application tier — or RDS Proxy, which we cover in Chapter 10 — is not optional for architectures with more than a few dozen concurrent clients.

Backup recovery time must be tested. Automated backups are only valuable if you have verified, under conditions resembling production, that you can actually restore from them in the time your SLA requires. This does not change because backups are managed. What changes is the restoration mechanism — you are restoring from RDS snapshots or native backups to S3 rather than running `RESTORE DATABASE` from your own backup files manually — but the discipline of testing restores remains entirely your responsibility.

1.8 Choosing a Platform: RDS, Azure SQL, and Cloud SQL

This book covers all three major clouds, and readers who are evaluating where to run their workloads deserve a direct framework for that decision. This is not a comprehensive feature comparison — those go stale within months — but a durable architectural framing.

Stick with your existing ecosystem first. If your organization is deeply invested in AWS — IAM, VPCs, CloudFormation, existing RDS instances — the switching cost to Azure or GCP is significant and the marginal benefit is usually small. Conversely, if your analytics workload already runs on BigQuery, Cloud SQL for SQL Server for transactional data keeps your data gravity in GCP. Platform consistency reduces operational complexity, and operational complexity is where production incidents are born.

Match the service tier to your operational maturity. For SQL Server workloads, the choice is between RDS for SQL Server and Azure SQL: RDS for SQL Server keeps you closest to the self-managed SQL Server you already know — same edition model, same T-SQL surface, same backup semantics — while Azure SQL Database and Managed Instance trade some compatibility for deeper platform integration. If you are migrating an existing on-premises SQL Server workload and want the fewest unknowns, RDS for SQL Server (or SQL Server on an EC2/VM you manage yourself) reduces the number of surprises.

Azure SQL Database vs. Azure SQL Managed Instance vs. SQL Server on Azure VM is a choice about how much of the instance you need to keep, covered in depth in Chapter 13. If your application was written for SQL Server — T-SQL stored procedures, SQL Server Agent

jobs, cross-database queries, linked servers, existing SSRS or SSIS dependencies — Managed Instance or SQL Server on VM is the natural target. If the workload is greenfield or already modernized, Azure SQL Database provides the most managed experience with the lowest operational overhead.

Google Cloud SQL for SQL Server vs. SQL Server on Compute Engine follows the managed-versus-self-managed pattern: Cloud SQL is SQL Server with managed infrastructure — the most compatible option for straightforward migrations — while SQL Server on Compute Engine gives you full OS and instance control at the cost of owning patching, availability, and backups yourself. We cover both in depth in Chapter 19.

The honest answer is: for most production SQL Server workloads that are not at extreme scale, RDS for SQL Server, Azure SQL Managed Instance, or Cloud SQL for SQL Server are excellent choices that will not limit you. Evaluate Azure SQL Hyperscale or a self-managed deployment on EC2/Compute Engine when you have specific requirements — storage growth beyond what standard managed services handle cost-effectively, read scale beyond what a few replicas provide, or sub-thirty-second failover RTO — and when your team has the operational capacity to understand the differences.

1.9 The DBA's New Toolkit

The tools change when the platform changes. A production cloud DBA in 2024 operates with a toolkit that is meaningfully different from the on-premises DBA toolkit, and the difference is not just the addition of cloud consoles.

Infrastructure as code (IaC) is not optional. Every database instance, parameter group, subnet group, security group, and IAM policy should be defined in Terraform, Pulumi, or CloudFormation. Clicking through the AWS console to create a production database is the equivalent of hand-editing server settings on a box that has no change management — it will work until it doesn't, and you will have no record of what you did. Chapter 2 covers Terraform patterns for RDS in depth.

The CLI is your command line. `aws`, `az`, and `gcloud` replace the SSH session. Being fluent in the CLI for your primary platform is as important as knowing `sqlcmd` commands. The examples throughout this book use CLI commands intentionally — they are scriptable, auditable, and version-controllable in ways that console clicks are not.

Secrets management replaces config files. AWS Secrets Manager, Azure Key Vault, and GCP Secret Manager are the production-grade replacements for the `DATABASE_URL` environment variable that contains a plaintext password. Rotation policies, audit logs, and IAM-controlled access make them categorically more secure. We cover integration patterns in Chapter 5.

Cost monitoring is a DBA function. On-premises, the DBA rarely owned infrastructure costs — that was procurement and finance's problem. In cloud environments where database spend is metered hourly and can surprise you at month-end, monitoring your RDS or Azure SQL spend against budget is part of database operations. AWS Cost Explorer, Azure Cost Management, and GCP Billing all provide service-level breakdowns that let you see exactly what your database fleet is costing. Setting budget alerts before they are needed, not after the first unexpected invoice, is the professional approach.

Documentation lives in the provider's release notes. SQL Server cumulative update release notes are the same whether you are on-premises or on RDS. But RDS-specific behavior, parameter limitations, and maintenance policies live in AWS's RDS documentation and change independently of SQL Server versions. Subscribing to AWS RDS release notes, Azure SQL Blog, and Cloud SQL release notes is operational discipline — these are the channels where you learn that a new behavior will affect your production instances before it affects your production instances.

Key Takeaways

- → Cloud managed databases shift the DBA's responsibility from infrastructure ownership to service consumption. The infrastructure layer — OS, storage, replication, patching — is managed by the provider. Schema design, query optimization, connection management, and cross-region availability architecture remain entirely your responsibility.
- → The configuration boundary is platform-specific and changes over time. RDS parameter groups, Azure Server Parameters, and Cloud SQL database flags map to familiar on-premises configuration mechanisms but enforce restrictions that do not exist on self-managed instances. Know which parameters are locked before you need them in an incident.
- → Networking and authentication are more layered in cloud environments than on-premises. VPC placement, IAM authentication, and private endpoints replace flat firewall rules and per-host login management. This architecture is more secure and more auditable, but it requires understanding how the layers compose to debug connectivity failures.
- → The database engine fundamentals do not change on managed infrastructure. Query plans, locking behavior, row versioning, index fragmentation, and connection overhead work identically. Performance problems that exist on-premises follow you to cloud managed databases unless you address them at the query and schema level.

- → Infrastructure as code, CLI fluency, secrets management, and cost monitoring are non-negotiable components of cloud database operations. Clicking through a console to manage production databases is not a sustainable operational practice, and treating cloud database cost as someone else's problem will produce unwelcome surprises at billing time.

Chapter 2: Managed vs Self-Managed: The Real Tradeoffs

Every DBA who moves to the cloud arrives with the same question: what exactly am I giving up, and what am I getting in return? The marketing answer is "you get managed infrastructure." The production answer is more nuanced. Managed database services abstract away the operating system, storage subsystem, replication mechanics, and backup scheduling — but they do not abstract away the need to understand them. This chapter maps the specific boundaries between what AWS RDS, Azure SQL, and Google Cloud SQL control versus what you still own, so you can make architecture decisions with your eyes open rather than discovering limits during an incident at 2 a.m.

What "Managed" Actually Means at the Infrastructure Layer

When you ran SQL Server on bare metal or VMware, your responsibility surface looked like this: hardware procurement, OS patching, disk layout, network configuration, SQL Server installation, `sp_configure` tuning, backup scheduling, Always On setup, failover scripting, and monitoring. In a managed service, roughly the bottom half of that stack disappears — but it disappears in specific ways that matter to how you work.

AWS RDS runs your database inside an EC2 instance that AWS owns. You never SSH into it. The OS is there — Amazon Linux 2 or a variant depending on the engine — but it is not your OS. You cannot install components that are not on the supported list — no SQL Server Agent, no SSIS, SSAS, or SSRS — you cannot add startup parameters or trace flags beyond the curated set AWS exposes, and you cannot change the SQL Server binaries. What you can do is control almost everything above that layer: parameter groups, option groups, subnet groups, security groups, IAM authentication, automated backup retention up to 35 days, and Multi-AZ deployment.

Azure SQL Database is further abstracted than RDS. Microsoft runs SQL Server inside a service fabric that you have zero visibility into. There is no concept of an underlying VM in the standard tiers. You configure DTUs or vCores, storage limits, firewall rules, Active Directory authentication, and Transparent Data Encryption. The SQL Server instance itself — its memory settings, its tempdb layout, its trace flags — is managed by Microsoft according to tier.

Google Cloud SQL sits closer to the RDS model than to Azure SQL Database. It runs SQL Server (among other engines) on Compute Engine VMs that Google manages. For SQL Server you get a small set of database flags (the equivalent of `sp_configure` parameters), authorized networks, private IP configuration, and automated backups. Like RDS, the OS is not yours.

The practical consequence for an experienced DBA: your tuning vocabulary shifts from OS-level and filesystem-level settings toward database-level configuration objects — parameter groups, database flags, elastic pool settings — and cloud-native controls like IAM policies and VPC routing.

The Access Boundary: What You Can and Cannot Touch

This is where most on-prem DBAs first hit friction. The following table summarizes the access boundaries that matter most in production:

Capability	Self-Managed SQL Server	RDS for SQL Server	Azure SQL	Cloud SQL for SQL Server	
---	---	---	---	---	
OS access	Full root	None	None	None	
Install arbitrary extensions	Yes	No (approved list only)	No	No (approved list)	
Manage logins and authentication rules at OS level	Yes	No (use IAM/security groups)	N/A	No (use authorized networks)	
Access the transaction log directly	Yes	No	N/A	No	
Native <code>BACKUP DATABASE</code> to external target	Yes	Yes (to S3 via the <code>SQLSERVER_BACKUP_RESTORE</code> option group)	N/A	Yes (to GCS)	
Custom superuser access	Full	<code>rds_superuser</code> role (limited)	<code>db_owner</code> (limited)	limited admin roles	
Set <code>user connections</code> freely	Yes	Yes (via parameter group)	Tier-dependent ceiling	Yes (via database flags)	
Tablespace on custom paths	Yes	No	No	No	

The master-user privilege point deserves emphasis. On RDS for SQL Server, the master user you create at provisioning time is **not** a full `sysadmin`. It can manage logins, databases, and most administrative functions, but it cannot grant itself server-level permissions that AWS

reserves, cannot install components outside the supported set, and cannot touch the OS. If you have built any tooling that depends on sysadmin-level OS interaction — for example, a maintenance script that shells out with `xp_cmdshell` or writes directly to the filesystem — that tooling requires redesign.

Azure SQL is a different situation entirely. The concept of a SQL Server instance owner with `sysadmin` rights does not exist in Azure SQL Database. The closest equivalent is `db_owner` on individual databases, plus server-level roles like `loginmanager` and `dbmanager` on the logical server. If your on-prem SQL Server processes rely on SQL Server Agent jobs with `xp_cmdshell`, those processes need to be rebuilt as Azure Functions, Azure Automation runbooks, or Elastic Jobs.

Parameter Tuning: From `sp_configure` to Parameter Groups

On a self-managed SQL Server instance, tuning `max server memory (MB)`, `cost threshold for parallelism`, `max degree of parallelism`, and `fill factor (%)` means running `sp_configure` and possibly restarting. In RDS, that workflow translates to parameter groups.

A **parameter group** in RDS is a named configuration object that you attach to one or more DB instances. AWS provides default parameter groups (read-only) for each engine version. You create a custom parameter group, modify the parameters you need, and associate it with your instance. Parameter changes fall into two categories: static (require a reboot) and dynamic (apply immediately). Knowing which is which before you plan a change window matters.

```
# Create a custom RDS parameter group for SQL Server 2022 (Standard Edition)
aws rds create-db-parameter-group \
  --db-parameter-group-name prod-sql2022-custom \
  --db-parameter-group-family sqlserver-se-16.0 \
  --description "Production SQL Server 2022 parameter group" \
  --region us-east-1

# Modify key parameters (the sp_configure equivalents)
aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-sql2022-custom \
  --parameters \
    "ParameterName=max server memory (MB),ParameterValue={DBInstanceClassMemory/1048576}*3/4,ApplyMethod=pending-reboot" \
    "ParameterName=cost threshold for parallelism,ParameterValue=50,ApplyMethod=immediate" \
    "ParameterName=max degree of parallelism,ParameterValue=8,ApplyMethod=immediate" \
  \
    "ParameterName=fill factor (%),ParameterValue=90,ApplyMethod=pending-reboot" \
    "ParameterName=optimize for ad hoc workloads,ParameterValue=1,ApplyMethod=immediate" \
  --region us-east-1

# Apply the parameter group to an existing instance
aws rds modify-db-instance \
  --db-instance-identifier prod-sql-primary \
  --db-parameter-group-name prod-sql2022-custom \
  --apply-immediately \
  --region us-east-1
```

On Google Cloud SQL for SQL Server, the equivalent concept is **database flags**. The syntax and application mechanism differ, but the intent is the same. Note that Cloud SQL for SQL Server exposes only a small flag surface — most instance tuning still happens with `sp_configure` inside the instance.

```
# Set database flags on a Cloud SQL for SQL Server instance
gcloud sql instances patch prod-sql-instance \
  --database-flags \
    max degree of parallelism=4 \
  --project my-gcp-project

# Verify current flags
gcloud sql instances describe prod-sql-instance \
  --project my-gcp-project \
  --format="json(settings.databaseFlags)"
```

On Azure SQL, the tuning model is fundamentally different. You do not set `user connections` because connection limits are determined by the service tier. For an 8-vCore General Purpose instance, Azure SQL allows approximately 800 concurrent connections. For Business Critical,

limits are higher. You tune at the query level using Query Store, Automatic Plan Correction, and resource governor equivalents — but you do not manage server memory allocation directly. Azure SQL's Resource Governor equivalent is elastic pool resource governance, and it operates at the pool level rather than the instance level.

One real gotcha with RDS parameter groups: if you create a Multi-AZ deployment and modify a static parameter, the pending reboot applies to both primary and standby simultaneously during the next maintenance window — or immediately if you force it. This means you need to treat static parameter changes with the same care as a planned failover, because that is effectively what AWS does to apply them.

High Availability: What You Delegate and What You Still Own

In a self-managed SQL Server environment, high availability required you to build a Windows Server Failover Cluster, configure Always On availability groups, set up the availability group listener, write the failover detection and alerting, and test it regularly. That is a significant operational surface.

RDS Multi-AZ reduces that surface dramatically. When you enable Multi-AZ on an RDS for SQL Server instance, AWS provisions a synchronous standby in a different Availability Zone using SQL Server's own high-availability technology under the hood, manages the replication, monitors the primary, and executes automatic failover — typically completing within 60 to 120 seconds for standard RDS. You do not configure any of this. The connection string uses the RDS endpoint, which automatically resolves to whichever instance is currently primary.

What you still own: **application reconnection logic**. RDS failover changes the underlying IP that the DNS endpoint points to. DNS TTL for RDS endpoints is 5 seconds, but client-side DNS caching in the JVM, in certain connection poolers, and in some OS-level resolvers can hold stale entries for minutes. If your application opens a connection and holds it across the failover boundary, that connection will break and must be reestablished. This is not RDS failing to do its job — it is the boundary between the managed service and the application tier.

```

# Python example: RDS-aware connection handling with retry logic
# This pattern is necessary because RDS failover breaks existing connections

import pyodbc
import time
import logging

logger = logging.getLogger(__name__)

CONN_TEMPLATE = (
    "DRIVER={{ODBC Driver 18 for SQL Server}};"
    "SERVER={host},{port};DATABASE={dbname};"
    "UID={user};PWD={password};"
    "Encrypt=yes;TrustServerCertificate=no;"
    "Connect Timeout=5;"
)

def get_rds_connection(host, port, dbname, user, password,
                       max_retries=5, retry_delay=2):
    """
    Establish a connection to RDS with retry logic suitable for
    Multi-AZ failover scenarios. On failover, DNS propagation
    may take 5-30 seconds; this loop handles the gap.
    """
    attempt = 0
    last_error = None

    while attempt < max_retries:
        try:
            conn = pyodbc.connect(
                CONN_TEMPLATE.format(host=host, port=port, dbname=dbname,
                                     user=user, password=password),
                autocommit=False,
            )
            logger.info(f"Connected to RDS at {host} on attempt {attempt + 1}")
            return conn

        except pyodbc.Error as e:
            last_error = e
            attempt += 1
            logger.warning(
                f"RDS connection attempt {attempt} failed: {e}. "
                f"Retrying in {retry_delay}s..."
            )
            time.sleep(retry_delay)
            retry_delay = min(retry_delay * 2, 30) # exponential backoff, cap at
30s

    raise RuntimeError(
        f"Could not connect to RDS after {max_retries} attempts. "
        f"Last error: {last_error}"
    )

```

```

def execute_with_reconnect(host, port, dbname, user, password, query, params=None):
    """
    Execute a query and handle connection failures that can occur
    mid-session during failover events.
    """
    conn = get_rds_connection(host, port, dbname, user, password)
    try:
        cur = conn.cursor()
        cur.execute(query, params or [])
        rows = cur.fetchall() if cur.description else None
        conn.commit()
        cur.close()
        return rows
    except pyodbc.Error as e:
        logger.error(f"Query failed, connection likely lost during failover: {e}")
        conn.close()
        # Re-raise so the caller can decide whether to retry at the business logic
level
        raise
    finally:
        try:
            conn.close()
        except Exception:
            pass

```

Azure SQL Business Critical tier uses an Always On Availability Group under the hood, with a hidden read-only secondary exposed via the `ApplicationIntent=ReadOnly` connection string parameter. Business Critical carries the highest published availability SLA of the Azure SQL tiers — check current Microsoft documentation for the exact figure, since it has changed over time and varies with zone redundancy. For General Purpose tier, Azure uses remote storage (Premium SSD) and failover involves attaching that storage to a new compute node — typically 20 to 30 seconds but potentially longer under high storage load.

For Cloud SQL, high availability within a region uses a standby instance with synchronous replication. Cross-region read replicas can be promoted manually but do not participate in automatic failover. For multi-region automatic failover of a SQL Server workload in Google Cloud, Cloud SQL for SQL Server is not the answer — you architect it with cross-region read replicas and manual promotion, or choose a platform whose SQL Server offering has it built in.

Backup, Restore, and the Point-in-Time Recovery Model

On a self-managed SQL Server instance, your backup strategy involved `BACKUP DATABASE` for full backups, transaction log backups to a network target, and periodic validation through `RESTORE VERIFYONLY` and full test restores. You controlled the retention schedule, the storage location, and the restore procedure.

In RDS, automated backups are configured with a retention period (1 to 35 days) and a backup window. RDS takes daily snapshots and continuously archives transaction logs to S3 — storage you never see but that AWS uses to serve Point-in-Time Recovery (PITR). Restoring to a point in time always creates a **new DB instance**; you cannot overwrite the existing one. This is one of the most operationally significant differences from self-managed recovery. If you need to restore to 30 minutes ago, you restore to a new endpoint and redirect traffic, or you use the restored instance to extract specific data with `bcp` or an SSMS export and load it into the production instance.

```
# Initiate a point-in-time restore on RDS to a new instance
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier prod-sql-primary \
  --target-db-instance-identifier prod-sql-pitr-restore-20240115 \
  --restore-time 2024-01-15T14:30:00Z \
  --db-instance-class db.r6i.2xlarge \
  --db-subnet-group-name prod-subnet-group \
  --vpc-security-group-ids sg-0abc123def456789 \
  --no-publicly-accessible \
  --region us-east-1

# Monitor the restore status
aws rds describe-db-instances \
  --db-instance-identifier prod-sql-pitr-restore-20240115 \
  --query 'DBInstances[0].{Status:DBInstanceStatus,Endpoint:Endpoint.Address}' \
  --region us-east-1
```

The storage costs for automated backups in RDS are worth understanding before you enable 35-day retention on large instances. AWS provides backup storage equal to the total provisioned storage of your RDS instances at no additional charge; storage beyond that is billed per GB-month at a rate you should confirm against current AWS pricing. For a 10 TB RDS instance with 35-day retention and high write volume, backup storage costs can exceed the compute costs of the instance itself. Audit your backup retention policies as deliberately as you would on-prem.

Azure SQL automated backups use a similar model: full weekly, differential daily, transaction log every 5 to 12 minutes. Retention for Azure SQL Database is 1 to 35 days for PITR, with long-term retention (LTR) available for up to 10 years. PITR restores, like RDS, create a new database — they do not overwrite the source.

One genuine improvement in managed services over self-managed setups for most teams: **backup validation is handled for you**. AWS verifies that automated snapshots are restorable. On-prem shops frequently discover corrupted backup chains during actual recovery events. That said, this does not replace the need to periodically test restore procedures yourself — automated backup verification confirms the backup file is intact, not that your application functions correctly on the restored data.

Terraform Provisioning: Encoding the Decisions You Cannot Delegate

Infrastructure as Code is not optional in a production cloud database environment. The decisions you make at provisioning time — instance class, storage type, Multi-AZ enablement, encryption, parameter group, deletion protection — are significantly harder to change after the fact than on-prem settings. Terraform locks these decisions into version-controlled code and makes them reproducible.

```

# Terraform configuration for a production-grade RDS for SQL Server instance
# This encodes the decisions that matter most at provisioning time.

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

# KMS key for RDS encryption at rest
resource "aws_kms_key" "rds_key" {
  description      = "RDS encryption key for production SQL Server"
  deletion_window_in_days = 30
  enable_key_rotation = true

  tags = {
    Environment = "production"
    ManagedBy   = "terraform"
  }
}

# Custom parameter group for SQL Server 2022 (Standard Edition)
resource "aws_db_parameter_group" "sql2022_prod" {
  name     = "prod-sql2022-custom"
  family   = "sqlserver-se-16.0"
  description = "Production SQL Server 2022 parameters"

  parameter {
    name      = "max server memory (MB)"
    value     = "{DBInstanceClassMemory/1048576}*3/4"
    apply_method = "pending-reboot"
  }

  parameter {
    name      = "cost threshold for parallelism"
    value     = "50"
    apply_method = "immediate"
  }

  parameter {
    name      = "max degree of parallelism"
    value     = "8"
    apply_method = "immediate"
  }

  parameter {
    name      = "fill factor (%)"
    value     = "90"
    apply_method = "pending-reboot"
  }
}

```



```

parameter {
    name          = "optimize for ad hoc workloads"
    value          = "1"
    apply_method  = "immediate"
}
}

# RDS subnet group
resource "aws_db_subnet_group" "prod" {
    name          = "prod-rds-subnet-group"
    subnet_ids    = var.private_subnet_ids
    description    = "Private subnets for RDS – no public route"

    tags = {
        Environment = "production"
    }
}

# Production RDS for SQL Server instance
resource "aws_db_instance" "prod_sql" {
    identifier      = "prod-sql-primary"
    engine          = "sqlserver-se"
    engine_version  = "16.00"
    instance_class  = "db.r6i.2xlarge" # 8 vCPU, 64 GB RAM

    # Storage: gp3 gives 3000 IOPS baseline, upgradeable independently of storage size
    allocated_storage      = 500
    max_allocated_storage  = 2000 # autoscaling ceiling in GB
    storage_type            = "gp3"
    iops                    = 12000 # provisioned IOPS for gp3
    storage_throughput      = 500 # MB/s throughput for gp3
    storage_encrypted       = true
    kms_key_id              = aws_kms_key.rds_key.arn

    db_name    = "appdb"
    username   = "dbadmin"
    # In production: use aws_secretsmanager_secret and reference here
    password   = var.db_master_password

    # HA and networking
    multi_az      = true
    db_subnet_group_name    = aws_db_subnet_group.prod.name
    vpc_security_group_ids  = [var.db_security_group_id]
    publicly_accessible     = false

    # Parameter group
    parameter_group_name = aws_db_parameter_group.sql2022_prod.name

    # Backup configuration
    backup_retention_period = 14
    backup_window            = "03:00-04:00" # UTC
    maintenance_window      = "sun:04:30-sun:05:30"

```

```

copy_tags_to_snapshot      = true
delete_automated_backups  = false

# Protection and logging
deletion_protection        = true
skip_final_snapshot        = false
final_snapshot_identifier  = "prod-sql-primary-final-snapshot"

enabled_cloudwatch_logs_exports = [
  "error",
  "agent"
]

# Performance Insights for query-level visibility
performance_insights_enabled      = true
performance_insights_retention_period = 7 # days; 731 for long-term (additional
cost)

tags = {
  Environment = "production"
  Owner       = "database-team"
  ManagedBy   = "terraform"
}
}

```

Two provisioning decisions in this template deserve attention for DBAs coming from on-prem.

First, `storage_type = "gp3"`. On RDS, gp3 replaced gp2 as the default general-purpose storage type. gp2 tied IOPS to storage size (3 IOPS per GB, bursting to 3000 for volumes under 1 TB). gp3 provides 3000 baseline IOPS and 125 MB/s throughput regardless of volume size, with independent scaling up to 16,000 IOPS and 1000 MB/s. For any database workload where you are not running io1 or io2 (the provisioned IOPS types), gp3 is almost always the right choice and is generally cheaper per GB than equivalent gp2 configurations — check current AWS pricing for the exact difference.

Second, `max_allocated_storage`. RDS Storage Autoscaling can expand your volume automatically when free space drops below 10% of allocated storage (or 5 GB, whichever is larger). Setting a ceiling with `max_allocated_storage` prevents runaway storage growth from a data incident — a bulk insert gone wrong, a logging misconfiguration — from producing a surprise billing event.

Key Takeaways

- → Managed services eliminate OS, storage, and replication management, but replace those responsibilities with cloud-native configuration objects — parameter groups, database flags, security groups, IAM policies — that require the same deliberate attention as the files they replace.
- → The RDS master user, ``cloudsqlsuperuser``, and Azure SQL ``db_owner`` roles are not equivalent to full sysadmin or OS-level access. Any automation, extension, or process that depends on superuser filesystem or binary access requires architectural redesign before it can run on a managed service.
- → PITR restores on RDS, Azure SQL, and Cloud SQL always create a new database instance, not an in-place overwrite. Recovery procedures must account for the time to provision a new instance, redirect connections, and validate data — typically 15 to 45 minutes beyond the recovery point itself.
- → RDS Multi-AZ failover typically completes in 60 to 120 seconds; Azure SQL failover is generally faster due to its different architecture — understand each platform's failover behavior before you size your application's retry logic and downtime tolerance around it.
- → Infrastructure as Code is not a convenience in cloud database environments — it is the mechanism through which provisioning-time decisions (encryption keys, storage type, Multi-AZ, deletion protection) become auditable, reproducible, and reviewable before a change reaches production.

Chapter 3: The SQL Server Cloud Landscape: AWS, Azure, and Google Cloud

Before choosing services, understand the terrain. Each cloud offers SQL Server in managed and self-managed forms, with different architectures under the hood, different identity systems, different backup mechanics, and different cost models. This chapter maps the landscape — the service taxonomy, the compute/storage architectures, authentication, backup, monitoring, and cost — so the chapters that follow have a foundation to build on. Everything here is SQL Server; the comparisons are between clouds and service models, not engines.

3.1 The Service Taxonomy: What Each Cloud Actually Offers

AWS: RDS for SQL Server — the managed service (automated backups, Multi-AZ failover, no readable secondaries, no IAM database authentication — Chapters 6 and 10). **SQL Server on EC2** — self-managed on virtual machines with full engine control, Always On AGs, and readable secondaries (Chapter 8). **RDS Proxy for SQL Server** — managed connection pooling (Chapter 10).

Azure: Azure SQL Database — the PaaS single-database and elastic-pool offering, no SQL Server Agent (Chapter 14). **Azure SQL Managed Instance** — near-complete engine compatibility with full Agent support, the lift-and-shift target (Chapter 12). **Azure SQL Hyperscale** — the disaggregated-storage architecture for very large databases (Chapter 15). **SQL Server on Azure VMs** — self-managed with Azure Hybrid Benefit and constrained vCPUs (Chapter 13).

Google Cloud: Cloud SQL for SQL Server — the managed service with HA via regional disks (Chapter 19). **SQL Server on Compute Engine** — self-managed VMs with AG support across zones (Chapter 19). The pattern across clouds is consistent: a managed service that trades control for convenience, and a VM offering that trades convenience for control. The choice between them (Chapter 7's framework) is the most consequential decision in the book.

3.2 Compute and Storage Architecture: What Changes Under the Hood

Under every managed service is the same physical reality — VMs and network-attached storage — with different abstractions. **RDS** runs SQL Server on EC2-equivalent hosts with EBS storage (gp3/io1/io2 — Chapter 36); Multi-AZ keeps a synchronous standby whose storage is independent. **Azure SQL** General Purpose uses remote storage with local SSD caching;

Business Critical uses local SSD with AG replication between nodes; Hyperscale disaggregates compute from page servers for near-instant scaling and fast restores. **Cloud SQL** uses regional Persistent Disk with automatic failover to a standby zone.

What this means for the DBA: storage latency characteristics differ by architecture (local SSD vs network-attached — measure PAGEIOLATCH waits, Chapter 35), failover times differ (RDS Multi-AZ 60–120 seconds, Azure BC faster, Cloud SQL 60–120 seconds), and the storage abstraction determines what you can observe (no iostat on managed storage — the platform's metrics are your instrumentation, Chapter 30). The architecture also determines the scaling axes: Hyperscale scales storage to 100 TB without the restore-time penalties of monolithic databases; read scale-out requires self-managed AG secondaries on AWS/GCP but is built into Azure's tiers.

3.3 Authentication and Access Control: Identity Is the New Perimeter

Each cloud has its own identity system, and SQL Server meets it differently: **Azure** leads with Microsoft Entra ID — native to Azure SQL, supporting managed identities (no passwords) and Entra-only authentication mode (Chapter 31). **AWS** offers Windows Authentication on RDS via Managed Microsoft AD join, but no IAM database authentication for SQL Server — SQL logins with Secrets Manager rotation fill the gap (Chapters 10, 31). **GCP** uses the Cloud SQL Auth Proxy with Cloud IAM for access control around SQL logins (Chapter 31). Self-managed SQL Server on any cloud's VMs can join Active Directory for full Kerberos parity with on-premises (Chapters 8, 13, 19).

The networking complement (Chapter 32): private subnets, private endpoints (PrivateLink, Azure Private Link, Private Service Connect), firewall least privilege, and TLS enforcement. The identity story and the network story are designed together — the database that requires Entra authentication but accepts connections from the public internet has solved half the problem.

3.4 Backup and Point-in-Time Recovery: What "Managed" Actually Means

"Managed backups" means the platform runs the schedule — it does not mean you can stop thinking about recovery. **RDS** automated backups with PITR up to 35 days, plus native backups to S3 via stored procedures for portability. **Azure SQL** fully automated full/differential/log backups with 7–35 day PITR and long-term retention to 10 years. **Cloud SQL** automated daily backups with PITR via log archiving. On self-managed VMs, you own the schedule — native backups to object storage with the full/differential/log cadence (Chapter 29).

The DBA's responsibilities don't change: verify restores (automated weekly restore tests — Chapter 29), understand RPO/RTO per mechanism (Chapter 16), keep backup copies in a second region and a second account, and know that PITR on managed services creates a *new* instance/database rather than overwriting in place — the recovery procedure must include the redirect.

3.5 Performance Monitoring: What You Get, What You Have to Build

Each platform provides the telemetry foundation: **CloudWatch + Performance Insights** (RDS/EC2), **Azure Monitor + Query Store + Intelligent Insights** (Azure SQL), **Cloud Monitoring + Query Insights** (Cloud SQL/GCE). What you get: host metrics, database metrics, and query-level telemetry with history. What you build: the dashboards (health, performance, capacity — Chapter 30), the alerting hierarchy (page/ticket/log with runbook links), and the cross-platform standardization so one DBA reads every cloud's metrics. The non-negotiable: query-level visibility enabled everywhere (Query Store, Performance Insights) — platform metrics show *that* it's slow; query telemetry shows *what* is slow (Chapter 35).

3.6 The Operational Cost Model: Thinking in Cloud Units

Cloud databases bill in units the on-premises DBA never saw: vCore-hours or DTUs, provisioned IOPS, storage GB-months, backup GB-months, data transfer GB, and (on serverless) seconds of compute. The cost disciplines from the book's final section: **right-sizing** from measured p95/peak (Chapter 39), **commitments** for the stable baseline (Chapter 38), **storage tier matching** (Chapter 36), **secondary economics** (Chapter 37), and the **unified cost-performance dashboard** (Chapter 40). The single most important cost habit: the monthly review where engineering and finance walk the numbers together. Everything else is technique; the review is the institution.

Key Takeaways

- → **Every cloud offers the same two shapes: managed service and self-managed VMs.** RDS/Azure SQL/Cloud SQL trade control for convenience; EC2/Azure VMs/GCE trade convenience for control. Learn both shapes on your primary cloud first.
- → **Architecture determines failover time, scaling axes, and observability.** Hyperscale's disaggregation, BC's local SSD, RDS's EBS — know what's under your database and what it implies.
- → **Identity differs per cloud; Entra ID leads on Azure.** No IAM database auth for SQL Server on RDS — plan secrets rotation. Design identity and networking together.

- → **Managed backups don't remove the DBA's recovery responsibilities.** Verify restores weekly, understand PITR mechanics (new instance, not in-place), keep cross-region and cross-account copies.
- → **Enable query-level telemetry everywhere; review cost monthly.** Query Store/Performance Insights are non-negotiable; the cost review with finance is the institution that makes all cost technique stick.

Chapter 4: The Cloud DBA Role: What Changes, What Stays

If you have spent years tuning SQL Server max server memory, managing Always On Availability Groups, and writing runbooks for failover procedures at 2 AM, the shift to managed cloud databases can feel disorienting. AWS, Azure, and Google Cloud take over large portions of the operational work you once owned completely — OS patching, storage provisioning, replication wiring — while introducing entirely new categories of responsibility that did not exist in your on-premises world. This chapter maps that transition precisely. It identifies what the cloud genuinely removes from your plate, what it hands back to you in a different form, and what it adds as net-new work. Understanding this boundary is the foundation for everything else in this book.

4.1 The Managed Service Contract: What AWS, Azure, and GCP Actually Own

When you move from self-managed SQL Server to Amazon RDS, you are entering into an implicit contract. The cloud provider owns the hypervisor, the operating system, the storage subsystem, the replication mechanism, and the software patching lifecycle. You own the database configuration, the schema design, the query performance, the access control model, and the backup retention policy. The line between those two domains is firm, and understanding it prevents both false assumptions and wasted effort.

What the provider owns:

On RDS for SQL Server, you will never run Windows Update on the host, restart the OS, or access the SQL Server installation directories directly. AWS handles patching on a schedule you influence but do not fully control unless you take explicit ownership of the maintenance window. The storage layer is EBS-backed with automatic replication within an Availability Zone. Multi-AZ deployments add a synchronous standby in a second AZ, and failover is handled automatically — typically completing in 60–120 seconds for RDS Multi-AZ (Azure SQL Business Critical generally fails over faster due to its AG-based architecture).

On Azure SQL Database, Microsoft manages the underlying SQL Server engine, the Always On availability group that provides HA, and the Intelligent Query Processing features that ship with each compatibility level upgrade. Google Cloud SQL similarly abstracts OS management and handles read replica replication lag monitoring internally.

What you still own:

The provider's ownership does not extend to logical database design, index strategy, connection pooling, application-layer retry logic, or query plans. A poorly written query that causes a full table scan on a 2 TB table will perform exactly as badly on RDS as it did on your on-premises server. Cloud-managed does not mean self-healing at the query level. Your expertise in execution plans, wait events, lock escalation, and statistics management transfers directly and remains essential.

The practical implication is this: you spend less time on infrastructure operations and more time on database architecture decisions, performance engineering, and cost optimization. That is a genuine shift in how a senior DBA allocates time, not a reduction in the need for DBA expertise.

4.2 Access Patterns and Permissions: From Server Logins to IAM

On-premises SQL Server access control flows through two layers: server logins control who can connect, and database users and roles control what they can do once connected. Windows Authentication through Active Directory ties both layers to corporate identity. In the cloud, these familiar mechanisms are extended — not replaced — by identity and access management layers that sit above the database engine.

IAM Authentication on RDS

With IAM database authentication enabled on RDS for SQL Server, a login connects using a short-lived authentication token generated by the AWS IAM service rather than a static password. The token is valid for 15 minutes and is generated using the AWS credentials of the calling identity — an EC2 instance profile, a Lambda execution role, or a developer's IAM user. The database-side configuration still uses SQL Server logins and database users, but the credential presented at connect time is the IAM token instead of the login's password.

```
# Enable IAM authentication on an existing RDS for SQL Server instance
aws rds modify-db-instance \
  --db-instance-identifier prod-sql-01 \
  --enable-iam-database-authentication \
  --apply-immediately

# Generate an authentication token for a login named "app_user"
aws rds generate-db-auth-token \
  --hostname prod-sql-01.cxyz1234.us-east-1.rds.amazonaws.com \
  --port 1433 \
  --region us-east-1 \
  --username app_user
```

On the database side, the login must exist on the instance and be associated with the IAM role:

```
-- Create the login for the application service account.
-- Keep a strong Secrets Manager-backed password as a break-glass fallback;
-- day-to-day authentication is token-based and never uses the static password.
CREATE LOGIN [app_user] WITH PASSWORD = '<managed-by-secrets-manager>';
-- Grant object-level privileges as you normally would
GRANT SELECT, INSERT, UPDATE ON SCHEMA::dbo TO [app_user];
```

This pattern replaces the practice of embedding static database passwords in application configuration files. The calling application retrieves a fresh token before each connection attempt (or connection pool initialization), using the AWS SDK. The conceptual analog is Kerberos authentication with Active Directory — the application proves its identity to a central authority and receives a time-limited credential.

Azure SQL and Azure Active Directory

Azure SQL Database integrates with Azure Active Directory using contained database users. The model is familiar if you have used Windows Authentication with SQL Server, but the administration happens through the Azure portal, Azure CLI, or Entra ID (formerly Azure AD) rather than through local Windows group policy.

```
# Set an Azure AD admin for an Azure SQL logical server
az sql server ad-admin create \
  --resource-group prod-rg \
  --server prod-sql-server-01 \
  --display-name "DBA-Team-Admin" \
  --object-id "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"
```

Inside the database, Entra-backed contained users are created without passwords:

```
-- Create an Azure AD-backed user in Azure SQL Database
CREATE USER [dba-engineer@company.com] FROM EXTERNAL PROVIDER;
ALTER ROLE db_datareader ADD MEMBER [dba-engineer@company.com];

-- For a managed identity (e.g., an Azure App Service)
CREATE USER [my-app-service] FROM EXTERNAL PROVIDER;
GRANT SELECT, INSERT, UPDATE ON SCHEMA::dbo TO [my-app-service];
```

The Core Shift

What changes is not the concept of authentication and authorization — it is the locus of control. You now manage database access through the same IAM, RBAC, and identity systems that govern access to every other cloud resource. Permissions are auditable through CloudTrail (AWS), Azure Monitor Activity Log (Azure), and Cloud Audit Logs (GCP). Rotating credentials means updating IAM policies, not editing server logins and coordinating password rotations across application servers. This is a genuine improvement in auditability and a reduction in credential sprawl, but it requires the DBA to become conversant in cloud IAM, which is a new skill domain.

4.3 Configuration Management: Parameter Groups, Server Parameters, and What You Cannot Touch

On self-managed SQL Server, you run `sp_configure` directly and control every engine setting from `max server memory (MB)` to `cost threshold for parallelism` to `fill factor (%)`. On RDS, that surface is owned by AWS. You configure the engine through Parameter Groups — a named collection of engine parameters that you apply to one or more DB instances.

RDS Parameter Groups

The default parameter group for any RDS engine version is managed by AWS and is not modifiable. You create a custom parameter group, copy defaults into it, modify what you need, and associate it with your instance. Changes to dynamic parameters take effect immediately; changes to static parameters require an instance reboot.

```
# Create a custom parameter group for RDS SQL Server 2022
aws rds create-db-parameter-group \
  --db-parameter-group-name prod-sql2022-custom \
  --db-parameter-group-family sqlserver-se-16.0 \
  --description "Production SQL Server 2022 tuning"

# Modify key parameters
aws rds modify-db-parameter-group \
  --db-parameter-group-name prod-sql2022-custom \
  --parameters \
    "ParameterName=max server memory (MB),ParameterValue={DBInstanceClassMemory}/1048576}*3/4,ApplyMethod=pending-reboot" \
    "ParameterName=cost threshold for parallelism,ParameterValue=50,ApplyMethod=immediate" \
    "ParameterName=max degree of parallelism,ParameterValue=8,ApplyMethod=immediate" \
    "ParameterName=fill factor (%),ParameterValue=90,ApplyMethod=pending-reboot" \
    "ParameterName=optimize for ad hoc workloads,ParameterValue=1,ApplyMethod=immediate"

# Associate with an instance
aws rds modify-db-instance \
  --db-instance-identifier prod-sql-01 \
  --db-parameter-group-name prod-sql2022-custom \
  --apply-immediately
```

Note the formula syntax for `max server memory (MB)`. RDS supports arithmetic expressions using `{DBInstanceClassMemory}` as a variable — this is how AWS recommends setting memory-relative parameters so the configuration remains valid if you resize the instance class. For a `db.r6i.2xlarge` with 64 GB RAM, leaving roughly a quarter of memory for the OS is the standard SQL Server practice, which the formula encodes directly.

What You Cannot Configure

This is where experienced DBAs encounter real friction. Several parameters are locked in RDS because they control behaviors that AWS manages on your behalf:

- → Many ``sp_configure`` options are restricted to the curated set AWS exposes; arbitrary trace flags and startup parameters are unavailable
- → Transaction log backup scheduling is managed by AWS; you cannot run your own log backup chain on the instance
- → Network protocols and endpoints are managed by AWS
- → TLS is forced on — you cannot disable encryption on RDS connections
- → ``sysadmin`` access does not exist on RDS; the master user is a functional substitute but does not grant OS-level access
- → Components must be on the supported list — no SQL Server Agent, no SSIS, SSAS, or SSRS

The absence of true sysadmin is a significant operational change. On RDS for SQL Server you get a master user with broad but not unlimited privileges — the `rdsadmin` account owns the instance internals, and tasks like killing sessions owned by other logins or accessing server-level internals may require the RDS-specific stored procedures rather than the T-SQL you know. Plan your privilege requirements before migration: anything depending on full sysadmin (certain Agent subsystem access, server-level DDL triggers, OS-level operations) needs a self-managed EC2 deployment instead.

Azure SQL Server-Level vs. Database-Level Parameters

Azure SQL Database presents a different model. Many parameters that SQL Server DBAs manage at the server level — max server memory, MAXDOP, cost threshold for parallelism — are controlled through database-scoped configurations or are set by Microsoft at the service tier level.

```
-- Azure SQL: set database-scoped configuration
ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 4;
ALTER DATABASE SCOPED CONFIGURATION SET LEGACY_CARDINALITY_ESTIMATION = OFF;
ALTER DATABASE SCOPED CONFIGURATION SET QUERY_OPTIMIZER_HOTFIXES = ON;

-- Check current database-scoped configuration
SELECT name, value, value_for_secondary
FROM sys.database_scoped_configurations
ORDER BY name;
```

Maximum memory is determined by the service tier and compute size — there is no `sp_configure 'max server memory'` you can run. Business Critical generally provides a higher memory-to-vCore ratio than General Purpose, since it runs on local SSD-backed hardware rather than remote storage, but the exact figures vary by hardware generation — check current Microsoft documentation rather than assuming a fixed number. What matters operationally is that these ceilings are fixed for the tier and not tunable.

4.4 High Availability and Failover: Understanding What the Cloud Gives You

HA in on-premises environments is expensive to build and maintain. A self-managed SQL Server Always On availability group with a Windows Server Failover Cluster, listener, and quorum configuration requires significant operational investment. Self-managed failover clustering demands quorum votes, witness configuration, and careful patch sequencing across nodes. In the cloud, HA is a checkbox during instance creation, and the complexity of maintaining it disappears — but understanding what you got is essential because the behavior is not identical to what you built yourself.

RDS Multi-AZ: What It Is and Is Not

RDS Multi-AZ deploys a synchronous standby in a second Availability Zone. Writes are committed to both the primary and standby before returning to the application. Failover is automatic and DNS-based — the endpoint DNS name resolves to the standby after promotion, typically within 60–120 seconds including DNS TTL propagation. The standby is not readable. It is a hot standby only for failover purposes.

This surprises many DBAs coming from Always On availability groups where secondaries serve read traffic. On RDS for SQL Server Multi-AZ, you pay for the standby instance in full and receive zero read scaling from it — and unlike other RDS engines, SQL Server has no read-replica option on RDS at all. Read scaling for SQL Server means readable AG secondaries on self-managed EC2/VMs (Chapter 8) or the built-in readable secondaries in Azure SQL's Business Critical and Hyperscale tiers (Chapter 7).

Azure SQL: A Different Architecture

Azure SQL changes this model substantially. The Business Critical tier runs an Always On availability group across compute nodes: a primary plus replicas with synchronous commit, one of which is readable at no additional replica cost. Failover promotes a replica that already holds a synchronized copy of the data — typically completing faster than RDS Multi-AZ's storage-level failover — and the readable secondary doubles as the read-scale target (Chapter 7). Hyperscale goes further, disaggregating compute from page servers so storage scales independently and restores complete in minutes regardless of database size (Chapter 15).

```
# Create an RDS for SQL Server instance with Multi-AZ and encryption
aws rds create-db-instance \
  --db-instance-identifier prod-sqlserver \
  --engine sqlserver-se \
  --engine-version 16.00.4125.1.v1 \
  --db-instance-class db.r7i.2xlarge \
  --allocated-storage 500 --storage-type gp3 \
  --multi-az --storage-encrypted \
  --master-username dbadmin \
  --master-user-password "$(aws secretsmanager get-secret-value \
    --secret-id prod/rds-sqlserver/master-password \
    --query SecretString --output text)" \
  --db-subnet-group-name prod-subnet-group \
  --vpc-security-group-ids sg-0abc123 \
  --backup-retention-period 7
```

Azure SQL Business Critical vs. General Purpose

Azure SQL HA behavior depends on the service tier. General Purpose uses remote storage (Azure Premium SSD) with a database availability SLA achieved through Azure infrastructure-level redundancy. Failover in the General Purpose tier can take 25–30 seconds as data files are remounted on a new compute node. Business Critical deploys an Always On availability group across three replicas using local SSD storage. One of those replicas is readable and accessible to your application — unlike RDS Multi-AZ standby, the Business Critical secondary serves read workloads without additional configuration.

The read-scale secondary in Business Critical is accessed via a connection string that includes `ApplicationIntent=ReadOnly`:

```
-- Connect string for Azure SQL Business Critical read-only replica
-- Server=prod-sql-server-01.database.windows.net;
-- Database=prod-db;
-- Authentication=Active Directory Default;
-- ApplicationIntent=ReadOnly;

-- Verify you are on the secondary
SELECT DATABASEPROPERTYEX(DB_NAME(), 'Updateability') AS replica_role;
-- Returns: READ_ONLY when connected to secondary
-- Returns: READ_WRITE when connected to primary
```

The HA tiers, failover behaviors, and read-scale capabilities across cloud providers have direct cost implications. Choosing between RDS for SQL Server Multi-AZ, SQL Server on EC2 with Always On AGs, Azure SQL General Purpose, and Business Critical is as much an architectural decision as a financial one, and it belongs in the DBA's scope, not just the infrastructure team's.

4.5 Monitoring: CloudWatch, Azure Monitor, and Cloud Logging vs. What You Had Before

On-premises DBAs typically own their full monitoring stack — Datadog agents, custom scripts querying `sys.dm_exec_requests`, `sys.dm_os_wait_stats`, and `sys.dm_tran_locks`. In the cloud, the provider delivers a baseline monitoring layer automatically, and the DBA's job shifts from building that layer to interpreting it, extending it, and setting appropriate thresholds.

CloudWatch and Enhanced Monitoring on RDS

Every RDS instance publishes a set of CloudWatch metrics automatically with no configuration required: `CPUUtilization`, `DatabaseConnections`, `FreeableMemory`, `ReadIOPS`, `WriteIOPS`, `ReadLatency`, `WriteLatency`, `ReplicaLag`. These metrics refresh at 60-second intervals by default.

Enhanced Monitoring is an additional feature that installs an agent on the DB instance OS (which you cannot access directly) and publishes 50+ OS-level metrics at 1-second to 60-second granularity to CloudWatch Logs. These include per-process CPU, memory, disk I/O broken out by device, and file system metrics. Enable it for production instances — the cost is negligible compared to the diagnostic value during an incident.

```
# Enable Enhanced Monitoring with 15-second granularity
aws rds modify-db-instance \
  --db-instance-identifier prod-sql-01 \
  --monitoring-interval 15 \
  --monitoring-role-arn arn:aws:iam::123456789012:role/rds-monitoring-role \
  --apply-immediately

# Query CloudWatch for replica lag over the last hour
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name ReplicaLag \
  --dimensions Name=DBInstanceIdentifier,Value=prod-sql-reader-1 \
  --start-time $(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%SZ) \
  --period 60 \
  --statistics Maximum \
  --output table
```

Performance Insights

Performance Insights is the cloud analog to Oracle's Active Session History and a significant improvement over what most self-managed SQL Server deployments have out of the box. It records database load in terms of active sessions broken down by wait event, SQL statement, host, and user. The default retention is 7 days free; extending to up to 2 years incurs an additional per-vCPU-hour charge — check current AWS pricing.

For a production workload investigation, Performance Insights replaces the manual practice of sampling `sys.dm_exec_requests` every few seconds and correlating the results. The UI presents a visual timeline of database load against the `max_vCPUs` line — anything above that line indicates the database is CPU-bound or waiting on a bottleneck that requires investigation.


```

# Retrieve Performance Insights data via boto3
import boto3
from datetime import datetime, timedelta, timezone

pi_client = boto3.client('pi', region_name='us-east-1')

response = pi_client.get_resource_metrics(
    ServiceType='RDS',
    Identifier='db-ABCDEFGHIJ1234567', # DBI resource ID, not instance ID
    MetricQueries=[
        {
            'Metric': 'db.load.avg',
            'GroupBy': {
                'Group': 'db.wait_event',
                'Limit': 10
            }
        }
    ],
    StartTime=datetime.now(timezone.utc) - timedelta(hours=1),
    EndTime=datetime.now(timezone.utc),
    PeriodInSeconds=60
)

for result in response['MetricList']:
    print(f"Metric: {result['Key']}")
    for dp in result['DataPoints'][-5:]: # last 5 data points
        print(f"  {dp['Timestamp']}: {dp.get('Value', 'N/A'):.4f}")

```

Azure Monitor and Query Performance Insight

Azure SQL Database surfaces equivalent functionality through Azure Monitor metrics (DTU percentage, CPU percentage, data I/O percentage, log write percentage, connection counts) and the Query Performance Insight blade, which aggregates data from Query Store — a feature that has been running automatically in Azure SQL since it was introduced in SQL Server 2016.

Query Store is particularly valuable because it captures query plan changes over time. A query that performed well for three months and suddenly regressed after an auto-statistics update will show the plan change in Query Store with the before and after execution statistics.

```
-- Azure SQL: find the top 10 queries by total CPU consumption in the last 24 hours
SELECT TOP 10
    q.query_id,
    qt.query_sql_text,
    SUM(rs.avg_cpu_time * rs.count_executions) AS total_cpu_ms,
    SUM(rs.count_executions) AS total_executions,
    SUM(rs.avg_duration * rs.count_executions) AS total_duration_ms
FROM sys.query_store_query_text qt
JOIN sys.query_store_query q ON qt.query_text_id = q.query_text_id
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
JOIN sys.query_store_runtime_stats_interval rsi
    ON rs.runtime_stats_interval_id = rsi.runtime_stats_interval_id
WHERE rsi.start_time >= DATEADD(HOUR, -24, GETUTCDATE())
GROUP BY q.query_id, qt.query_sql_text
ORDER BY total_cpu_ms DESC;
```

The monitoring shift is real: you spend less time building monitoring infrastructure and more time interpreting richer, already-collected data. The tradeoff is that cloud monitoring tools have gaps — they do not expose lock wait graphs as clearly as `sys.dm_tran_locks` queries do on self-managed SQL Server, and correlating application traces with database wait events across CloudWatch, X-Ray, and RDS Performance Insights requires more configuration than running a single monitoring agent that sees both layers.

4.6 The New DBA Work: Cost Management, IaC, and Architecture Ownership

The reduction in operational work does not leave a vacuum. Cloud DBAs acquire a meaningful set of responsibilities that did not exist in their on-premises roles: database infrastructure as code, cost engineering, and cross-service architecture design.

Infrastructure as Code for Databases

In on-premises environments, provisioning a SQL Server VM involved coordinating with a server team, a storage team, and a network team, filling out a change request, and waiting. In the cloud, a DBA with appropriate IAM permissions can provision a multi-AZ RDS for SQL Server instance with encrypted storage, automated backups, and VPC-isolated networking in under 10 minutes using Terraform. The DBA who understands both the database requirements and the infrastructure configuration is in a stronger position than one who waits for an infrastructure team to provision resources.

```
# Terraform: production RDS for SQL Server instance
resource "aws_db_instance" "prod" {
  identifier          = "prod-sqlserver"
  engine              = "sqlserver-se"
  engine_version      = "16.00.4125.1.v1"
  instance_class      = "db.r7i.2xlarge"
  allocated_storage   = 500
  storage_type        = "gp3"
  multi_az            = true
  storage_encrypted    = true
  backup_retention_period = 7
  deletion_protection = true
}
```

Treating database infrastructure as code gives you version history, peer review, repeatable deployments across environments, and the ability to reconstruct a destroyed environment from source control. These are practices that are standard in software engineering and are now expected of cloud DBAs. If your team does not already do this, starting with a simple Terraform module for a non-production cluster is the practical entry point.

Cost Engineering

On-premises database costs are largely fixed — server hardware depreciates, storage is bought in bulk, and the marginal cost of running a query is invisible. In the cloud, every instance-hour, every GB of storage, every I/O operation (on gp2 storage), and every GB of data transferred across AZs has a line-item cost. A cloud DBA who understands the cost model can make architecture recommendations that have direct financial impact.

The most common cost levers for RDS for SQL Server and Azure SQL are:

Instance sizing and Reserved Instances / Savings Plans. On-demand pricing for a `db.r7i.2xlarge` running SQL Server Standard in us-east-1 is meaningful at monthly scale — SQL Server licensing is a large share of that line item. Purchasing a one-year Reserved Instance (or Azure's one-/three-year reservations with Hybrid Benefit) for the same size produces a substantial discount compared to on-demand — consult the current pricing calculator for exact figures, as prices change. Most production workloads with stable compute requirements should run on Reserved Instances. A common failure pattern is running development and staging instances on on-demand because they were provisioned ad hoc, then leaving them running indefinitely.

Storage autoscaling. RDS supports storage autoscaling — storage grows automatically when free space falls below a threshold. This prevents the 2 AM call about a full disk but can mask unexpected data growth. Set a `MaxAllocatedStorage` limit and alert on storage growth rate, not just absolute free space.

Readable-secondary sizing (self-managed). On SQL Server on EC2 or Azure VMs, Always On secondaries that serve reporting workloads do not need to match the primary's instance class. Size secondaries for the read workload — reporting-heavy secondaries may want more memory for sorts and hashes, while simple lookup secondaries can be smaller. (On RDS for SQL Server there are no readable secondaries, so this lever applies to the VM deployments.)

```
# List all running RDS instances and their costs-relevant attributes
aws rds describe-db-instances \
  --query 'DBInstances[*].{
    ID:DBInstanceIdentifier,
    Class:DBInstanceClass,
    Engine:Engine,
    EngineVersion:EngineVersion,
    MultiAZ:MultiAZ,
    Storage:AllocatedStorage,
    StorageType:StorageType,
    Status:DBInstanceStatus
  }' \
  --output table

# Identify instances without Reserved Instance coverage using Cost Explorer API
aws ce get-reservation-coverage \
  --time-period Start=2024-01-01,End=2024-01-31 \
  --granularity MONTHLY \
  --filter '{
    "Dimensions": {
      "Key": "SERVICE",
      "Values": ["Amazon Relational Database Service"]
    }
  }' \
  --query 'CoveragesByTime[*].Groups[*].{
    Instance:Attributes.instanceType,
    CoverageHours:Coverage.CoverageHours.CoverageHoursPercentage
  }' \
  --output table
```

Architecture Ownership

The cloud DBA's most expansive new responsibility is architecture ownership — the ability to design and advocate for database solutions that span multiple cloud services rather than a single database engine. An on-premises DBA might optimize a single SQL Server cluster indefinitely. A cloud DBA is frequently asked whether data should live in RDS for SQL Server, SQL Server on EC2, Azure SQL Database, or be offloaded to columnstore analytics (Chapter 22), and the answer requires understanding access patterns, query shapes, consistency requirements, and cost at scale.

This does not mean the cloud DBA becomes a generalist who knows everything shallowly. It means the DBA's deep expertise in relational database behavior is the anchor for evaluating other options. Understanding why a particular workload is poorly suited for a key-value store, or why an analytical query pattern belongs in a columnar store rather than a row store, is exactly the kind of judgment that deep database expertise develops.

4.7 What Genuinely Does Not Change

It is worth being direct about this, because hype around cloud services can create the impression that managed databases are fundamentally different systems that require entirely new skills.

Query optimization does not change. Actual execution plans on RDS for SQL Server are produced by the same optimizer as on self-managed SQL Server. The same principles govern index selection, join ordering, statistics quality, and plan regression. Query Store on Azure SQL and Performance Insights on RDS give you better visibility into query behavior than most on-premises deployments had, but interpreting that data requires the same foundational knowledge of execution plans and cost estimation.

```
-- SQL Server: plan and IO analysis works identically on RDS, Azure SQL, and self-
managed
SET STATISTICS IO, TIME ON;
SELECT
    o.order_id,
    o.created_at,
    c.email,
    SUM(oi.quantity * oi.unit_price) AS order_total
FROM dbo.orders o
JOIN dbo.customers c ON o.customer_id = c.customer_id
JOIN dbo.order_items oi ON o.order_id = oi.order_id
WHERE o.created_at >= DATEADD(DAY, -30, GETUTCDATE())
    AND o.status = 'completed'
GROUP BY o.order_id, o.created_at, c.email
ORDER BY o.created_at DESC;
```

Schema design does not change. Normalization, denormalization tradeoffs, partitioning strategies, and foreign key design are engine-level concerns that the cloud provider does not touch. A poorly designed schema that causes page splits through inefficient clustered index choices will fragment just as effectively on RDS as on a bare-metal SQL Server.

Backup and recovery reasoning does not change. The mechanics of backup change — you do not run `BACKUP DATABASE` manually on RDS for SQL Server, and you do not schedule SQL Server maintenance plan jobs on Azure SQL. But the reasoning about RPO, RTO, backup

retention, and recovery testing does not change. The cloud DBA must still define backup retention windows, understand what PITR can recover and what it cannot, and test restore procedures regularly.

Transaction management and isolation levels do not change. `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE` — these are engine behaviors that the cloud wrapper does not modify. Application developers who write code with implicit assumptions about transaction boundaries will cause the same data integrity problems on RDS that they would cause on your on-premises server.

Locking and concurrency design does not change. Application locks via `sp_getapplock`, row-level locking, table-level locking, deadlock detection — these behave identically. The cloud DBA who knows how to diagnose a lock contention issue using `sys.dm_tran_locks` and `sys.dm_exec_requests` on self-managed SQL Server will apply exactly the same skills on RDS.

```
-- SQL Server: identify blocking chains (works on RDS, Azure SQL, self-managed
identically)
-- SQL Server equivalent (Azure SQL)
SELECT
    blocked_req.session_id AS blocked_session,
    blocked_req.wait_type,
    blocked_req.wait_time / 1000.0 AS wait_seconds,
    blocked_req.blocking_session_id AS blocking_session,
    blocked_sql.text AS blocked_query,
    blocking_sql.text AS blocking_query,
    blocked_req.status
FROM sys.dm_exec_requests blocked_req
CROSS APPLY sys.dm_exec_sql_text(blocked_req.sql_handle) blocked_sql
JOIN sys.dm_exec_sessions blocking_sess
    ON blocking_sess.session_id = blocked_req.blocking_session_id
CROSS APPLY sys.dm_exec_sql_text(blocking_sess.most_recent_sql_handle) blocking_sql
WHERE blocked_req.blocking_session_id > 0
ORDER BY blocked_req.wait_time DESC;
```

The cloud changes where your database runs and who manages the infrastructure around it. It does not change how relational databases work or what makes them perform well. The DBA who has mastered the fundamentals carries those skills directly into the cloud environment and adds cloud-specific operational knowledge on top of that foundation.

4.8 Organizational Changes: The DBA in Cloud-Native Teams

Beyond the technical changes, the cloud DBA role shifts in organizational context. In many organizations that move to cloud-managed databases, the traditional operations-focused DBA headcount decreases while the need for database engineers embedded in development teams or platform teams increases.

The distinction matters. An operations DBA monitors a fleet of databases, responds to incidents, performs maintenance, and keeps the lights on. A database engineer (the role that cloud-managed services tend to create demand for) contributes to architectural decisions, reviews schema changes, automates database provisioning and migrations, builds the observability layer, and advises development teams on data model and query design choices. If you are currently an operations DBA in an organization moving to managed cloud databases, understanding this shift is important for positioning your career.

The practical advice is to develop skills in three areas that complement your existing database expertise and increase your value in a cloud context:

1. **Infrastructure as Code fluency.** Terraform or Pulumi for database infrastructure is a baseline expectation on mature cloud teams. If you can write and review Terraform for an RDS SQL Server instance, security groups, parameter groups, and Secrets Manager integration, you can contribute directly to platform work rather than waiting for infrastructure teams to provision what you need.
2. **Observability and alerting design.** Building CloudWatch alarms, Azure Monitor alert rules, and dashboard configurations that accurately surface real problems without generating noise is a skill. Knowing which thresholds matter (replica lag exceeding a threshold appropriate for your RPO, connection pool saturation, storage growth rate), and knowing how to page on the right signals, is a DBA-specific contribution to the observability stack.
3. **Database migration engineering.** Moving data between environments — from on-premises to cloud, between cloud providers, or between database engines — is a domain where deep database knowledge is essential and where mistakes have serious consequences. AWS Database Migration Service, Azure Database Migration Service, native backup/restore to object storage, and log shipping are all in scope. Chapter 9 covers this in depth.

Key Takeaways

- → The cloud provider owns the OS, storage subsystem, replication wiring, and patching lifecycle; the DBA owns configuration, schema design, query performance, access control policy, and backup retention — this division of responsibility is firm and understanding it prevents both misplaced effort and dangerous assumptions.
- → Authentication and authorization move from engine-native mechanisms (SQL Server logins, Windows authentication) to cloud IAM layers (AWS IAM database authentication, Azure Entra ID contained users), requiring the DBA to become conversant in cloud identity systems while retaining all existing knowledge of database-level privilege management.
- → Configuration management shifts from editing flat files to managing Parameter Groups (RDS) or database-scoped configurations (Azure SQL), with certain parameters locked because the provider manages them on your behalf — most notably, true superuser access does not exist on RDS, which affects extension management and some administrative operations.
- → Core DBA expertise — query optimization, execution plan analysis, schema design, locking and concurrency management, transaction isolation reasoning, and backup/recovery strategy — transfers without modification to managed cloud databases; cloud wrappers change where the database runs, not how relational engines work.
- → Cloud DBAs acquire meaningful net-new responsibilities in infrastructure as code, cost engineering (Reserved Instances, storage tier selection, I/O optimization), and cross-service architecture design, while the organizational context shifts toward database engineering roles embedded in development and platform teams rather than purely operations-focused roles.

Chapter 5: SQL Server Cloud Architecture Patterns

Cloud SQL Server deployments repeat a small set of architectural patterns: how availability is achieved, how reads scale, how storage grows, how the network is laid out, how tuning happens when the OS is not yours, how serverless and auto-scaling work, how multi-region DR is designed, how observability is assembled, how cost is engineered, and what each provider constrains. This chapter catalogs the patterns with SQL Server as the engine throughout.

5.1 High Availability and Failover Architecture

RDS for SQL Server Multi-AZ deploys a synchronous standby in a second Availability Zone using database mirroring internals; the endpoint fails over via DNS, typically in 60–120 seconds. The standby is not readable. **Azure SQL** offers multiple HA tiers: General Purpose uses remote storage with a hot standby (failover ~20–30 seconds); Business Critical runs an Always On availability group across nodes (failover typically faster, with a readable secondary included); Hyperscale disaggregates compute from page servers, making failover fast regardless of database size. **Cloud SQL for SQL Server** uses regional Persistent Disk with synchronous replication to a standby zone — failover is automatic, typically 60–120 seconds.

Self-managed HA: SQL Server on EC2, Azure VMs, or GCE supports full Always On availability groups — synchronous replicas across zones, readable secondaries, and (with a witness and stretched configuration) multi-region AGs. This is the HA pattern that gives you readable secondaries on AWS/GCP, where the managed services don't offer them (Chapter 8).

```
# Terraform: RDS for SQL Server with Multi-AZ and backup retention
resource "aws_db_instance" "prod" {
  identifier          = "prod-sqlserver"
  engine              = "sqlserver-se"
  engine_version      = "16.00.4125.1.v1"
  instance_class      = "db.r7i.2xlarge"
  allocated_storage   = 1000
  storage_type        = "gp3"
  multi_az            = true
  storage_encrypted    = true
  backup_retention_period = 14
  deletion_protection = true
}
```

5.2 Read Scaling and Replica Patterns

RDS for SQL Server has **no read replicas** — the managed service on AWS is single-writer with a non-readable standby. Read scaling on AWS means either SQL Server on EC2 with Always On readable secondaries, or offloading reads to a different tier (Chapter 22's columnstore analytics pattern). **Azure SQL Hyperscale** supports named replicas (up to 30) that share the underlying page-server storage — Microsoft's answer to read scale-out, with replica lag measured in milliseconds because replicas read from shared pages. **Azure SQL Database** geo-replicas are readable secondaries in other regions for both DR and read offload (Chapter 16). **Cloud SQL** read replicas serve reads but promote-to-standalone for DR rather than preserving a global endpoint (Chapter 19).

Connection pooling pattern: a standalone pooler in front of a replica pool doesn't translate to managed services directly — RDS Proxy (Chapter 10) is the managed alternative for SQL Server connection fan-in on AWS, while Azure and GCP expect application-level pooling with driver-level failover awareness. When replicas have lag (self-managed AG secondaries under heavy write load, cross-region replicas), route lag-tolerant reads to secondaries and lag-sensitive reads to the primary — never assume replica reads are current.

5.3 Storage Architecture and Auto-Scaling

RDS storage is EBS-backed: gp3 (baseline performance, provisionable IOPS), io1/io2 (high IOPS). Storage increases are permanent — you cannot shrink. Scaling a Multi-AZ instance's storage triggers a failover-like event; plan the operational window. **Azure SQL Hyperscale** storage grows to 100 TB independently of compute, and restores complete in minutes regardless of database size because only metadata is rehydrated. **Cloud SQL** storage auto-scales to 64 TB. The operational rules: set `MaxAllocatedStorage` on RDS to bound auto-scaling cost, alert on growth rate not just free space, and remember that storage tier selection (gp3 vs io2 — Chapter 36) is a performance decision disguised as a cost decision.

```
-- T-SQL: compare logical database sizes against provisioned storage
SELECT DB_NAME(database_id) AS db_name,
       SUM(size) * 8.0 / 1024 / 1024 AS total_gb,
       SUM(FILEPROPERTY(name, 'SpaceUsed')) * 8.0 / 1024 / 1024 AS used_gb
FROM sys.master_files
GROUP BY database_id
ORDER BY total_gb DESC;
```

5.4 Network Architecture: VPC, Private Link, and Connectivity

Every managed instance lives inside the cloud's network: **RDS** in a VPC via DB subnet groups spanning at least two AZs, with security groups as stateful firewalls. **Azure SQL** reachable via public endpoint with firewall rules, or privately via Azure Private Link with private DNS. **Cloud SQL** via authorized networks or private IP through Private Service Connect. The architecture patterns: put databases in private subnets with no internet route; use private endpoints/links so traffic never crosses the public internet; peer or hub-and-spoke VNets/VPCs for cross-region and on-premises connectivity (ExpressRoute, Direct Connect, Cloud Interconnect); and model egress costs — cross-AZ and cross-region traffic is billed (Chapter 40).

SQL Server Private Network Topology

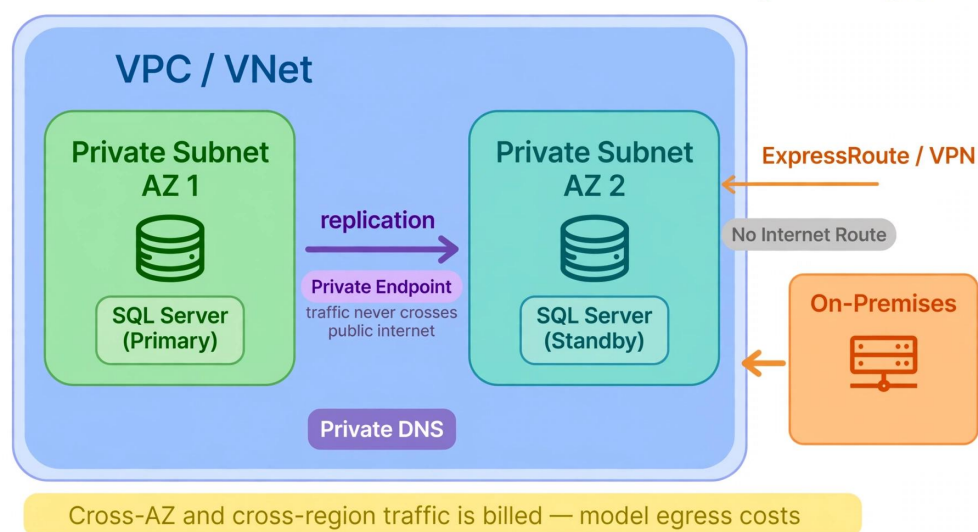


Figure 5.1 — SQL Server in a private network: VPC topology, private endpoints, and on-premises connectivity.

How to read it: the **blue VPC/VNet zone** is the network boundary — everything inside it stays off the public internet. Each **green/teal private subnet** spans a different availability zone and holds a SQL Server instance, with the **purple replication arrow** keeping the standby current. The **purple Private Endpoint** pill is the connection path applications use: traffic enters the VPC but never crosses the public internet, and **Private DNS** resolves the database name to that private address. The **orange On-Premises zone** connects back via ExpressRoute or VPN, and the **gray "No Internet Route" badge** is the security guarantee — the databases have no path to or from the internet at all. The **yellow banner** is the cost rule: replication and cross-zone traffic is billed, so place secondaries where the economics favor them (Chapter 37) and model egress before you scale out.

5.5 Parameter Tuning in Managed Environments

Managed services expose a curated subset of engine settings: **RDS** via DB parameter groups (the `SQLSERVER_2019` / `SQLSERVER_2022` families — Chapter 6); **Azure SQL** via database-scoped configuration and portal settings; **Cloud SQL for SQL Server** via a narrow documented flag set (most tuning happens through `sp_configure` inside the instance). The tuning pattern: apply changes through IaC (parameter group in Terraform, database-scoped config in scripts) so configuration is versioned; test non-dynamic settings in staging first; and accept that settings the provider doesn't expose (startup parameters, trace flags on some services) are off-limits — design around them (Chapter 35).

5.6 Serverless and Auto-Scaling Patterns

Azure SQL Database serverless auto-scales vCores between a floor and ceiling and auto-pauses after inactivity — the pattern for intermittent workloads (Chapter 11). Compute auto-scaling for SQL Server is otherwise limited: managed services don't vertically scale on load automatically, so the pattern is **scheduled scaling** (scale up before the business day, down at night) and **read-scale routing** (direct reporting to Hyperscale replicas or AG secondaries). For containers and Lambda-like workloads generating connection storms, RDS Proxy (Chapter 10) absorbs the fan-in. The serverless decision rule: variable, intermittent workloads with pausable demand belong on serverless; steady production belongs on provisioned with commitments (Chapter 38).

5.7 Multi-Region and Disaster Recovery Architecture

Cross-region DR patterns by platform: **RDS for SQL Server** uses cross-region native backups to S3 plus restore automation, or log shipping to a second region's instance (Chapter 16) — there is no managed cross-region replica for SQL Server on RDS. **Azure SQL** offers auto-failover groups: a readable geo-secondary, DNS-level failover, and typically sub-minute RTO for planned failover (Chapter 16). **Cloud SQL** cross-region read replicas exist but promote to standalone instances — your automation must handle the endpoint switch (Chapter 19).

The DR table: for RPO near zero and RTO under minutes, Azure failover groups are the strongest native option; for AWS, cross-region log shipping with tested restore automation is the production pattern (Chapter 16). Whatever the mechanism, DR is only real if tested quarterly — an untested DR plan is a hope (Chapter 29).

5.8 Observability Architecture: Metrics, Logs, and Tracing

Assemble the three pillars per platform: **metrics** (CloudWatch, Azure Monitor, Cloud Monitoring — host and database metrics with dashboards per Chapter 30), **logs** (SQL Server error logs exported to CloudWatch Logs / Log Analytics / Cloud Logging, with alerts on severity), and **query tracing** (Query Store on Azure, Performance Insights on RDS, Query Insights on Cloud SQL — the query-level layer that identifies *what* is slow). The architecture pattern: platform metrics answer "is the database healthy," query telemetry answers "which query is the problem," and the alerting hierarchy (page/ticket/log — Chapter 30) routes each signal to the right urgency.

5.9 Cost Architecture: Designing for Efficiency

Design cost in, don't bolt it on: choose the storage tier for the I/O profile (gp3 vs io2 — Chapter 36), commit the stable baseline to reservations (Chapter 38), right-size from measured peak not provisioned headroom (Chapter 39), place secondaries where the economics favor them (Chapter 37), and consolidate where density pays (elastic pools — Chapter 14). The io2-vs-gp3 decision alone can swing storage cost substantially for I/O-heavy workloads — pull actual IOPS from monitoring and price both models (Chapter 36). Cost architecture is capacity architecture done honestly.

5.10 Feature and Component Constraints by Cloud Provider

The constraints that shape architecture: **RDS for SQL Server** — no readable secondaries, no IAM database authentication, no SQL Server Agent on some editions (check current docs), limited sysadmin. **Azure SQL Database** — no Agent, no cross-database queries without elastic queries, DTU vs vCore model choice. **Cloud SQL for SQL Server** — narrower flag surface, regional HA model. **Self-managed VMs** — full engine, but you own patching, backups, and AG operations. Every architecture decision in this book is a negotiation with one of these constraint sets (Chapters 6–19 detail each platform).

Key Takeaways

- → **Cloud HA is managed abstraction, not magic:** RDS Multi-AZ gives 60–120 second failover with a non-readable standby; Azure Business Critical is faster with a readable secondary included; Hyperscale makes failover fast at any size. Know your platform's numbers.
- → **Read scaling is platform-specific:** RDS for SQL Server has no read replicas — reads scale via AG secondaries on EC2 or Azure's Hyperscale named replicas. Design the read path per platform.

- → **Storage scaling is one-directional and has operational windows:** RDS storage increases are permanent and scaling a Multi-AZ instance triggers an event — plan windows and set `MaxAllocatedStorage`.
- → **Parameter tuning is curated and versioned:** use DB parameter groups, database-scoped configuration, and IaC so configuration is reviewable and reproducible.
- → **DR is only real if tested:** cross-region mechanisms differ per platform (Azure failover groups lead natively); quarterly DR tests turn the architecture into a capability.
- → **Every architecture decision negotiates with provider constraints:** no readable secondaries on RDS, no Agent on Azure SQL Database, narrow flags on Cloud SQL — Chapters 6–19 detail each constraint set.

Chapter 6: RDS Deep Dive: Engines, Instance Types, and Storage

Amazon RDS is where most DBAs begin their cloud journey, and for good reason — it takes the engine you already know — SQL Server — and wraps it in a managed layer that handles provisioning, patching, and backups. But "managed" does not mean "transparent." The choices you make about engine version, instance family, and storage type have cascading effects on performance, cost, and operational flexibility that cannot be undone without downtime or data migration. This chapter breaks down those choices with the specificity that production systems demand: which instance families to use and why, how RDS storage behaves differently from what you know on-premises, where the hard limits are, and what you give up when AWS takes over the OS layer.

6.1 Engine Selection and Version Strategy

RDS supports several database engines; this book covers RDS for SQL Server, which runs on a managed EC2 instance underneath, and the distinction between "RDS for SQL Server" and "self-managed SQL Server on EC2" is not just operational convenience — it represents a fundamental trade-off in control.

What you lose versus self-managed SQL Server on EC2:

- → No OS-level access. You cannot SSH into the underlying host.
- → No direct server-property editing. Authentication rules are enforced through security groups, IAM database authentication, and TLS requirements set via parameter groups.
- → No sysadmin. The master user grants most capabilities, but it is not true sysadmin — you cannot enable arbitrary trace flags, use the dedicated admin connection freely, or load unapproved CLR assemblies.
- → No transaction log file access. You cannot read the raw LDF files directly; transaction log backups are managed by RDS.
- → No `BACKUP DATABASE` to local disk. Backups are managed by RDS; native backups to S3 require the SQLSERVER_BACKUP_RESTORE option group.

What you gain:

- → Automated backups with point-in-time recovery (PITR) up to 35 days.
- → Automated minor version upgrades on a configurable maintenance window.

- → Multi-AZ failover handled by the platform without you managing Windows Server Failover Clustering.
- → Monitoring metrics surfaced through CloudWatch without running exporters.

Engine version selection is not just about features — it determines your upgrade path. RDS maintains a list of supported minor versions, and AWS periodically deprecates older ones, issuing mandatory upgrades. SQL Server 2012 reached end of extended support and AWS deprecated it on RDS — instances still running deprecated versions receive forced upgrade notices. You should never run more than one major version behind current in production.

Check available versions before provisioning:

```
# List all available SQL Server engine versions in us-east-1
aws rds describe-db-engine-versions \
  --engine sqlserver-se \
  --query 'DBEngineVersions[*].[EngineVersion,DBEngineVersionDescription]' \
  --output table \
  --region us-east-1

# Check which versions support a specific feature (e.g., Kerberos auth)
aws rds describe-db-engine-versions \
  --engine sqlserver-se \
  --query 'DBEngineVersions[?contains(SupportedFeatureNames, `kerberos`) == `true`].
[EngineVersion]' \
  --output text \
  --region us-east-1
```

SQL Server licensing deserves separate attention. RDS SQL Server offers two licensing models: License Included (LI) and Bring Your Own License (BYOL). License Included is available for Standard and Enterprise editions, with the license cost embedded in the hourly rate. Enterprise edition on a large instance class adds a substantial license-included premium to the hourly rate — check current AWS pricing for the specific instance class and edition. If your organization already has SQL Server Enterprise Agreement licenses, BYOL reduces that cost significantly — but BYOL requires Software Assurance, and AWS audits eligibility.

Oracle on RDS follows the same LI vs. BYOL model. Oracle Enterprise Edition is only available as BYOL. Standard Edition 2 is available as License Included. The moment you need features like Partitioning, Advanced Compression, or Real Application Testing, you're in Enterprise Edition territory, which means BYOL and your own Oracle licensing agreement.

Version pinning strategy: In production, pin your minor version explicitly. Do not enable auto minor version upgrade on database instances that have application dependencies on specific behavior (system behavior changed between SQL Server 2019 and 2022, for example). Enable auto minor version upgrade only on development and QA instances.


```
# Disable auto minor version upgrade on an existing instance
aws rds modify-db-instance \
  --db-instance-identifier prod-sql-01 \
  --no-auto-minor-version-upgrade \
  --apply-immediately \
  --region us-east-1

# Verify the change
aws rds describe-db-instances \
  --db-instance-identifier prod-sql-01 \
  --query 'DBInstances[0].AutoMinorVersionUpgrade' \
  --region us-east-1
```

6.2 Instance Families and Sizing

The RDS instance naming convention follows the same pattern as EC2: `db.{family}{generation}.{size}`. Understanding the family is more important than the size, because each family has different CPU architecture, memory ratios, and network characteristics.

Current generation families:

Family	vCPU:Memory Ratio	Use Case	Example
db.t3	Burstable, 1:2	Dev/test, small workloads	db.t3.medium: 2 vCPU, 4 GB
db.m7i	General purpose, 1:4	Web applications, moderate OLTP	db.m7i.4xlarge: 16 vCPU, 64 GB
db.r7i	Memory optimized, 1:8	Large buffers, analytics	db.r7i.8xlarge: 32 vCPU, 256 GB
db.r7i	Memory optimized, Intel	SQL Server (no ARM support)	db.r7i.16xlarge: 64 vCPU, 512 GB
db.x2idn	Memory intensive, 1:32	SAP HANA, large in-memory	db.x2idn.16xlarge: 64 vCPU, 2048 GB

The `g` suffix means Graviton (ARM-based). Graviton-based instance classes are not available for SQL Server on RDS — choose the Intel/AMD `i`-family classes (e.g., `db.r7i`, `db.m7i`) for SQL Server workloads.

T-series instances (burstable) in production: The T-series uses CPU credits. When the instance is idle, it accumulates credits. Under sustained load, it burns through credits and throttles to its baseline CPU (which is 20–40% of nominal). This behavior is catastrophic for production OLTP workloads that have sustained query pressure. The only legitimate production use of T-series is for small dev/test instances or internal tooling databases with infrequent queries. (T-series is unavailable for RDS for SQL Server Multi-AZ production workloads — check current docs for edition support.)

You can detect credit exhaustion in CloudWatch:

```
# Check CPU credit balance for a burstable instance
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name CPUCreditBalance \
  --dimensions Name=DBInstanceIdentifier,Value=dev-sqlserver-t4g \
  --start-time 2024-01-15T00:00:00Z \
  --end-time 2024-01-15T23:59:59Z \
  --period 3600 \
  --statistics Minimum \
  --output table

# Also check CPU credit usage rate
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name CPUCreditUsage \
  --dimensions Name=DBInstanceIdentifier,Value=dev-sqlserver-t4g \
  --start-time 2024-01-15T00:00:00Z \
  --end-time 2024-01-15T23:59:59Z \
  --period 3600 \
  --statistics Sum \
  --output table
```

Sizing recommendations for SQL Server migration: When migrating a self-managed SQL Server instance, the most important metrics from your source system describe buffer pool behavior — Page Life Expectancy (PLE), buffer cache hit ratio, and lazy writes/sec. On RDS for SQL Server, max server memory is managed by AWS based on instance class. A db.r7i.4xlarge has 128 GB of RAM. If your on-premises system has a 40 GB buffer pool with a healthy PLE and a buffer cache hit ratio above 95%, move to db.r7i.8xlarge (256 GB RAM) with headroom.

The right way to size using actual database metrics:

```
-- Run this on your SOURCE SQL Server instance before migration
-- to understand buffer pool behavior

-- Buffer Manager counters (PLE healthy range scales with RAM: roughly 300s per 4GB)
SELECT counter_name, cntr_value
FROM sys.dm_os_performance_counters
WHERE object_name LIKE '%Buffer Manager%'
    AND counter_name IN (
        'Buffer cache hit ratio',
        'Page life expectancy',
        'Lazy writes/sec',
        'Checkpoint pages/sec'
    );

-- Working set estimate: buffer pool usage by database
SELECT DB_NAME(database_id) AS database_name,
       COUNT(*) * 8 / 1024 AS buffer_pool_mb
FROM sys.dm_os_buffer_descriptors
GROUP BY database_id
ORDER BY buffer_pool_mb DESC;

-- Memory pressure signals: stolen pages and out-of-memory notifications
SELECT counter_name, cntr_value
FROM sys.dm_os_performance_counters
WHERE object_name LIKE '%Memory Manager%'
    AND counter_name IN ('Stolen Server Memory (KB)', 'Out of memory errors');
```

Changing instance class: RDS instance class changes require a reboot. On Single-AZ, this means downtime of 3–10 minutes. On Multi-AZ, AWS performs the change on the standby first, then fails over — total visible downtime is typically 30–120 seconds for the failover. Plan instance resizing during low-traffic windows regardless of Multi-AZ, because the failover itself causes connection drops.

6.3 RDS Storage: gp2, gp3, io1, io2, and Magnetic

Storage in RDS is EBS-backed, and the storage type you choose determines IOPS characteristics, throughput ceilings, and cost. Getting this wrong is the most common cause of RDS performance problems, because storage bottlenecks in managed services are much harder to diagnose than on-premises where you can run `iostat` directly.

Available storage types:

gp2 (General Purpose SSD): The legacy default. Delivers 3 IOPS per GB of storage, with a burst pool for volumes under 1 TB. A 100 GB gp2 volume gets 300 IOPS baseline with burst to 3,000 IOPS using credits (same credit model as T-series CPU). Above roughly 5.3 TB (where

3 IOPS/GB reaches the 16,000 IOPS ceiling), you get the full ceiling without burst dependency. Maximum throughput: 250 MB/s. Do not use gp2 for new deployments — it has been superseded entirely by gp3.

gp3 (General Purpose SSD v2): The current recommended default. Provides a baseline of 3,000 IOPS and 125 MB/s throughput regardless of volume size. You can independently provision additional IOPS up to 16,000 and throughput up to 1,000 MB/s at additional cost. A 100 GB gp3 volume gets 3,000 IOPS — the same as a 1 TB gp2 volume, at one-tenth the storage cost. For most OLTP workloads under 16,000 IOPS, gp3 is the right choice.

io1 (Provisioned IOPS SSD): Allows IOPS provisioning independent of storage size, up to 64,000 IOPS on supported instance types. Maximum throughput: 1,000 MB/s. More expensive than gp3 for the same IOPS level. io1 made sense before gp3 existed; today, it's primarily for workloads needing more IOPS than gp3 provides but that don't require io2's higher durability or ceiling.

io2 (Provisioned IOPS SSD v2): Higher durability (99.999% vs 99.8% for io1), same IOPS ceiling. For SQL Server and Oracle, io2 supports up to 256,000 IOPS. For most SQL Server workloads, gp3 with provisioned IOPS is sufficient and cheaper.

Magnetic (standard): Legacy spinning disk. Do not use this for any workload. It exists only for backward compatibility.

Storage autoscaling: RDS supports automatic storage scaling. When free storage drops below 10% of allocated storage (or below 5 GB, whichever is smaller), RDS automatically increases storage. Autoscaling only increases storage — it never shrinks it. You cannot reduce allocated storage on an RDS instance without creating a new instance and migrating data. Set your initial storage thoughtfully; you will pay for every GB permanently.

Configure a new SQL Server instance with gp3 and autoscaling via Terraform:

```

# terraform/rds_instance.tf
# Production SQL Server RDS with gp3 storage and appropriate settings

resource "aws_db_instance" "prod_sql" {
  identifier      = "prod-sql-primary"
  engine          = "sqlserver-se"
  engine_version  = "16.00"
  instance_class  = "db.r7i.4xlarge"

  # Storage configuration
  storage_type      = "gp3"
  allocated_storage = 500          # GB - initial allocation
  max_allocated_storage = 2000     # GB - autoscaling ceiling
  iops              = 12000        # Additional provisioned IOPS above gp3
  baseline (3000)
  storage_throughput = 500          # MB/s - above gp3 baseline (125 MB/s)
  storage_encrypted  = true
  kms_key_id         = aws_kms_key.rds_encryption.arn

  # Engine configuration
  db_name = "appdb"
  username = "dbadmin"
  password = var.db_password # Use Secrets Manager in production

  # High availability
  multi_az      = true
  availability_zone = "us-east-1a" # for single-AZ; ignored when
multi_az=true

  # Backup and maintenance
  backup_retention_period = 14
  backup_window           = "03:00-04:00"
  maintenance_window      = "sun:04:30-sun:05:30"
  deletion_protection     = true

  # Network
  db_subnet_group_name = aws_db_subnet_group.private.name
  vpc_security_group_ids = [aws_security_group.rds_sqlserver.id]
  publicly_accessible   = false

  # Parameter and option groups
  parameter_group_name = aws_db_parameter_group.sqlserver2022_prod.name

  # Monitoring
  monitoring_interval      = 60 # Enhanced monitoring every 60 seconds
  monitoring_role_arn      = aws_iam_role.rds_enhanced_monitoring.arn
  performance_insights_enabled = true
  performance_insights_retention_period = 731 # 2 years

  # Upgrade behavior
  auto_minor_version_upgrade = false
  apply_immediately         = false # Apply changes during maintenance window

```

```

tags = {
  Environment = "production"
  Team       = "platform-db"
  CostCenter  = "engineering"
}
}

resource "aws_db_parameter_group" "sqlserver2022_prod" {
  name     = "sqlserver2022-prod"
  family   = "sqlserver-se-16.0"

  parameter {
    name = "max server memory (MB)"
    value = "{DBInstanceClassMemory/1048576}*3/4" # ~75% of instance memory
  }

  parameter {
    name = "max degree of parallelism"
    value = "8"
  }

  parameter {
    name = "cost threshold for parallelism"
    value = "50"
  }

  parameter {
    name = "fill factor (%)"
    value = "90"
  }

  parameter {
    name = "optimize for ad hoc workloads"
    value = "1"
  }

  parameter {
    name = "backup compression default"
    value = "1"
  }

  parameter {
    name = "blocked process threshold (s)"
    value = "30" # capture blocking chains in the error log
  }
}

```

IOPS math for production sizing: On gp3, the baseline 3,000 IOPS costs nothing extra, and each additional IOPS above that baseline is billed incrementally — check current AWS pricing for the exact rate, since it changes. Provisioned IOPS on io1/io2 are billed per-IOPS-month at a materially higher rate than gp3's incremental IOPS pricing, which is why gp3 is the default recommendation for most workloads that don't need io2's higher ceiling or durability.

Monitoring storage saturation in CloudWatch:

```
# Check storage I/O metrics for an RDS instance
# ReadIOPS and WriteIOPS tell you current throughput
# ReadLatency and WriteLatency tell you if storage is saturating

aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name ReadLatency \
  --dimensions Name=DBInstanceIdentifier,Value=prod-sql-primary \
  --start-time 2024-01-15T09:00:00Z \
  --end-time 2024-01-15T10:00:00Z \
  --period 60 \
  --statistics Average Maximum \
  --output table

# DiskQueueDepth > 1 consistently indicates storage saturation
aws cloudwatch get-metric-statistics \
  --namespace AWS/RDS \
  --metric-name DiskQueueDepth \
  --dimensions Name=DBInstanceIdentifier,Value=prod-sql-primary \
  --start-time 2024-01-15T09:00:00Z \
  --end-time 2024-01-15T10:00:00Z \
  --period 60 \
  --statistics Average Maximum \
  --output table
```

If `DiskQueueDepth` averages above 1 and `WriteLatency` exceeds 10ms consistently, your provisioned IOPS are insufficient. Increase provisioned IOPS without downtime on gp3 — the change is applied immediately (though it takes up to 6 hours for the modification to complete depending on volume size).

The storage full emergency: RDS will put your database in read-only mode when storage reaches 100%. At that point, you cannot write data including transaction log records. The sequence of events is: free storage drops → autoscaling triggers (if enabled) → scaling takes 5–15 minutes → if autoscaling is not enabled or is at ceiling, the database goes read-only. Set a CloudWatch alarm at 20% free storage, not 10%, so you have time to respond before hitting autoscaling or the ceiling.

6.4 Parameter Groups: The Replacement for `sp_configure`

On a self-managed SQL Server instance, you run `sp_configure` followed by `RECONFIGURE` or restart the service. On RDS, that surface is read-only outside the parameter group. All configuration changes go through Parameter Groups — a named collection of configuration values that you associate with one or more DB instances.

RDS for SQL Server uses DB Parameter Groups (instance-level settings).

A parameter group has a family (e.g., `sqlserver-se-16.0`, `sqlserver-ee-16.0`, `sqlserver-ex-16.0`) and contains every configurable parameter for that engine version. AWS ships a `default.sqlserver-se-16.0` parameter group that contains AWS-optimized defaults. You cannot modify the default group — you must create a custom one and attach it to your instance.

Parameters have two apply types:

- → **Static:** Requires a database restart to take effect. Examples: ``max server memory (MB)``, ``max degree of parallelism``, ``fill factor (%)``.
- → **Dynamic:** Takes effect immediately on ``apply_immediately`` or after the maintenance window. Examples: ``cost threshold for parallelism``, ``optimize for ad hoc workloads``, ``backup compression default``.

Common SQL Server parameter customizations for RDS:


```
# Create a custom parameter group and configure key parameters

aws rds create-db-parameter-group \
  --db-parameter-group-name sqlserver2022-prod \
  --db-parameter-group-family sqlserver-se-16.0 \
  --description "Production SQL Server 2022 parameters" \
  --region us-east-1

# Apply multiple parameters in one call
aws rds modify-db-parameter-group \
  --db-parameter-group-name sqlserver2022-prod \
  --parameters \
    "ParameterName=cost threshold for
parallelism,ParameterValue=50,ApplyMethod=immediate" \
    "ParameterName=max degree of parallelism,ParameterValue=8,ApplyMethod=immediate"
\
    "ParameterName=optimize for ad hoc
workloads,ParameterValue=1,ApplyMethod=immediate" \
    "ParameterName=backup compression
default,ParameterValue=1,ApplyMethod=immediate" \
    "ParameterName=blocked process threshold
(s),ParameterValue=30,ApplyMethod=immediate" \
  --region us-east-1

# Static parameters require pending reboot – apply these separately
aws rds modify-db-parameter-group \
  --db-parameter-group-name sqlserver2022-prod \
  --parameters \
    "ParameterName=max server memory (MB),ParameterValue={DBInstanceClassMemory/
1048576}*3/4,ApplyMethod=pending-reboot" \
    "ParameterName=fill factor (%),ParameterValue=90,ApplyMethod=pending-reboot" \
  --region us-east-1

# Apply the parameter group to the instance (takes effect after maintenance window
# unless you use --apply-immediately, which causes a brief interruption)
aws rds modify-db-instance \
  --db-instance-identifier prod-sql-primary \
  --db-parameter-group-name sqlserver2022-prod \
  --region us-east-1
```

Verifying parameter group status: After applying a parameter group, check whether the instance requires a reboot for any parameters to take effect:

```
# Check for pending reboot parameters
aws rds describe-db-instances \
  --db-instance-identifier prod-sql-primary \
  --query 'DBInstances[0].DBParameterGroups' \
  --region us-east-1
```

The response includes a `ParameterApplyStatus` field. A value of `pending-reboot` means at least one static parameter has been changed and the instance must be restarted before it takes effect. A value of `in-sync` means all parameters are active.

You can also verify parameter values from inside SQL Server after applying:

```
-- SQL Server: Check current effective values of key parameters
SELECT name, value, value_in_use, description, is_dynamic
FROM sys.configurations
WHERE name IN (
    'max server memory (MB)',
    'max degree of parallelism',
    'cost threshold for parallelism',
    'fill factor (%)',
    'optimize for ad hoc workloads',
    'backup compression default',
    'blocked process threshold (s)',
    'remote query timeout (s)'
)
ORDER BY name;
```

For SQL Server on RDS, parameter groups work somewhat differently. SQL Server exposes fewer runtime-adjustable parameters through the RDS parameter group interface than a self-managed instance exposes through `sp_configure`. Many SQL Server settings that you'd configure via `sp_configure` on a self-managed instance are either fixed by RDS, exposed as parameter group settings, or must be set at the database level.

SQL Server parameter group equivalent — key settings:

```
-- SQL Server: Check current sp_configure settings on RDS
-- (These reflect what RDS has applied from the parameter group)
EXEC sp_configure 'show advanced options', 1;
RECONFIGURE;

EXEC sp_configure;
GO

-- Check settings that affect query performance
SELECT
    name,
    value,
    value_in_use,
    description
FROM sys.configurations
WHERE name IN (
    'max degree of parallelism',
    'cost threshold for parallelism',
    'max server memory (MB)',
    'min server memory (MB)',
    'optimize for ad hoc workloads',
    'remote query timeout (s)'
)
ORDER BY name;
```

Parameter group gotchas:

1. **max server memory (MB) uses a formula syntax on RDS.** You cannot set it to a raw value like `64000` without considering the instance class. Instead, RDS uses expressions like `{DBInstanceClassMemory/1048576}*3/4` which resolve at runtime to roughly 75% of instance memory. This is actually better than a hardcoded value because when you resize the instance, the memory setting scales automatically.
2. **user connections defaults to 0 (dynamic self-configuration) on RDS for SQL Server.** Leave it there unless you have a specific reason to cap it. Connection pressure on SQL Server shows up as worker thread exhaustion and `THREADPOOL` waits, not a connection-count error. If you're running RDS Proxy, watch `DatabaseConnections` in CloudWatch and size the proxy to your workload's actual concurrency.
3. **blocked process threshold (s) is one of the most impactful parameters to set immediately.** The default is `0` (disabled). Set it to `30` seconds in production so blocking chains are captured in the error log — exported to CloudWatch Logs via the `error` log type. The blocked process report is how you identify locking regressions during deployments.

4. **Parameter group changes on a Multi-AZ standby require failover or re-application.**

The standby does not automatically pick up primary parameter-group changes without an instance reboot cycle. If you want replicas to use the same configuration, create a common parameter group and attach it to each instance individually.

6.5 Multi-AZ, Read Replicas, and High Availability Architecture

RDS offers two distinct HA constructs that serve different purposes: Multi-AZ deployments and Read Replicas. They are often conflated by engineers new to RDS, and using them incorrectly produces systems that are either more expensive than necessary or less resilient than assumed.

Multi-AZ: Synchronous standby for failover

In a Multi-AZ deployment, RDS maintains a synchronous standby replica in a different Availability Zone. Every write to the primary is synchronously replicated to the standby before the write is acknowledged to the application. This means:

- → RPO is effectively zero (no committed transactions are lost on failover).
- → RTO is 60–120 seconds for automatic failover (DNS propagation + database promotion).
- → The standby cannot serve reads. It exists solely for failover.
- → You pay for two instances but get one connection endpoint.

The Multi-AZ standby is completely transparent to applications — they connect to the instance endpoint (e.g., `prod-sql-primary.abc123.us-east-1.rds.amazonaws.com`), and that DNS record is automatically updated to point to the new primary after failover.

Multi-AZ DB Cluster is not available for SQL Server: AWS offers a newer Multi-AZ option called Multi-AZ DB Cluster with two readable standby instances — for other engines. It is *not* available for RDS for SQL Server. For SQL Server, Multi-AZ uses SQL Server Always On availability groups: the primary replica accepts writes, and the secondary replica exists solely for automatic failover. The secondary is not readable and cannot be queried directly.

Creating a Multi-AZ SQL Server instance:

```
# Create a Multi-AZ SQL Server instance (Always On availability group under the hood)
aws rds create-db-instance \
  --db-instance-identifier prod-sql-primary \
  --db-instance-class db.r7i.4xlarge \
  --engine sqlserver-se \
  --engine-version 16.00 \
  --master-username dbadmin \
  --master-user-password "${DB_PASSWORD}" \
  --allocated-storage 500 \
  --storage-type gp3 \
  --multi-az \
  --backup-retention-period 14 \
  --vpc-security-group-ids sg-0abc123def456789 \
  --db-subnet-group-name private-db-subnets \
  --db-parameter-group-name sqlserver2022-prod \
  --region us-east-1
```

Read scaling on RDS for SQL Server: there are no read replicas

RDS for SQL Server does *not* offer read replicas — `create-db-instance-read-replica` is not supported for the `sqlserver-*` engines. This is the single biggest architectural difference to internalize: on RDS for SQL Server, read scaling must be designed explicitly. Your options are:

- → **Self-managed readable secondaries on EC2:** run SQL Server on EC2 with an Always On availability group and add readable secondary replicas for reporting and analytics offload (covered in Chapter 8).
- → **Snapshot-refreshed reporting copies:** restore a production snapshot to a separate RDS instance on a schedule (nightly or hourly) for reporting workloads that tolerate stale data.
- → **Cross-region DR via snapshot copy:** copy automated or manual snapshots to a second region and restore them there — this is your geographic DR story, not replica promotion.
- → **Reduce primary read load first:** before building read-scale infrastructure, verify the primary is not doing avoidable work — missing indexes, unparameterized ad hoc queries, and reporting queries that belong on a copy.

Multi-AZ is not a read-scaling feature on RDS for SQL Server. The secondary replica cannot serve reads, so a Multi-AZ primary does nothing for read pressure. The correct production architecture is a Multi-AZ primary for failover *plus* an explicit read-scaling strategy (readable AG secondaries on EC2, or snapshot-refreshed reporting copies) sized to your reporting workload.

Cross-region snapshot copy for DR (the RDS-for-SQL-Server alternative to replica promotion):

```
# RDS for SQL Server has no create-db-instance-read-replica.
# For cross-region DR, copy the latest automated snapshot and restore it.
# Run the copy in the TARGET region:
aws rds copy-db-snapshot \
  --source-db-snapshot-identifier arn:aws:rds:us-east-1:123456789012:snapshot:rds:prod-sql-primary-2024-01-15-03-00 \
  --target-db-snapshot-identifier prod-sql-dr-copy-20240115 \
  --kms-key-id arn:aws:kms:us-west-2:123456789012:key/mrk-abc123 \
  --region us-west-2

# Restore it as a DR/test instance in the target region
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier prod-sql-dr-usw2 \
  --db-snapshot-identifier prod-sql-dr-copy-20240115 \
  --db-instance-class db.r7i.2xlarge \
  --no-multi-az \
  --region us-west-2
```

Monitoring Multi-AZ health (the secondary is not directly queryable):

```
# There is no ReplicaLag metric for RDS for SQL Server Multi-AZ.
# Monitor failover and AG health through RDS events instead.
# Subscribe to failover-related events:
aws rds create-event-subscription \
  --subscription-name prod-sql-failover-events \
  --sns-topic-arn arn:aws:sns:us-east-1:123456789012:db-alerts \
  --source-type db-instance \
  --source-ids prod-sql-primary \
  --event-categories "failover" "failure" \
  --region us-east-1

# Review recent failover/failure events
aws rds describe-events \
  --source-identifier prod-sql-primary \
  --source-type db-instance \
  --start-time 2024-01-14T00:00:00Z \
  --query 'Events[*].[Date,Message]' \
  --output table \
  --region us-east-1
```

```
-- On RDS Multi-AZ for SQL Server the secondary replica is NOT queryable.
-- For reference, this is the AG health query you would run on a
-- self-managed / EC2 Always On availability group:
SELECT ar.replica_server_name,
       rs.role_desc,
       rs.connected_state_desc,
       rs.synchronization_state_desc,
       DB_NAME(drs.database_id) AS database_name,
       drs.synchronization_state_desc AS db_sync_state,
       drs.log_send_queue_size AS log_send_queue_kb,
       drs.redo_queue_size AS redo_queue_kb
FROM sys.dm_hadr_availability_replica_states rs
JOIN sys.availability_replicas ar
     ON rs.replica_id = ar.replica_id
JOIN sys.dm_hadr_database_replica_states drs
     ON drs.replica_id = rs.replica_id;

-- On RDS, confirm Multi-AZ status from the control plane instead:
-- aws rds describe-db-instances --db-instance-identifier prod-sql-primary
-- --query 'DBInstances[0].[MultiAZ, SecondaryAvailabilityZone]'
```

RDS Proxy: Connection pooling as a managed service

RDS Proxy sits between your application and the RDS instance and multiplexes database connections. It maintains a warm pool of connections to the database, allowing thousands of application connections to share a smaller number of actual database connections. This is most impactful for:

- → Serverless workloads (Lambda functions that open and close connections on every invocation)
- → Applications using a connection-per-request pattern
- → Instances where connection churn or worker-thread pressure is a bottleneck

RDS Proxy also provides a secondary benefit: during Multi-AZ failover, the proxy handles connection re-routing, reducing application-visible downtime from 60–120 seconds to as low as a few seconds.

```
# Create an RDS Proxy for the SQL Server instance
aws rds create-db-proxy \
  --db-proxy-name prod-sql-proxy \
  --engine-family SQLSERVER \
  --auth '[{
    "AuthScheme": "SECRETS",
    "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:prod/
sqlserver/dbadmin",
    "IAMAuth": "DISABLED"
  }]' \
  --role-arn arn:aws:iam::123456789012:role/rds-proxy-role \
  --vpc-subnet-ids subnet-0abc123 subnet-0def456 subnet-0ghi789 \
  --vpc-security-group-ids sg-0proxy123 \
  --require-tls \
  --region us-east-1

# Register the target (point proxy at the RDS instance)
aws rds register-db-proxy-targets \
  --db-proxy-name prod-sql-proxy \
  --db-instance-identifiers prod-sql-primary \
  --region us-east-1
```

RDS Proxy for SQL Server authenticates through Secrets Manager — IAM database authentication is not available for SQL Server on RDS. Known limitations for the SQL Server engine family: Windows (Kerberos/NTLM) authentication is not supported through the proxy, so applications must use SQL authentication with the secret; and some connection-string behaviors differ behind the proxy, so test failover and transaction behavior thoroughly before putting Proxy in front of a production SQL Server instance.

6.6 Backup, Restore, and PITR

RDS automated backups use EBS volume snapshots plus transaction log backups to enable point-in-time recovery. Understanding the mechanics matters because the recovery behavior has important implications for your RTO.

How automated backups work:

1. During the backup window, RDS takes an EBS snapshot of the storage volume. For Multi-AZ, the snapshot is taken from the standby to avoid I/O impact on the primary.
2. Transaction log backups are taken continuously (roughly every 5 minutes) and uploaded to S3.
3. PITR works by restoring the nearest snapshot before the target time, then replaying logs forward to the exact target timestamp.

Retention period: Backups are retained for 0–35 days. Setting retention to 0 disables automated backups (and PITR). Never set retention to 0 on a production instance. The standard retention for production is 14 days; compliance-heavy environments often require 35 days.

Manual snapshots are independent of the retention period. They persist until you explicitly delete them. Before any major change (schema migration, engine upgrade, instance resizing), take a manual snapshot:

```
# Take a manual snapshot before a risky operation
aws rds create-db-snapshot \
  --db-instance-identifier prod-sql-primary \
  --db-snapshot-identifier prod-sql-before-migration-20240115 \
  --region us-east-1

# Wait for the snapshot to complete
aws rds wait db-snapshot-completed \
  --db-snapshot-identifier prod-sql-before-migration-20240115 \
  --region us-east-1

echo "Snapshot complete – safe to proceed with migration"
```

Point-in-time restore: PITR creates a new RDS instance restored to the specified timestamp. It does not restore in-place — you always get a new instance. This means you must update your application's connection string after a PITR recovery.

```
# Restore to a specific point in time (creates a NEW instance)
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier prod-sql-primary \
  --target-db-instance-identifier prod-sql-recovered-20240115 \
  --restore-time 2024-01-15T08:30:00Z \
  --db-instance-class db.r7i.4xlarge \
  --storage-type gp3 \
  --multi-az \
  --region us-east-1

# Monitor restoration progress
aws rds describe-db-instances \
  --db-instance-identifier prod-sql-recovered-20240115 \
  --query 'DBInstances[0].[DBInstanceStatus,PercentProgress]' \
  --output table \
  --region us-east-1
```

The PITR recovery window: The earliest restorable time is limited by the oldest retained backup snapshot. You can check the exact recovery window:

```
# Check the earliest and latest restorable times
aws rds describe-db-instances \
  --db-instance-identifier prod-sql-primary \
  --query 'DBInstances[0].{
    EarliestRestorableTime: EarliestRestorableTime,
    LatestRestorableTime: LatestRestorableTime,
    BackupRetentionPeriod: BackupRetentionPeriod
  }' \
  --output table \
  --region us-east-1
```

Restoring a snapshot to a different instance class or storage type: This is useful when you want to test a configuration change. Restore a production snapshot to a smaller instance in a non-production environment, validate, and then apply the same change to production.

```
# Restore from a manual snapshot to a different (cheaper) instance class
# for testing or data recovery purposes
aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier staging-sql-from-prod-snap \
  --db-snapshot-identifier prod-sql-before-migration-20240115 \
  --db-instance-class db.r7i.xlarge \
  --storage-type gp3 \
  --no-multi-az \
  --no-publicly-accessible \
  --vpc-security-group-ids sg-0staging123 \
  --db-subnet-group-name staging-db-subnets \
  --region us-east-1
```

Cross-region backup copies: Automated backups do not automatically copy to other regions. For DR, you need to either configure cross-region automated backup copy or copy manual snapshots. AWS recently added automated cross-region backup to RDS, but manual snapshot copy remains the most flexible approach:

```
# Copy a manual snapshot to another region for DR
aws rds copy-db-snapshot \
  --source-db-snapshot-identifier \
    arn:aws:rds:us-east-1:123456789012:snapshot:prod-sql-before-migration-20240115 \
  --target-db-snapshot-identifier prod-sql-before-migration-20240115-copy \
  --kms-key-id arn:aws:kms:us-west-2:123456789012:key/mrk-abc123 \
  --region us-west-2
```

Note that cross-region snapshot copies transfer data across AWS backbone networks and incur data transfer costs. For large databases, automate periodic snapshot copies rather than copying every snapshot.

6.7 Upgrade Paths and Maintenance Windows

RDS engine upgrades come in two types: minor version upgrades (e.g., 16.2 → 16.3) and major version upgrades (e.g., 15 → 16). The operational complexity and risk differ significantly.

Minor version upgrades:

- → Applied automatically if `AutoMinorVersionUpgrade` is enabled, during the maintenance window.
- → Can be applied manually at any time.
- → For Multi-AZ: zero-downtime upgrade (standby upgraded first, then failover, then original primary upgraded).
- → For Single-AZ: brief downtime (~5 minutes) during the upgrade.
- → Generally safe with no application changes required.

Major version upgrades:

- → Always require explicit action — no automatic major version upgrades.
- → Require a compatible parameter group for the new version (you must create one in advance).
- → In-place major version upgrades on RDS work for some engine combinations (SQL Server 2019 → 2022 is supported) but not all.
- → For SQL Server, the in-place upgrade runs the SQL Server setup upgrade path underneath. RDS takes a snapshot first, runs the upgrade, and if it fails, rolls back to the snapshot.
- → Estimated downtime: 10–60 minutes depending on database size and version gap.

Pre-upgrade checklist for a SQL Server major version upgrade:

```

-- Run these on the CURRENT version before upgrading
-- to identify potential incompatibilities

-- Check for deprecated features in use (nonzero = in use since last restart)
SELECT object_name, counter_name, cntr_value
FROM sys.dm_os_performance_counters
WHERE object_name LIKE '%Deprecated Features%'
      AND cntr_value > 0;

-- Check database compatibility levels (old levels may block upgrade paths)
SELECT name, compatibility_level
FROM sys.databases
WHERE compatibility_level < 150;

-- Inventory features RDS does not support: linked servers and CLR assemblies
SELECT name, product, provider, data_source
FROM sys.servers
WHERE is_linked = 1;

SELECT name, permission_set_desc
FROM sys.assemblies
WHERE is_user_defined = 1;

```

Performing a major version upgrade with pre-upgrade snapshot:

```

# Step 1: Take a manual snapshot before upgrading
aws rds create-db-snapshot \
  --db-instance-identifier prod-sql-primary \
  --db-snapshot-identifier pre-upgrade-sql2019-to-sql2022-$(date +%Y%m%d) \
  --region us-east-1

aws rds wait db-snapshot-completed \
  --db-snapshot-identifier pre-upgrade-sql2019-to-sql2022-$(date +%Y%m%d) \
  --region us-east-1

# Step 2: Create the parameter group for the new major version FIRST
aws rds create-db-parameter-group \
  --db-parameter-group-name sqlserver2022-prod \
  --db-parameter-group-family sqlserver-se-16.0 \
  --description "Production SQL Server 2022 parameters" \
  --region us-east-1

# Re-apply all your custom parameters to the new group (use the same
# modify-db-parameter-group commands from section 6.4)

# Step 3: Initiate the upgrade
aws rds modify-db-instance \
  --db-instance-identifier prod-sql-primary \
  --engine-version 16.00 \
  --db-parameter-group-name sqlserver2022-prod \
  --allow-major-version-upgrade \
  --apply-immediately \
  --region us-east-1

# Step 4: Monitor upgrade progress
watch -n 30 'aws rds describe-db-instances \
  --db-instance-identifier prod-sql-primary \
  --query "DBInstances[0].[DBInstanceStatus,EngineVersion,PendingModifiedValues]" \
  --output table \
  --region us-east-1'

```

Maintenance windows deserve careful thought in production. The default maintenance window is a 30-minute window during off-peak hours in the instance's region. AWS applies minor patches, security updates, and approved configuration changes during this window. For global applications with no true off-peak window, schedule maintenance during the lowest-traffic window you have, and ensure your application handles connection drops gracefully (retry logic, connection pool reconnect on error).

```
# Update the maintenance window to Sunday 2am-3am UTC
aws rds modify-db-instance \
  --db-instance-identifier prod-sql-primary \
  --preferred-maintenance-window "sun:02:00-sun:03:00" \
  --region us-east-1

# Check pending maintenance actions (AWS may schedule security patches)
aws rds describe-pending-maintenance-actions \
  --region us-east-1 \
  --query 'PendingMaintenanceActions[*].{
    ResourceIdentifier: ResourceIdentifier,
    Action: PendingMaintenanceActionDetails[0].Action,
    AutoAppliedAfterDate: PendingMaintenanceActionDetails[0].AutoAppliedAfterDate,
    Description: PendingMaintenanceActionDetails[0].Description
  }' \
  --output table
```

If AWS schedules a forced patch with an `AutoAppliedAfterDate`, that patch will apply automatically after that date regardless of your maintenance window preference. Monitor this endpoint and apply patches proactively before AWS forces them, so you control the timing.

6.8 RDS Operational Monitoring and Alerting

The monitoring stack for RDS has three tiers: CloudWatch Basic Metrics (free, 1-minute granularity), Enhanced Monitoring (paid, 1–60 second granularity, OS-level metrics), and Performance Insights (paid, query-level attribution).

CloudWatch Basic Metrics include CPU utilization, database connections, free storage, read/write IOPS, read/write latency, replica lag, and network throughput. These are sufficient for capacity trend monitoring but insufficient for diagnosing query-level performance issues.

Enhanced Monitoring adds OS-level metrics that the basic metrics don't expose: per-process CPU, memory by component (RDS process, OS, file system cache), and disk I/O broken down by read/write and user/kernel. Enhanced Monitoring sends data to CloudWatch Logs rather than CloudWatch Metrics, which means you query it differently:

```
# Enhanced Monitoring is in the RDSOSMetrics log group
# Get the latest OS metrics for an instance
aws logs filter-log-events \
  --log-group-name "RDSOSMetrics" \
  --log-stream-name-prefix "prod-sql-primary" \
  --start-time $(date -d '5 minutes ago' +%s000) \
  --limit 5 \
  --query 'events[*].message' \
  --output text \
  --region us-east-1
```

The output is JSON with detailed OS metrics. Parse it with `jq` for usable output:

```
aws logs filter-log-events \
  --log-group-name "RDSOSMetrics" \
  --log-stream-name-prefix "prod-sql-primary" \
  --start-time $(date -d '5 minutes ago' +%s000) \
  --limit 1 \
  --query 'events[0].message' \
  --output text \
  --region us-east-1 | \
jq '{
  timestamp: .timestamp,
  cpu_idle: .cpuUtilization.idle,
  cpu_user: .cpuUtilization.user,
  memory_free_mb: (.memory.free / 1024 / 1024),
  disk_read_iops: .diskIO[0].readIOsPS,
  disk_write_iops: .diskIO[0].writeIOsPS,
  load_average_1m: .loadAverageMinute.one
}'
```

Performance Insights is the most powerful diagnostic tool for RDS. It collects query-level wait event data and lets you answer "what was the database doing at 2:34am when latency spiked?" It shows the database load broken down by wait event, SQL statement, user, host, or application. For SQL Server, it leverages DMVs such as `sys.dm_exec_query_stats` and `sys.dm_os_wait_stats` internally.

Set Performance Insights retention to at least 7 days (the free tier) and ideally to 731 days (2 years, paid) for production systems where you need to compare behavior across deployments and seasonal traffic patterns.

Essential CloudWatch alarms for any production RDS instance:

```

# Alarm: CPU utilization above 80% for 10 minutes
aws cloudwatch put-metric-alarm \
  --alarm-name rds-prod-sql-cpu-high \
  --metric-name CPUUtilization \
  --namespace AWS/RDS \
  --dimensions Name=DBInstanceIdentifier,Value=prod-sql-primary \
  --statistic Average \
  --period 300 \
  --evaluation-periods 2 \
  --threshold 80 \
  --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:db-alerts \
  --region us-east-1

# Alarm: Free storage below 20% (500 GB instance -> alarm at 100 GB free)
aws cloudwatch put-metric-alarm \
  --alarm-name rds-prod-sql-low-storage \
  --metric-name FreeStorageSpace \
  --namespace AWS/RDS \
  --dimensions Name=DBInstanceIdentifier,Value=prod-sql-primary \
  --statistic Average \
  --period 300 \
  --evaluation-periods 1 \
  --threshold 107374182400 \
  --comparison-operator LessThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:db-alerts \
  --region us-east-1

# Alarm: Freeable memory below 2 GB (memory pressure / buffer pool squeeze)
aws cloudwatch put-metric-alarm \
  --alarm-name rds-prod-sql-low-memory \
  --metric-name FreeableMemory \
  --namespace AWS/RDS \
  --dimensions Name=DBInstanceIdentifier,Value=prod-sql-primary \
  --statistic Average \
  --period 300 \
  --evaluation-periods 2 \
  --threshold 2147483648 \
  --comparison-operator LessThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:db-alerts \
  --region us-east-1

# Alarm: Database connections above 400 (of 500 max)
aws cloudwatch put-metric-alarm \
  --alarm-name rds-prod-sql-connections-high \
  --metric-name DatabaseConnections \
  --namespace AWS/RDS \
  --dimensions Name=DBInstanceIdentifier,Value=prod-sql-primary \
  --statistic Maximum \
  --period 60 \
  --evaluation-periods 2 \
  --threshold 400 \
  --comparison-operator GreaterThanThreshold \

```



```
--alarm-actions arn:aws:sns:us-east-1:123456789012:db-alerts \  
--region us-east-1
```

Querying Performance Insights via API for automated reporting:

```
# Get top SQL by average active sessions (last hour)  
aws pi get-resource-metrics \  
  --service-type RDS \  
  --identifier db-ABCDEFGHJIJ1234567 \  
  --start-time $(date -d '1 hour ago' --iso-8601=seconds) \  
  --end-time $(date --iso-8601=seconds) \  
  --period-in-seconds 3600 \  
  --metric-queries '[  
    {  
      "Metric": "db.load.avg",  
      "GroupBy": {  
        "Group": "db.sql",  
        "Limit": 10  
      }  
    }  
  ]' \  
  --region us-east-1
```

The `identifier` in Performance Insights API calls is the DBI resource ID, not the instance name. Get it with:

```
aws rds describe-db-instances \  
  --db-instance-identifier prod-sql-primary \  
  --query 'DBInstances[0].DbiResourceId' \  
  --region us-east-1
```

Key Takeaways

- → **Engine version and licensing choices are harder to reverse than they appear.** Pin minor versions in production, stay within one major version of current, and understand the SQL Server and Oracle licensing models before committing to an instance size and edition — the hourly cost difference between editions on large instances is substantial.
- → **Instance family selection matters more than raw size.** Graviton (g-suffix) instances do not support SQL Server at all — for RDS for SQL Server always choose Intel families (db.r7i, db.m7i). Never use T-series burstable instances for production OLTP workloads that sustain CPU pressure — CPU credit exhaustion causes silent, unpredictable throttling.

- → **gp3 storage is the correct default for nearly all new RDS deployments.** It decouples IOPS and throughput from storage size, eliminating the gp2 credit model at lower cost. Monitor `DiskQueueDepth` and `WriteLatency` to detect storage saturation; provision additional IOPS on gp3 before reaching the throughput ceiling rather than after observing latency spikes.
- → **Multi-AZ is failover, not read scaling — and RDS for SQL Server has no read replicas.** Multi-AZ provides synchronous failover with near-zero RPO but the secondary cannot serve reads. Plan read scaling explicitly: readable Always On secondaries on EC2, or snapshot-refreshed reporting copies. Production architectures pair a Multi-AZ primary with a deliberate read-offload strategy.
- → **RDS backup and restore behavior requires architectural awareness.** PITR always produces a new instance — recovery requires updating application connection strings, not just waiting for an in-place restore. Automated backups do not copy cross-region by default. Take manual snapshots before any major change, verify the earliest restorable time matches your RPO requirements, and set CloudWatch alarms at 20% free storage rather than relying solely on autoscaling.

Chapter 7: RDS for SQL Server vs Azure SQL: When to Choose Which

If you run SQL Server, you have a choice few other database platforms offer: two major clouds provide a first-class managed SQL Server, and both want your workload. Amazon RDS for SQL Server and Azure SQL (Database and Managed Instance) are mature, production-proven platforms — but they are built on different philosophies, metered by different licensing models, and missing different features. Picking wrong does not just cost money; it can force an application rewrite or a painful re-migration eighteen months later when you discover the feature your workload depends on does not exist on the platform you chose.

This chapter is the decision guide. It compares the three real options — RDS for SQL Server, Azure SQL Database, and Azure SQL Managed Instance — across architecture, licensing and cost, feature compatibility, high availability, performance and scaling, security, operations, and migration paths, and ends with a framework you can apply to your own estate.

7.1 Why This Comparison Is the One That Matters

Most SQL Server estates arriving in the cloud fall into one of three situations. First, the AWS-centric organization: the company standardized on AWS years ago, and the SQL Server workload has to live there. RDS for SQL Server is the natural answer, but "natural" is not the same as "optimal" — the licensing math and the feature gaps deserve a real evaluation before you commit. Second, the Microsoft-centric organization: Active Directory, Microsoft 365, existing Enterprise Agreements with Software Assurance. Azure SQL is the natural answer here, and Azure Hybrid Benefit can make it dramatically cheaper than it first appears. Third, the genuinely undecided: a new project, an acquisition, or a renewal moment where either cloud is on the table.

The comparison is asymmetric in an important way. On AWS there is exactly one managed SQL Server: RDS. On Azure there are two, and they are very different products: Azure SQL Database (a PaaS database-as-a-service) and Azure SQL Managed Instance (a PaaS instance-as-a-service with near-complete engine compatibility). Treating "Azure SQL" as one thing is the most common mistake in this evaluation. Throughout this chapter, "Azure SQL Database" and "Managed Instance" (MI) are discussed separately wherever their behavior differs.

A note on Google Cloud: Cloud SQL for SQL Server exists and is covered in Chapter 19. It is a solid option, but in practice the RDS-versus-Azure decision covers the overwhelming majority of SQL Server cloud evaluations, because licensing leverage (your existing Microsoft agreements) and organizational gravity (where the rest of your infrastructure lives) dominate the decision.

7.2 The Three Contenders

Amazon RDS for SQL Server. A managed DB instance running SQL Server 2016 through 2022 in Express, Web, Standard, and Enterprise editions. You choose License Included (the license cost is folded into the hourly rate) or Bring Your Own License via License Mobility (requires Software Assurance). Multi-AZ deployments use Always On availability groups for automatic failover. Storage is EBS-backed (gp3, io1, io2) up to 16 TB. There are no read replicas for SQL Server on RDS, and several engine-adjacent components — SSIS, SSAS, SSRS, and SQL Server Agent — are either unavailable or restricted compared to a full installation. What you get is the relational engine, managed: automated backups, patching windows, CloudWatch monitoring, and Performance Insights.

Azure SQL Database. A pure PaaS offering: you get databases, not a server. There is no OS, no instance-level surface, no SQL Server Agent. You buy single databases or elastic pools (a pool of compute shared across many databases with unpredictable peaks). Purchasing models are vCore-based (the modern model, with provisioned or serverless compute) or the legacy DTU model. The Hyperscale tier scales to 100 TB with rapid scaling and named replicas for read offload. High availability is built in — every database runs with multiple replicas and zone redundancy is a checkbox. This is the most "cloud-native" of the three options and also the furthest from an on-premises SQL Server.

Azure SQL Managed Instance. A managed instance of the SQL Server engine with near-100% compatibility with on-premises SQL Server: SQL Server Agent, linked servers, CLR, Service Broker, Database Mail, cross-database queries, and four-part naming all work. It lives inside your virtual network, which makes hybrid scenarios (Azure ExpressRoute to on-premises, Active Directory integration) natural. It is the lift-and-shift target: the platform where an existing application migrates with the fewest changes. The trade-off is that it is heavier than SQL Database — provisioning takes longer, scaling is an instance-level operation, and you manage (or at least think about) more surface area.

7.3 Architecture Under the Hood

RDS for SQL Server is an instance on managed infrastructure. Underneath, your database runs on an EC2 host that AWS manages; you get a DB instance endpoint, EBS storage, and a managed layer that handles failover, patching, and backups. Multi-AZ adds a synchronous Always On secondary in another Availability Zone. The mental model is familiar to any DBA: it is a SQL Server instance, with the OS and the failover cluster taken away from you.

Azure SQL Database has no instance at all in your mental model. Your database's compute and storage are separated: compute nodes run the query engine, and data lives in Azure Storage with multiple replicas. Failover is a control-plane operation that reattaches storage to a new compute node — typically completing in seconds, with no availability group for you to monitor. The Hyperscale tier takes this further with a page-server architecture that allows near-instant scaling and up to 100 TB per database. You trade control for resilience: there is no AG to tune, no secondary to lag, and no failover cluster to patch.

Managed Instance sits between the two. It is a dedicated set of VMs running the full SQL Server engine inside your VNet, with storage on Azure Premium disks (or faster tiers). High availability uses Always On availability groups internally — the same technology as RDS Multi-AZ — but the replica topology is managed for you, and Business Critical tier instances include a free readable secondary replica you can actually use for reporting. General Purpose tier uses remote storage with local SSD cache; Business Critical uses local SSD with multiple replicas.

The practical consequence: on RDS you still think in instances, endpoints, and failover; on Azure SQL Database you think in databases, elastic pools, and service tiers; on Managed Instance you think in instances again, but with VNet integration and near-complete engine fidelity.

7.4 Licensing and Cost: Where the Decision Is Often Made

For SQL Server, licensing is not a footnote — it is frequently the largest line item and the decisive factor. The three options meter licenses completely differently, and your existing Microsoft agreements change the math more than any technical feature.

RDS licensing: License Included vs BYOL. With License Included, you pay an hourly rate that bundles the SQL Server license; Enterprise Edition LI on a large instance is one of the most expensive ways to run any database on AWS. With BYOL via License Mobility (which requires active Software Assurance), you pay only for the EC2 infrastructure and apply your existing licenses — often cutting the bill dramatically. The catch: License Mobility has rules. You must have Software Assurance, you must report usage, and license mobility through SA

does not cover every scenario (short-term or burst BYOL without SA is not permitted). Web and Express editions have their own constraints: Express is free but capped (10 GB per database, limited memory and CPU), Web is LI-only and cannot use BYOL.

Azure SQL and Azure Hybrid Benefit. This is Azure's killer feature in the SQL Server decision. If you have SQL Server licenses with active Software Assurance, Azure Hybrid Benefit lets you apply them to Azure SQL Database and Managed Instance and pay only the compute rate — no SQL Server license cost in the meter. For an organization already paying for SQL Server Enterprise under an Enterprise Agreement, AHB can reduce the Azure SQL bill by 40% or more compared to license-included pricing. There is also a dev/test variant: Azure Dev/Test subscriptions offer steep discounts for non-production workloads. If your organization holds Microsoft licenses with SA and you are comparing RDS License Included against Azure SQL without AHB in the model, you are not comparing real prices.

Purchasing models compared. RDS charges per instance-hour by instance class, plus storage, IOPS, and backup storage — the model Chapter 6 dissected. Azure SQL Database vCore model charges per vCore-hour (provisioned) or per vCore-second with auto-pause (serverless, which can pause compute entirely during idle periods — ideal for dev databases and intermittent workloads). Elastic pools let dozens of small databases share a pool of eDTUs or vCores, which is dramatically cheaper than provisioning each database separately when their peaks do not coincide. Managed Instance charges per vCore-hour at the instance level, with General Purpose and Business Critical tiers at very different price points.

A cost comparison is only honest if it includes the same things on both sides: compute, storage, IOPS/throughput, backup retention beyond the included window, data transfer, and the license. Model a representative month — including a failover test, a PITR drill, and your actual backup retention — before committing. The cheapest option on a pricing calculator is often not the cheapest option in production.

```
# Azure: create a General Purpose SQL Database with serverless compute
# (auto-pauses during idle – good for dev/test and intermittent workloads)
az sql db create \
  --resource-group rg-prod \
  --server sqlsrv-prod \
  --name appdb \
  --edition GeneralPurpose \
  --family Gen5 \
  --capacity 4 \
  --compute-model Serverless \
  --auto-pause-delay 60 \
  --min-capacity 0.5

# Azure: create a Managed Instance (VNet-integrated, near-full engine compat)
az sql mi create \
  --resource-group rg-prod \
  --name mi-prod \
  --vnet-name vnet-prod \
  --subnet snet-sqlmi \
  --capacity 8 \
  --edition GeneralPurpose \
  --family Gen5 \
  --storage 256 \
  --license-type BasePrice # BasePrice = Azure Hybrid Benefit applied
```

7.5 Feature Compatibility: What Breaks Where

This is the section to read with your application inventory in hand. Run the Data Migration Assistant (DMA) or Azure Data Studio's assessment against your databases before you decide — but the high-level map below tells you where the bodies are buried.

RDS for SQL Server gaps. The engine is complete, but the surrounding components are not: no SSIS, SSAS, or SSRS on the RDS instance (run them on EC2 and connect to RDS, or use AWS-native alternatives). SQL Server Agent exists on RDS but with restrictions — notably, replication-related jobs and some job types are limited, and you cannot RDP to the host to manage the Agent directly. CLR is supported in SAFE mode only. Linked servers are supported but restricted to specific providers and targets. Filestream/FileTable, PolyBase scale-out, and machine learning services have limited or no support. Native backup to S3 works well, which partially compensates — you can take native `BACKUP DATABASE ... TO URL` -style backups to S3 for portability.

Azure SQL Database gaps. This is the longest list, because SQL Database is the least "SQL Server"-like. No SQL Server Agent (use elastic jobs, which are similar but not identical, or Azure Automation). No linked servers (use elastic queries for cross-database reads). No CLR. No Service Broker. No cross-database transactions in the general case. No three- or four-part naming across databases. `msdb` does not exist as you know it. SQL Profiler-style server traces

are replaced by Extended Events scoped to the database. If your application was written against a full SQL Server instance with Agent jobs, linked servers, and cross-database joins, migrating it to SQL Database is a rewrite project, not a migration project.

Managed Instance: the compatibility play. MI supports SQL Server Agent, linked servers, CLR, Service Broker, Database Mail, distributed transactions (via MS DTC in recent updates), and cross-database queries with three-part naming. The remaining gaps are narrow: no SSIS/SSAS/SSRS on the instance itself (use Azure Data Factory / SSIS in ADF), no Filestream in early versions (supported in newer updates), and some server-level settings you cannot change. For lift-and-shift migrations, MI is almost always the right Azure target — and the DMA assessment will confirm it quantitatively.

- → **Inventory before you choose.** Query `msdb.dbo.sysjobs` for Agent jobs, `sys.servers` for linked servers, `sys.assemblies` for CLR, and check for cross-database references and Service Broker usage. Every row in those results is a migration work item on SQL Database and a non-issue on MI or RDS.
- → **SSIS/SSAS/SSRS need a home everywhere.** None of the three options hosts these components on the database service itself. On AWS, run them on EC2; on Azure, use Azure Data Factory (with the SSIS integration runtime) or VMs. Factor that into the architecture, not just the database decision.

7.6 High Availability and Disaster Recovery Compared

RDS Multi-AZ gives you a synchronous Always On secondary in a second Availability Zone, automatic failover in 60–120 seconds, and no readable secondary. Cross-region DR is snapshot copy and restore — an RPO measured in snapshot frequency and an RTO measured in restore time. It is simple, well-understood, and exactly what Chapter 6 described.

Azure SQL Database builds HA in by default: every database has multiple replicas, and zone redundancy (replicas across Availability Zones) is a configuration choice, not an architecture project. Failover is typically measured in seconds. For DR, active geo-replication lets you create up to four readable secondary databases in other regions, and auto-failover groups provide a single listener endpoint with automatic failover across regions, including a grace period you configure to avoid flapping. For a team that needs cross-region DR with readable secondaries and automatic failover, this is the most complete story of the three — and it is included in the service rather than assembled from parts.

Managed Instance uses Always On availability groups internally. General Purpose tier keeps a single replica with remote storage (failover reattaches storage, similar in spirit to SQL Database); Business Critical tier runs multiple replicas including one free readable secondary

you can point reporting at. Failover groups for MI provide cross-region DR with a listener endpoint, mirroring the SQL Database story at instance scope. Failover times are typically under a minute for planned failovers and a few minutes for unplanned.

The honest summary: if cross-region DR with readable secondaries and automatic failover groups matters to you, Azure's story is more complete out of the box. If single-region HA with a well-understood AG failover is enough, RDS Multi-AZ is simpler to reason about and monitor.

7.7 Performance, Scaling, and Limits

Scaling up. On RDS you scale by changing instance class (minutes of downtime for Single-AZ, near-zero for Multi-AZ with failover) and by provisioning IOPS on gp3/io2. On Azure SQL Database you change service tier or vCores online — Hyperscale can scale compute in seconds without moving data, because compute and storage are separated. Managed Instance scales vCores and storage with brief downtime windows. For workloads with sharp, unpredictable peaks, Hyperscale's rapid scaling and serverless auto-pause are unmatched by either RDS or MI.

Read scaling. RDS for SQL Server has no read replicas — Chapter 6 covered the workarounds. Azure SQL Database Hyperscale offers named replicas: up to 30 readable secondaries you can scale independently for read offload, each with its own compute. MI Business Critical includes one readable secondary. If your workload is read-heavy and you want first-class read scale-out without managing replicas yourself, Hyperscale named replicas are the strongest option in this comparison.

Limits that surprise people. RDS for SQL Server caps database storage at 16 TB per instance and has no serverless option — you pay for the instance whether it is busy or idle. Azure SQL Database single databases top out at 100 TB on Hyperscale (far beyond RDS), but the General Purpose and Business Critical tiers have much lower per-database limits, and `tempdb` sizing and log throughput caps per tier cause the classic "it worked on-premises" performance incidents. Managed Instance caps at 16 TB (General Purpose) with instance-level resource governance. Log rate limits on Azure tiers are the most common surprise for write-heavy migrations: measure your log generation rate on-premises (`sys.dm_io_virtual_file_stats` on the log file, or PerfMon log bytes flushed/sec) and compare it against the tier's documented throughput before you migrate.

7.8 Security and Compliance

Both clouds cover the fundamentals: encryption at rest (KMS on AWS, Azure Key Vault / Microsoft-managed keys on Azure), encryption in transit (TLS), private network isolation (VPC security groups and PrivateLink on AWS; VNet service endpoints and Private Endpoint on Azure), and auditing. The differences are in identity integration and the surrounding security services.

- → **Windows Authentication.** RDS for SQL Server supports Windows Authentication via AWS Managed Microsoft AD — your domain users and groups can authenticate to RDS. Azure SQL Database and Managed Instance authenticate via Microsoft Entra ID (formerly Azure AD), including managed identities for Azure services, which eliminates connection-string passwords for app-to-database calls. If your application relies on Windows integrated authentication with an on-premises AD, both paths work but the trust topology differs: AWS needs AD connectivity to its managed AD; Azure needs Entra Connect / federation.
- → **TDE and key control.** Transparent Data Encryption is available on all three. RDS uses KMS (AWS-managed or customer-managed keys); Azure SQL enables TDE by default on new databases with Microsoft-managed keys and supports customer-managed keys in Key Vault. Both satisfy the "customer controls the key" compliance requirement.
- → **Threat detection.** RDS leans on GuardDuty and CloudTrail for the control plane plus Performance Insights for workload anomalies. Azure has Microsoft Defender for SQL, which bundles vulnerability assessment, advanced threat protection, and data discovery/classification into the database service itself. If your compliance regime wants vulnerability assessment reports out of the box, Defender for SQL is the more integrated story.
- → **Auditing.** RDS for SQL Server publishes audit logs to CloudWatch/S3 via option settings; Azure SQL auditing writes to Log Analytics, Event Hub, or storage with a single policy switch. Both are straightforward; Azure's policy-driven auditing across an entire estate (via Azure Policy) scales better for large fleets.

7.9 Operations: Patching, Backups, Monitoring

Patching. RDS patches during your maintenance window; you control the window and can defer, but AWS will eventually force critical patches. Azure SQL Database and Managed Instance are patched automatically by the platform with minimal downtime — you configure a maintenance window for MI to control timing, but there is no "patch Tuesday" you manage. Teams that want patching to disappear entirely prefer Azure; teams that want to correlate patches with their change process prefer RDS's explicit windows.

Backups. RDS automated backups: snapshots plus log backups, 0–35 day retention, PITR to a new instance, manual snapshots retained indefinitely. Azure SQL automated backups: full/differential/log backups managed entirely by the platform, 7–35 day PITR retention

depending on tier, plus long-term retention (LTR) policies up to 10 years for compliance — a first-class feature with no RDS equivalent. Azure backups restore to a new database (like RDS restores to a new instance), and geo-redundant backup storage gives you cross-region restore capability built in. If your compliance team asks for 7-year retention, Azure LTR answers it natively; on RDS you build it from snapshots and lifecycle policies.

Monitoring. RDS gives you CloudWatch metrics, Enhanced Monitoring, and Performance Insights (Chapter 6). Azure gives you Azure Monitor metrics, Log Analytics, Query Store (on by default — a flight recorder for query plans and runtime stats that RDS does not have natively), and Intelligent Insights for automatic anomaly detection. Query Store deserves emphasis: automatic plan regression detection and forced-plan remediation are built into the engine on Azure, while on RDS you assemble the equivalent from Query Store (available in SQL Server 2016+ on RDS, but you manage it) plus Performance Insights.

7.10 Migration Paths to Each

To RDS for SQL Server: AWS Database Migration Service (DMS) with ongoing replication for near-zero downtime, or native backup/restore to S3 for simpler cutovers (take a native backup on-premises, upload to S3, restore with `rds_restore_database`). AWS Schema Conversion Tool matters less for SQL Server-to-SQL Server than for heterogeneous migrations. Test the restore path first: native restore validates edition compatibility and exposes unsupported features early.

To Azure SQL: assess with DMA or the Azure Data Studio migration extension, then choose the target the assessment points to (usually MI for existing estates). Migrate with Azure Database Migration Service, or use the **MI link** — a near-zero-downtime migration path that replicates from your on-premises SQL Server to Managed Instance over Always On technology and fails over when you are ready. The MI link is the smoothest SQL Server cloud migration path that exists on any cloud: it is online, it is built on AG replication you already understand, and rollback is failing back to the primary.

Whichever target you choose, the migration checklist from Chapter 3 applies: inventory Agent jobs, linked servers, CLR, cross-database dependencies, and SSIS packages; measure log generation rate and peak IOPS; and run a production-scale performance test on the target tier before cutover. The platform decision and the migration plan are the same project.

7.11 The Decision Framework

Apply these in order. The first one that fires usually decides it:

- → **Existing Microsoft licenses with Software Assurance?** Price Azure SQL with Azure Hybrid Benefit against RDS BYOL/LI. AHB frequently wins by a wide margin — this single factor decides more evaluations than all the technical criteria combined.
- → **Where does the rest of the estate live?** Data gravity and operational gravity matter. A workload deeply integrated with AWS services (Lambda, Kinesis, S3 data lakes) usually belongs on RDS; a workload tied to Active Directory, Microsoft 365, and ExpressRoute usually belongs on Azure.
- → **How much application change is acceptable?** Zero-to-minimal change points to RDS for SQL Server or Managed Instance. Willingness to modernize (elastic pools, serverless, Hyperscale) opens up Azure SQL Database, which rewards cloud-native design with the best scaling and DR story.
- → **What does the compatibility inventory say?** Heavy SQL Agent, linked server, CLR, or cross-database usage eliminates SQL Database and leaves RDS vs MI — decided by the first two questions.
- → **Do you need read scale-out or 100 TB databases?** Hyperscale named replicas and 100 TB ceilings are unmatched; if you need them, the answer is Azure SQL Database Hyperscale.
- → **Compliance retention?** Multi-year backup retention requirements favor Azure's built-in long-term retention.

Key Takeaways

- → **"Azure SQL" is two products, not one.** Azure SQL Database (PaaS, no instance, elastic scale, longest feature-gap list) and Managed Instance (near-full engine compatibility, VNet-integrated, the lift-and-shift target) must be evaluated separately. Most existing estates belong on Managed Instance, not SQL Database.
- → **Licensing usually decides before technology does.** Azure Hybrid Benefit for licenses with Software Assurance, versus RDS License Included or BYOL via License Mobility — model the real prices with your actual agreements before comparing features. The cheapest technically-sufficient option wins.
- → **Inventory compatibility before choosing.** Agent jobs, linked servers, CLR assemblies, cross-database queries, and SSIS packages each eliminate or complicate options. Run DMA or the Azure Data Studio assessment and let the inventory, not intuition, rule out platforms.
- → **For DR depth and read scale-out, Azure's story is more complete.** Active geo-replication, auto-failover groups, Hyperscale named replicas, and built-in long-term retention are first-class features; on RDS you assemble equivalents from snapshots, copies, and EC2-based secondaries.
- → **For operational familiarity on AWS, RDS is the simpler mental model.** Instances, endpoints, parameter groups, and Multi-AZ failover map directly onto what SQL Server DBAs already know — at the cost of no read replicas, no serverless option, and a 16 TB ceiling.

Chapter 8: SQL Server on EC2: Self-Managed in the Cloud

RDS is the default answer for SQL Server on AWS, and Chapter 6 showed why: automated backups, Multi-AZ failover, and patching without effort. But "default" is not "universal." A meaningful fraction of SQL Server workloads cannot live on RDS — they need components RDS does not offer, control RDS does not grant, or architectures RDS does not support. For those workloads, the answer is SQL Server on EC2: the full product, on infrastructure you control, with cloud-native tooling wrapped around it.

This chapter is the EC2 playbook: when self-managed beats RDS, how licensing works on EC2, how to size instances and storage, how to build Always On availability groups across Availability Zones, and how to handle patching, backup, and monitoring without the RDS managed layer doing it for you.

8.1 When EC2 Beats RDS

Choose EC2 over RDS when you need something RDS structurally cannot give you:

- → **Engine-adjacent components.** SSIS, SSAS, and SSRS do not run on RDS. If your ETL packages, cubes, or paginated reports must stay close to the database, EC2 hosts the full stack — often on the same instance or a dedicated companion instance in the same AZ.
- → **Readable secondaries.** RDS for SQL Server has no readable secondary (Chapter 6). An Always On availability group on EC2 gives you up to 8 readable secondaries for reporting offload — the standard answer to read pressure on SQL Server.
- → **Instance consolidation.** RDS is one engine per instance. On EC2 you can consolidate multiple instances or dozens of databases per VM, which is often dramatically cheaper for estates with many small databases.
- → **Patch-level control.** RDS patches on its schedule within your window. If your application is certified against a specific cumulative update, or you need a GDR security fix the day it ships, EC2 lets you patch exactly when and to what you choose.
- → **Unrestricted engine surface.** UNSAFE CLR assemblies, linked servers to any provider, replication (transactional, merge, snapshot), PolyBase, Machine Learning Services, FileStream — all fully available on EC2, restricted or absent on RDS.
- → **Custom backup targets.** Back up to S3, to FSx, to a cross-account bucket with your own lifecycle and immutability policies — not just the RDS backup window.

The honest cost of EC2 is operational: you own the OS, the patching, the failover cluster, and the 3 AM pages. Chapter 2 quantified that trade-off. This chapter assumes you have decided the control is worth it and shows how to do it well.

8.2 Licensing on EC2: License Included vs BYOL

EC2 offers two licensing paths for SQL Server, and the choice dwarfs most technical decisions in dollar terms.

License Included (LI) AMIs. AWS publishes Windows Server AMIs with SQL Server pre-installed (Express, Web, Standard, Enterprise). The license cost is folded into the hourly EC2 rate. This is the simplest path: no license paperwork, no true-up, per-second billing. It is also the most expensive per hour for Standard and especially Enterprise. LI is ideal for short-lived, elastic, or uncertain workloads — dev/test, burst capacity, migrations in flight.

Bring Your Own License (BYOL). Deploy SQL Server on a plain Windows Server EC2 instance using your existing licenses. For production, this almost always means **License Mobility through Software Assurance**: with active SA, you may run your SQL Server licenses on shared-tenancy EC2. Without SA, BYOL on shared tenancy is generally not permitted — you would need Dedicated Hosts or Dedicated Instances, which changes the economics. The rules here are a licensing conversation with your Microsoft reseller, not an AWS configuration: get the answer in writing before you build.

The per-core math that matters. SQL Server Enterprise is licensed per core (4-core minimum per VM). On EC2, vCPUs map to hyperthreads — SQL Server licensing counts physical cores, so a VM with 8 vCPUs generally needs 4 core licenses. Size the VM to the license footprint, not just the workload: an r6i.2xlarge (8 vCPU) needs 4 Enterprise core licenses whether it is 10% busy or 90% busy. Consolidation is the lever — putting four departmental databases on one licensed VM instead of four VMs can quarter the license cost.

```
# Find the latest AWS-provided SQL Server 2022 Enterprise AMI (License Included)
aws ec2 describe-images \
  --owners amazon \
  --filters "Name=name,Values=Windows_Server-2022-English-Full-SQL_2022_Enterprise*" \
  --query 'Images | sort_by(@, &CreationDate) | [-1].[ImageId,Name,CreationDate]' \
  --output table \
  --region us-east-1

# For BYOL: start from a plain Windows Server AMI and install your media
aws ec2 describe-images \
  --owners amazon \
  --filters "Name=name,Values=Windows_Server-2022-English-Full-Base*" \
  --query 'Images | sort_by(@, &CreationDate) | [-1].[ImageId,Name]' \
  --output table \
  --region us-east-1
```

8.3 Instance and Storage Sizing

Instance families. The same guidance as RDS applies, minus the managed layer: R-family (r7i, r6i) for memory-heavy OLTP, M-family for balanced workloads, and dedicated hosts only if your licensing requires them. Two EC2-specific notes: first, EBS-optimized instances are effectively standard on modern families, but verify `efs0ptimized` is enabled — without it, EBS traffic contends with network traffic and your storage benchmarks will lie to you. Second, for the most demanding log-write workloads, consider instances with NVMe instance-store volumes for tempdb (more below).

Storage layout: separate everything. On RDS, storage is one volume you cannot see into. On EC2, you design the layout: data files on one EBS volume (or striped set), log files on another, tempdb on a third — or on NVMe instance-store volumes for maximum throughput. The rules are the on-premises rules, enforced with EBS knobs:

- → **Data:** gp3 with provisioned IOPS, or io2 for the highest durability and IOPS ceilings. Stripe multiple volumes with Windows Storage Spaces or dynamic disks if a single volume's throughput ceiling binds you.
- → **Log:** dedicated gp3/io2 volume sized for write latency, not capacity. Log writes are sequential and latency-sensitive — this is the volume where provisioned IOPS pays for itself.
- → **Tempdb:** NVMe instance-store volumes (ephemeral — data lost on stop/start, which is fine for tempdb) or a dedicated gp3 volume. Multiple equally-sized tempdb data files, one per logical processor up to 8, as ever.
- → **Backups:** a dedicated volume as a staging area, then lifecycle to S3. Never back up to the data or log volume — a runaway backup filling the log volume is a classic self-inflicted outage.

Format for performance. Use 64 KB allocation unit size on SQL Server data and log volumes — the same best practice as on-premises. It is easy to forget on cloud-provisioned volumes and costs measurable throughput on write-heavy workloads.

```
# PowerShell: verify EBS optimization and volume layout on the instance
Get-EC2Instance -InstanceId i-0abc123def456789 |
  Select-Object -ExpandProperty Instances |
  Select-Object InstanceType, EbsOptimized

# List attached volumes with their device mapping
aws ec2 describe-volumes \
  --filters Name=attachment.instance-id,Values=i-0abc123def456789 \
  --query 'Volumes[*].[VolumeId,Size,VolumeType,Iops,Attachments[0].Device]' \
  --output table \
  --region us-east-1
```

8.4 Provisioning: AMIs, SysPrep, and Infrastructure as Code

Do not build SQL Server EC2 instances by hand in the console. Build a golden AMI pipeline: start from the AWS SQL Server AMI (or your BYOL base + slipstreamed media), apply your standard configuration (data/log/tempdb volume layout, 64K formatting, `max server memory`, backup jobs, CloudWatch agent, SSM agent), generalize with SysPrep, and bake the AMI. New instances — including AG secondaries and DR nodes — launch from that AMI in minutes with identical configuration. Configuration drift between "identical" manually-built instances is one of the most common sources of failover surprises.

Terraform or CloudFormation should own the instance, volumes, security groups, and IAM roles. The SQL Server configuration itself (AG creation, logins, jobs) belongs in a configuration management tool or a tested PowerShell DSC/Ansible playbook run at launch via user data. Keep the two layers separate: infrastructure code builds the VM, configuration code builds the SQL Server.

8.5 Always On Availability Groups Across Availability Zones

This is the reason most teams choose EC2: a real Always On availability group with readable secondaries, spanning AZs for resilience. The architecture is standard AG technology with cloud-specific adaptations for networking and quorum.

Topology. Two or three replicas in different Availability Zones (three AZs with one replica each is the resilient pattern — it survives an AZ loss with quorum intact). Each replica is an EC2 instance running SQL Server Standard or Enterprise. Note the edition math: Basic

availability groups (Standard Edition) support one database per group and no readable secondaries; full AGs with readable secondaries require Enterprise. If read offload is the goal, Enterprise is the requirement — price it accordingly.

The listener problem. On-premises, the AG listener is a clustered network name with a single IP that floats. In a VPC, an IP cannot float across AZs — subnets are AZ-scoped. The solution is a **multi-subnet listener**: the listener gets one IP per AZ subnet, DNS returns all of them, and the client connection string uses `MultiSubnetFailover=True` so the driver tries all IPs in parallel and connects to whichever is live. Without that connection-string parameter, clients wait out TCP timeouts on dead IPs during failover — the single most common AG-on-AWS client misconfiguration.

Quorum without shared storage. Traditional failover clustering wants a shared disk witness. In AWS, use a **file share witness** — an SMB share on a small third-AZ instance or on Amazon FSx for Windows File Server. For a two-replica AG, the witness holds the deciding vote; without it, losing one AZ takes the AG offline. With three replicas across three AZs, node majority alone sustains quorum through a single AZ loss, and the witness becomes belt-and-suspenders.

```
-- T-SQL: verify AG health the same way on EC2 as on-premises
-- (this is the query RDS Multi-AZ never lets you run)
SELECT ar.replica_server_name,
       rs.role_desc,
       rs.connected_state_desc,
       rs.synchronization_state_desc,
       drs.log_send_queue_size AS log_send_queue_kb,
       drs.redo_queue_size AS redo_queue_kb,
       drs.last_commit_time
FROM sys.dm_hadr_availability_replica_states rs
JOIN sys.availability_replicas ar ON rs.replica_id = ar.replica_id
JOIN sys.dm_hadr_database_replica_states drs ON drs.replica_id = rs.replica_id
WHERE drs.is_local = 0;

-- Failover Cluster Manager equivalent: check quorum configuration
-- (run in PowerShell on any cluster node)
Get-ClusterQuorum | Select-Object QuorumResource, QuorumType
```

Failover Cluster Instances (FCI): an alternative to AGs when you need instance-level protection (system databases, Agent jobs, linked servers fail over together). FCI requires shared storage — on EC2 that means EBS multi-attach (io1/io2 volumes attached to multiple instances in the same AZ, so FCIs are single-AZ) or a third-party shared-storage solution. Because FCI is single-AZ on native EBS, most cloud AG designs prefer availability groups over FCIs. Use FCI only when instance-level failover semantics are a hard requirement.

8.6 Patching Without RDS

On EC2, patching is yours: Windows updates and SQL Server cumulative updates. The disciplined pattern is a monthly cycle driven by AWS Systems Manager (SSM): patch AMIs first, test the AMI against a restored copy of production, then roll the fleet — secondaries before primaries, with AG failover orchestrating the primary patch. Never patch the primary first; the rollback story is always "fail back to the unpatched secondary."

Slipstream CUs into your golden AMI pipeline so new instances launch current. And keep a tested rollback: for AGs, rollback is failing over to a replica you have not patched yet — which means staging patches one replica at a time and validating the workload on the patched secondary (readable secondaries earn their keep here as canaries) before committing the primary.

8.7 Backup to S3: Native Backups, Your Lifecycle

Without RDS automated backups, you run native SQL Server backups — which is a feature, not just a burden. Native `BACKUP DATABASE ... TO URL` (SQL Server 2016+ with the S3-compatible credential, or via AWS Systems Manager / custom tooling writing to S3) gives you portable backup files you can restore anywhere: another EC2 instance, RDS (via `rds_restore_database`), or back on-premises. That portability is your exit strategy and your DR story in one.

```
-- Native backup with compression to a local staging volume,
-- then lifecycle to S3 (run as a SQL Agent job)
BACKUP DATABASE [appdb]
TO DISK = 'E:\Backups\appdb_full.bak'
WITH COMPRESSION, CHECKSUM,
    STATS = 10;
GO

-- Verify the backup before it leaves the building
RESTORE VERIFYONLY FROM DISK = 'E:\Backups\appdb_full.bak';
GO

-- Log backups every 15 minutes keep RPO tight
BACKUP LOG [appdb]
TO DISK = 'E:\Backups\appdb_log.trn'
WITH COMPRESSION, CHECKSUM;
GO
```

Push staging backups to S3 with versioning and Object Lock (compliance mode) for immutability — ransomware cannot encrypt what it cannot overwrite. Cross-region replication on the bucket gives you geographic DR for the price of storage. And test restores monthly with an automated job that restores the latest backup to a throwaway instance and runs `DBCC CHECKDB`: an untested backup is a rumor, not a recovery plan.

8.8 Monitoring: What RDS Was Hiding From You

On EC2 you get everything back: full PerfMon counters, Extended Events sessions of your design, the SQL Server error log on disk, Windows event logs, and OS-level metrics via the CloudWatch agent. The monitoring stack to build:

- → **SQL Server counters:** `Page life expectancy`, `Buffer cache hit ratio`, `Batch Requests/sec`, `Lock Waits/sec`, log flush waits — the same counters as on-premises, shipped to CloudWatch via the agent.
- → **AG health:** the DMV query from section 8.5 on a schedule, alerting on `log_send_queue_kb` growth and `synchronization_state_desc != 'SYNCHRONIZED'` for synchronous replicas.
- → **OS and EBS:** disk queue depth and latency per volume (your early warning for EBS contention), memory pressure, and network throughput on the replication path between AZs.
- → **Backup verification:** alert on missed backup jobs and failed `RESTORE VERIFYONLY` results — the managed backup safety net is gone, so this alert is load-bearing.

8.9 The Cost Comparison, Honestly

EC2 is cheaper than RDS per compute-hour for the same instance class — AWS charges a management premium for RDS. But the honest comparison includes: your engineering time for patching, monitoring, and failover testing; the license footprint (consolidation favors EC2, per-VM licensing punishes sprawl); and the cost of the secondaries. An RDS Multi-AZ deployment and a two-replica EC2 AG both run two copies of the workload — the EC2 version adds your labor and subtracts the RDS premium. For a single production database with standard requirements, RDS usually wins on total cost. For consolidated estates, readable-secondary architectures, or workloads needing full engine surface, EC2 wins. Model both for your actual topology; the answer varies more than the marketing suggests.

8.10 Common EC2 Mistakes

Self-managed means self-inflicted outages are available in new varieties. The recurring ones:

- → **Forgetting `MultiSubnetFailover=True`**. Without it, client failover across AZs takes as long as TCP timeouts allow — minutes of apparent outage for a failover that completed in seconds. Audit every connection string, including the ones in reporting tools and ETL packages that nobody remembers.
- → **Tempdb on EBS without thinking**. Tempdb on a general-purpose EBS volume shared with data files is the most common EC2 storage mistake. Tempdb allocation contention shows up as `PAGELATCH_UP` waits on allocation pages — give it dedicated throughput or instance-store NVMe.
- → **No quorum vote planning**. A two-node AG without a witness loses quorum when either AZ fails — the exact event you built the AG to survive. The witness is not optional; it is the third vote.
- → **Patching the primary first**. Always roll secondaries first and validate the workload on a patched readable secondary before failing over. A bad CU on the primary with no clean rollback is a resume-generating event.
- → **Treating S3 backup copy as backup testing**. The backup reaching S3 proves the copy worked. Only an automated restore plus `DBCC CHECKDB` proves the backup is restorable. Schedule the restore test; alert on its failure like any production alert.

Key Takeaways

- → **Choose EC2 when RDS structurally cannot serve the workload**. Readable secondaries, SSIS/SSAS/SSRS, unrestricted engine surface (CLR, replication, linked servers), instance consolidation, and patch-level control are the legitimate reasons — not vague discomfort with managed services.
- → **Licensing dominates the EC2 economics**. License Included AMIs are simplest; BYOL via License Mobility with Software Assurance is usually cheapest for steady production. Size VMs to the license footprint and consolidate aggressively — per-core Enterprise licensing punishes sprawl.
- → **AGs on EC2 need cloud-specific design**. Multi-subnet listeners with `MultiSubnetFailover=True`, file share witnesses for quorum, three AZs for real resilience. The on-premises AG knowledge transfers; the networking and quorum pieces do not.
- → **Own the undifferentiated heavy lifting with automation**. Golden AMI pipelines, IaC for infrastructure, config management for SQL Server, SSM-driven patching with secondaries-first rollout. EC2 without automation is just on-premises with extra latency.
- → **Native backups to S3 are a feature**. Portable backup files, Object Lock immutability, cross-region replication, and automated restore testing give you a DR and exit story that managed backups cannot match — at the cost of owning the backup jobs yourself.

Chapter 9: Failover Testing and Game Days: Proving Your SQL Server HA Works

Every architecture in Chapters 6 through 8 — RDS Multi-AZ, Always On availability groups on EC2, Azure failover groups — makes a promise: when the primary fails, the workload continues. That promise is worth exactly as much as your last successful test of it. Untested high availability is not high availability; it is optimism with a diagram. This chapter is about turning the diagrams into evidence: how to test failovers safely, how to measure what actually happens, how to make applications survive the transition, and how to run game days that find the gaps before your customers do.

9.1 The Failover You Never Tested

The pattern repeats across the industry: a team builds Multi-AZ or an availability group, watches one clean failover during the initial setup, and never tests again. Eighteen months later the primary fails at 2 AM and the team discovers that the application connection strings never had `MultiSubnetFailover=True`, the reporting job points at the old primary's DNS name cached in a config file, the logins on the secondary are missing three service accounts created last quarter, and the runbook describes an architecture that was redesigned six months ago. The failover "worked" — the database came up on the secondary — and the application was down for 47 minutes anyway.

Failover testing has a half-life. Connection strings change, logins drift, DNS TTLs get modified, application retry logic gets refactored away by someone who never saw the original incident. A failover test is a measurement of the entire system — database, network, application, and operations — at a point in time. Repeat it on a schedule or it decays.

9.2 What to Test on Each Platform

RDS Multi-AZ: reboot with failover. The safe, supported test is `reboot-db-instance --force-failover`. This exercises the real failover path: DNS update, AG replica promotion, and application reconnect. Do it during a maintenance window the first time, then promote it to a quarterly scheduled test. Watch the actual client-observed downtime, not the RDS event timestamps — the gap between "RDS says failover complete" and "application serving traffic" is the number that matters.

```
# Planned failover test on RDS Multi-AZ (safe: exercises the real path)
aws rds reboot-db-instance \
  --db-instance-identifier prod-sql-primary \
  --force-failover \
  --region us-east-1

# Watch the failover events in real time
aws rds describe-events \
  --source-identifier prod-sql-primary \
  --source-type db-instance \
  --start-time $(date -u -d '10 minutes ago' +%Y-%m-%dT%H:%M:%SZ) \
  --query 'Events[*].[Date,Message]' \
  --output table \
  --region us-east-1
```

Always On AGs on EC2: manual failover. Use `ALTER AVAILABILITY GROUP ... FAILOVER` for planned tests — it validates the listener, the quorum, and the client drivers without the chaos of a real crash. Then, once planned failovers are boring, test unplanned: stop the SQL Server service on the primary (not the whole VM — you want to test database-level failure first, host-level failure second). Verify that automatic failover triggers within your configured timeout and that the application reconnects through the multi-subnet listener.

```
-- Planned AG failover test (run on the primary replica)
ALTER AVAILABILITY GROUP [ag_prod] FAILOVER;
GO

-- Verify the new primary and replica roles after failover
SELECT ar.replica_server_name,
       rs.role_desc,
       drs.synchronization_state_desc,
       drs.last_commit_time
FROM sys.dm_hadr_availability_replica_states rs
JOIN sys.availability_replicas ar ON rs.replica_id = ar.replica_id
JOIN sys.dm_hadr_database_replica_states drs ON drs.replica_id = rs.replica_id
WHERE drs.is_local = 1;
```

Azure: failover group drills. For Managed Instance and SQL Database failover groups, use a planned failover from the Azure portal or CLI during a window, then graduate to forced failover (which simulates data loss — verify your RPO expectations against reality). Azure's grace period setting exists to prevent flapping; test with the grace period you run in production, because it directly adds to observed downtime.

```
# Planned failover of an Azure failover group (test the listener endpoint)
az sql failover-group set-primary \
  --resource-group rg-prod \
  --server sqlsrv-secondary \
  --name fg-prod \
  --allow-data-loss false
```

9.3 Measuring RTO Honestly

Vendor RTO numbers measure the database. Your RTO is measured at the client. Instrument the full chain: a canary job that executes a trivial write every 10 seconds and records gaps; application-level success-rate dashboards during the test; DNS resolution time for the endpoint (RDS endpoint DNS changes propagate in 60–120 seconds, and clients with aggressive DNS caching see longer); and connection-pool drain time (pools full of dead connections to the old primary must detect failure, discard, and reconnect — misconfigured pools turn a 60-second database failover into a 10-minute application outage).

Record three numbers from every test: detection time (failure to failover initiation), promotion time (failover initiation to new primary accepting writes), and client recovery time (new primary accepting writes to application fully serving traffic). The third number is always the largest, and it is the only one your users experience. If client recovery dominates, your next investment is application resilience (section 9.4), not database tuning.

9.4 Application Resilience: Surviving the Transition

The database can fail over perfectly and the application can still fall over. The application-side requirements:

- → **Retry logic with backoff.** Transient failures during failover (connection refused, transport errors, 40613 on Azure, 40197 during reconfiguration) must be retried, not surfaced to users. .NET's `SqlConnection` with `ConnectRetryCount / ConnectRetryInterval` handles connection-phase retries; command-phase retries belong in your data access layer or a library like Polly.
- → **MultiSubnetFailover=True on every connection string.** For AG listeners (EC2) and Azure failover group listeners. Audit relentlessly — the ETL package, the BI tool, and the legacy Windows service are where this gets missed.
- → **Idempotent writes.** A write that succeeded on the old primary but whose acknowledgment was lost will be retried by the client. If the retry is not idempotent, failover creates duplicate orders, duplicate payments, duplicate rows. Design for at-least-once delivery at the database boundary.

- → **Read-only intent where possible.** Reporting connections using `ApplicationIntent=ReadOnly` route to readable secondaries where they exist (EC2 AGs, MI Business Critical) and keep the primary's connection budget for writes during the chaos of a failover.

9.5 Designing a Game Day

A game day is a scheduled, controlled failure exercise with observers, a timeline, and a blameless review. Start small and escalate across quarters:

- → **Level 1: planned failover.** Announced, during business hours, full team watching. Success criterion: client-observed downtime within the RTO budget and zero manual steps beyond the planned ones.
- → **Level 2: unplanned primary kill.** Stop the SQL Server service (or reboot the primary VM) without warning the application team — but with the DBA team ready. Success criterion: automatic failover completes, application recovers via retry logic, and nobody pages anyone.
- → **Level 3: AZ failure simulation.** Block network traffic to the primary's AZ (security group rules or route manipulation) to simulate AZ impairment rather than clean host death. This tests quorum behavior, witness function, and the difference between "host down" and "network partition" — the scenario that causes split-brain anxiety.
- → **Level 4: region drill.** Fail over the DR topology (restore from cross-region snapshot, or fail over the Azure failover group with data loss allowed) and run the business's critical transactions against it. This is the only test that validates your actual disaster recovery, as opposed to your high availability.

Rules for all levels: never run a game day you cannot abort; have an explicit rollback decision-maker; record everything; and hold the blameless review within 48 hours while memories are fresh. The review's output is not a grade — it is work items: the missing login, the un-audited connection string, the runbook paragraph that lied.

9.6 Post-Failover Validation Checklist

The failover completing is the midpoint, not the end. Run this validation before declaring victory — automate it as a post-failover job:

- → **Replica roles and sync state.** Confirm the old primary is now a synchronized secondary (not stuck in `NOT SYNCHRONIZING`), and that all databases joined the AG cleanly. A database that failed to join the group after failover is a data-loss incident wearing a success costume.
- → **Logins and jobs.** Compare `sys.server_principals` and `msdb.dbo.sysjobs` between replicas. Logins and Agent jobs live outside the availability group — drift here is the number one cause of "failover worked but the app is broken."

- → **Database integrity spot-check.** Run `DBCC CHECKDB` on the most critical database, or at minimum verify the latest backup chain is intact on the new primary.
- → **Application smoke tests.** Execute the business's critical transactions end to end — login, search, checkout, report — not just a `SELECT 1`. Wire these into your deployment pipeline so they exist before you need them.
- → **Monitoring re-baselining.** Confirm alerts, dashboards, and backup jobs are pointing at the new primary. Backup jobs that kept targeting the old primary's hostname silently stop protecting you.

```
-- Post-failover drift check: logins present on primary but missing on secondary
-- Run on the secondary; compare against the primary's list
SELECT sp.name AS login_name, sp.type_desc
FROM sys.server_principals sp
WHERE sp.type IN ('S', 'U', 'G')
      AND sp.name NOT LIKE '##%'
      AND NOT EXISTS (
          SELECT 1 FROM [PRIMARY_LINKED].master.sys.server_principals p
          WHERE p.name = sp.name
      );
-- (Replace PRIMARY_LINKED with a linked server to the primary,
-- or export both lists and diff them in your automation.)
```

9.7 When Failover Goes Wrong: Real Failure Modes

Game days exist because failovers fail in specific, repeatable ways. Knowing the failure modes in advance turns a 3 AM mystery into a checklist:

- → **Quorum loss.** The cluster loses majority — typically an AZ failure in a two-AZ design without a working witness, or the witness share becoming unreachable at the worst moment. Symptom: the AG goes offline everywhere instead of failing over. Prevention: three AZs or a verified witness, and a game day that actually kills an AZ rather than a SQL Server service.
- → **Database not joined.** A database on the new primary sits in `NOT SYNCHRONIZING` because a log backup chain broke, a file path differs between replicas, or the database was added to the instance but never joined to the group. The AG reports healthy; the database is not protected. Your validation checklist must check per-database state, not just replica state.
- → **Listener DNS staleness.** Clients cache the old listener IPs past the DNS TTL — Java JVMs cache DNS forever by default unless `networkaddress.cache.ttl` is set, and some connection pools hold resolved addresses for the pool's lifetime. After a failover, a subset of application servers keep hammering the dead primary while others work fine — the most confusing partial-outage pattern in this space.

- → **Azure grace period surprise.** A failover group with a one-hour grace period will not fail over automatically for one hour after an outage begins — by design, to ride out transient blips. If your RTO budget is 15 minutes, the grace period must be 15 minutes or less. This is a configuration value that directly contradicts the RTO unless someone sets it deliberately.
- → **RDS failover during heavy write load.** Failover time stretches when the secondary has redo backlog — the promotion must complete crash recovery before accepting writes. If your workload generates log faster than the secondary redoes it, your tested 90-second failover becomes a 12-minute one under production load. Test failover under load, not just on a quiet Sunday.
- → **Split-brain writes.** True split-brain (two primaries accepting writes) is prevented by quorum on AGs and by the platform on RDS/Azure — but application-level split-brain happens when a "failed" primary is actually still alive and serving a subset of clients that never noticed the failover (see DNS staleness above). Fence the old primary: revoke its client network access or stop the service as part of the failover procedure.

For each failure mode, your runbook should name the symptom, the detection query or alarm, and the remediation command. The team that has rehearsed quorum loss in a game day recovers in minutes; the team meeting it for the first time at 3 AM recovers in hours.

9.8 Automating Failover Tests

Manual failover tests happen twice and then stop. Automated ones happen on schedule. Build a quarterly pipeline: trigger the failover (SSM automation document for EC2, a scheduled Lambda for RDS `reboot-db-instance --force-failover`, an Azure DevOps pipeline for failover groups), run the canary workload, collect the three RTO numbers, run the validation checklist, and publish the results to the team channel. Alert on regression: if client recovery time grows quarter over quarter, something drifted — find it while it is a graph, not an incident.

Keep the automation's blast radius explicit: it should only ever target the instances in its scope, require an explicit approval gate for production (at least until the team trusts it), and log every action with timestamps for the review. The goal is boring, repeatable evidence — not chaos for its own sake.

9.9 The Runbook Is a Product

Every game day produces runbook updates. Treat the failover runbook as a product with users (the on-call DBA at 3 AM), versioning, and a test suite (the game days themselves). It should contain: the exact commands for planned and forced failover on your platform; the expected timeline with the three RTO numbers from the last test; the validation checklist; the

escalation path when failover does not complete; and the known gaps with their work-item links. A runbook nobody has tested is documentation; a runbook validated by last quarter's game day is an operations asset.

Key Takeaways

- → **Untested HA is optimism, not engineering.** Failover capability decays as connection strings, logins, DNS, and application code drift. Test on a schedule — quarterly at minimum — or accept that your RTO is a guess.
- → **Measure client-observed recovery, not database promotion.** Detection time, promotion time, and client recovery time are three different numbers; the third is the largest and the only one users feel. Instrument it with canary writes and application dashboards.
- → **Applications must be failover-ready, not just failover-adjacent.** Retry logic with backoff, `MultiSubnetFailover=True` on every connection string, idempotent writes, and read-only intent where possible. Audit the forgotten connection strings — ETL, BI, legacy services.
- → **Escalate game days from planned to adversarial.** Planned failover, then unplanned primary kill, then AZ impairment, then full region drill. Each level tests a different failure mode; only the region drill validates disaster recovery rather than high availability.
- → **Validate after every failover and automate the tests.** Replica sync state, login/job drift, integrity spot-checks, application smoke tests, monitoring re-baselining. Automate the whole cycle quarterly and alert on RTO regression.

Chapter 10: RDS Proxy for SQL Server: Connection Management at Scale

Connection management is a problem every production DBA has solved before — connection pooling in the application tier, disciplined connection lifetimes, enough headroom for peaks. In the cloud, the problem changes shape: serverless functions open and close connections on every invocation, auto-scaled fleets multiply connection counts unpredictably, and a Multi-AZ failover that should take 60 seconds turns into a 10-minute outage because thousands of clients reconnect simultaneously and overwhelm the new primary. RDS Proxy exists for exactly these problems: a managed connection pooler that sits between your applications and RDS for SQL Server, multiplexing thousands of client connections onto a small pool of database connections and smoothing over failovers.

A note on history: early versions of RDS Proxy did not support SQL Server, and older documentation — including earlier drafts of this book — said so. That changed: RDS Proxy now supports RDS for SQL Server (engine family `SQLSERVER`), with Secrets Manager-based authentication. This chapter covers the SQL Server reality: creating and configuring the proxy, authentication, monitoring and connection pinning, failover behavior, sizing, limitations, and a runbook for diagnosing pool exhaustion with T-SQL.

10.1 Why Connection Management Is Different in the Cloud

On-premises, you controlled both ends of the connection. You knew how many application servers existed, each ran a fixed-size ADO.NET connection pool, and the total was a number you could compute on a whiteboard. In the cloud, three things break that model:

- → **Serverless connection churn.** An AWS Lambda function that queries SQL Server on every invocation opens a connection, runs one batch, and closes it. At 500 concurrent executions, that is 500 connection setups per burst — each one a TCP handshake, TLS negotiation, and SQL Server login sequence. Login storms like this show up as `PREEMPTIVE_OS_AUTHENTICATIONOPS` waits and CPU spikes on the database, not in your application metrics.
- → **Elastic fleet scaling.** Auto Scaling groups and ECS/EKS workloads scale connection counts with traffic. Your database's `user connections` setting and worker thread pool are fixed; the fleet is not. The failure mode is `THREADPOOL` waits — new connections queue because no worker thread is free to run the login task.

- → **Failover reconnect storms**. During a Multi-AZ failover, every client connection breaks at once and every client reconnects at once. Without a proxy, the new primary absorbs a thundering herd of logins while it is still completing crash recovery. With a proxy, the proxy holds the client connections and re-establishes its smaller pool to the new primary — clients see a pause, not an error.

ADO.NET connection pooling helps within a single process, but it does not help across hundreds of Lambda execution environments or dozens of containers. RDS Proxy pools across all of them, at the infrastructure layer.

10.2 Creating and Configuring RDS Proxy for SQL Server

A proxy needs: a name, the `SQLSERVER` engine family, Secrets Manager credentials for the database login, an IAM role the proxy assumes, subnets and security groups, and TLS enforcement. Create it with the CLI:

```
# Create an RDS Proxy for a SQL Server instance
aws rds create-db-proxy \
  --db-proxy-name prod-sql-proxy \
  --engine-family SQLSERVER \
  --auth '[{
    "AuthScheme": "SECRETS",
    "SecretArn": "arn:aws:secretsmanager:us-east-1:123456789012:secret:prod/
sqlserver/dbadmin",
    "IAMAuth": "DISABLED"
  }]' \
  --role-arn arn:aws:iam::123456789012:role/rds-proxy-role \
  --vpc-subnet-ids subnet-0abc123 subnet-0def456 subnet-0ghi789 \
  --vpc-security-group-ids sg-0proxy123 \
  --require-tls \
  --idle-client-timeout 1800 \
  --debug-logging \
  --region us-east-1

# Register the RDS for SQL Server instance as the proxy target
aws rds register-db-proxy-targets \
  --db-proxy-name prod-sql-proxy \
  --db-instance-identifiers prod-sql-primary \
  --region us-east-1
```

Key configuration choices:

- → **idle-client-timeout**. How long the proxy keeps an idle client connection before closing it (default 1800 seconds). Serverless workloads benefit from shorter timeouts; long-lived application pools can use the default. Setting it too low causes reconnect churn; too high holds proxy capacity for dead clients.

- → **Connection pool size.** The proxy maintains its own pool to the database, sized as a percentage of the database's capacity (`MaxConnectionsPercent`). Start conservative — the proxy multiplexes efficiently, and the most common misconfiguration is sizing the proxy pool as large as the old direct-connection count, which defeats the purpose.
- → **TLS.** `--require-tls` enforces encryption between clients and the proxy. Your client connection strings should already use `Encrypt=True` ; the proxy does not excuse plaintext credentials on the wire.

In Terraform, the equivalent is `aws_db_proxy` with `engine_family = "SQLSERVER"` and an `auth` block pointing at the Secrets Manager secret. Keep the proxy definition in the same Terraform stack as the RDS instance so target registration and security group wiring stay consistent.

10.3 Authentication: Secrets Manager, Not IAM

RDS Proxy does not support IAM database authentication for the SQL Server engine, and that restriction extends through the proxy. SQL Server proxy authentication goes through Secrets Manager, period.

The operational pattern: store the SQL login credentials in Secrets Manager, enable automatic rotation on the secret, and let the proxy pick up rotated credentials without restarts. Rotation without application coordination is the entire point — the application authenticates to the proxy, the proxy authenticates to the database with the current secret value, and neither side needs a deployment when the password changes.

```
# Enable automatic rotation on the database secret (30-day rotation)
aws secretsmanager rotate-secret \
  --secret-id prod/sqlserver/dbadmin \
  --rotation-lambda-arn arn:aws:lambda:us-east-1:123456789012:function:SecretsManagerRDSRotation \
  --rotation-rules AutomaticallyAfterDays=30 \
  --region us-east-1

# Verify the proxy can still authenticate after rotation
aws rds describe-db-proxies \
  --db-proxy-name prod-sql-proxy \
  --query 'DBProxies[0].[Status,EngineFamily]' \
  --output table \
  --region us-east-1
```

Windows (Kerberos/NTLM) authentication is not supported through RDS Proxy — applications connecting via the proxy must use SQL authentication with the Secrets Manager secret. If your application currently relies on Windows integrated authentication, the proxy

migration includes an authentication migration: create dedicated SQL logins for the proxied workloads, scope their permissions, and store them in Secrets Manager. Plan this before cutover, not during.

10.4 Monitoring Proxy Performance and Connection Pinning

The proxy exposes CloudWatch metrics under the `AWS/RDS` namespace with proxy dimensions. The ones that matter:

- → **DatabaseConnections VS ClientConnections**. The ratio is your multiplexing efficiency. Hundreds of client connections on dozens of database connections is healthy. If the two converge, the proxy is not multiplexing — something is pinning connections.
- → **DatabaseConnectionsCurrentlyBorrowed**. How many database connections are checked out right now. Sustained high values against the pool ceiling mean the pool is too small or pinned connections are leaking.
- → **QueryResponseTime / ClientConnectionWaitTime**. Time clients wait for a proxy connection. Spikes here with a healthy database mean pool saturation, not query slowness — do not tune queries for a pooling problem.

Connection pinning is the proxy's fundamental limitation: when a client connection uses a feature the proxy cannot multiplex safely, the proxy pins that client to a dedicated database connection for the session's lifetime. For SQL Server, the classic pinning causes are open transactions held across batches, temporary tables that must persist across batches on the same connection, session-level `SET` options the proxy cannot track, and MARS (Multiple Active Result Sets) sessions. An application that opens a transaction, does application work, then commits — holding the transaction open for seconds — will pin one database connection per client and get almost no multiplexing benefit.

Diagnose pinning by correlating proxy metrics with the database side: query

`sys.dm_exec_sessions` for sessions with `open_transaction_count > 0` and old

`last_request_end_time` — those are the pinned sessions. The fix is almost always in the application: shorten transaction scope, avoid holding transactions across network round trips, and do not use temp tables as a cross-batch communication mechanism through the proxy.

10.5 Failover Behavior: What RDS Proxy Changes

During a Multi-AZ failover without a proxy, every client connection breaks and the application tier generates a reconnect storm against the new primary. With a proxy, the architecture changes: clients hold connections to the proxy, the proxy holds a smaller pool to the database. When failover happens, the proxy detects the topology change, re-establishes its

database connections to the new primary, and client connections experience a pause rather than an error. AWS documents failover time reductions from 60–120 seconds of client-visible errors to a few seconds of elevated latency.

What the proxy does not change: in-flight transactions are still rolled back (the new primary recovered without them), DNS still propagates on its own timeline for direct connections that bypass the proxy, and the application still needs retry logic for the pause window. The proxy converts failover from "every connection errors and every client retries at once" to "connections stall briefly and resume." Test it with the `reboot-db-instance --force-failover` procedure from Chapter 9 and measure client-observed recovery with and without the proxy in the path — that measurement is the business case.

10.6 Sizing and Scaling Considerations

Size the proxy pool from the database's sustainable concurrency, not from the client's connection count. A SQL Server instance that comfortably runs 200 concurrent active sessions does not need a proxy pool of 2,000 — it needs a pool slightly above its proven active concurrency, with the proxy queueing the rest. Oversizing the proxy pool reproduces the exact `THREADPOOL` exhaustion the proxy was meant to prevent.

RDS Proxy scales by adding capacity behind the endpoint; monitor `ClientConnectionWaitTime` as the scaling signal. One proxy per VPC is the common pattern; separate proxies per workload (OLTP vs reporting vs serverless) let you set different idle timeouts and pool sizes and isolate a misbehaving workload's connection churn from everyone else. And remember the proxy is priced per vCPU-hour of proxy capacity plus data transfer — it is not free, so the cost comparison in your business case should include it.

10.7 Limitations and When Not to Use RDS Proxy

- → **No Windows authentication.** Kerberos/NTLM through the proxy is unsupported. Workloads requiring integrated authentication must connect directly.
- → **No IAM database authentication for SQL Server.** Secrets Manager only. If your security standard mandates IAM auth, the proxy cannot deliver it for SQL Server.
- → **Pinning-prone workloads gain little.** Applications with long-held transactions, cross-batch temp tables, or MARS will pin connections and see minimal multiplexing. Fix the application patterns first; the proxy amplifies good connection hygiene, it does not replace it.
- → **Single-AZ proxy endpoints.** The proxy itself is multi-AZ, but understand your application's availability requirements end to end — the proxy is one more component in the path, with its own (small) latency addition per query.

- → **When direct is better.** Steady-state applications with well-tuned ADO.NET pooling, predictable connection counts, and no failover sensitivity often see no benefit from a proxy — just added latency and cost. The proxy earns its keep for serverless, highly elastic, and failover-sensitive workloads.

10.8 Practical Runbook: Diagnosing Connection Pool Exhaustion

Symptom: application errors — timeouts acquiring connections, `THREADPOOL` waits on the server, or proxy `ClientConnectionWaitTime` spikes. Work this sequence:

Step 1: Is the proxy multiplexing or pinned? Compare `ClientConnections` to `DatabaseConnections` in CloudWatch. A collapsing ratio means pinning or pool saturation.

Step 2: Connect directly to the database (bypassing the proxy) and find the sessions. Keep a direct administrative path that does not go through the proxy — you reserved it for exactly this scenario.

```

-- T-SQL: connect directly to the RDS instance (not via proxy)
-- Who is connected and what are they doing?
SELECT s.session_id,
       s.login_name,
       s.host_name,
       s.program_name,
       s.status,
       s.open_transaction_count,
       r.wait_type,
       r.wait_time AS wait_ms,
       DATEDIFF(SECOND, r.start_time, GETDATE()) AS running_seconds,
       SUBSTRING(t.text, 1, 100) AS query_snippet
FROM sys.dm_exec_sessions s
LEFT JOIN sys.dm_exec_requests r ON s.session_id = r.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE s.is_user_process = 1
ORDER BY running_seconds DESC;

-- Count by program and status: find the connection hog
SELECT s.program_name,
       s.status,
       COUNT(*) AS session_count,
       SUM(s.open_transaction_count) AS open_transactions
FROM sys.dm_exec_sessions s
WHERE s.is_user_process = 1
GROUP BY s.program_name, s.status
ORDER BY session_count DESC;

-- Pinned-session suspects: open transactions idle for a long time
SELECT s.session_id, s.login_name, s.program_name,
       s.open_transaction_count,
       DATEDIFF(SECOND, s.last_request_end_time, GETDATE()) AS idle_seconds
FROM sys.dm_exec_sessions s
WHERE s.is_user_process = 1
      AND s.open_transaction_count > 0
      AND s.status = 'sleeping'
      AND DATEDIFF(SECOND, s.last_request_end_time, GETDATE()) > 30
ORDER BY idle_seconds DESC;

```

Step 3: If idle open transactions are the cause, kill the blockers. Preview first, kill second, and fix the application third — killing treats the symptom.

```

-- T-SQL: terminate a runaway session (use sparingly; prefer fixing the app)
-- First identify the session_id from the query above, then:
KILL 87; -- replace 87 with the offending session_id
GO

-- Prevent recurrence: alert on sessions idle with open transactions
-- (add to your monitoring; threshold to your workload's transaction profile)

```

Step 4: Decide whether the pool or the application needs to change. If sessions are legitimately busy (high `running_seconds`, real wait types), the workload exceeds the pool — increase `MaxConnectionsPercent` cautiously or scale the database. If sessions are idle-with-open-transactions or sleeping hordes from a chatty ORM, the application needs connection discipline — the proxy cannot fix a leak, it can only queue it more gracefully.

Key Takeaways

- → **RDS Proxy supports SQL Server — with Secrets Manager auth, not IAM.** The old "not supported" guidance is outdated. Plan for SQL authentication through the proxy; Windows authentication must bypass it.
- → **The proxy's value is multiplexing and failover smoothing, not magic.** It converts thousands of client connections into a small database pool and turns failover reconnect storms into brief pauses. It does not fix bad transaction hygiene.
- → **Watch the multiplexing ratio and pinning.** `ClientConnections` VS `DatabaseConnections` tells you whether the proxy is working. Long-held transactions, cross-batch temp tables, and MARS pin connections — diagnose with `sys.dm_exec_sessions` and fix the application.
- → **Size the pool from database concurrency, not client count.** Oversizing the proxy pool reproduces the `THREADPOOL` exhaustion it was meant to prevent. Scale on `ClientConnectionWaitTime`, isolate workloads with separate proxies.
- → **Keep a direct administrative path.** When the proxy is the problem, you need a way into the database that does not go through it. Reserve direct connections for diagnostics and run the T-SQL runbook above.

Chapter 11: Serverless SQL Server: Azure SQL Serverless and Cost-Efficient Alternatives

Not every database needs to run at full power 24 hours a day. Development databases sit idle overnight. Departmental applications spike for two hours each morning. SaaS products with a handful of customers have databases that do nothing for long stretches. Paying for provisioned compute through all of that idle time is one of the quietest wastes in cloud database spending — and serverless SQL Server is the answer, with caveats that matter.

The honest framing first: there is no serverless RDS for SQL Server, and there is no serverless Managed Instance. True serverless SQL Server exists in exactly one place — Azure SQL Database's serverless compute tier. Everything else in this chapter is about achieving serverless-like economics for the workloads that cannot use it: auto-pause patterns, scheduled start/stop, and elastic pools. If your workload can live on Azure SQL Database, the serverless tier may be the cheapest way to run SQL Server in any cloud.

11.1 Azure SQL Database Serverless: How It Works

Serverless is a compute tier of Azure SQL Database, not a separate product. You configure min and max vCores and an auto-pause delay. When the database is idle for the delay period, compute pauses entirely — you pay only for storage. When a connection arrives, compute resumes automatically. Billing is per vCore-second of compute used, with a minimum billable quantum per resume.

The details that determine whether it fits:

- → **Resume latency.** Waking a paused database takes seconds (typically 30–60 seconds for the first connection, sometimes longer for larger databases). Interactive applications with human users notice; batch jobs and dev workflows do not. Set the auto-pause delay to match the workload's real idle pattern — pausing a database that wakes every 20 minutes costs more in resume overhead than it saves.
- → **Min vCores floor.** The minimum vCore setting (as low as 0.5) bounds how far compute scales down before pausing. A database that is lightly but constantly busy never pauses and bills like a small provisioned database — serverless only wins when there are genuine idle periods.
- → **Feature parity.** Serverless supports the same Azure SQL Database feature set (with the SQL Database limitations from Chapter 7 — no Agent, no linked servers). It is not a back door to full SQL Server compatibility.

- → **Geo-replication limits.** Historically, serverless databases had restrictions on active geo-replication. Verify current support before designing DR around a serverless primary — the safe pattern is serverless for dev/test and lower environments, provisioned for production DR topologies.

```
# Create a serverless database: 0.5–4 vCores, auto-pause after 60 min idle
az sql db create \
  --resource-group rg-dev \
  --server sqlsrv-dev \
  --name appdb-dev \
  --edition GeneralPurpose \
  --family Gen5 \
  --min-capacity 0.5 \
  --capacity 4 \
  --compute-model Serverless \
  --auto-pause-delay 60

# Convert an existing provisioned database to serverless
az sql db update \
  --resource-group rg-dev \
  --server sqlsrv-dev \
  --name appdb-dev \
  --compute-model Serverless \
  --min-capacity 0.5 \
  --auto-pause-delay 120
```

11.2 The Workloads Serverless Fits (and the Ones It Does Not)

Ideal fits: development and QA databases (idle 14+ hours a day — the classic 70% cost reduction), departmental applications with business-hours usage, batch-oriented workloads that run on schedules, SaaS databases for small tenants with sporadic usage, and any database where you can tolerate a cold-start delay on first connection.

Poor fits: latency-sensitive OLTP with steady traffic (never pauses, bills like provisioned, adds resume risk), databases behind user-facing applications where a 30-second cold start is a support ticket, workloads needing SQL Agent jobs on a schedule (use elastic jobs or provisioned), and anything requiring features outside Azure SQL Database's compatibility surface.

The decision rule: look at a week of CPU utilization. If the database is below 5% utilization for more than half the week, serverless almost certainly wins. If it never drops below 20%, provisioned is cheaper and more predictable. Between those, model both — the Azure pricing calculator has a serverless mode, but validate with a month of real billing, because resume frequency dominates the math.

11.3 Elastic Pools: Serverless Economics for Many Databases

When you have dozens or hundreds of small databases — the multi-tenant SaaS pattern from Chapter 14 — serverless per database becomes operationally noisy. Elastic pools are the alternative: a shared pool of vCores (or eDTUs) across many databases, where each database bursts against the pool and you pay for the pool's aggregate capacity rather than each database's peak. The economics work when peaks do not coincide: 100 databases that each peak at 2 vCores but average 0.1 vCores fit comfortably in a 20-vCore pool.

Pools and serverless solve different problems. Serverless optimizes one database's idle time. Pools optimize many databases' uncorrelated peaks. For the SaaS scenario with hundreds of tenant databases, pools are usually the answer; for the single dev database that sleeps all night, serverless is.

11.4 Serverless-Like Economics on AWS: Scheduled Start/Stop

RDS for SQL Server has no auto-pause. But an RDS instance that does not need to run overnight can simply be stopped — you pay for storage while stopped, not compute. For dev, QA, and training environments on RDS, scheduled stop/start is the serverless equivalent, and the savings are comparable: stopping nights and weekends cuts compute cost by roughly two-thirds.

```
# Stop a dev RDS instance (no compute charges while stopped)
aws rds stop-db-instance \
  --db-instance-identifier dev-sql-01 \
  --region us-east-1

# Start it again in the morning
aws rds start-db-instance \
  --db-instance-identifier dev-sql-01 \
  --region us-east-1
```

Automate it with EventBridge Scheduler: a cron rule that stops dev instances at 7 PM and starts them at 7 AM on weekdays, with a tag-based scope (`Schedule=dev-hours`) so production is never in scope. Two caveats: RDS instances stopped for more than 7 days are automatically restarted by AWS (a guardrail against forgotten instances, which also breaks the "stopped all month" plan — handle it in the automation), and stopping does not pause the 7-day backup retention clock semantics the way you might expect — verify restore windows for long-stopped instances.

For EC2-based SQL Server, the same pattern uses EC2 stop/start (even cheaper, since you can also drop to smaller volumes or snapshot-and-terminate for long idle periods). An EC2 dev instance stopped nights and weekends, with its EBS volumes snapshotted monthly, costs a fraction of a 24/7 RDS instance.

11.5 Containers and Ephemeral SQL Server

The most serverless-adjacent pattern for SQL Server on AWS is containers: SQL Server Linux container images running on ECS Fargate or EKS, started on demand for batch jobs, integration tests, or ephemeral review environments, and terminated when done. You pay per-second for the task runtime plus storage. This does not replace a persistent database — it replaces the persistent dev/test databases that sit idle between CI runs. A CI pipeline that spins up a SQL Server container, runs migrations and tests, and tears it down uses minutes of compute per run instead of a 24/7 instance.

Limits are real: containers on Fargate have no persistent local storage guarantees across restarts (use EFS or accept ephemerality), Windows containers remain second-class on most orchestrators (use the Linux SQL Server images), and production OLTP on Fargate is not a serious architecture. But for ephemeral workloads, it is the closest thing to serverless SQL Server outside Azure.

11.6 Cold-Start Mitigation Patterns

When the workload almost fits serverless but the resume latency hurts, you have options short of going provisioned:

- → **Keep-warm queries.** A scheduled Azure Function or Logic App that opens a connection and runs `SELECT 1` every 30 minutes during business hours prevents auto-pause when users are active, while still allowing overnight pauses. You are explicitly buying responsiveness during the hours that matter.
- → **Lengthen the auto-pause delay.** A 6-hour delay instead of 1 hour means the database survives lunch and meeting gaps without pausing. The cost of a few extra billed hours is often less than the support tickets from mid-day cold starts.
- → **Warm the connection in the application.** If the application knows a user session is starting (login page rendered, report requested), open the database connection eagerly in the background rather than waiting for the first query. The resume happens during page render, not during the query.
- → **Split the workload.** The user-facing OLTP database stays provisioned; the reporting, ETL staging, and dev copies go serverless. Cold starts on a staging database that feeds a morning report nobody watches live are free money.

11.7 Monitoring and Billing Guardrails

Serverless billing is variable by design, which means it needs guardrails:

- → **Alert on resume frequency.** A serverless database resuming 50 times a day is a misconfigured workload — either the pause delay is too short or the workload is steadier than assumed. Track resumes via Azure Monitor and convert chronic resumers to provisioned.
- → **Alert on vCore-hour spend.** Set a budget alert at the expected monthly cost; a runaway process holding a serverless database awake for a month can exceed the provisioned price. The failure mode of serverless is not downtime — it is a surprise bill.
- → **Monitor pause/resume latency as an SLI.** If you promise users responsiveness, measure resume time from the client's perspective and treat regressions like any other latency regression.
- → **Tag and review quarterly.** Workloads change: the dev database that slept all night gets a nightly ETL job, and suddenly it never pauses. A quarterly review of serverless candidates against actual utilization keeps the fleet honest.

11.8 Choosing Your Idle-Time Strategy

- → **On Azure SQL Database and the workload tolerates cold starts:** serverless tier. Simplest, true per-second billing, zero automation to maintain.
- → **Many small databases with uncorrelated peaks:** elastic pools, on either provisioned or (where supported) serverless pool configurations.
- → **RDS dev/test with business-hours usage:** EventBridge-scheduled stop/start. Two cron rules replace a serverless tier.
- → **Ephemeral CI/test databases:** SQL Server containers on Fargate/EKS, created and destroyed per run.
- → **Steady production OLTP:** none of the above — provisioned compute, right-sized (Chapter 39), possibly reserved (Chapter 38).

Key Takeaways

- → **True serverless SQL Server exists in one place: Azure SQL Database's serverless tier.** Per-vCore-second billing with auto-pause is the cheapest way to run idle-heavy SQL Server workloads — if the workload fits SQL Database's compatibility surface and tolerates resume latency.
- → **Serverless wins on idle time, not on size.** A database below 5% utilization for half the week is the ideal candidate. Steady workloads should stay provisioned — serverless adds resume risk without saving money.

- → **On AWS, scheduled stop/start is the serverless equivalent for RDS.** EventBridge-driven nights-and-weekends shutdown cuts dev/test compute cost by two-thirds. Watch the 7-day auto-restart behavior.
- → **Elastic pools solve the many-databases problem; containers solve the ephemeral problem.** Match the tool to the idle pattern: correlated single-database idleness → serverless; uncorrelated multi-database peaks → pools; per-run ephemerality → containers.
- → **Production OLTP stays provisioned.** None of the idle-time strategies apply to steady production workloads — right-size them and consider reservations instead (Chapters 38–39).

Chapter 12: SQL Server Agent, Jobs, and Automation in the Cloud

Ask a SQL Server DBA what runs the business at night and the answer is SQL Server Agent: maintenance plans, ETL schedules, index rebuilds, backup jobs, alerting, and the hundred small automations accumulated over a decade. Agent is also one of the first things that breaks — or vanishes — in a cloud migration. RDS restricts it, Azure SQL Database removes it, and Managed Instance keeps it but changes the operational context. This chapter maps the automation surface on each platform and shows how to rebuild your job estate wherever you land.

12.1 What SQL Server Agent Does for You Today

Before replacing anything, inventory what you have. Agent typically carries five workload types, each with different migration needs:

- → **Database maintenance:** integrity checks, index maintenance, statistics updates, backup jobs. These translate directly to any platform — the schedule and the T-SQL survive, only the host changes.
- → **ETL and data movement:** SSIS package executions, bulk loads, cross-server data syncs. These depend on components (SSIS, linked servers, file shares) that may not exist on the target — the highest-risk category.
- → **Business logic on a schedule:** nightly aggregations, billing runs, archiving, notification generation. The T-SQL usually ports cleanly; the scheduling mechanism changes.
- → **Monitoring and alerting:** jobs that check replication lag, disk space, failed logins, and long-running queries. These get rebuilt on cloud-native monitoring (CloudWatch alarms, Azure Monitor alerts) rather than ported.
- → **Operational glue:** jobs calling `xp_cmdshell`, PowerShell steps, SSIS-adjacent scripting. These almost never survive migration unchanged — each one needs a redesign decision.

```
-- Inventory every Agent job: name, owner, schedule, and step types
-- Step types tell you what will survive migration and what will not
SELECT j.name AS job_name,
       j.enabled,
       sc.name AS schedule_name,
       STUFF((
           SELECT ', ' + s.subsystem
           FROM msdb.dbo.sysjobsteps s
           WHERE s.job_id = j.job_id
           GROUP BY s.subsystem
           FOR XML PATH('')
       ), 1, 2, '') AS step_subsystems,
       j.date_created
FROM msdb.dbo.sysjobs j
LEFT JOIN msdb.dbo.sysjobschedules js ON j.job_id = js.job_id
LEFT JOIN msdb.dbo.sysschedules sc ON js.schedule_id = sc.schedule_id
ORDER BY j.name;
```

The `step_subsystems` column is your triage key: `TSQL` steps port almost anywhere; `SSIS`, `CmdExec`, and `PowerShell` steps need platform-specific rebuilds.

12.2 SQL Server Agent on RDS: Available but Restricted

RDS for SQL Server includes SQL Server Agent, and for T-SQL maintenance jobs it works the way you expect: create jobs, schedules, and alerts through SSMS or T-SQL against `msdb`. The restrictions are at the edges:

- → **No CmdExec or PowerShell steps that touch the OS.** There is no OS to touch — no file system access, no `xp_cmdshell`, no local scripts. Jobs that shell out must be reimplemented (see section 12.5).
- → **Replication jobs are limited.** RDS supports SQL Server replication in specific configurations only; Agent jobs tied to replication need verification against the RDS-supported topology.
- → **Multi-AZ failover and msdb.** Agent jobs live in `msdb`, which is a system database — it fails over with the instance on RDS Multi-AZ, so jobs survive failover. But job *history* and in-flight executions do not transfer; design jobs to be re-runnable and idempotent.
- → **Alerting integration.** Agent alerts (severity-based, performance-condition-based) work, but route notifications through Database Mail to SNS or your mail system — the on-premises operator-to-pager chain does not exist.

Practical rule: if your Agent estate is 90% T-SQL maintenance and business logic, RDS Agent carries it with minor changes. If it is heavy on `CmdExec`, `PowerShell`, and `SSIS`, you are rebuilding the automation layer regardless of the database platform — start that design early.

12.3 No Agent on Azure SQL Database: Elastic Jobs and Alternatives

Azure SQL Database has no SQL Server Agent. The native replacement is **elastic jobs**: an Azure-hosted job service (backed by a dedicated job database) that executes T-SQL across one or many databases on schedules you define. Conceptually it is Agent's T-SQL scheduling extracted into a service:

- → **What transfers:** T-SQL steps, schedules, multi-database targeting (run the same index maintenance across 200 tenant databases — elastic jobs are genuinely better at this than Agent ever was).
- → **What does not:** SSIS steps, CmdExec, PowerShell, replication agents, and anything requiring server scope. Elastic jobs run T-SQL against databases, nothing else.
- → **Operational differences:** job history lives in the job database's tables (query it like any data), credentials use database-scoped credentials, and private connectivity requires the job agent's VNet integration for databases behind private endpoints.

```
# Create the Elastic Job agent (one per logical server region topology)
az sql elastic-job-agent create \
  --resource-group rg-prod \
  --server sqlsrv-prod \
  --name jobagent-prod

-- T-SQL: define a job that runs index maintenance across all tenant databases
-- (run in the job database)
EXEC jobs.sp_add_job @job_name = 'NightlyIndexMaintenance';
EXEC jobs.sp_add_jobstep
  @job_name = 'NightlyIndexMaintenance',
  @command = N'EXEC dbo.usp_AdaptiveIndexMaintenance;',
  @credential_name = 'job_credential',
  @target_group_name = 'TenantDatabases';
EXEC jobs.sp_add_jobschedule
  @job_name = 'NightlyIndexMaintenance',
  @enabled = 1,
  @schedule_interval_type = 'Days',
  @schedule_interval_count = 1,
  @schedule_start_time = '2024-01-01 02:00:00';
```

For non-T-SQL automation on Azure, the surrounding platform takes over: Azure Automation runbooks (PowerShell/Python with schedules and webhooks), Logic Apps (event-driven workflows with 400+ connectors), and Azure Data Factory (the SSIS replacement — including the SSIS integration runtime for lift-and-shift of existing packages). The pattern is consistent: the database runs queries; the platform runs everything else.

12.4 SQL Server Agent on Managed Instance: The Familiar Path

Managed Instance runs a full SQL Server Agent — jobs, schedules, alerts, operators, proxies, T-SQL steps, SSIS steps (via the SSIS IR integration), and CmdExec/PowerShell steps within the instance's sandbox. For lift-and-shift migrations, this is the path of least resistance: script your jobs from `msdb` on-premises and recreate them on MI. The differences from on-premises Agent are operational, not functional: no OS access (so CmdExec steps cannot touch a file system — they run in a constrained context), Database Mail routes through Azure's mail infrastructure, and job history retention follows the instance's storage.

One MI-specific consideration: failover groups. Agent jobs live in `msdb` on each instance — they are not replicated to the secondary. After a failover, the secondary has its own copy of the jobs only if you deployed them there. The standard pattern is to deploy identical jobs to both instances and gate execution: jobs check `sys.dm_hadr_database_replica_states` or the failover group's role and run only on the primary. Skip this and your maintenance jobs either run twice or not at all after a failover.

```
-- MI failover-group-safe job step: only run on the primary replica
-- Add as the first step of every scheduled job in a failover group topology
IF EXISTS (
    SELECT 1
    FROM sys.dm_hadr_database_replica_states rs
    JOIN sys.databases d ON rs.database_id = d.database_id
    WHERE d.name = 'appdb'
        AND rs.is_local = 1
        AND rs.role_desc <> 'PRIMARY'
)
BEGIN
    RAISERROR('Not the primary replica – skipping job execution.', 16, 1);
    RETURN;
END
-- ... rest of the job step follows
```

12.5 Rebuilding OS-Dependent Automation

Every estate has jobs that touch the operating system: copy a backup file, run a PowerShell ETL step, call a web service, manipulate files on a share. None of the managed platforms give you an OS. The rebuild patterns:

- → **File operations → object storage + platform services.** A job that copies backups to a file share becomes a job that writes to S3 (RDS native backup to S3) or Azure Blob Storage, with lifecycle policies replacing the cleanup script. A job that bulk-loads from a file becomes `BULK INSERT` from cloud storage or an ADF/S3-triggered pipeline.

- → **PowerShell/CmdExec steps → Automation runbooks or Lambda.** On AWS, a Lambda function on a schedule (EventBridge) replaces the PowerShell step — it has IAM permissions, VPC access, and logging built in. On Azure, an Automation runbook with a Hybrid Worker or managed identity does the same. The logic ports; the host changes.
- → **SSIS packages → ADF or EC2.** Existing packages run in Azure Data Factory's SSIS integration runtime (lift-and-shift) or on an EC2 instance with SSIS installed (AWS). New ETL should be evaluated against cloud-native orchestration (ADF pipelines, Step Functions + Lambda/Glue) rather than reflexively rehosting SSIS.
- → **Custom alerting scripts → native monitoring.** The Agent job that emails when disk space is low becomes a CloudWatch alarm or Azure Monitor alert. Do not port monitoring scripts — replace them with the platform's monitoring, which is more reliable than a job on the thing being monitored.

12.6 Maintenance Plans in the Cloud

Maintenance plans deserve explicit attention because every SQL Server estate has them and the cloud changes the calculus. The classic plan — nightly full backup, integrity check, index rebuild, statistics update — needs rethinking:

- → **Backups:** on RDS, automated backups replace most Agent backup jobs (keep Agent jobs only for extra copies or special handling). On Azure SQL, platform backups replace them entirely. On EC2/MI, Agent backup jobs remain — writing to S3/Blob storage.
- → **Integrity checks:** keep them everywhere. `DBCC CHECKDB` on a schedule is non-negotiable on every platform — managed storage can corrupt too, and early detection is the difference between a restore and a resume event.
- → **Index maintenance:** replace blanket rebuilds with intelligent maintenance (Ola Hallengren's scripts or equivalent) that rebuilds only fragmented indexes above a threshold. On cloud storage with provisioned IOPS, a nightly rebuild of every index is both a waste of IOPS and a source of log generation that can hit tier throughput caps.
- → **Statistics:** auto-update statistics handles most cases; add manual updates only for the tables where the optimizer demonstrably needs them (large tables with skewed distributions, ascending keys).

12.7 The Migration Sequence

Migrate the automation estate in this order: first, inventory and triage with the query from section 12.1. Second, migrate pure T-SQL maintenance jobs — they are the easiest win and de-risk the schedule. Third, rebuild monitoring and alerting on native platform tools rather than porting scripts. Fourth, tackle ETL and data movement (the long pole — SSIS/ADF design takes real time). Fifth, handle the operational glue last, replacing each OS-dependent

step with its platform equivalent. Throughout, keep the on-premises Agent running in parallel until the cloud automation has executed successfully on its own schedule at least twice — "it ran once" is not proven.

Key Takeaways

- → **Inventory before you migrate.** Query `msdb` for every job, owner, schedule, and step subsystem. T-SQL steps port easily; SSIS, CmdExec, and PowerShell steps each need a redesign decision.
- → **Each platform has a different automation story.** RDS: Agent with T-SQL, no OS access. Azure SQL Database: no Agent — elastic jobs for T-SQL scheduling, Azure Automation/Logic Apps/ADF for everything else. Managed Instance: full Agent, the lift-and-shift path, with failover-group gating for jobs.
- → **Do not port monitoring scripts or OS glue — replace them.** CloudWatch alarms, Azure Monitor alerts, Lambda, and Automation runbooks are more reliable than jobs running on the thing they monitor.
- → **Make jobs idempotent and failover-aware.** Jobs must survive being interrupted by failover and must not double-execute in failover-group topologies. Gate on replica role; design for re-runnability.
- → **Rethink maintenance plans for cloud storage.** Intelligent, threshold-based index maintenance instead of blanket rebuilds; platform backups where they exist; `DBCC CHECKDB` on a schedule everywhere, without exception.

Chapter 13: SQL Server on Azure VMs: Self-Managed on Azure

Chapter 8 built SQL Server on EC2. This chapter is its Azure twin: SQL Server on Azure Virtual Machines. The motivations rhyme — full engine control, readable secondaries, SSIS/SSAS/SSRS, consolidation — but the implementation is Azure-specific in ways that matter: the load balancer in front of the availability group listener, the cloud witness for quorum, Ultra Disk and caching policies for storage, and Azure Hybrid Benefit applied to VMs. If your organization is Microsoft-centric but your workload needs self-managed SQL Server, this is the chapter.

13.1 When Azure VMs Beat Azure SQL

The decision tree from Chapter 7 points here when: you need the full SQL Server surface (replication topologies Azure SQL does not support, specific CLR or linked-server configurations, PolyBase scale-out); you are consolidating many instances or databases per VM for licensing efficiency; you need a SQL Server version or CU that the PaaS offerings do not offer; or your operational model already manages Windows Server fleets and the team prefers IaaS consistency over PaaS abstraction. Azure VMs are also the natural home for SSIS, SSAS, and SSRS in a Microsoft-centric estate — the same components that pushed EC2 in Chapter 8.

The cost of IaaS is the same on both clouds: you own patching, availability, backup verification, and monitoring. What changes is the tooling around you — and several Azure features exist specifically to make SQL Server on VMs less painful than raw IaaS.

13.2 Licensing: Azure Hybrid Benefit on VMs

SQL Server on Azure VMs has the same two licensing paths as EC2, with Azure's twist.

License-included images come from the Azure Marketplace with SQL Server pre-installed (per-minute billing that folds in the license). **Azure Hybrid Benefit (AHB)** lets you bring existing licenses with Software Assurance and pay only the VM compute rate — the same benefit from Chapter 7, applied to IaaS. For steady-state production VMs, AHB is usually the cheapest compliant path, and unlike AWS License Mobility, it is a first-class Azure billing toggle, not a paperwork exercise.

The licensing lever unique to Azure VMs is **constrained vCPUs**: several VM sizes can be deployed with fewer active vCPUs than the hardware provides, reducing the SQL Server core-license footprint without sacrificing memory or I/O bandwidth. A VM with 64 vCPUs constrained to 16 active vCPUs needs 16 core licenses instead of 64 — while keeping the

memory and disk throughput of the larger size. For Enterprise Edition licensing, this single feature can change the VM selection entirely: choose the size for memory and throughput, constrain vCPUs for licensing.

```
# Deploy a SQL Server VM from a Marketplace image with AHB applied
az vm create \
  --resource-group rg-prod \
  --name sqlvm-prod-01 \
  --image MicrosoftSQLServer:sql2022-ws2022:enterprise:latest \
  --size Standard_E16s_v5 \
  --vcpus-available 4 \
  --vcpus-per-core 2 \
  --license-type Windows_Server \
  --vnet-name vnet-prod \
  --subnet snet-sql \
  --nsg sg-sql \
  --admin-username sqladmin

# Apply Azure Hybrid Benefit for SQL Server to an existing VM
az sql vm update \
  --resource-group rg-prod \
  --name sqlvm-prod-01 \
  --license-type AHUB
```

13.3 VM Sizing: E, D, and M Series

The workhorse series for SQL Server on Azure are **E-series** (memory-optimized, the default choice for OLTP), **D-series** (balanced compute), and **M-series** (very large memory for the biggest workloads). Two Azure-specific sizing notes: first, check the uncached disk throughput limits per VM size — your storage design cannot exceed what the VM can push, and the limit is per-VM, not per-disk. Second, Premium SSD v2 and Ultra Disk change the throughput math versus classic Premium SSD; Ultra Disk offers sub-millisecond latency and independently configurable IOPS/throughput, making it the choice for log volumes on demanding workloads.

13.4 Storage: Caching Policies and the Temp Drive

Azure storage caching is the setting EC2 does not have, and getting it wrong costs performance silently. The rules:

- → **Data files: ReadOnly caching.** The local SSD cache in the VM accelerates reads; writes bypass the cache to Premium storage. This is the recommended and highest-performing setting for data volumes.

- → **Log files: no caching (None).** Write-through latency matters more than read caching for the log, and cached writes risk data loss semantics. Set caching to None on log volumes — this is the most commonly misconfigured setting.
- → **Tempdb: the D: drive.** Azure VMs have an ephemeral temp drive (D: on Windows) backed by local SSD — fast, free, and wiped on VM move/redeploy. Point tempdb at D: for maximum tempdb throughput at zero storage cost. The trade-off is ephemerality: tempdb is recreated on SQL Server start anyway, so this is safe — but anything else you put on D: is not.

Stripe data volumes across multiple Premium/Ultra disks when a single disk's IOPS or throughput ceiling binds you, and use 64 KB allocation units exactly as on EC2. For the highest-end workloads, Ultra Disk with sub-millisecond latency replaces the "local NVMe" conversation from Chapter 8 — with the advantage of persistence that instance-store volumes lack.

13.5 Availability Groups on Azure VMs: The Load Balancer Listener

Always On AGs on Azure VMs work like AGs on EC2 — multi-subnet listeners, quorum planning — with two Azure-specific adaptations. First, the **AG listener goes behind an Azure Load Balancer** (internal load balancer for VNet-internal listeners). The load balancer holds the listener IP as a frontend, with a health probe (conventionally TCP port 59999) that each replica answers only when it is the primary. Enable **floating IP (direct server return)** on the load balancing rule so the primary responds directly to clients. Without the load balancer, cross-subnet listener failover does not work on Azure — this is the architectural piece with no EC2 equivalent, and the most common source of "AG works, listener doesn't" incidents.

Second, **quorum uses a cloud witness**: a blob storage endpoint that holds the quorum vote, replacing the file-share witness. No third VM to manage, no SMB share to maintain — a storage account and a configuration step. For two-replica AGs, the cloud witness is the deciding vote; for three-replica AGs across three zones, it is resilience depth.

```
# Create the internal load balancer for the AG listener
az network lb create \
  --resource-group rg-prod \
  --name lb-aglistener \
  --sku Standard \
  --vnet-name vnet-prod \
  --subnet snet-sql \
  --private-ip-address 10.0.1.50 \
  --frontend-ip-name fe-aglistener

# Health probe: replicas answer only when primary
az network lb probe create \
  --resource-group rg-prod \
  --lb-name lb-aglistener \
  --name probe-ag \
  --protocol Tcp \
  --port 59999 \
  --interval 5 \
  --threshold 2

# Load balancing rule with floating IP enabled
az network lb rule create \
  --resource-group rg-prod \
  --lb-name lb-aglistener \
  --name rule-aglistener \
  --protocol Tcp \
  --frontend-port 1433 \
  --backend-port 1433 \
  --frontend-ip-name fe-aglistener \
  --backend-pool-name be-agreplicas \
  --probe-name probe-ag \
  --floating-ip true \
  --idle-timeout 30
```

On each replica, the probe port is served by a lightweight mechanism that returns success only on the primary — commonly a PowerShell-driven firewall rule or the cluster's probe response. Test listener failover explicitly: the AG can fail over cleanly while the load balancer still sends traffic to the old primary if the probe configuration is wrong.

13.6 Patching: Automated Patching and Maintenance Configurations

Azure offers **automated patching** for SQL Server VMs via the SQL IaaS Agent extension: Windows and SQL Server updates install on a schedule you define (e.g., monthly, third Tuesday, 2 AM), with pre/post-patch scripting hooks. For availability groups, enable health-check-driven patching so the platform patches secondaries first and fails over gracefully — the same secondaries-first discipline as Chapter 8, automated. **Maintenance configurations**

give you control over when Azure's own host maintenance (not SQL Server patching) can reboot the VM — pin it to your maintenance window so host updates never surprise a production workload.

13.7 Backup: Automated Backup to Blob Storage

The SQL IaaS Agent extension provides **automated backup**: full and log backups on a schedule you configure, written directly to Azure Blob storage with configurable retention (point-in-time restore within the retention window). This is the closest Azure IaaS gets to RDS automated backups — and unlike RDS, the backup files are native `.bak` files in your storage account, restorable anywhere. Combine with blob immutability policies (time-based retention / legal hold) for ransomware-resistant backups, and geo-redundant storage for the cross-region copy.

13.8 Monitoring and the SQL IaaS Extension

Register every SQL Server VM with the **SQL IaaS Agent extension** (lightweight mode at minimum). Registration unlocks automated patching, automated backup, and Azure Monitor integration — including SQL-specific metrics and the Azure portal's SQL VM management blade. Unregistered VMs are just Windows VMs that happen to run SQL Server; registered VMs get the manageability layer that narrows the gap with PaaS. For deep diagnostics, the same stack as EC2 applies: PerfMon counters to Azure Monitor, Extended Events, Query Store, and the AG health DMVs from Chapter 8.

13.9 Cost Comparison: Azure VM vs the Alternatives

With AHB, a SQL Server VM's cost is the VM compute plus storage — often substantially less than Managed Instance's vCore-hour rate for the same capacity, and less than RDS License Included. The VM wins on raw infrastructure cost and loses on operational cost: your team owns everything Chapter 8 described. The honest comparison, as always, includes engineering time. For consolidated estates (many databases per VM) with AHB, Azure VMs are frequently the cheapest compliant way to run SQL Server in the cloud — which is exactly why the decision deserves the full analysis rather than a PaaS default.

Key Takeaways

- → **Azure VMs are the self-managed answer for Microsoft-centric estates.** Full engine control, readable AG secondaries, SSIS/SSAS/SSRS, and consolidation — with Azure-specific implementation for networking, quorum, and storage.

- → **Licensing is the lever: AHB plus constrained vCPUs.** Apply Azure Hybrid Benefit to pay compute-only, and constrain vCPUs to size the license footprint independently of memory and throughput. Get the licensing answer in writing before building.
- → **Storage caching policies are load-bearing.** Data volumes ReadOnly, log volumes None, tempdb on the ephemeral D: drive. Ultra Disk for sub-millisecond log latency on demanding workloads.
- → **AG listeners go behind an internal load balancer with floating IP.** The health probe answers only on the primary. Cloud witness replaces the file-share witness for quorum. Test listener failover explicitly — AG failover and listener failover are two different things.
- → **Register every VM with the SQL IaaS Agent extension.** Automated patching, automated backup to Blob storage, and Azure Monitor integration narrow the IaaS manageability gap. Unregistered VMs are just Windows VMs.

Chapter 14: Elastic Pools: Multi-Tenant Cost Optimization

The SaaS problem: you have 300 customers, each with their own database. Each database averages 2% CPU utilization but peaks at 60% — at different times. Provisioned individually, you pay for 300 peaks. In an elastic pool, you pay for the aggregate: a shared pool of vCores (or eDTUs) that databases draw from as needed. When peaks do not coincide, the pool costs a fraction of the sum of its databases. This chapter covers how elastic pools work, how to size them, how to keep tenants from hurting each other, and the operational patterns for running hundreds of databases as a fleet.

14.1 How Elastic Pools Work

An elastic pool is a pool of compute and storage resources shared by a set of single databases on a logical server. You set the pool's total vCores (or eDTUs), per-database min/max limits, and storage cap. Each database consumes what it needs moment to moment, up to its max; the pool's total is the only thing you pay for. The key insight: you are buying the *sum of average utilization plus headroom for coincident peaks*, not the sum of peaks.

Two purchasing models: **vCore-based** (the modern model — provisioned compute with per-database min/max vCores, available in General Purpose and Business Critical tiers) and **DTU-based** (the legacy bundled model — eDTUs combining CPU, memory, and I/O). New pools should use vCore; DTU pools exist for legacy estates. Per-database settings matter enormously: `min vCores per database` guarantees each tenant a floor (set it above zero for databases that must always be responsive), and `max vCores per database` caps any single tenant from consuming the pool.

```
# Create an elastic pool: 50 vCores shared, per-DB guardrails
az sql elastic-pool create \
  --resource-group rg-saas \
  --server sqlsrv-saas \
  --name pool-tenants \
  --edition GeneralPurpose \
  --family Gen5 \
  --capacity 50 \
  --db-dtu-max 5 \
  --db-dtu-min 0 \
  --storage 500

# Move a database into the pool
az sql db update \
  --resource-group rg-saas \
  --server sqlsrv-saas \
  --name tenant_042 \
  --elastic-pool pool-tenants
```

14.2 Sizing: The Math That Makes Pools Win

Size from measurements, not guesses. For each candidate database, capture a week of CPU, data I/O, and log I/O utilization (Azure Monitor metrics per database, or Query Store wait statistics). The pool size formula:

- → **Sum the averages** across all databases — this is the pool's baseline load.
- → **Add headroom for coincident peaks.** If peaks are uncorrelated, 30–50% headroom over the summed average is typical. If tenants share usage patterns (all B2B customers active 9–5 in one timezone), peaks correlate — size closer to the sum of peaks and reconsider whether pooling fits.
- → **Set per-database max to bound blast radius.** A common starting point: max vCores per database at 25–50% of pool vCores. One tenant's runaway query should degrade that tenant, not exhaust the pool.

Example: 200 tenant databases averaging 0.25 vCores each (50 vCores of average load), peaks uncorrelated. A 75-vCore pool (50% headroom) replaces 200 individually-provisioned databases — typically a 60–80% cost reduction. The break-even analysis against 200 serverless databases (Chapter 11) depends on idle patterns: pools win when databases are lightly busy around the clock; serverless wins when they are truly idle for long stretches.

14.3 The Noisy Neighbor Problem

Pooling's risk is resource contention: one tenant's workload starves the others. The defenses are layered:

- → **Per-database max caps** (section 14.2) — the first and most important control.
- → **Resource governance alerts.** Alert on pool-level CPU, data I/O, and log I/O utilization, and on per-database utilization approaching its max. The alert should identify the offending database — pool-level metrics alone just tell you the pool is hot.
- → **Query Store per database.** When a tenant degrades, Query Store on that database identifies the regressed query in minutes. This is the diagnostic that makes pools operable at scale.
- → **Tenant isolation tiers.** Not all tenants belong in the pool: your largest tenants get dedicated provisioned databases (or their own pool), the long tail shares the pool. Revisit placement quarterly as tenants grow — a tenant that outgrows the pool's per-database max needs promotion, not a bigger cap.

14.4 Multi-Tenant Schema Patterns

Elastic pools assume the **database-per-tenant** pattern. Two alternatives worth knowing: **schema-per-tenant** (one database, one schema per tenant — cheaper, but no per-tenant resource isolation and harder per-tenant restore) and **shared schema with tenantId** (one database, row-level security — cheapest, weakest isolation). Database-per-tenant in a pool is the sweet spot for most SaaS: per-tenant backup/restore, per-tenant Query Store diagnostics, per-tenant max caps, and straightforward tenant onboarding/offboarding (add/remove database from the pool). Use elastic queries or a catalog database for cross-tenant reporting — cross-database queries from the pool work through elastic query, with the performance characteristics of remote queries.

14.5 Fleet Operations: Managing Hundreds of Databases

At pool scale, you do not manage databases — you manage the fleet:

- → **Schema deployment:** elastic jobs (Chapter 12) execute migrations across all pool databases from a single job definition. Version-gate the rollout: canary tenants first, then waves.
- → **Monitoring:** Azure Monitor at pool scope plus per-database Query Store. Build a fleet dashboard: pool utilization, top databases by CPU/I/O, databases hitting per-database max, and tenant growth trends.
- → **Onboarding/offboarding:** automate database creation from a template (bacpac or copy), pool assignment, elastic job enrollment, and monitoring registration. Tenant provisioning should be an API call, not a ticket.
- → **Backup and PITR per tenant:** Azure SQL backs up each database independently — per-tenant point-in-time restore works even inside a pool. Verify restore drills cover a single-tenant restore, not just pool-level scenarios.

14.6 Disaster Recovery for Pools

Pools participate in the same DR mechanisms as single databases, with fleet-scale considerations. **Failover groups** replicate entire pools: add the pool to a failover group and every database in it geo-replicates to the secondary region, failing over together behind one listener endpoint. This is the right model when tenants share a region-failure fate — which, for most SaaS, they do. The secondary pool must be sized to absorb the primary's load; an under-provisioned secondary turns a regional failover into a pool-wide brownout.

For per-tenant recovery, **geo-restore** works per database: restore any single tenant's database from geo-redundant backup storage to any region, without touching the rest of the pool. This covers the "one tenant needs yesterday's data" scenario that failover groups cannot address. Drill both: a full-pool failover annually, a single-tenant geo-restore quarterly. And remember the RPO/RTO asymmetry — geo-restore RPO is up to an hour (backup replication lag), while active geo-replication RPO is seconds. Choose per scenario, not per habit.

14.7 Moving Databases Between Pools and Tiers

Pool membership is not permanent. The lifecycle operations you will run regularly:

- → **Promoting a tenant out.** When a database consistently hits its per-database max, move it to a dedicated provisioned database (or Hyperscale). The move is online; update the tenant's connection string and remove it from the pool's elastic job targets.
- → **Splitting a hot pool.** When pool-level utilization stays above 80%, split by tenant cohort (e.g., by region, plan tier, or signup cohort) into two pools rather than just adding vCores. Smaller pools have smaller blast radii and clearer noisy-neighbor attribution.
- → **Tier changes.** Moving a pool from General Purpose to Business Critical (for the readable secondary or lower latency) is an online operation but changes the price significantly — model it against the dedicated-database alternative first.
- → **Storage autoscaling.** Pool storage caps apply to the aggregate; monitor total pool storage, not just per-database sizes. A single tenant's data growth can push the whole pool toward its cap — set alerts at 70% and have the split/promote runbook ready.

Automate placement decisions: a monthly job that reviews per-database utilization against pool headroom and recommends promotions, splits, or vCore adjustments. At fleet scale, placement is a continuous optimization, not a one-time design.

14.8 When Pools Do Not Fit

Do not pool when: tenants have correlated peaks (the pool must be sized near the sum of peaks, erasing the savings); a single tenant needs more than the per-database max the pool can offer; tenants need different service tiers or compliance boundaries (a HIPAA tenant and a dev tenant do not share a pool); or databases need features outside the pool's tier. The honest alternative is a hybrid: pool for the long tail, dedicated provisioned or Hyperscale databases for the head. Most mature SaaS estates end up exactly here.

Key Takeaways

- → **Elastic pools monetize uncorrelated peaks.** Size from measured averages plus headroom for coincident peaks, not from summed peaks. The typical win is 60–80% over individual provisioning for long-tail SaaS tenants.
- → **Per-database min/max caps are the control plane.** Max caps bound noisy neighbors; min guarantees keep tenants responsive. Set them deliberately and revisit as tenants grow.
- → **Operate the fleet, not the databases.** Elastic jobs for schema rollout, pool-scope monitoring with per-database Query Store diagnostics, automated tenant provisioning, and per-tenant PITR drills.
- → **Tier your tenants.** Pool the long tail; give large, correlated, or compliance-sensitive tenants dedicated databases or their own pools. Promote tenants that outgrow the pool.
- → **Pools vs serverless is an idle-pattern question.** Lightly-busy-around-the-clock fleets belong in pools; truly-idle-for-hours databases belong on serverless. Measure, then choose.

Chapter 15: Azure SQL Hyperscale: When You Need It

Azure SQL Hyperscale is Microsoft's answer to a problem that on-prem DBAs know well: what happens when your database outgrows your storage architecture? Traditional SQL Server installations hit ceiling after ceiling — SAN I/O limits, filegroup management headaches, backup windows measured in hours, and the constant anxiety of a database that has grown beyond what your infrastructure was originally sized to handle. Azure SQL Hyperscale reimagines the storage layer entirely, decoupling compute from storage and introducing a distributed log service and page server architecture that lets you scale to 100 TB without a weekend maintenance window. This chapter covers when Hyperscale is the right choice, how it differs architecturally from standard Azure SQL Database tiers, how to provision and configure it correctly, what the production gotchas look like, and how to migrate into — and out of — it with your eyes open.

What Hyperscale Actually Is (And What It Isn't)

Before you provision a Hyperscale database, you need to understand that it is not simply "a bigger Azure SQL Database." It is a fundamentally different storage architecture that happens to present a familiar SQL Server surface. If you treat it like a standard Azure SQL tier with a higher storage cap, you will be surprised by behaviors that don't match your mental model.

The Architecture

In a traditional SQL Server instance — on-prem or in an Azure SQL General Purpose or Business Critical tier — your compute node owns the data files. Pages live on local or attached storage that the database engine reads and writes directly. Backups are full snapshots of that storage. Replica setup involves copying or synchronizing those files. Everything is tightly coupled.

Hyperscale decouples these layers. The architecture has four distinct components:

1. **Compute replicas** — These are your SQL Server engine instances. They hold the buffer pool (in-memory pages) and execute queries, but they do not own the data files.

2. **Log service** — A distributed, append-only transaction log service that sits between compute and storage. When your primary compute node commits a transaction, it writes to the log service first. This is where the acknowledged commit happens. Page servers and secondary replicas consume from this log asynchronously.

3. **Page servers** — Multiple independent page server processes that maintain portions of the database's page space and serve pages to compute nodes on demand. When a compute node needs a page not in its buffer pool, it fetches it from a page server over the network rather than from local disk. Each page server handles a shard of the database's page space, and they scale out as the database grows.

4. **Azure Storage** — The durable backing store underneath the page servers. This is Azure Premium Storage, providing the actual durability guarantee.

The practical consequence: your compute node's buffer pool is a cache in front of a distributed page store. A cold query that misses the buffer pool on a Hyperscale instance will be slower than the equivalent query on a Business Critical instance with local SSD, because it pays a network round-trip to a page server. On hot, well-cached workloads, Hyperscale performs comparably or better than other tiers for reads because you can add named replicas that each have their own independent buffer pool.

Azure SQL Hyperscale Architecture

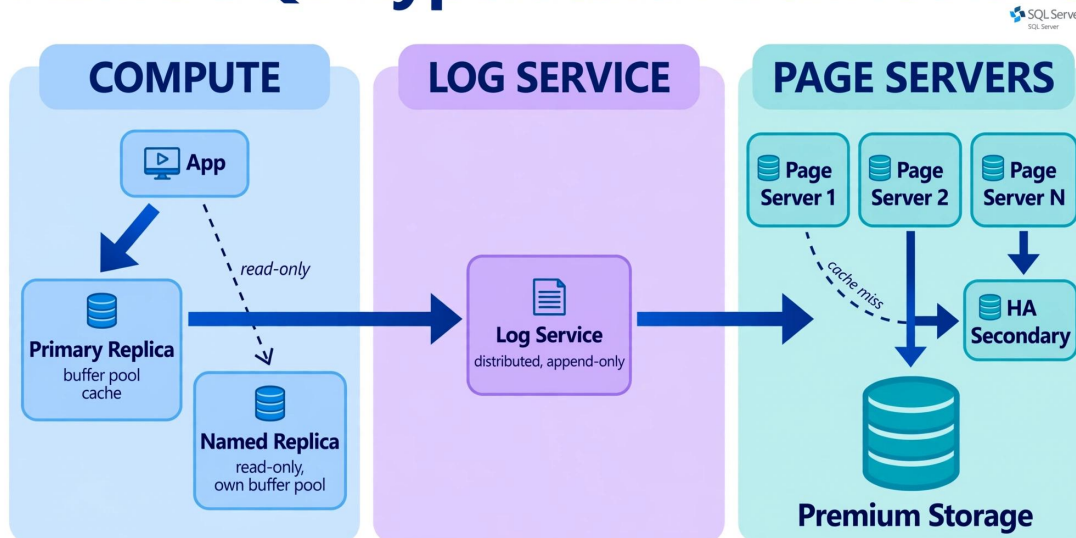


Figure 15.1 — Azure SQL Hyperscale Architecture.

Follow the arrows left to right: your app talks to the compute tier, where the primary replica serves writes from its buffer pool and named replicas serve read-only traffic from their own independent pools. Every write lands in the distributed, append-only log service — the single source of truth — which then fans out to HA secondaries and page servers. The page servers

own shards of the data on Premium Storage, and the primary fetches pages from them only on a buffer-pool cache miss, which is why Hyperscale storage can scale to 100 TB without touching compute.

Hyperscale's decoupled layers: compute replicas hold only a buffer-pool cache; durable page data lives in sharded page servers backed by Azure Storage, with the log service as the single write path all replicas consume from.

What It Is Designed For

Hyperscale makes sense when one or more of the following conditions apply:

- → Your database exceeds 4 TB (the limit for General Purpose and Business Critical tiers)
- → You need near-instant database creation and restore times regardless of database size
- → Your backup and restore windows are a compliance or operational problem — a 50 TB Hyperscale database restores in roughly 5–10 minutes because Azure only needs to instantiate compute and point it at existing page server snapshots
- → You have a mixed workload where you want to offload read-heavy reporting or analytics to a separate compute replica with its own memory and CPU, without the overhead of maintaining a full replica
- → You have bursty storage growth patterns and don't want to over-provision

What It Is Not Designed For

Hyperscale is not a general upgrade from Business Critical. Business Critical uses local SSD with in-memory OLTP support and has lower read latency for cold pages because storage is local. If your workload is primarily OLTP with a well-tuned buffer pool hit ratio above 98%, Business Critical will often outperform Hyperscale for raw query latency. Hyperscale also has restrictions that on-prem DBAs find surprising: as of the current GA release, you cannot change a Hyperscale database back to another service tier. That decision is one-directional. You migrate in, and if you need to migrate out, you do it through export/import or backup/restore — not through a service tier change.

The no-downgrade restriction is not a technical limitation that might be lifted later. It reflects the fact that Hyperscale storage format is fundamentally different from the other tiers. Plan accordingly before you migrate production.

Provisioning Hyperscale: Configuration That Matters

When you create a Hyperscale database, the decisions that matter most are compute generation, vCore count, read replica topology, and backup redundancy. Storage is automatic and does not require sizing decisions the way you were accustomed to on-prem.

Creating a Hyperscale Database with Azure CLI

```
# Create a resource group and logical server first if not already present
az group create \
  --name rg-hyperscale-prod \
  --location eastus2

az sql server create \
  --name sql-hs-prod-eastus2 \
  --resource-group rg-hyperscale-prod \
  --location eastus2 \
  --admin-user sqladmin \
  --admin-password "$(az keyvault secret show \
    --vault-name kv-prod \
    --name sql-admin-password \
    --query value -o tsv)"

# Create the Hyperscale database
# Edition: Hyperscale
# Service objective: HS_Gen5_8 (Gen5, 8 vCores)
# High-availability replicas: 1 (adds a secondary compute replica for HA)
# Backup storage redundancy: Zone (zone-redundant backup storage)
az sql db create \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --name OrdersDB \
  --edition Hyperscale \
  --service-objective HS_Gen5_8 \
  --ha-replicas 1 \
  --backup-storage-redundancy Zone \
  --read-scale Enabled

# Verify the database properties
az sql db show \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --name OrdersDB \
  --query "{tier:sku.tier, size:sku.name, maxSize:maxSizeBytes,
haReplicas:highAvailabilityReplicaCount}" \
  --output table
```

Hyperscale with Terraform

```

terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.80"
    }
  }
}

provider "azurerm" {
  features {}
}

resource "azurerm_resource_group" "hs_rg" {
  name     = "rg-hyperscale-prod"
  location = "East US 2"
}

resource "azurerm_mssql_server" "hs_server" {
  name                                = "sql-hs-prod-eastus2"
  resource_group_name                 = azurerm_resource_group.hs_rg.name
  location                           = azurerm_resource_group.hs_rg.location
  version                             = "12.0"
  administrator_login                 = "sqladmin"
  administrator_login_password       = var.sql_admin_password
  minimum_tls_version                 = "1.2"

  azuread_administrator {
    login_username = "DBA-Admins"
    object_id      = var.aad_admin_group_object_id
  }
}

resource "azurerm_mssql_database" "orders_hs" {
  name      = "OrdersDB"
  server_id = azurerm_mssql_server.hs_server.id

  sku_name = "HS_Gen5_8" # Hyperscale, Gen5, 8 vCores

  # Hyperscale-specific settings
  # high_availability_replica_count: 0 = no HA replica (cheaper, higher RT0)
  #                                     1 = one HA replica (recommended for production)
  #                                     2 = two HA replicas (for AZ failure tolerance)
  high_availability_replica_count = 1

  # read_replica_count: named replicas for read scale-out
  # These are separately provisioned and charged
  # Set to 0 here; add named replicas separately when needed

  # Backup storage redundancy: Geo, GeoZone, Local, Zone
  geo_backup_enabled = true

  tags = {

```

```

    Environment = "Production"
    ManagedBy    = "Terraform"
    CostCenter   = "Engineering"
  }
}

# Private endpoint for Hyperscale server
resource "azurerm_private_endpoint" "hs_pe" {
  name                = "pe-sql-hs-prod"
  location            = azurerm_resource_group.hs_rg.location
  resource_group_name = azurerm_resource_group.hs_rg.name
  subnet_id           = var.private_endpoint_subnet_id

  private_service_connection {
    name                = "psc-sql-hs-prod"
    private_connection_resource_id = azurerm_mssql_server.hs_server.id
    subresource_names     = ["sqlServer"]
    is_manual_connection   = false
  }
}

```

Compute Tiers and Sizing

Hyperscale compute uses the same vCore-based sizing as other Azure SQL tiers, but the service objective naming prefix is `HS_`. Common choices:

Service Objective	vCores	Memory	Notes
HS_Gen5_4	4	20.4 GB	Minimum recommended for production
HS_Gen5_8	8	40.8 GB	Good starting point for moderate OLTP
HS_Gen5_16	16	83 GB	High-concurrency workloads
HS_Gen5_32	32	166 GB	Large in-memory working sets
HS_Gen5_80	80	408 GB	Very large working sets, high throughput

There are also Fsv2 (compute-optimized) and M-series (memory-optimized) hardware generations available in select regions. The M-series configurations go up to 128 vCores and 3.7 TB of memory, which is territory that previously required dedicated hardware and significant capital expenditure.

You can scale compute up or down online. Scaling to a different vCore count within the same hardware generation takes approximately 2–3 minutes with minimal disruption. Changing hardware generation (Gen5 to Fsv2, for example) takes longer and involves a reconfiguration event.

Backup Storage Redundancy — Read This Before You Deploy

The `--backup-storage-redundancy` flag matters more than it appears. The options are:

- → `Local` — Backups stored in a single datacenter. Lowest cost, lowest durability. Not appropriate for most production workloads.
- → `Zone` — Zone-redundant backup storage. Appropriate for single-region deployments that need AZ tolerance.
- → `Geo` — Geo-redundant backup storage. Replicates backups to a paired region. Default for most Azure SQL tiers historically.
- → `GeoZone` — Zone-redundant in primary region plus geo-replication. Highest durability.

The critical gotcha: you can only set backup storage redundancy when you create the Hyperscale database. You cannot change it afterward without creating a new database. If you provision with `Local` and later realize you need geo-redundant backups for your DR policy, you are rebuilding the database.

Named Replicas: Read Scale-Out Without the Traditional Overhead

One of the most operationally useful Hyperscale features is named replicas. A named replica is a Hyperscale compute node pointed at the same page servers as the primary. It sees a consistent, near-real-time view of the data (typically within 1–5 seconds of the primary, sometimes less). Critically, it is a fully independent Azure SQL Database resource with its own service tier, its own connection string, its own buffer pool, and its own independent security configuration.

On-prem, you might have handled this with Always On readable secondaries. The fundamental difference is that a named replica in Hyperscale shares storage infrastructure with the primary — it is not maintaining a full copy of the data files. Adding a named replica takes minutes regardless of database size, because Azure is not copying 50 TB of data. It is instantiating a new compute node that can already read from the existing page servers.

When Named Replicas Make Sense

Named replicas are the right choice when:

- → You have a large reporting or analytics workload that competes with OLTP on the primary
- → You need to give a third-party team or application its own connection endpoint with no risk of their workload affecting production OLTP
- → You need read scale-out during peak periods and want to spin replicas up and down dynamically

Named replicas are billed as separate Hyperscale databases with their own vCore cost. If you provision a named replica as `HS_Gen5_16`, you pay for those 16 vCores separately. For pure read offloading, you can often provision named replicas at a lower vCore count than the primary.

```
# Create a named replica of an existing Hyperscale database
# The named replica lives in the same logical server
# It uses a different service objective than the primary if needed

az sql db replica create \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --name OrdersDB \
  --partner-database OrdersDB_ReportingReplica \
  --partner-server sql-hs-prod-eastus2 \
  --service-objective HS_Gen5_4 \
  --partner-resource-group rg-hyperscale-prod

# Verify replication link
az sql db replica list-links \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --name OrdersDB \
  --output table

# Connect to the named replica using its own connection string
# Named replica endpoint: sql-hs-prod-eastus2.database.windows.net
# Database: OrdersDB_ReportingReplica
# Application read intent:
# Server=sql-hs-prod-eastus2.database.windows.net;
# Database=OrdersDB_ReportingReplica;
# ApplicationIntent=ReadOnly;
```

Replication Lag Monitoring

Named replicas and HA secondary replicas consume the log service asynchronously. You need to monitor replication lag in production, particularly for workloads where reporting queries need data that is close to real-time.

```

-- Query replication lag on the primary (connect to primary, not replica)
-- sys.dm_database_replica_states is your tool here
SELECT
    r.replica_id,
    r.role_desc,
    r.secondary_lag_seconds,
    r.log_send_queue_size,
    r.log_send_rate,
    r.redo_queue_size,
    r.redo_rate,
    r.synchronization_health_desc,
    r.last_hardened_lsn,
    r.last_received_lsn
FROM sys.dm_hadr_database_replica_states r
WHERE r.is_local = 0; -- Non-local = secondary replicas

-- For Hyperscale-specific log service metrics
-- Check log generation rate and log service throughput
SELECT
    database_id,
    log_bytes_used,
    log_bytes_used_percent,
    log_committed_bytes,
    log_committed_percent,
    log_recovery_size_mb,
    log_since_last_log_backup_mb,
    log_since_last_checkpoint_mb
FROM sys.dm_db_log_stats(DB_ID());

-- Monitor page server health and latency from compute node perspective
-- This DMV shows remote page fetches (page server hits vs buffer pool hits)
SELECT
    page_server_reads,
    page_server_reads_in_progress,
    page_server_read_in_progress_ms,
    page_server_read_elapsed_ms,
    checkpoint_phase_desc,
    checkpoint_start_time
FROM sys.dm_exec_requests
WHERE page_server_reads > 0
ORDER BY page_server_reads DESC;

```

A replication lag above 30 seconds warrants investigation. Common causes are high log generation rate on the primary (large bulk operations, high-frequency small transactions), temporary page server pressure, or network congestion within the Azure fabric. You cannot directly tune the log service, but you can reduce peak log generation rate by batching large DML operations.

Migrating Into Hyperscale: Patterns That Work in Production

Migrating to Hyperscale from an on-prem SQL Server or from another Azure SQL tier requires more planning than a tier change, because Hyperscale is one-directional and the migration itself requires database-level operations rather than just a service tier slider.

Option 1: Backup and Restore via Azure Blob Storage

For on-prem databases below roughly 1 TB, this is the fastest path. You take a COPY_ONLY backup, upload to Azure Blob Storage, and restore into a Hyperscale database using `RESTORE DATABASE ... FROM URL .`

```

-- On-premises SQL Server: Create credential and backup to Azure Blob Storage
-- Step 1: Create SAS credential on source SQL Server
CREATE CREDENTIAL [https://storprodmigration.blob.core.windows.net/backups]
WITH IDENTITY = 'SHARED ACCESS SIGNATURE',
SECRET =
'sv=2023-01-03&ss=b&srt=sco&sp=rwdlacupiytfx&se=2024-12-31T00:00:00Z&st=2024-01-01T0
0:00:00Z&spr=https&sig=<your-sas-token>';
GO

-- Step 2: Full backup with COPY_ONLY (non-disruptive to existing backup chain)
BACKUP DATABASE [OrdersDB]
TO URL = 'https://storprodmigration.blob.core.windows.net/backups/
OrdersDB_full_20240115.bak'
WITH
    COPY_ONLY,
    COMPRESSION,
    STATS = 10,
    BLOCKSIZE = 65536,
    MAXTRANSFERSIZE = 4194304;
GO

-- For large databases, stripe the backup across multiple files
BACKUP DATABASE [OrdersDB]
TO
    URL = 'https://storprodmigration.blob.core.windows.net/backups/
OrdersDB_stripe1.bak',
    URL = 'https://storprodmigration.blob.core.windows.net/backups/
OrdersDB_stripe2.bak',
    URL = 'https://storprodmigration.blob.core.windows.net/backups/
OrdersDB_stripe3.bak',
    URL = 'https://storprodmigration.blob.core.windows.net/backups/
OrdersDB_stripe4.bak'
WITH
    COPY_ONLY,
    COMPRESSION,
    STATS = 5;
GO

-- Step 3: On the Hyperscale target (via SSMS or Azure Cloud Shell with sqlcmd)
-- Create credential on target server
CREATE CREDENTIAL [https://storprodmigration.blob.core.windows.net/backups]
WITH IDENTITY = 'SHARED ACCESS SIGNATURE',
SECRET =
'sv=2023-01-03&ss=b&srt=sco&sp=rwdlacupiytfx&se=2024-12-31T00:00:00Z&st=2024-01-01T0
0:00:00Z&spr=https&sig=<your-sas-token>';
GO

-- Step 4: Restore into Hyperscale
-- Note: You must restore to a NEW database name, not overwrite an existing one
RESTORE DATABASE [OrdersDB]
FROM
    URL = 'https://storprodmigration.blob.core.windows.net/backups/
OrdersDB_stripe1.bak',

```

```
URL = 'https://storprodmigration.blob.core.windows.net/backups/
OrdersDB_stripe2.bak',
URL = 'https://storprodmigration.blob.core.windows.net/backups/
OrdersDB_stripe3.bak',
URL = 'https://storprodmigration.blob.core.windows.net/backups/
OrdersDB_stripe4.bak'
WITH
RECOVERY,
STATS = 5;
GO
```

For minimal-downtime migrations, you restore the full backup, then apply differential and log backups during a short cutover window. This is functionally the same as the SQL Server DBA's traditional migration playbook — the cloud just adds the credential layer.

Option 2: Azure Database Migration Service

For larger databases or when you want a more managed migration process, Azure DMS handles the orchestration. You configure it to perform an initial load followed by continuous replication of changes until you cut over. It is comparable in concept to setting up log shipping or replication to a new instance, with the monitoring and retry logic handled for you.

```
# Create an Azure DMS instance (Premium tier for online migrations)
az dms create \
  --resource-group rg-hyperscale-prod \
  --name dms-migration-prod \
  --location eastus2 \
  --sku-name Premium_4vCores \
  --subnet /subscriptions/<sub-id>/resourceGroups/rg-network/providers/
Microsoft.Network/virtualNetworks/vnet-prod/subnets/snet-dms \
  --storage-account /subscriptions/<sub-id>/resourceGroups/rg-hyperscale-prod/
providers/Microsoft.Storage/storageAccounts/storprodmigration

# Create a migration project
az dms project create \
  --resource-group rg-hyperscale-prod \
  --service-name dms-migration-prod \
  --name proj-ordersdb-migration \
  --source-platform SQL \
  --target-platform AzureSqlDb \
  --location eastus2

# Create a migration task (online migration with continuous sync)
az dms project task create \
  --resource-group rg-hyperscale-prod \
  --service-name dms-migration-prod \
  --project-name proj-ordersdb-migration \
  --name task-ordersdb-migrate \
  --task-type OnlineMigration \
  --source-connection-json @source_connection.json \
  --target-connection-json @target_connection.json \
  --database-options-json @database_migration_options.json

# Monitor migration status
az dms project task show \
  --resource-group rg-hyperscale-prod \
  --service-name dms-migration-prod \
  --project-name proj-ordersdb-migration \
  --name task-ordersdb-migrate \
  --expand output \
  --query "output[0].migrationState"
```

Pre-Migration Compatibility Check

Hyperscale supports most SQL Server features, but there are exclusions. Run the Database Experimentation Assistant (DEA) or use the assessment feature in Azure Migrate before committing to a migration timeline. Common compatibility gaps that surface for on-prem databases:

- → Linked servers — not supported in Azure SQL Database including Hyperscale. Replace with OPENDATASOURCE, external tables, or application-level joins.
- → SQL Server Agent — not available. Use Azure Elastic Jobs or Azure Automation.

- → Filestream / Filetable — not supported.
- → Cross-database queries within the same instance — Azure SQL is database-scoped. If your application uses three-part names to query across databases, that needs rearchitecting.
- → BULK INSERT from local file paths — requires Azure Blob Storage as the source.

These are not Hyperscale-specific limitations. They are Azure SQL Database limitations that apply across all tiers including Hyperscale. The point is that you discover them during migration planning, not after you have committed to the migration.

Production Gotchas and Operational Patterns

Hyperscale has behaviors that will surprise experienced SQL Server DBAs because they are not documented in the places you look first. This section covers the ones that cause the most friction in real production deployments.

Cold Query Latency and Buffer Pool Warming

After a compute replica failover — whether from a planned scaling event, an unplanned node failure, or the HA replica being promoted to primary — the new compute node starts with a cold buffer pool. Pages must be fetched from page servers on demand. For a large database, the period immediately following a failover will have noticeably higher read latency than steady state.

On Business Critical, failover also lands you on a cold buffer pool on the new node if it was an unplanned failover, but the page fetch is from local SSD. On Hyperscale, it is a network round-trip to a page server. The latency is typically measured in single-digit milliseconds per page fetch, but queries that scan large ranges and miss the buffer pool can feel sluggish for several minutes to an hour after failover, depending on your working set size and query patterns.

The mitigation strategies are:

1. Schedule planned failovers (compute scaling, maintenance) during low-traffic periods when possible.
2. Use connection retry logic in your application so that the brief (~30 second) failover window does not produce user-visible errors.
3. For workloads where cold start latency is particularly damaging, consider pre-warming by running a scheduled query after known maintenance windows that touches your most-accessed indexes.


```
-- Simple buffer pool pre-warming query pattern
-- Run this after a planned failover or compute scaling event
-- Adjust the table and index list to match your actual hot working set

-- Pre-warm a specific clustered index by scanning it
-- This forces pages into the buffer pool proactively
SELECT COUNT_BIG(*) FROM dbo.Orders WITH (NOLOCK);
SELECT COUNT_BIG(*) FROM dbo.OrderItems WITH (NOLOCK);
SELECT COUNT_BIG(*) FROM dbo.Customers WITH (NOLOCK);

-- You can also pre-warm specific index ranges if full scans are too expensive
-- Warm the last 90 days of orders (typical hot working set for OLTP)
SELECT COUNT_BIG(*)
FROM dbo.Orders WITH (NOLOCK)
WHERE OrderDate >= DATEADD(DAY, -90, GETDATE());

-- Monitor buffer pool state after warming
-- Check what percentage of a table is currently cached
SELECT
    t.name AS TableName,
    i.name AS IndexName,
    i.type_desc AS IndexType,
    COUNT(1) AS CachedPages,
    COUNT(1) * 8 / 1024 AS CachedMB,
    SUM(CASE WHEN b.is_modified = 1 THEN 1 ELSE 0 END) AS DirtyPages
FROM sys.dm_os_buffer_descriptors b
INNER JOIN sys.allocation_units au
    ON b.allocation_unit_id = au.allocation_unit_id
INNER JOIN sys.partitions p
    ON au.container_id = p.hobt_id
INNER JOIN sys.indexes i
    ON p.object_id = i.object_id AND p.index_id = i.index_id
INNER JOIN sys.tables t
    ON p.object_id = t.object_id
WHERE b.database_id = DB_ID()
GROUP BY t.name, i.name, i.type_desc
ORDER BY CachedMB DESC;
```

The Checkpoint Behavior Difference

On standard SQL Server, checkpoints flush dirty pages from the buffer pool to disk at the storage layer. On Hyperscale, checkpoint behavior is different because the primary compute node does not own the storage. Checkpoints on Hyperscale flush dirty pages from the compute buffer pool to the page servers. This means the write path goes: application DML → log service (commit acknowledged here) → page server receives log record and applies it → dirty page flushed from compute buffer pool on checkpoint.

The practical consequence is that I/O patterns around checkpoint look different in wait statistics. You will see `WRITELOG` waits that correspond to log service throughput, not local disk throughput. If you have historically used `WRITELOG` waits as a proxy for your storage I/O ceiling, you need to recalibrate — the constraint in Hyperscale is the log service throughput, which scales with your vCore tier.

```
-- Monitor log throughput and wait statistics relevant to Hyperscale
SELECT
    wait_type,
    waiting_tasks_count,
    wait_time_ms,
    max_wait_time_ms,
    signal_wait_time_ms,
    wait_time_ms - signal_wait_time_ms AS resource_wait_time_ms
FROM sys.dm_os_wait_stats
WHERE wait_type IN (
    'WRITELOG',
    'LOG_RATE_GOVERNOR',
    'HADR_LOGCAPTURE_WAIT',
    'HADR_WORK_QUEUE',
    'PAGEIOLATCH_SH',
    'PAGEIOLATCH_EX',
    'PAGEIOLATCH_UP',
    'REMOTEBUF_IO_WAIT' -- Hyperscale-specific: wait for remote page server I/O
)
ORDER BY wait_time_ms DESC;

-- LOG_RATE_GOVERNOR is particularly important for Hyperscale
-- It indicates your workload is hitting the log generation rate limit for your
vCore tier
-- If you see high LOG_RATE_GOVERNOR waits, you need to scale up vCores or reduce
log throughput
```

LOG_RATE_GOVERNOR: The Throttle You Will Hit

Each Hyperscale vCore tier has a maximum log generation rate measured in MB/s. When your transaction workload generates log records faster than this limit, SQL Server itself throttles new log-generating transactions, producing `LOG_RATE_GOVERNOR` waits. This is not a bug — it is an intentional throttle that protects the log service and downstream page servers.

The log rate limits increase with vCore count. If you are routinely hitting `LOG_RATE_GOVERNOR` waits, the options are:

1. Scale up vCores to a tier with a higher log rate limit.
2. Reduce log generation per unit of work — batch your DML operations, minimize unnecessary row-by-row processing, use minimally logged operations where appropriate.

3. Spread bulk operations over time rather than running them in large single transactions.

```
-- Identify the heaviest log-generating operations currently running
SELECT
    r.session_id,
    r.status,
    r.command,
    r.wait_type,
    r.wait_time,
    r.total_elapsed_time,
    r.reads,
    r.writes,
    r.logical_reads,
    t.text AS QueryText,
    qp.query_plan
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
CROSS APPLY sys.dm_exec_query_plan(r.plan_handle) qp
WHERE r.wait_type = 'LOG_RATE_GOVERNOR'
ORDER BY r.total_elapsed_time DESC;

-- Historical log generation rate from ring buffer
-- This gives you a picture of log throughput over recent minutes
SELECT
    end_time,
    delta_log_bytes_used / 1048576.0 AS log_MB_generated,
    avg_log_rate_value / 1048576.0 AS log_rate_MB_per_sec
FROM sys.dm_db_resource_stats
ORDER BY end_time DESC;
```

Snapshot Isolation and Read Committed Snapshot

Hyperscale databases have RCSI (Read Committed Snapshot Isolation) enabled by default and it cannot be disabled. For on-prem DBAs who have spent years managing version store pressure on tempdb, this is actually welcome news — Hyperscale stores version records in the page server layer, not in tempdb. The version store does not create the tempdb pressure that causes headaches on busy on-prem instances.

However, if your application was written against a database with `READ_COMMITTED_SNAPSHOT OFF` and relies on blocking behavior for correctness (for example, using locks as a coordination mechanism between application processes), you need to verify that RCSI does not change the behavior. Applications that take explicit `UPDLOCK` hints or use `SERIALIZABLE` transactions are not affected. Applications that relied on reader-writer blocking as an implicit synchronization mechanism will behave differently.

Tempdb Is Still Shared on Standard Compute

Unlike on-prem SQL Server where each instance has its own tempdb, Azure SQL Database shares tempdb resources across databases on the same logical server. On standard vCore tiers this is per the Azure SQL infrastructure. This is not a Hyperscale-specific behavior, but it catches DBAs who come from dedicated instances. Heavy tempdb consumers (large sorts, hash joins, spill-heavy plans, excessive temp table use) can see unexpected tempdb pressure from neighboring databases.

Monitor tempdb usage and address spills in your query plans:

```
-- Identify queries spilling to tempdb on Hyperscale
SELECT TOP 20
    qs.total_spills / qs.execution_count AS avg_spills_per_exec,
    qs.total_spills,
    qs.execution_count,
    qs.total_elapsed_time / qs.execution_count / 1000 AS avg_elapsed_ms,
    SUBSTRING(st.text, (qs.statement_start_offset / 2) + 1,
        ((CASE qs.statement_end_offset
            WHEN -1 THEN DATALENGTH(st.text)
            ELSE qs.statement_end_offset
        END - qs.statement_start_offset) / 2) + 1) AS QueryText
FROM sys.dm_exec_query_stats qs
CROSS APPLY sys.dm_exec_sql_text(qs.sql_handle) st
WHERE qs.total_spills > 0
ORDER BY qs.total_spills DESC;
```

Monitoring Hyperscale in Production

Standard Azure SQL monitoring applies to Hyperscale, with some additional dimensions. You should be collecting metrics from Azure Monitor, DMV-based monitoring from within the database, and query store data for workload trends.

Azure Monitor Metrics for Hyperscale

The metrics that matter most for Hyperscale, beyond the standard CPU and connection count, are:

- → `log\_write\_percent` — how close you are to the log rate limit. Watch for sustained values above 80%.
- → `storage\_percent` — Hyperscale does not have a fixed storage cap in the traditional sense, but this metric reflects page server utilization.
- → `xtp\_storage\_percent` — In-Memory OLTP storage (if you use memory-optimized tables).
- → `dwu\_consumption\_percent` — effective compute consumption.
- → `sessions\_percent` — connection count relative to the limit for your tier.

```
# Pull Hyperscale-specific metrics via Azure CLI
az monitor metrics list \
  --resource "/subscriptions/<sub-id>/resourceGroups/rg-hyperscale-prod/providers/Microsoft.Sql/servers/sql-hs-prod-eastus2/databases/OrdersDB" \
  --metric
"log_write_percent,cpu_percent,physical_data_read_percent,sessions_percent" \
  --interval PT1M \
  --start-time "$(date -u -d '1 hour ago' '+%Y-%m-%dT%H:%MZ')" \
  --end-time "$(date -u '+%Y-%m-%dT%H:%MZ')" \
  --output table

# Set up an alert for log rate throttling
az monitor metrics alert create \
  --name "hs-log-rate-high" \
  --resource-group rg-hyperscale-prod \
  --scopes "/subscriptions/<sub-id>/resourceGroups/rg-hyperscale-prod/providers/Microsoft.Sql/servers/sql-hs-prod-eastus2/databases/OrdersDB" \
  --condition "avg log_write_percent > 80" \
  --window-size 5m \
  --evaluation-frequency 1m \
  --action-group /subscriptions/<sub-id>/resourceGroups/rg-hyperscale-prod/providers/Microsoft.Insights/actionGroups/ag-dba-oncall \
  --description "Hyperscale log rate sustained above 80% - risk of LOG_RATE_GOVERNOR throttling"
```

Query Store for Workload Trend Analysis

Query Store is enabled by default on Hyperscale and is your primary tool for identifying plan regressions, tracking query performance over time, and understanding whether a performance problem is from a plan change or a data volume change.

```

-- Configure Query Store for production Hyperscale workloads
-- The defaults are reasonable but you may want to increase retention
ALTER DATABASE [OrdersDB]
SET QUERY_STORE = ON (
    OPERATION_MODE = READ_WRITE,
    CLEANUP_POLICY = (STALE_QUERY_THRESHOLD_DAYS = 90),
    DATA_FLUSH_INTERVAL_SECONDS = 900,
    INTERVAL_LENGTH_MINUTES = 15,
    MAX_STORAGE_SIZE_MB = 4096,
    QUERY_CAPTURE_MODE = AUTO,
    MAX_PLANS_PER_QUERY = 200,
    SIZE_BASED_CLEANUP_MODE = AUTO,
    WAIT_STATS_CAPTURE_MODE = ON
);
GO

-- Find queries with the highest average elapsed time in the last 24 hours
SELECT TOP 20
    q.query_id,
    qt.query_sql_text,
    rs.avg_duration / 1000.0 AS avg_elapsed_ms,
    rs.avg_cpu_time / 1000.0 AS avg_cpu_ms,
    rs.avg_logical_io_reads,
    rs.avg_physical_io_reads, -- High value here = page server fetches
    rs.count_executions,
    p.plan_id,
    p.last_force_failure_reason_desc
FROM sys.query_store_query q
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
JOIN sys.query_store_runtime_stats_interval i ON rs.runtime_stats_interval_id =
i.runtime_stats_interval_id
WHERE i.start_time >= DATEADD(HOUR, -24, GETUTCDATE())
ORDER BY rs.avg_duration DESC;

-- Find plan regressions: queries that have gotten slower recently
WITH recent_stats AS (
    SELECT
        q.query_id,
        AVG(rs.avg_duration) AS recent_avg_duration,
        MAX(rs.avg_duration) AS recent_max_duration,
        SUM(rs.count_executions) AS recent_executions
    FROM sys.query_store_query q
    JOIN sys.query_store_plan p ON q.query_id = p.query_id
    JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
    JOIN sys.query_store_runtime_stats_interval i ON rs.runtime_stats_interval_id =
i.runtime_stats_interval_id
    WHERE i.start_time >= DATEADD(HOUR, -4, GETUTCDATE())
    GROUP BY q.query_id
),
baseline_stats AS (
    SELECT

```

```

        q.query_id,
        AVG(rs.avg_duration) AS baseline_avg_duration
    FROM sys.query_store_query q
    JOIN sys.query_store_plan p ON q.query_id = p.query_id
    JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
    JOIN sys.query_store_runtime_stats_interval i ON rs.runtime_stats_interval_id =
i.runtime_stats_interval_id
    WHERE i.start_time >= DATEADD(HOUR, -28, GETUTCDATE())
        AND i.start_time < DATEADD(HOUR, -4, GETUTCDATE())
    GROUP BY q.query_id
)
SELECT
    r.query_id,
    qt.query_sql_text,
    b.baseline_avg_duration / 1000.0 AS baseline_avg_ms,
    r.recent_avg_duration / 1000.0 AS recent_avg_ms,
    r.recent_avg_duration / NULLIF(b.baseline_avg_duration, 0) AS regression_ratio
FROM recent_stats r
JOIN baseline_stats b ON r.query_id = b.query_id
JOIN sys.query_store_query q ON r.query_id = q.query_id
JOIN sys.query_store_query_text qt ON q.query_text_id = qt.query_text_id
WHERE r.recent_avg_duration > b.baseline_avg_duration * 2 -- 2x regression
threshold
    AND r.recent_executions >= 10 -- Only queries with meaningful execution count
ORDER BY regression_ratio DESC;

```

Page Server Latency DMV

Hyperscale exposes a DMV specifically for page server operations that has no equivalent in standard Azure SQL tiers:

```

-- sys.dm_user_db_resource_governance includes Hyperscale-specific limits
-- For page server I/O characteristics, use this combination:
SELECT
    io_stall,
    io_stall_read_ms,
    io_stall_write_ms,
    num_of_reads,
    num_of_writes,
    num_of_bytes_read,
    num_of_bytes_written,
    io_stall_queued_read_ms,    -- time spent waiting to issue reads
    io_stall_queued_write_ms   -- time spent waiting to issue writes
FROM sys.dm_io_virtual_file_stats(DB_ID(), NULL)
ORDER BY io_stall DESC;

-- Hyperscale-specific: check page server fetch activity
-- This is available in sys.dm_exec_requests during active queries
SELECT
    r.session_id,
    r.wait_type,
    r.wait_time,
    r.page_server_reads,
    r.page_server_reads_in_progress,
    r.page_server_read_elapsed_ms,
    CASE
        WHEN r.page_server_reads > 0
        THEN r.page_server_read_elapsed_ms / r.page_server_reads
        ELSE 0
    END AS avg_page_server_read_ms,
    SUBSTRING(t.text, r.statement_start_offset / 2 + 1,
        (CASE r.statement_end_offset
            WHEN -1 THEN DATALENGTH(t.text)
            ELSE r.statement_end_offset
        END - r.statement_start_offset) / 2 + 1) AS current_statement
FROM sys.dm_exec_requests r
CROSS APPLY sys.dm_exec_sql_text(r.sql_handle) t
WHERE r.page_server_reads > 0
ORDER BY r.page_server_reads DESC;

```

If your `avg_page_server_read_ms` values are consistently above 5–10 ms for a heavily accessed query, it is a signal that your working set does not fit in the buffer pool and you would benefit from more memory — either by scaling vCores up or by adding a named replica that caches a subset of pages independently.

Migrating Out of Hyperscale

Because the tier change from Hyperscale back to another Azure SQL tier is not supported through the portal or API, you need a data movement strategy. This is not a frequent operation, but the scenarios where it arises include consolidation, cost reduction after a data archiving project that brought the database below 4 TB, or a strategic decision to move to a different database service entirely.

Export to BACPAC

BACPAC export is the simplest path for databases that are not extremely large. It exports the schema and data to an Azure Blob Storage file that can be imported into any Azure SQL tier.

```

# Export Hyperscale database to BACPAC
# Note: For very large databases, this can take many hours
# Monitor progress via the export operation status

az sql db export \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --name OrdersDB \
  --admin-user sqladmin \
  --admin-password "$(az keyvault secret show --vault-name kv-prod --name sql-admin-
password --query value -o tsv)" \
  --storage-key "$(az storage account keys list --resource-group rg-hyperscale-prod
--account-name storprodmigration --query '[0].value' -o tsv)" \
  --storage-key-type StorageAccessKey \
  --storage-uri "https://storprodmigration.blob.core.windows.net/exports/
OrdersDB_export.bacpac"

# Monitor the export operation
az sql db op list \
  --resource-group rg-hyperscale-prod \
  --server sql-hs-prod-eastus2 \
  --database OrdersDB \
  --query "[?operation=='ExportDatabase']" \
  --output table

# Import BACPAC to a new General Purpose database
az sql db import \
  --resource-group rg-target \
  --server sql-gp-prod \
  --name OrdersDB \
  --admin-user sqladmin \
  --admin-password "$(az keyvault secret show --vault-name kv-prod --name sql-admin-
password --query value -o tsv)" \
  --storage-key "$(az storage account keys list --resource-group rg-hyperscale-prod
--account-name storprodmigration --query '[0].value' -o tsv)" \
  --storage-key-type StorageAccessKey \
  --storage-uri "https://storprodmigration.blob.core.windows.net/exports/
OrdersDB_export.bacpac" \
  --edition GeneralPurpose \
  --service-objective GP_Gen5_8

```

BACPAC export is not point-in-time consistent for active databases. If the database is still receiving writes during export, the resulting BACPAC may have a logically inconsistent state for tables that were modified while the export was running. For a clean migration out of Hyperscale, the safest approach is to quiesce writes during export or use the backup-and-restore path with a log backup applied at cutover.

Transactional Replication for Live Cutover

For large databases or workloads that cannot tolerate extended downtime, transactional replication from the Hyperscale source to a target database in a different tier provides continuous synchronization until cutover. You configure Hyperscale as the publisher, the target as the subscriber, let replication catch up, then flip the connection string and remove the replication topology.

This approach requires careful attention to replication topology setup, but it is the same replication mechanism that on-prem DBAs have used for decades. Hyperscale supports being both a publisher and a subscriber for transactional replication, with the same T-SQL configuration commands that work on-prem.

Cost Modeling: When Hyperscale Is the Right Economic Choice

Hyperscale is not always the cheapest choice even for large databases, and it is worth doing the arithmetic before committing.

The cost components for a Hyperscale database are:

1. **Compute:** vCore-hours for the primary compute replica.
2. **HA replicas:** Each HA replica is billed at the same vCore rate as the primary. One HA replica effectively doubles your compute cost.
3. **Named replicas:** Each named replica is billed at its own vCore rate.
4. **Storage:** Hyperscale storage pricing is per-GB per-month beyond the first 100 GB. Storage consumption grows with your database and does not shrink automatically when you delete data (pages are not immediately reclaimed from page servers after DELETE operations — space reclamation happens through index rebuilds and DBCC SHRINKFILE, which should be used judiciously).
5. **Backup storage:** Billed per GB-month beyond 1x the database size.
6. **Log service egress:** Generally included in the tier, but cross-region configurations may have network transfer costs.

The inflection point where Hyperscale becomes economically competitive with Business Critical is roughly at the point where:

- → The database exceeds 4 TB (forced migration anyway for General Purpose or Business Critical)

- → Or the named replica topology you need would require multiple Always On nodes that would each need to be Business Critical size

For databases below 2 TB with no read scale-out requirement and a stable, OLTP-dominant workload, Business Critical often delivers better price-performance because of local SSD and the tighter latency profile.

Do the vCore math with your actual workload size, your replica topology needs, and the current Azure SQL pricing for your region. Prices change, reserved instance discounts alter the comparison significantly, and the Dev/Test pricing for non-production Hyperscale instances reduces the cost of running staging and QA environments on the same tier as production.

Key Takeaways

- → **Hyperscale is architecturally distinct, not just a storage upgrade.** Its decoupled compute, log service, and page server model gives you near-instant backup and restore at any size, but introduces cold-page network latency that does not exist in Business Critical. Understand the tradeoff before provisioning.
- → **The migration into Hyperscale is one-directional through the service tier API.** You can migrate out, but only through data movement operations like BACPAC export, backup/restore, or transactional replication — not through a tier change. Make the decision with full awareness of this constraint.
- → **Named replicas are a genuinely different read scale-out model.** Because they share the page server layer rather than maintaining full storage copies, you can add or remove them in minutes regardless of database size. This makes dynamic read scale-out practical in ways that traditional readable secondaries are not.
- → **`LOG\_RATE\_GOVERNOR` is your primary throughput ceiling indicator.** High `LOG\_RATE\_GOVERNOR` wait times mean your log generation rate exceeds what your vCore tier allows. The response is either to scale vCores up or to reduce log generation through batching and minimally logged patterns.
- → **Backup storage redundancy and RCSI-on behavior are set at creation and cannot be changed.** Validate your DR requirements and your application's isolation level assumptions before the first provisioning command runs — these are not settings you can tune after the fact without rebuilding the database.

Chapter 16: SQL Server Cross-Region Disaster Recovery

High availability keeps you running when a server or zone fails. Disaster recovery keeps the business alive when a region fails — or when a logical catastrophe (mass deletion, ransomware, corruption replicated everywhere) makes every copy in the primary region untrustworthy. This chapter designs cross-region DR for SQL Server on each cloud: the replication mechanisms, the RPO/RTO trade-offs, the failover procedures, and the validation that proves it works. It builds on Chapter 9's testing discipline and applies it to the regional scale.

16.1 The DR Problem: What You Are Actually Solving

Define the scenarios before choosing mechanisms. **Regional infrastructure failure** (the cloud region is impaired): you need a runnable copy of the database in another region and a way to redirect clients. **Logical disaster** (data deleted or corrupted, then replicated): you need a point in time *before* the corruption, which replication cannot give you — only backups can. **Prolonged degradation** (the region works but badly): you need a controlled, preferably reversible failover. Most DR designs handle the first scenario well and the second poorly. Your design must cover both: replication for RTO, backups for RPO-behind-corruption.

Set the targets explicitly: RPO (how much data loss is acceptable — seconds, minutes, hours?) and RTO (how long until the business is running again?). Every mechanism below lands somewhere on that grid. Do not let anyone say "we need zero RPO and zero RTO" without pricing it — synchronous cross-region replication exists (and its latency cost is real), but most businesses choose async with eyes open.

16.2 AWS: Snapshot Copy, S3 Backups, and EC2 AGs

RDS for SQL Server has no cross-region read replica, so DR on AWS is assembled from backup-based mechanisms:

- → **Cross-region snapshot copy.** Copy automated or manual snapshots to a second region on a schedule (Chapter 6). RPO equals the snapshot/copy frequency (typically hours); RTO equals restore time (tens of minutes to hours depending on size). Automate the copy with EventBridge and verify the copies complete — a DR plan whose copies silently stopped six months ago is the classic finding.

- → **Native backups to S3 with cross-region replication.** For EC2-based SQL Server (and as a complement to RDS snapshots), native backup files in S3 with CRR on the bucket give you portable, restorable copies in the second region. RPO equals the log backup frequency (15 minutes is typical); RTO equals the restore-and-recovery time. This is the finest RPO available for SQL Server DR on AWS without a running secondary.
- → **Distributed availability groups on EC2.** For near-zero RPO/RTO on AWS, run a distributed AG: a local AG in each region, linked by cross-region replication. This is the most complete — and most operationally demanding — SQL Server DR architecture on AWS. It requires EC2 (RDS cannot participate), careful network design (inter-region latency directly affects async replication lag), and the failover orchestration from Chapter 9.

```
# Automate daily cross-region snapshot copy with EventBridge Scheduler
aws scheduler create-schedule \
  --name dr-snapshot-copy-daily \
  --schedule-expression "cron(0 2 * * ? *)" \
  --target '{
    "Arn": "arn:aws:lambda:us-east-1:123456789012:function:copy-rds-snapshot-dr",
    "RoleArn": "arn:aws:iam::123456789012:role/scheduler-execution-role"
  }' \
  --flexible-time-window '{"Mode": "OFF"}' \
  --region us-east-1

# The Lambda copies the latest automated snapshot to us-west-2
# and prunes copies older than the retention policy
```

16.3 Azure: Failover Groups and Geo-Restore

Azure's DR story is the most integrated of the three clouds. **Auto-failover groups** (for SQL Database and Managed Instance) bundle databases or instances into a group with a primary region and a secondary region, a single listener endpoint, and automatic failover policy with a configurable grace period. The secondary is a continuously replicated, readable copy. RPO is seconds (async replication lag); RTO is the grace period plus promotion time — typically single-digit minutes. Readable secondaries in the DR region also serve reporting, so the DR investment does double duty.

```
# Create a failover group for a Managed Instance pair
az sql mi failover-group create \
  --resource-group rg-prod \
  --name fg-mi-prod \
  --primary-server mi-prod-east \
  --partner-servers mi-prod-west \
  --partner-resource-group rg-prod-west \
  --failover-policy Automatic \
  --grace-period 60

# Planned failover drill (Chapter 9 discipline applies at regional scale)
az sql mi failover-group set-primary \
  --resource-group rg-prod-west \
  --name fg-mi-prod \
  --server mi-prod-west \
  --allow-data-loss false
```

For the logical-disaster scenario, **geo-restore** recovers any database from geo-redundant backup storage to any region, at any point within the retention window — independent of replication. And **long-term retention** backups in paired-region storage give you the compliance-depth copies. The Azure DR checklist is therefore two tracks: failover groups for regional failure (tested with drills), geo-restore/LTR for logical disaster (tested with restore exercises).

16.4 The Client Redirection Problem

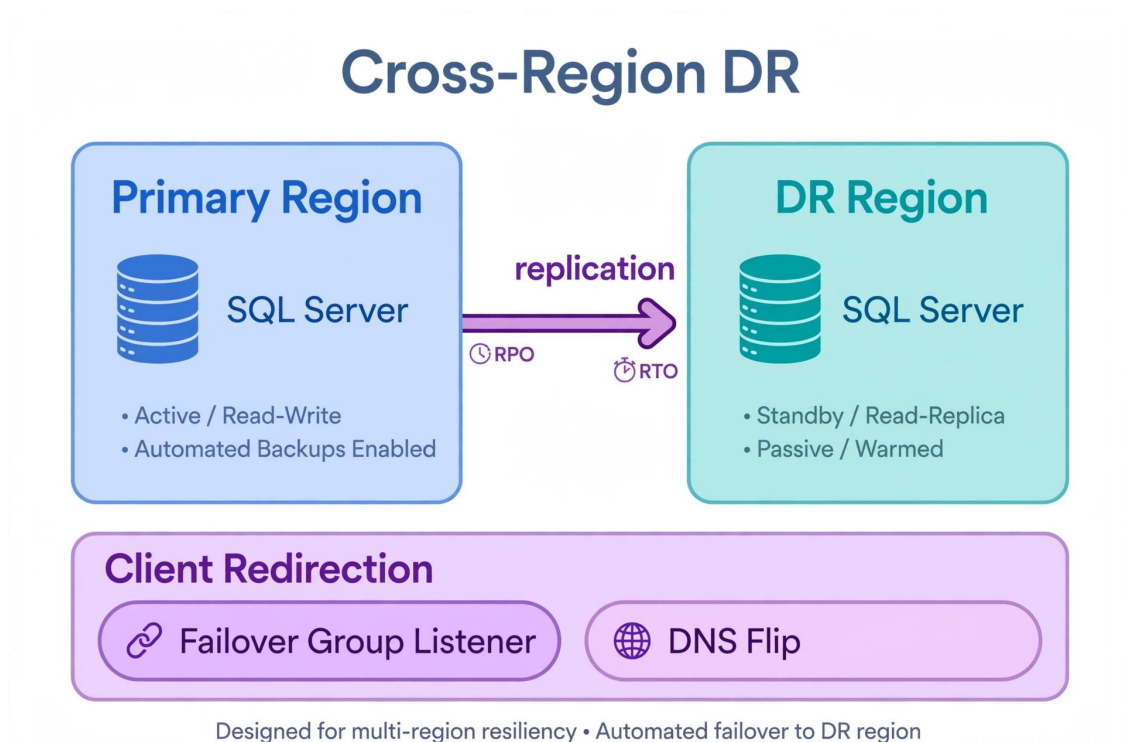


Figure 16.1 — Cross-Region DR Topology. Replicate the data, then redirect the clients — the bottom lane is the half most DR plans forget.

The top half is the data plane: the primary region's SQL Server replicates to the DR region, and the RPO/RTO badges on the replication arrow are the contract that mechanism buys you — know those two numbers cold before you promise anything to the business. The bottom lane is the part teams discover missing during the first real drill: client redirection. The failover group listener (Azure) abstracts the region behind one stable name; the DNS flip (AWS/Route 53) moves the hostname but is gated by TTL. Data replicated without a client-redirection plan is a DR plan that fails its first real test.

The database failing over regions is half the problem; getting clients to the new region is the other half. Options in increasing order of sophistication:

- → **Failover group listener.** On Azure, the listener endpoint abstracts the region — clients connect to one name and the platform routes them. This is the simplest correct answer where available.
- → **DNS failover.** On AWS, update a Route 53 record (or use Route 53 health checks with failover routing) to point the database hostname at the DR region's endpoint after restore/promotion. DNS TTL controls propagation time — set the database hostname TTL low (60 seconds) permanently, not just during drills.
- → **Application configuration.** The application reads the database endpoint from configuration and you redeploy or flip a feature flag. Slowest but most explicit — acceptable for RTO budgets measured in hours, not minutes.

Whichever mechanism you choose, the DR drill must test it end to end: a database restored in the second region that no client can reach is not recovered. Include client connectivity in the drill's success criteria.

16.5 RPO/RTO Reality Check

Mechanism	Typical RPO	Typical RTO	Notes
RDS cross-region snapshot copy	Hours	Restore time (30+ min)	Simplest; automate and verify copies
Native backup to S3 + CRR	Minutes (log backup frequency)	Restore + recovery time	Portable; best RPO without running secondary
Distributed AG (EC2)	Seconds	Minutes (manual/automated)	Most complete; most operational cost
Azure failover group (auto)	Seconds	Grace period + promotion	Integrated; readable secondary included
Azure geo-restore	Up to 1 hour	Restore time	

Mechanism	Typical RPO	Typical RTO	Notes
			Per-database; covers logical disaster

Treat these as starting points for your own measurement, not guarantees. Inter-region latency, database size, and restore throughput all move the numbers. The only RPO/RTO that counts is the one from your last drill.

16.6 The DR Drill: Regional Game Day

Run the regional drill at least annually (Chapter 9, level 4). The script: declare the drill, fail over (or restore) in the secondary region, redirect clients via your chosen mechanism, run the business's critical transactions against the secondary, measure the actual RTO, then fail back. Record every manual step — each one is an automation candidate. Common drill findings: the secondary region's capacity was never reserved (failover succeeds technically but the undersized secondary falls over under load); KMS keys or Key Vault access do not exist in the secondary region (encrypted restores fail); the runbook's endpoint names are stale; and nobody told the security team, whose anomaly detection pages on the "unexpected" cross-region traffic.

16.7 Logical Disaster: The Scenario Replication Cannot Save

Replication faithfully copies corruption, deletion, and ransomware encryption to every replica — that is its job. Defense in depth for logical disaster:

- → **Immutable backups.** S3 Object Lock (compliance mode) or Azure immutable blob storage for backup copies. An attacker who compromises the database credentials cannot delete what the storage policy forbids deleting.
- → **Delayed replica (EC2).** An async AG secondary with intentional apply delay (e.g., 1 hour) gives you a window to stop replication before the corruption arrives. It costs a replica and operational complexity; for the most critical databases it is worth it.
- → **Long retention.** Azure LTR or extended snapshot/backup retention gives you clean points far behind the corruption event. Know your maximum tolerable "time to detection" for logical corruption and set retention beyond it.
- → **Separate security domains.** Backup storage in a separate account/subscription with separate credentials, so a compromised production identity cannot reach the backups. This is the single highest-value DR security control.

Key Takeaways

- → **Design for two disasters, not one.** Regional failure needs replication (RTO); logical disaster needs point-in-time backups (RPO behind corruption). Replication alone is half a DR plan.
- → **Match the mechanism to the RPO/RTO target.** Snapshot copy for simple/hour-scale DR, S3 native backups with CRR for minute-scale RPO, distributed AGs or Azure failover groups for second-scale RPO with automatic failover. Price each honestly.
- → **Client redirection is part of the drill.** Listener endpoints, low-TTL DNS failover, or config flips — untested redirection is the most common reason a technically successful failover still means downtime.
- → **Immutable, separate-domain backups are the ransomware answer.** Object Lock / immutable blob storage in a separate account, with retention beyond your worst-case detection time.
- → **Drill regionally, annually, end to end.** Fail over, redirect clients, run critical transactions, measure real RTO, fail back. Every manual step found is an automation work item.

Chapter 17: Azure SQL Security: TDE, Always Encrypted, Ledger

Azure SQL Database ships with encryption features that would have required significant infrastructure investment in an on-premises SQL Server environment. Transparent Data Encryption was always available on-prem, but managing certificates, key rotation, and backup encryption was a full operational concern. Always Encrypted required application-side key management that most teams deferred indefinitely. Azure SQL Ledger didn't exist at all. In the cloud, these three technologies form a layered security posture that satisfies everything from SOC 2 to PCI DSS to HIPAA — but only if you understand where each layer applies, what it actually protects, and what its operational costs are. This chapter walks through each feature from first principles to production deployment, with the gotchas that the official documentation skips.

17.1 Transparent Data Encryption: What It Does (and Doesn't) Protect

If you've managed SQL Server on-prem, you know TDE: it encrypts the data files, log files, and backup files at rest. Azure SQL Database has TDE enabled by default on every new database. That sentence creates a false sense of security that has burned more than a few DBAs during audit preparation.

What TDE protects: Data files on disk. An attacker who steals raw storage media or data files gets ciphertext. Backups are encrypted automatically. This satisfies the "data at rest" requirement in most compliance frameworks.

What TDE does not protect: Data in memory, data in transit, data from a compromised SQL login, or a DBA with elevated permissions. TDE encryption and decryption happens transparently at the storage layer. Once the database engine decrypts data for query execution, TDE offers no protection. A rogue DBA running `SELECT * FROM dbo.CreditCards` gets plaintext every time.

The default TDE configuration uses a service-managed key: Microsoft generates and manages the Database Encryption Key (DEK) and the certificate that protects it. For most applications this is acceptable. For regulated industries, you need Customer-Managed Keys (CMK) via Azure Key Vault — where you control key lifecycle, rotation, and revocation.

Enabling TDE with Customer-Managed Key via Azure CLI

```

# Step 1: Create or identify the Azure Key Vault
az keyvault create \
  --name "prod-sqlsecurity-kv" \
  --resource-group "rg-prod-eastus" \
  --location "eastus" \
  --sku premium \
  --enable-purge-protection true \
  --retention-days 90

# Step 2: Create the key used as the TDE protector
az keyvault key create \
  --vault-name "prod-sqlsecurity-kv" \
  --name "sql-tde-protector" \
  --kts RSA-HSM \
  --size 2048

# Step 3: Assign the SQL Server managed identity access to the key vault
# First get the SQL Server's managed identity principal ID
SQL_IDENTITY=$(az sql server show \
  --name "prod-sql-server" \
  --resource-group "rg-prod-eastus" \
  --query identity.principalId \
  --output tsv)

az keyvault set-policy \
  --name "prod-sqlsecurity-kv" \
  --object-id "$SQL_IDENTITY" \
  --key-permissions get wrapKey unwrapKey list

# Step 4: Set the TDE protector to the customer-managed key
KEY_ID=$(az keyvault key show \
  --vault-name "prod-sqlsecurity-kv" \
  --name "sql-tde-protector" \
  --query key.kid \
  --output tsv)

az sql server tde-key set \
  --resource-group "rg-prod-eastus" \
  --server "prod-sql-server" \
  --server-key-type AzureKeyVault \
  --kid "$KEY_ID"

# Step 5: Verify TDE is active on the database
az sql db tde show \
  --resource-group "rg-prod-eastus" \
  --server "prod-sql-server" \
  --database "prod-app-db"

```

The output of that last command will show `"state": "Enabled"` when TDE is active. During the initial encryption scan — which runs as a background process — you'll see `"state": "Enabling"`. The scan duration scales with database size and can run for hours on multi-

terabyte databases. Queries continue running during this window, but expect elevated I/O and some added query latency on write-heavy workloads until the scan completes — measure it against your own workload rather than assuming a fixed overhead.

Key Rotation

With CMK-based TDE, key rotation is a two-phase process: add the new key version, then set it as the active protector. The old key stays in Key Vault until you explicitly confirm rotation is complete — this is intentional, because any backup encrypted under the old key still needs the old key for restore.

```
# Rotate the TDE protector key
az keyvault key rotate \
  --vault-name "prod-sqlsecurity-kv" \
  --name "sql-tde-protector"

# Get the new key version ID
NEW_KEY_ID=$(az keyvault key show \
  --vault-name "prod-sqlsecurity-kv" \
  --name "sql-tde-protector" \
  --query key.kid \
  --output tsv)

# Update the server TDE protector to use the new version
az sql server tde-key set \
  --resource-group "rg-prod-eastus" \
  --server "prod-sql-server" \
  --server-key-type AzureKeyVault \
  --kid "$NEW_KEY_ID"
```

Production gotcha: If the SQL Server managed identity loses access to Key Vault — an expired policy, a deleted vault, a misconfigured network rule — the server goes into an inaccessible state within minutes. You cannot query the database, cannot restore from backup, cannot do anything until Key Vault access is restored. Set up Azure Monitor alerts on Key Vault access failures for the SQL Server managed identity before you go live with CMK.

17.2 Always Encrypted: Column-Level Encryption the Right Way

Always Encrypted addresses the threat model that TDE ignores: the privileged insider or compromised DBA account. With Always Encrypted, sensitive column data is encrypted client-side before it ever reaches the SQL Server engine. The database engine sees ciphertext. It stores ciphertext. It returns ciphertext. The plaintext never exists on the server.

Think of it this way: TDE protects your data from someone who steals the storage hardware. Always Encrypted protects your data from someone who steals the database credentials or has DBA access. These are different attackers in different positions.

The architecture has three components:

- → **Column Encryption Key (CEK):** Encrypts the actual column data. One CEK per column group.
- → **Column Master Key (CMK):** Encrypts the CEKs. Stored in Azure Key Vault (or Windows Certificate Store for dev). The database stores only metadata pointing to the CMK location — never the key itself.
- → **Client application:** Holds the Azure Key Vault credentials. The ADO.NET or JDBC driver handles encryption/decryption transparently when the connection string includes `Column Encryption Setting=enabled`.

Setting Up Always Encrypted with Azure Key Vault

```

-- Step 1: Create the Column Master Key metadata in SQL Server
-- The actual key material lives in Azure Key Vault, not here
-- This T-SQL runs in SSMS or sqlcmd after configuring AKV access

CREATE COLUMN MASTER KEY [AKV_CMK_Production]
WITH (
    KEY_STORE_PROVIDER_NAME = 'AZURE_KEY_VAULT',
    KEY_PATH = 'https://prod-sqlsecurity-kv.vault.azure.net/keys/always-encrypted-
cmk/abc123def456'
);
GO

-- Step 2: Create the Column Encryption Key
-- This operation contacts Key Vault to wrap the CEK with the CMK
-- Requires the executing account to have Key Vault Wrap permissions

CREATE COLUMN ENCRYPTION KEY [CEK_PII_Data]
WITH VALUES (
    COLUMN_MASTER_KEY = [AKV_CMK_Production],
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = 0x016E000001630075007200720065006E... -- wrapped CEK bytes
);
GO

-- Step 3: Create a table with encrypted columns
-- DETERMINISTIC encryption: same plaintext always produces same ciphertext
-- Supports equality comparisons (WHERE SSN = @ssn)
-- RANDOMIZED encryption: same plaintext produces different ciphertext each time
-- More secure but supports NO predicates – SELECT only by returning the column

CREATE TABLE dbo.CustomerPII (
    CustomerID          INT          NOT NULL PRIMARY KEY,
    FirstName           NVARCHAR(100) NOT NULL, -- plaintext, used in UI
    LastName            NVARCHAR(100) NOT NULL, -- plaintext
    SSN                 CHAR(11)     COLLATE Latin1_General_BIN2
                        ENCRYPTED WITH (
                            COLUMN_ENCRYPTION_KEY = CEK_PII_Data,
                            ENCRYPTION_TYPE = DETERMINISTIC,
                            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
                        ) NOT NULL,
    CreditCardNumber    NVARCHAR(19) COLLATE Latin1_General_BIN2
                        ENCRYPTED WITH (
                            COLUMN_ENCRYPTION_KEY = CEK_PII_Data,
                            ENCRYPTION_TYPE = RANDOMIZED,
                            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
                        ) NULL,
    DateOfBirth         DATE         ENCRYPTED WITH (
                            COLUMN_ENCRYPTION_KEY = CEK_PII_Data,
                            ENCRYPTION_TYPE = RANDOMIZED,
                            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
                        ) NULL
);

```

```
);  
GO
```

Notice the `COLLATE Latin1_General_BIN2` requirement on the NVARCHAR columns. Always Encrypted requires BIN2 collation for deterministic encryption on string columns. If you're retrofitting Always Encrypted onto an existing table that uses `Latin1_General_CI_AS`, you'll need to rebuild those columns. That's a blocking operation on large tables.

Application Integration


```

# Python application connecting to Always Encrypted columns
# Requires: pip install pyodbc, azure-identity, azure-keyvault-keys

import pyodbc
from azure.identity import DefaultAzureCredential
from azure.keyvault.keys.crypto import CryptographyClient

# The connection string does the heavy lifting
# Column Encryption Setting=enabled tells the driver to handle crypto
connection_string = (
    "Driver={ODBC Driver 18 for SQL Server};"
    "Server=prod-sql-server.database.windows.net;"
    "Database=prod-app-db;"
    "Authentication=ActiveDirectoryManagedIdentity;"
    "Column Encryption Setting=enabled;"
    "KeyVaultClientId=<managed-identity-client-id>;"
    "KeyVaultClientSecret=notneeded;" # Not needed with managed identity
    "Encrypt=yes;"
    "TrustServerCertificate=no;"
)

conn = pyodbc.connect(connection_string)
cursor = conn.cursor()

# INSERT: application sends ciphertext, SQL Server stores ciphertext
# The ODBC driver encrypts the SSN value before it leaves the client machine
cursor.execute(
    "INSERT INTO dbo.CustomerPII (CustomerID, FirstName, LastName, SSN) "
    "VALUES (?, ?, ?, ?)",
    (1001, 'Jane', 'Smith', '123-45-6789') # SSN encrypted by driver
)

# SELECT with parameterized query on deterministic column: works
# The driver encrypts the parameter value, SQL Server compares ciphertext
cursor.execute(
    "SELECT CustomerID, FirstName, LastName, SSN FROM dbo.CustomerPII "
    "WHERE SSN = ?",
    ('123-45-6789',)
)

row = cursor.fetchone()
if row:
    # SSN comes back as plaintext – decrypted by the driver on the client
    print(f"Found customer: {row.FirstName} {row.LastName}, SSN: {row.SSN}")

conn.commit()
conn.close()

```

The Operational Constraints You Must Know Before Deploying

Always Encrypted has constraints that are not obvious from the documentation until you run into them in production:

You cannot do arithmetic on encrypted columns. `WHERE Salary > 50000` on an encrypted Salary column fails. RANDOMIZED encrypted columns support no predicates whatsoever. DETERMINISTIC columns support only equality predicates (`=`, `IN`).

Joins on encrypted columns work only for DETERMINISTIC encryption. `JOIN ON t1.SSN = t2.SSN` functions correctly with DETERMINISTIC. It fails with RANDOMIZED because the same plaintext produces different ciphertexts.

Stored procedures that accept encrypted column values must use parameterized execution. The stored procedure definition looks normal in T-SQL; the application must pass values as parameters, not as inline SQL literals.

GROUP BY, ORDER BY, and index ranges on encrypted columns: GROUP BY works with DETERMINISTIC. ORDER BY on encrypted data returns results ordered by ciphertext, which is meaningless for the application. There's no way around this. If you need sorted results on an encrypted column, sort client-side after decryption.

Column size expansion: Every encrypted value is larger than the plaintext. A `CHAR(11)` SSN value becomes a 65-byte binary blob. Size your columns and row sizes accordingly. Encryption adds approximately 54 bytes of overhead plus the encrypted value size, rounded up to the nearest AES block.

17.3 Always Encrypted with Secure Enclaves

The encryption predicate limitations in classic Always Encrypted (described above) are severe enough that many teams avoid the feature entirely. Azure SQL's Always Encrypted with Secure Enclaves removes most of them.

An enclave is a protected memory region in the SQL Server process — implemented via VBS (Virtualization Based Security) enclaves in Azure SQL Database. Sensitive cryptographic operations happen inside the enclave, where even the database engine process cannot inspect the memory. The application sends a CEK into the enclave via an attestation-verified channel. Query processing on encrypted columns happens inside the enclave, so the engine never exposes plaintext to the wider process.

With enclave-enabled Always Encrypted, you gain:

- → Range queries on encrypted columns (`WHERE Age BETWEEN 30 AND 45`)

- → Pattern matching (``LIKE '%smith%`` on RANDOMIZED encrypted columns with enclave-enabled CEKs)
- → Index creation on encrypted columns
- → In-place column encryption without data movement

The tradeoff is complexity: the application connection must include attestation configuration pointing to a Microsoft Azure Attestation endpoint, and the Key Vault CMK must be enclave-enabled.

```
# Enable enclave support on the Azure SQL logical server
az sql server update \
  --name "prod-sql-server" \
  --resource-group "rg-prod-eastus" \
  --set properties.enabledSecureEnclaveType="VBS"

# Verify the enclave type
az sql server show \
  --name "prod-sql-server" \
  --resource-group "rg-prod-eastus" \
  --query properties.enabledSecureEnclaveType
```

The attestation URL for VBS enclaves in Azure SQL follows the pattern `https://shareduks.uks.attest.azure.net` (region-specific). The connection string needs:

```
Enclave Attestation Url=https://shareduks.uks.attest.azure.net/attest/AzureIntelSGX;
Column Encryption Setting=enabled;
```

Enclave-enabled queries carry a performance cost: the attestation handshake adds ~5–10ms per new connection. For long-lived connection pools this is negligible. For high-frequency short-lived connections it's measurable. Test with your actual connection pool configuration before production rollout.

17.4 Azure SQL Ledger: Tamper-Evident Tables for Compliance

Azure SQL Ledger is the newest of the three technologies in this chapter, and it solves a different problem than encryption. Ledger doesn't hide your data — it makes your data tamper-evident. Any modification to ledger table data leaves a cryptographically verifiable trail that can prove, to an external auditor, that the data has not been altered since it was written.

The use cases are specific: financial transaction records, audit logs, configuration history, contract terms, any dataset where "we can prove this hasn't been modified" is a legal or regulatory requirement.

Ledger works by maintaining a SHA-256 hash tree (similar to a blockchain Merkle tree) over table rows. Each database transaction produces a block containing a hash of all changed rows. Each block includes the hash of the previous block. The result is a chain: modifying any historical row invalidates all subsequent hashes. An auditor can verify the entire chain in seconds.

Azure SQL Ledger has two table types:

Updatable Ledger Tables: Support INSERT, UPDATE, DELETE. Maintain a separate `ledger_history` table containing the full history of every change, with who made it and when. The current table shows current state. The history table is append-only and protected by the hash chain.

Append-Only Ledger Tables: Support INSERT only. No UPDATE, no DELETE. Appropriate for immutable audit logs, transaction records, event streams.

```

-- Create an updatable ledger table for account balances
-- The LEDGER = ON clause activates all ledger machinery
CREATE TABLE dbo.AccountBalance (
    AccountID          INT          NOT NULL,
    CustomerName       NVARCHAR(100) NOT NULL,
    Balance            DECIMAL(18,2) NOT NULL,
    LastModified       DATETIME2    NOT NULL DEFAULT SYSUTCDATETIME()
)
WITH (
    SYSTEM_VERSIONING = ON,
    LEDGER = ON
);
GO

-- Azure SQL automatically creates dbo.AccountBalance_Ledger
-- and the ledger history table with these system columns:
-- ledger_start_transaction_id BIGINT
-- ledger_end_transaction_id   BIGINT
-- ledger_start_sequence_number BIGINT
-- ledger_end_sequence_number  BIGINT
-- ledger_operation_type       TINYINT (1=INSERT, 2=DELETE for update tracking)
-- ledger_operation_type_desc  NVARCHAR(128)

-- Create an append-only ledger table for payment events
CREATE TABLE dbo.PaymentAudit (
    PaymentID          UNIQUEIDENTIFIER NOT NULL DEFAULT NEWSEQUENTIALID(),
    AccountID          INT              NOT NULL,
    Amount             DECIMAL(18,2)    NOT NULL,
    PaymentTimestamp   DATETIME2        NOT NULL DEFAULT SYSUTCDATETIME(),
    PaymentMethod      NVARCHAR(50)     NOT NULL,
    ProcessorRef       NVARCHAR(100)    NULL
)
WITH (
    LEDGER = ON (APPEND_ONLY = ON)
);
GO

-- Verify the ledger configuration
SELECT
    name,
    is_ledger,
    ledger_type_desc,
    is_dropped_ledger_table
FROM sys.tables
WHERE name IN ('AccountBalance', 'PaymentAudit');
GO

```

Verifying the Ledger

Verification is the feature that makes auditors comfortable. The `sp_verify_database_ledger` procedure recomputes the entire hash chain and reports any tampering.

```

-- Generate the database ledger digest (send to external immutable storage)
-- This digest is what you share with auditors as the "known good" state
EXECUTE sp_generate_database_ledger_digest;
GO
-- Returns JSON:
-- {
--   "database_name": "prod-app-db",
--   "block_id": 42,
--   "hash": "0x3A7F8B2C...",
--   "last_transaction_commit_time": "2024-11-15T14:22:11.123Z",
--   "digest_version": 1
-- }

-- Verify ledger integrity using a previously stored digest
-- Pass the digest JSON retrieved from your immutable storage (Azure Blob with WORM)
EXECUTE sp_verify_database_ledger
    N'["database_name":"prod-app-db","block_id":
42,"hash":"0x3A7F8B2C...","last_transaction_commit_time":"2024-11-15T14:22:11.123Z",
"digest_version":1]';
GO
-- Returns: The database ledger has been successfully verified.

-- View the full audit history for a specific account
-- This queries the automatically maintained history table
SELECT
    h.AccountID,
    h.CustomerName,
    h.Balance,
    h.LastModified,
    h.ledger_operation_type_desc AS Operation,
    t.commit_time AS ChangeTime,
    t.principal_name AS ChangedBy
FROM dbo.AccountBalance_Ledger h
INNER JOIN sys.database_ledger_transactions t
    ON h.ledger_start_transaction_id = t.transaction_id
WHERE h.AccountID = 10042
ORDER BY h.ledger_start_sequence_number;
GO

```

Digest Storage Strategy

A ledger without an independently stored digest is less useful than it appears. If an attacker can modify both the ledger data and the digest stored in the same database, they can reconstruct a consistent but fraudulent chain. Digests must be stored in immutable external storage:

- → **Azure Blob Storage with immutable storage policies (WORM):** The native Azure integration. Use `sp\_generate\_database\_ledger\_digest` on a schedule (hourly or after each significant batch) and write digests to a locked container.

- → **Azure Confidential Ledger:** A purpose-built service using hardware-backed distributed ledger. Digests written here are independently verified by an infrastructure you don't control.

```
# Configure automatic digest storage to Azure Blob immutable storage
az sql db ledger-digest-uploads enable \
  --resource-group "rg-prod-eastus" \
  --server "prod-sql-server" \
  --database "prod-app-db" \
  --endpoint "https://prodauditdigests.blob.core.windows.net/ledger-digests"

# Verify digest upload status
az sql db ledger-digest-uploads show \
  --resource-group "rg-prod-eastus" \
  --server "prod-sql-server" \
  --database "prod-app-db"
```

17.5 Layering the Security Stack: TDE + Always Encrypted + Ledger Together

These three features protect against different attackers and different threat vectors. In a production system handling regulated data, you often need all three:

Feature	Protects Against	Doesn't Protect Against
TDE	Stolen storage media, backup files	Privileged DB users, in-memory access
Always Encrypted	Privileged DBAs, compromised credentials	Tampered historical records
Ledger	Tampering with historical data	Data exposure to privileged users

---|---|---

| TDE | Stolen storage media, backup files | Privileged DB users, in-memory access |

| Always Encrypted | Privileged DBAs, compromised credentials | Tampered historical records |

| Ledger | Tampering with historical data | Data exposure to privileged users |

The combination creates defense in depth: TDE ensures backups are useless without your Key Vault key. Always Encrypted ensures DBAs see ciphertext on sensitive columns. Ledger ensures that even if someone does gain write access, any modification is detectable.

Layered SQL Server Security Stack

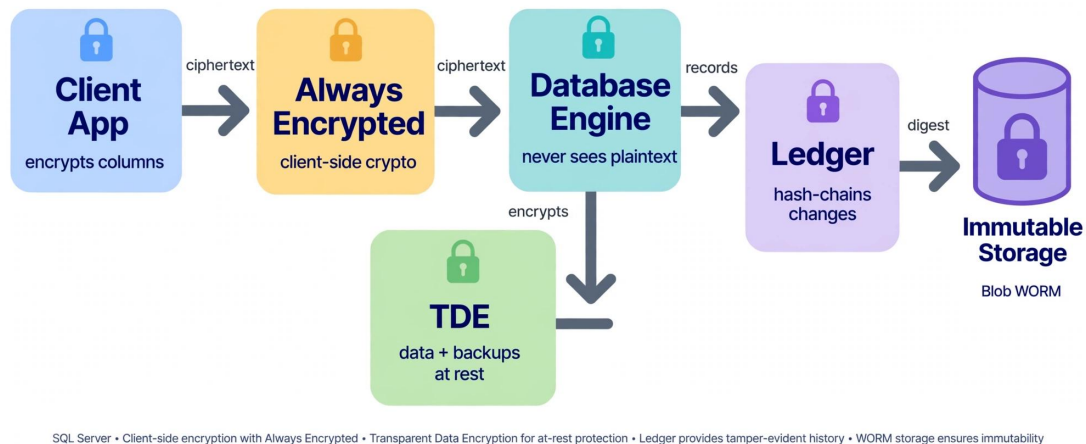


Figure 17.1 — Layered SQL Server Security Stack.

Read left to right along the top row: the client app encrypts sensitive columns before anything leaves the machine, so only ciphertext ever crosses the wire into Always Encrypted. The database engine itself never sees plaintext, while TDE quietly encrypts data and backup files at rest underneath. Ledger hash-chains every modification and ships periodic digests to immutable Blob WORM storage — so even a rogue admin with full sysadmin rights cannot rewrite history without the tampering being provable.

Each layer defends against a different attacker: TDE stops storage theft, Always Encrypted stops privileged-user data exposure, and Ledger makes any tampering with historical data provable — none of the three substitutes for the others.

Terraform: Provisioning a Secured Azure SQL Database


```

# terraform/azure_sql_secure.tf
# Provisions Azure SQL with TDE (CMK), auditing, and threat protection

terraform {
  required_providers {
    azurerm = {
      source  = "hashicorp/azurerm"
      version = "~> 3.80"
    }
  }
}

resource "azurerm_resource_group" "prod" {
  name      = "rg-prod-eastus"
  location  = "East US"
}

# Key Vault for TDE and Always Encrypted keys
resource "azurerm_key_vault" "sql_security" {
  name                        = "prod-sqlsecurity-kv"
  resource_group_name        = azurerm_resource_group.prod.name
  location                   = azurerm_resource_group.prod.location
  tenant_id                  = data.azurem_client_config.current.tenant_id
  sku_name                   = "premium"
  soft_delete_retention_days = 90
  purge_protection_enabled   = true

  network_acls {
    default_action = "Deny"
    bypass         = "AzureServices"
    ip_rules       = var.allowed_ip_ranges
  }
}

# SQL Server with system-assigned managed identity
resource "azurerm_mssql_server" "prod" {
  name                        = "prod-sql-server"
  resource_group_name        = azurerm_resource_group.prod.name
  location                   = azurerm_resource_group.prod.location
  version                    = "12.0"
  administrator_login        = var.sql_admin_login
  administrator_login_password = var.sql_admin_password
  minimum_tls_version        = "1.2"

  identity {
    type = "SystemAssigned"
  }

  azuread_administrator {
    login_username = var.aad_admin_username
    object_id      = var.aad_admin_object_id
  }
}

```

```

# Grant SQL Server managed identity access to Key Vault
resource "azurerm_key_vault_access_policy" "sql_server" {
  key_vault_id = azurerm_key_vault.sql_security.id
  tenant_id    = data.azurerm_client_config.current.tenant_id
  object_id    = azurerm_mssql_server.prod.identity[0].principal_id

  key_permissions = [
    "Get", "WrapKey", "UnwrapKey", "List"
  ]
}

# TDE protector key in Key Vault
resource "azurerm_key_vault_key" "tde_protector" {
  name          = "sql-tde-protector"
  key_vault_id = azurerm_key_vault.sql_security.id
  key_type      = "RSA-HSM"
  key_size      = 2048

  key_opts = ["wrapKey", "unwrapKey"]

  depends_on = [azurerm_key_vault_access_policy.sql_server]
}

# Set the TDE protector on the SQL Server
resource "azurerm_mssql_server_transparent_data_encryption" "prod" {
  server_id      = azurerm_mssql_server.prod.id
  key_vault_key_id = azurerm_key_vault_key.tde_protector.id
}

# Production database
resource "azurerm_mssql_database" "prod_app" {
  name          = "prod-app-db"
  server_id     = azurerm_mssql_server.prod.id
  collation     = "SQL_Latin1_General_CP1_CI_AS"
  sku_name      = "GP_Gen5_4"
  max_size_gb   = 256
  zone_redundant = true

  threat_detection_policy {
    state          = "Enabled"
    email_account_admins = "Enabled"
    retention_days  = 90
  }

  depends_on = [azurerm_mssql_server_transparent_data_encryption.prod]
}

# Storage account for audit logs with immutable policy
resource "azurerm_storage_account" "audit" {
  name          = "prodsqldataauditlogs"
  resource_group_name = azurerm_resource_group.prod.name
  location      = azurerm_resource_group.prod.location
}

```

```

account_tier          = "Standard"
account_replication_type = "GRS"
min_tls_version       = "TLS1_2"

blob_properties {
  versioning_enabled = true
}
}

resource "azurerm_storage_container" "audit_logs" {
  name                  = "sqlauditlogs"
  storage_account_name = azurerm_storage_account.audit.name
  container_access_type = "private"
}

# Server-level auditing to immutable storage
resource "azurerm_mssql_server_extended_auditing_policy" "prod" {
  server_id                  = azurerm_mssql_server.prod.id
  storage_endpoint           =
  azurerm_storage_account.audit.primary_blob_endpoint
  storage_account_access_key =
  azurerm_storage_account.audit.primary_access_key
  storage_account_access_key_is_secondary = false
  retention_in_days          = 90
  log_monitoring_enabled     = true
}

data "azurerm_client_config" "current" {}

```

17.6 Operational Monitoring and Alerting for Security Features

Deploying TDE, Always Encrypted, and Ledger is only half the work. The features need ongoing monitoring because several failure modes are silent until they become critical.

Key Vault Access Failures

The most operationally dangerous failure mode with CMK-based TDE is losing Key Vault access. Azure SQL will enter an inaccessible state — all connections refused — if it cannot reach the Key Vault key. This can happen due to:

- → Key Vault network rules blocking the SQL Server managed identity
- → The managed identity's access policy being accidentally deleted
- → The key being disabled or deleted in Key Vault
- → Key Vault going into a soft-delete state

Set up Azure Monitor alerts on the Key Vault diagnostic logs before going live:

```
# Enable Key Vault diagnostic logging
az monitor diagnostic-settings create \
  --name "keyvault-to-loganalytics" \
  --resource "/subscriptions/<sub-id>/resourceGroups/rg-prod-eastus/providers/
Microsoft.KeyVault/vaults/prod-sqlsecurity-kv" \
  --workspace "/subscriptions/<sub-id>/resourceGroups/rg-prod-eastus/providers/
Microsoft.OperationalInsights/workspaces/prod-law" \
  --logs '[{"category": "AuditEvent", "enabled": true}]' \
  --metrics '[{"category": "AllMetrics", "enabled": true}]'

# Create an alert for failed Key Vault operations from the SQL Server identity
az monitor scheduled-query create \
  --name "SQL-KeyVault-Access-Failure" \
  --resource-group "rg-prod-eastus" \
  --scopes "/subscriptions/<sub-id>/resourceGroups/rg-prod-eastus/providers/
Microsoft.OperationalInsights/workspaces/prod-law" \
  --condition "count 'Placeholder' > 0" \
  --condition-query "AzureDiagnostics
| where ResourceType == 'VAULTS'
| where ResourceId contains 'prod-sqlsecurity-kv'
| where ResultType == 'Unauthorized' or ResultType == 'Forbidden'
| where CallerIPAddress != ''
| where TimeGenerated > ago(5m)" \
  --evaluation-frequency "5m" \
  --window-size "5m" \
  --severity 1 \
  --action-groups "/subscriptions/<sub-id>/resourceGroups/rg-prod-eastus/providers/
Microsoft.Insights/actionGroups/prod-oncall"
```

Ledger Verification on a Schedule

Ledger verification should run on a schedule, not just during audits. A daily verification job catches tampering early. Wrap `sp_verify_database_ledger` in an Azure SQL elastic job or a simple Azure Function with a timer trigger:

```

-- Stored procedure to run verification and log results
-- Deploy this in your admin/DBA database or a monitoring schema

CREATE OR ALTER PROCEDURE dbo.usp_VerifyLedgerIntegrity
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @DigestJSON NVARCHAR(MAX);
    DECLARE @VerificationResult NVARCHAR(500);
    DECLARE @ErrorMessage NVARCHAR(1000);

    BEGIN TRY
        -- Retrieve the most recent stored digest from your audit table
        -- In production this would pull from Azure Blob or Confidential Ledger
        SELECT TOP 1 @DigestJSON = DigestJSON
        FROM dbo.LedgerDigestArchive
        ORDER BY CapturedAt DESC;

        IF @DigestJSON IS NULL
        BEGIN
            INSERT INTO dbo.LedgerVerificationLog (RunTime, Status, Message)
            VALUES (SYSUTCDATETIME(), 'WARNING', 'No stored digest found for
verification.');
```

```

            RETURN;
        END

        -- Run verification
        EXEC sp_verify_database_ledger @DigestJSON;

        INSERT INTO dbo.LedgerVerificationLog (RunTime, Status, Message)
        VALUES (SYSUTCDATETIME(), 'SUCCESS', 'Ledger verification passed.');
```

```

    END TRY
    BEGIN CATCH
        SET @ErrorMessage = ERROR_MESSAGE();

        INSERT INTO dbo.LedgerVerificationLog (RunTime, Status, Message)
        VALUES (SYSUTCDATETIME(), 'FAILURE', @ErrorMessage);

        -- Re-raise so the calling agent job marks the step as failed
        -- and triggers an alert
        THROW;
    END CATCH
END;
GO

-- Supporting tables
CREATE TABLE dbo.LedgerDigestArchive (
    ArchiveID      INT IDENTITY PRIMARY KEY,
    DigestJSON     NVARCHAR(MAX) NOT NULL,
    CapturedAt     DATETIME2      NOT NULL DEFAULT SYSUTCDATETIME(),
    BlockID        INT            NOT NULL,

```

```

        StoredBy      NVARCHAR(128) NOT NULL DEFAULT SYSTEM_USER
    );

    CREATE TABLE dbo.LedgerVerificationLog (
        LogID          INT IDENTITY PRIMARY KEY,
        RunTime        DATETIME2      NOT NULL,
        Status          NVARCHAR(20) NOT NULL,  -- SUCCESS, FAILURE, WARNING
        Message         NVARCHAR(1000) NULL
    );
GO

```

Azure Defender for SQL

Azure Defender for SQL (now part of Microsoft Defender for Cloud) provides threat detection on top of the base security features — anomalous query patterns, SQL injection attempts, unusual data access volumes. Enable it at the server level and configure alert routing to your security team's channel, not just the DBA team:

```

# Enable Microsoft Defender for SQL on the server
az sql server microsoft-support-auditing-policy update \
    --resource-group "rg-prod-eastus" \
    --server "prod-sql-server" \
    --state Enabled

az security pricing create \
    --name "SqlServers" \
    --tier "Standard"

# Set security contact for Defender alerts
az security contact create \
    --name "security-team-contact" \
    --email "security-alerts@yourcompany.com" \
    --phone "+1-555-0100" \
    --alert-notifications "0n" \
    --alerts-to-admins "0n"

```

17.7 Migration Scenarios: Adding Security to Existing Databases

Enabling these features on a greenfield database is straightforward. Adding them to an existing production database — with live application traffic — requires careful sequencing.

Adding Always Encrypted to an Existing Table

The challenge: existing plaintext data must be encrypted in-place, or you must shadow-copy the data through an encryption pass. Azure SQL provides the `ALTER TABLE ... ALTER COLUMN` path with Always Encrypted, but it requires an offline migration window on older configurations, or can use the in-place encryption available with enclave-enabled Always Encrypted.

```
-- With enclave-enabled Always Encrypted, encrypt existing plaintext columns
-- WITHOUT taking the table offline (online encryption, available with VBS enclave)

-- First, verify the server has enclave enabled
SELECT
    SERVERPROPERTY('EnabledSecureEnclaveType') AS EnclaveType;
-- Should return: VBS

-- Verify the CEK is enclave-enabled
SELECT
    cek.name,
    cekv.enclave_computations_allowed
FROM sys.column_encryption_keys cek
JOIN sys.column_encryption_key_values cekv
    ON cek.column_encryption_key_id = cekv.column_encryption_key_id
WHERE cek.name = 'CEK_PII_Data';

-- Perform online encryption of an existing column
-- This requires an enclave-enabled connection (Enclave Attestation Url in
connection string)
ALTER TABLE dbo.CustomerPII
ALTER COLUMN SSN CHAR(11) COLLATE Latin1_General_BIN2
    ENCRYPTED WITH (
        COLUMN_ENCRYPTION_KEY = CEK_PII_Data,
        ENCRYPTION_TYPE = DETERMINISTIC,
        ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
    ) NOT NULL
WITH (ONLINE = ON);
GO
```

The `WITH (ONLINE = ON)` clause for Always Encrypted column encryption is only available with enclave-enabled configurations. Without it, the `ALTER COLUMN` operation takes a schema modification lock that blocks reads and writes for the duration of the re-encryption scan. On a table with tens of millions of rows, that lock duration makes the classic path impractical during business hours.

Converting a Regular Table to a Ledger Table

Once created without ledger enabled, a regular SQL table cannot be converted to a ledger table in-place. There is no `ALTER TABLE ... SET LEDGER = ON`. The migration path is:

1. Create the new ledger table with the same schema

2. Insert the existing data from the old table (this seeds the ledger with the historical state)
3. Swap application traffic to the new table
4. Optionally drop or archive the old table

```
-- Migration: regular AccountBalance to ledger AccountBalance
-- Step 1: Create ledger version alongside the existing table
CREATE TABLE dbo.AccountBalance_Ledger_New (
    AccountID          INT          NOT NULL,
    CustomerName       NVARCHAR(100) NOT NULL,
    Balance            DECIMAL(18,2) NOT NULL,
    LastModified       DATETIME2    NOT NULL DEFAULT SYSUTCDATETIME()
)
WITH (
    SYSTEM_VERSIONING = ON,
    LEDGER = ON
);
GO

-- Step 2: Migrate existing data in batches to avoid log pressure
DECLARE @BatchSize INT = 10000;
DECLARE @Offset INT = 0;
DECLARE @Rows INT = 1;

WHILE @Rows > 0
BEGIN
    INSERT INTO dbo.AccountBalance_Ledger_New
        (AccountID, CustomerName, Balance, LastModified)
    SELECT
        AccountID, CustomerName, Balance, LastModified
    FROM dbo.AccountBalance
    ORDER BY AccountID
    OFFSET @Offset ROWS
    FETCH NEXT @BatchSize ROWS ONLY;

    SET @Rows = @@ROWCOUNT;
    SET @Offset += @BatchSize;

    -- Brief pause to reduce I/O pressure during business hours
    WAITFOR DELAY '00:00:00.100';
END;
GO

-- Step 3: Rename tables in a single transaction to minimize cutover window
BEGIN TRANSACTION;
    EXEC sp_rename 'dbo.AccountBalance', 'dbo.AccountBalance_Old';
    EXEC sp_rename 'dbo.AccountBalance_Ledger_New', 'dbo.AccountBalance';
COMMIT;
GO
```


Note that the data inserted during migration appears in the ledger as a batch of INSERT operations performed by the migration login. The ledger will correctly show that all initial rows were written at migration time by a specific principal — which is accurate and acceptable for audit purposes. Document the migration event in your compliance records so auditors understand the ledger history starts at the migration timestamp.

Key Takeaways

- → **TDE encrypts data at rest and satisfies storage-layer compliance requirements, but it provides zero protection against a privileged database user or compromised credentials.** Use Customer-Managed Keys via Azure Key Vault when your compliance framework requires key ownership, and set up Key Vault access failure alerts before the first day of production traffic — losing access to the CMK makes the database completely inaccessible.
- → **Always Encrypted moves the encryption boundary to the client application, ensuring the database engine never sees plaintext on sensitive columns.** The operational constraints — no range queries on classic Always Encrypted, mandatory BIN2 collation on string columns, and size expansion of every encrypted value — must be evaluated against your query patterns before committing to the schema. Secure Enclaves lift the range query restriction at the cost of an attestation handshake on each new connection.
- → **Azure SQL Ledger provides cryptographic proof of data integrity, not data confidentiality.** A tamper-evident audit trail is only meaningful if the digest is stored in immutable external storage (Azure Blob WORM policy or Azure Confidential Ledger) that the database owner cannot modify — otherwise a sophisticated attacker who compromises the database can also reconstruct the digest chain.
- → **The three features protect different threat vectors and are intended to be layered, not chosen between.** TDE covers storage-layer attackers, Always Encrypted covers insider threats and credential compromise, and Ledger covers retrospective tampering with historical records. A production system facing multiple compliance requirements will frequently need all three active simultaneously.
- → **Migration to these features on existing production databases requires careful sequencing.** Always Encrypted column encryption without Secure Enclaves requires a schema modification lock that can block reads and writes for hours on large tables; enclave-enabled `ALTER COLUMN ... WITH (ONLINE = ON)` is the production-safe path. Ledger tables cannot be altered in-place from regular tables — plan for a create-migrate-rename cutover with batched data movement to control transaction log growth.

Chapter 18: Azure Arc-Enabled SQL Server: Hybrid Cloud

Azure Arc-Enabled SQL Server extends Azure's management plane to SQL Server instances running anywhere — on-premises data centers, colocation facilities, edge locations, or competing cloud providers. For DBAs managing hybrid environments, this changes the operational calculus: instead of maintaining separate toolchains for on-prem and cloud databases, you get unified monitoring, policy enforcement, security scanning, and licensing management through a single Azure portal and API surface. This chapter covers how Arc-Enabled SQL Server works architecturally, how to onboard existing SQL Server instances, how to configure Azure services like Microsoft Defender and Azure Monitor against non-Azure hosts, and how to use Arc as a bridge during phased cloud migrations rather than a permanent destination.

18.1 Understanding Azure Arc Architecture for SQL Server

Before Arc, hybrid database management meant running parallel monitoring stacks — perhaps SCOM or Datadog for on-premises SQL Server alongside Azure Monitor for Azure SQL. Security assessments ran separately. Licensing compliance was tracked in spreadsheets. Arc collapses this by projecting non-Azure resources into Azure Resource Manager (ARM) as first-class objects.

The architecture rests on two components: the **Connected Machine Agent** (the Arc agent installed on the host OS) and the **Arc SQL Server extension**. The Connected Machine agent establishes an outbound HTTPS connection to Azure's Arc service endpoints on port 443. No inbound firewall rules are required. The agent authenticates using a service principal or managed identity, receives its configuration from ARM, and reports telemetry back to Azure. The SQL Server extension layer sits on top, communicating with the local SQL Server instance via a Windows service (`SqlServerExtension`) or `systemd` service on Linux.

On-Premises Host

- └─ SQL Server 2019 (or 2022, 2017, 2016)
- └─ Arc Connected Machine Agent
 - └─ Heartbeat → Azure Arc Service Endpoints (TCP 443)
- └─ Arc SQL Server Extension
 - └─ Reads SQL metadata, license info, configuration
 - └─ Sends to Azure Monitor Log Analytics
 - └─ Applies policies from Azure Policy

From ARM's perspective, your on-premises SQL Server appears under `Microsoft.AzureArcData/sqlServerInstances` as a resource type. You can tag it, apply RBAC, include it in Azure Policy assignments, and view it in the Azure portal alongside Azure SQL Managed Instances and Azure SQL Databases. This ARM projection is what makes the rest of Arc's capabilities possible — every feature (Defender, Monitor, Best Practices Assessment) hooks into ARM resource identity.

What Arc does not do: It does not move your data. It does not replicate your databases to Azure. It does not change how SQL Server operates internally. The Arc agent has read-only access to SQL Server metadata; it cannot execute DML or DDL. DBAs coming from environments where any agent was treated as a security risk should understand this boundary clearly.

Network requirements differ from what DBAs typically manage. Arc agents must reach several Azure endpoints:

Endpoint	Purpose
---	---
<code>management.azure.com</code>	ARM operations
<code>login.microsoftonline.com</code>	Azure AD authentication
<code>*.his.arc.azure.com</code>	Hybrid Identity Service
<code>*.guestconfiguration.azure.com</code>	Policy guest configuration
<code>dc.services.visualstudio.com</code>	Telemetry

In air-gapped environments, Arc supports proxy server configuration — the agent can route all outbound traffic through an authenticated HTTP proxy. Some highly secure environments route Arc traffic through Azure Private Link by deploying a Private Link scope for Azure Monitor and Arc endpoints, though this requires significant network engineering.

Supported SQL Server versions span from SQL Server 2012 through 2022. Extended Security Updates (ESUs) through Arc are a significant licensing benefit for shops still running 2012 or 2014: Microsoft provides ESU patches free of charge for Arc-connected instances (subject to Software Assurance or Azure subscription conditions). For DBAs managing legacy infrastructure that cannot be migrated immediately, this alone often justifies the Arc onboarding effort.

18.2 Onboarding SQL Server Instances to Arc

Onboarding has three paths: single-machine (interactive), at-scale via script, and policy-driven. Interactive onboarding suits labs and initial testing. At-scale is what production environments use.

Single-machine onboarding using Azure CLI:

```
# Prerequisites: Azure CLI with arcdata extension
az extension add --name arcdata
az extension add --name connectedmachine

# Create a resource group for Arc resources
az group create \
  --name rg-arc-sqlserver-prod \
  --location eastus2

# Create a service principal for Arc onboarding
# Store credentials securely - this SP will be used by agents
az ad sp create-for-rbac \
  --name "sp-arc-onboarding" \
  --role "Azure Connected Machine Onboarding" \
  --scopes "/subscriptions/<subscription-id>/resourceGroups/rg-arc-sqlserver-prod"

# Output:
# {
#   "appId": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
#   "displayName": "sp-arc-onboarding",
#   "password": "xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx",
#   "tenant": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
# }

# Download the onboarding script from Azure portal or generate via CLI
az connectedmachine connect \
  --resource-group rg-arc-sqlserver-prod \
  --location eastus2 \
  --subscription <subscription-id> \
  --tenant-id <tenant-id> \
  --service-principal-id <appId> \
  --service-principal-secret <password>
# Run this command ON the target SQL Server host
```

For at-scale onboarding across dozens or hundreds of servers, the recommended approach is a PowerShell script deployed via configuration management (Ansible, Chef, SCCM, or GPO):

```

# Deploy-ArcAgent.ps1
# Run this on each SQL Server host via your configuration management tool

param(
    [Parameter(Mandatory=$true)]
    [string]$TenantId,

    [Parameter(Mandatory=$true)]
    [string]$SubscriptionId,

    [Parameter(Mandatory=$true)]
    [string]$ResourceGroup,

    [Parameter(Mandatory=$true)]
    [string]$Location,

    [Parameter(Mandatory=$true)]
    [string]$ServicePrincipalId,

    [Parameter(Mandatory=$true)]
    [string]$ServicePrincipalSecret,

    [string]$Environment = "Production",
    [string]$CostCenter = "DBA-Ops"
)

# Download the Arc agent installer
$AgentInstallerUrl = "https://aka.ms/AzureConnectedMachineAgent"
$InstallerPath = "$env:TEMP\AzureConnectedMachineAgent.msi"

Write-Host "Downloading Arc Connected Machine Agent..."
Invoke-WebRequest -Uri $AgentInstallerUrl -OutFile $InstallerPath

# Install silently
Write-Host "Installing Arc Connected Machine Agent..."
msiexec.exe /i $InstallerPath /l*v "$env:TEMP\arc_install.log" /qn

# Wait for service to start
Start-Sleep -Seconds 30

# Connect to Azure Arc
$ArcConnectArgs = @(
    "connect",
    "--tenant-id", $TenantId,
    "--subscription-id", $SubscriptionId,
    "--resource-group", $ResourceGroup,
    "--location", $Location,
    "--service-principal-id", $ServicePrincipalId,
    "--service-principal-secret", $ServicePrincipalSecret,
    "--tags",
    "Environment=$Environment,CostCenter=$CostCenter,OnboardedBy=AutoScript"
)

```

```
& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" @ArcConnectArgs

if ($LASTEXITCODE -eq 0) {
    Write-Host "Arc agent connected successfully for $env:COMPUTERNAME"

    # Install SQL Server Arc extension
    # This happens automatically if the agent detects SQL Server,
    # but we can trigger it explicitly
    $ExtensionArgs = @(
        "extensions", "install",
        "--name", "WindowsAgent.SqlServer",
        "--publisher", "Microsoft.AzureData",
        "--version", "latest"
    )
    & "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" @ExtensionArgs
} else {
    Write-Error "Arc agent connection failed for $env:COMPUTERNAME with exit code $LASTEXITCODE"
    exit 1
}
```

After onboarding, verify the Arc SQL Server resource exists:

```
# Verify Arc-connected SQL Server instances
az sql server-arc list \
  --resource-group rg-arc-sqlserver-prod \
  --output table

# Get details of a specific instance
az sql server-arc show \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod

# List all databases discovered on the Arc SQL Server
az sql db-arc list \
  --server-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --output table
```

License type configuration is one of the most operationally important settings. Arc tracks whether each instance uses License Included, Azure Hybrid Benefit, or Pay-As-You-Go licensing. Setting this incorrectly leads to billing surprises or compliance gaps:

```
# Set Azure Hybrid Benefit license type for an Arc SQL instance
# Use this if you have existing SQL Server licenses with Software Assurance
az sql server-arc update \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --license-type LicenseIncluded

# License type options:
# LicenseIncluded - You're paying for the license through Azure (ESU scenario)
# PAYG            - Pay-as-you-go (useful for dev/test)
# Paid            - Azure Hybrid Benefit (bring your own SA license)
```

Gotcha: SQL Server instances with multiple named instances on the same host appear as separate Arc SQL Server resources, but they share the same Connected Machine Agent. If the agent goes offline, all instances become unreachable in Arc simultaneously. Monitor agent health separately from SQL Server health.

Gotcha: Linux-hosted SQL Server (SQL Server on RHEL, Ubuntu, SLES) is fully supported by Arc, but the extension service name differs (`mssql-server-arc` via `systemd`). Automation scripts that assume Windows service names will fail silently on Linux hosts.

18.3 Azure Monitor and Log Analytics Integration

Once Arc is connected, Azure Monitor becomes the unified observability layer for both your on-premises SQL Server and Azure SQL services. The key component is the **Log Analytics agent** (or its newer replacement, **Azure Monitor Agent / AMA**), which ships SQL Server performance counters and event log data to a Log Analytics workspace.

Configure Log Analytics data collection:

```

# Create a Log Analytics workspace
az monitor log-analytics workspace create \
  --resource-group rg-arc-sqlserver-prod \
  --workspace-name law-sqlserver-hybrid \
  --location eastus2 \
  --sku PerGB2018 \
  --retention-time 90

# Get workspace ID and key for agent configuration
WORKSPACE_ID=$(az monitor log-analytics workspace show \
  --resource-group rg-arc-sqlserver-prod \
  --workspace-name law-sqlserver-hybrid \
  --query customerId -o tsv)

WORKSPACE_KEY=$(az monitor log-analytics workspace get-shared-keys \
  --resource-group rg-arc-sqlserver-prod \
  --workspace-name law-sqlserver-hybrid \
  --query primarySharedKey -o tsv)

echo "Workspace ID: $WORKSPACE_ID"
echo "Workspace Key: $WORKSPACE_KEY"

# Enable monitoring extension on Arc SQL Server
az connectedmachine extension create \
  --machine-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --name MMA \
  --publisher Microsoft.EnterpriseCloud.Monitoring \
  --type MicrosoftMonitoringAgent \
  --settings "{\"workspaceId\": \"$WORKSPACE_ID\"}" \
  --protected-settings "{\"workspaceKey\": \"$WORKSPACE_KEY\"}"

```

Once data flows into Log Analytics, you write KQL queries against it exactly as you would for Azure SQL — the workspace abstraction normalizes the data regardless of origin:


```

// KQL: Find longest-running queries on Arc-connected SQL instances
// Run this in Log Analytics workspace
AzureDiagnostics
| where ResourceProvider == "MICROSOFT.AZUREARCDATA"
| where Category == "SQLQueryStoreRuntime"
| where TimeGenerated > ago(1h)
| extend ExecutionDurationMs = todouble(max_elapsed_time_d) / 1000
| where ExecutionDurationMs > 5000
| project
    TimeGenerated,
    Resource,
    database_name_s,
    query_id_d,
    ExecutionDurationMs,
    total_worker_time_d,
    execution_count_d
| order by ExecutionDurationMs desc
| take 50

// KQL: Wait stats comparison across hybrid fleet
Perf
| where ObjectName == "SQLServer:Wait Statistics"
| where CounterName in ("Lock waits", "Log write waits", "Network IO waits")
| where TimeGenerated > ago(24h)
| summarize AvgWait = avg(CounterValue) by Computer, CounterName, bin(TimeGenerated, 1h)
| render timechart

// KQL: Identify Arc instances with high CPU over 80%
Perf
| where ObjectName == "Processor Information"
| where CounterName == "% Processor Time"
| where TimeGenerated > ago(1h)
| summarize AvgCPU = avg(CounterValue) by Computer, bin(TimeGenerated, 5m)
| where AvgCPU > 80
| join kind=inner (
    AzureDiagnostics
    | where ResourceProvider == "MICROSOFT.AZUREARCDATA"
    | distinct Computer = Resource
) on Computer
| project TimeGenerated, Computer, AvgCPU
| order by AvgCPU desc

```

Best Practices Assessment is an Arc-specific capability that runs scheduled SQL Server configuration assessments and reports findings to Log Analytics. It checks a broad set of rules covering security configuration, performance settings, availability configuration, and deprecated features — consult current Microsoft documentation for the exact rule count and categories, since these are added to over time. Unlike ad-hoc assessment tools, this runs on a schedule (weekly by default, configurable to daily) and trends results over time.

```
# Enable Best Practices Assessment on Arc SQL Server
az sql server-arc assessment create \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --workspace-id $WORKSPACE_ID \
  --schedule "0 2 * * 0" # Weekly at 2 AM Sunday (cron format)

# View assessment results via CLI
az sql server-arc assessment show \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod

# Assessment findings appear in Log Analytics under SqlAssessment_CL table
```

Query assessment findings programmatically:

```
// KQL: Critical and High severity assessment findings
SqlAssessment_CL
| where TimeGenerated > ago(7d)
| where Severity_s in ("Critical", "High")
| where IsRuleInWarningState_b == true
| project
  TimeGenerated,
  Computer,
  RuleName_s,
  Severity_s,
  Category_s,
  Description_s,
  Recommendation_s
| order by Severity_s asc, TimeGenerated desc
```

Alert rules for Arc SQL Server instances use the same Azure Monitor alert framework as Azure SQL. The difference is that metric sources reference `microsoft.azurearcdata/sqlserverinstances` rather than `microsoft.sql/servers`. Some metrics available for Azure SQL (like DTU consumption) have no Arc equivalent since Arc instances aren't DTU-based.

```
# Create an alert rule for high CPU on Arc SQL instances
az monitor metrics alert create \
  --name "ArcSQL-HighCPU-Alert" \
  --resource-group rg-arc-sqlserver-prod \
  --scopes "/subscriptions/<sub-id>/resourceGroups/rg-arc-sqlserver-prod/providers/Microsoft.AzureArcData/sqlServerInstances/SQLPROD01" \
  --condition "avg Processor/PercentProcessorTime > 85" \
  --window-size 5m \
  --evaluation-frequency 1m \
  --action-group "/subscriptions/<sub-id>/resourceGroups/rg-arc-sqlserver-prod/actionGroups/dba-oncall" \
  --description "CPU exceeded 85% on Arc-connected SQL Server SQLPROD01" \
  --severity 2
```

18.4 Microsoft Defender for SQL via Arc

Microsoft Defender for SQL provides vulnerability assessment and advanced threat protection for SQL Server instances. Without Arc, Defender for SQL was limited to Azure SQL services. Arc extends it to on-premises and multicloud SQL Server instances under the same policy and billing framework.

Enable Defender for SQL on Arc instances:

```
# Enable Defender for SQL at subscription level
# This applies to all Arc SQL instances in the subscription
az security pricing create \
  --name SqlServerVirtualMachines \
  --tier Standard

# Verify Defender is active on specific Arc instance
az security sql-vulnerability-assessment list \
  --server-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod

# Run an on-demand vulnerability assessment
az security sql-vulnerability-assessment scan \
  --server-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --database-name HRDB
```

Defender for SQL performs two distinct functions. **Vulnerability Assessment** scans SQL Server configuration against a database of known misconfigurations — things like `sa` account enabled, CLR enabled without justification, `xp_cmdshell` exposed, databases with `TRUSTWORTHY ON`, weak password policies, and over-privileged logins. It produces a baseline on first run and flags deviations from that baseline on subsequent scans.

Advanced Threat Protection analyzes SQL Server audit logs for behavioral anomalies: SQL injection patterns, unusual data export volumes, logins from unexpected locations, and brute-force login patterns. This requires SQL Server Audit to be configured and shipping logs to a storage account or Log Analytics workspace.

```

-- Configure SQL Server Audit for Defender ATP integration
-- Run on the on-premises Arc-connected SQL Server instance

-- Create server audit pointing to Log Analytics
-- For Arc instances, use FILE audit and let Arc extension ship logs
USE master;
GO

CREATE SERVER AUDIT [ArcDefenderAudit]
TO FILE (
    FILEPATH = 'D:\SQLAudit\',
    MAXSIZE = 100 MB,
    MAX_ROLLOVER_FILES = 50,
    RESERVE_DISK_SPACE = OFF
)
WITH (
    QUEUE_DELAY = 1000,
    ON_FAILURE = CONTINUE
);
GO

ALTER SERVER AUDIT [ArcDefenderAudit] WITH (STATE = ON);
GO

-- Create audit specification for Defender-relevant events
CREATE SERVER AUDIT SPECIFICATION [ArcDefenderAuditSpec]
FOR SERVER AUDIT [ArcDefenderAudit]
ADD (FAILED_LOGIN_GROUP),
ADD (SUCCESSFUL_LOGIN_GROUP),
ADD (LOGOUT_GROUP),
ADD (DATABASE_PERMISSION_CHANGE_GROUP),
ADD (SERVER_PERMISSION_CHANGE_GROUP),
ADD (DATABASE_ROLE_MEMBER_CHANGE_GROUP),
ADD (SERVER_ROLE_MEMBER_CHANGE_GROUP),
ADD (USER_CHANGE_PASSWORD_GROUP),
ADD (SCHEMA_OBJECT_ACCESS_GROUP)
WITH (STATE = ON);
GO

-- Verify audit is active
SELECT
    a.name AS audit_name,
    a.status_desc,
    a.type_desc,
    s.name AS spec_name,
    s.is_state_enabled
FROM sys.server_audits a
JOIN sys.server_audit_specifications s
    ON a.audit_guid = s.audit_guid
WHERE a.name = 'ArcDefenderAudit';

```

Defender for SQL alerts surface in Azure Security Center (now Microsoft Defender for Cloud) under the same dashboard as Azure SQL alerts. A DBA managing 200 on-premises instances previously had no centralized security alert surface for SQL injection attempts. With Arc, a brute-force attack against an on-premises SQL Server in a Chicago data center triggers the same Defender for Cloud alert workflow as an attack against an Azure SQL Database in East US.

Gotcha: Defender for SQL is billed per protected instance per month — check current Azure pricing for your region and tier before enabling it broadly. For shops with large SQL Server estates, enabling Defender at subscription scope without an instance count audit first produces unexpected billing. Run the count before enabling:

```
# Count Arc SQL instances before enabling Defender
az sql server-arc list \
  --subscription <subscription-id> \
  --query "length(@)" \
  --output tsv

# List instances with vCore count to estimate Defender cost
az sql server-arc list \
  --subscription <subscription-id> \
  --query "[].{Name:name, Location:location, vCores:properties.vCores, Edition:properties.edition}" \
  --output table
```

18.5 Azure Policy for Hybrid SQL Server Governance

Azure Policy becomes your configuration enforcement mechanism across the hybrid fleet. Where you previously used SQL Server Policy-Based Management (PBM) for on-premises enforcement and separate tooling for Azure SQL, Arc unifies both under Azure Policy.

Built-in Arc SQL Server policies cover:

- → SQL Server should have Azure Defender enabled
- → SQL Server should have transparent data encryption enabled
- → SQL Server should have the latest cumulative update installed
- → SQL Server should not use the sa account
- → Arc-enabled SQL Server instances should have Azure AD authentication configured

You assign these policies to management groups, subscriptions, or resource groups. Instances that drift from policy show as non-compliant in the Defender for Cloud compliance dashboard.

```

# Assign built-in policy: Defender for SQL should be enabled on Arc instances
POLICY_DEFINITION_ID=$(az policy definition list \
  --query "[?displayName=='Microsoft Defender for SQL should be enabled for
unprotected SQL Servers'].id" \
  --output tsv)

az policy assignment create \
  --name "require-defender-arcsql" \
  --display-name "Require Defender for Arc SQL Servers" \
  --policy $POLICY_DEFINITION_ID \
  --scope "/subscriptions/<subscription-id>/resourceGroups/rg-arc-sqlserver-prod" \
  --enforcement-mode Default

# Create custom policy: Ensure minimum SQL Server version
cat > custom-policy-sqlversion.json << 'EOF'
{
  "mode": "All",
  "policyRule": {
    "if": {
      "allOf": [
        {
          "field": "type",
          "equals": "Microsoft.AzureArcData/sqlServerInstances"
        },
        {
          "field": "Microsoft.AzureArcData/sqlServerInstances/version",
          "notIn": ["SQL Server 2019", "SQL Server 2022"]
        }
      ]
    },
    "then": {
      "effect": "audit"
    }
  },
  "parameters": {}
}
EOF

az policy definition create \
  --name "require-sql-2019-or-later" \
  --display-name "Arc SQL Server must be 2019 or later" \
  --description "Audits Arc-connected SQL Servers running versions prior to SQL
Server 2019" \
  --rules custom-policy-sqlversion.json \
  --mode All

# Assign the custom policy
az policy assignment create \
  --name "enforce-sql-version" \
  --policy "require-sql-2019-or-later" \
  --scope "/subscriptions/<subscription-id>"

```

Policy remediation tasks handle non-compliant resources automatically where possible. For example, a policy requiring TDE on all Arc SQLServer databases can include a `deployIfNotExists` effect that triggers a remediation task to enable TDE without manual DBA intervention:

```
# Create remediation task for non-compliant Arc SQL instances
az policy remediation create \
  --name "remediate-defender-arcsql" \
  --policy-assignment "require-defender-arcsql" \
  --resource-group rg-arc-sqlserver-prod \
  --resource-discovery-mode ReEvaluateCompliance

# Monitor remediation task progress
az policy remediation show \
  --name "remediate-defender-arcsql" \
  --resource-group rg-arc-sqlserver-prod \
  --query "{Status:properties.provisioningState,
Succeeded:properties.resourceCounts.compliant,
Failed:properties.resourceCounts.noncompliant}"

# List all non-compliant Arc SQL resources across subscription
az policy state list \
  --subscription <subscription-id> \
  --filter "complianceState eq 'NonCompliant' and resourceType eq
'Microsoft.AzureArcData/sqlServerInstances'" \
  --query "[].{Resource:resourceId, Policy:policyDefinitionName,
State:complianceState}" \
  --output table
```

Guest Configuration policies extend enforcement into the OS and SQL Server configuration layer. Unlike standard Azure Policy which validates ARM resource properties, Guest Configuration runs inside the machine and can audit or enforce settings like SQL Server service account configuration, SQL Server error log retention, and SQL Agent job configurations. These policies deploy a Guest Configuration agent to the Arc-connected machine and run DSC-based configuration checks.


```
# List available Guest Configuration policy definitions for SQL Server
az policy definition list \
  --query "[?contains(displayName, 'SQL') && policyType=='BuiltIn' && contains(metadata.category, 'Guest Configuration')].{Name:displayName, Id:name}" \
  --output table

# Assign Guest Configuration policy for SQL Server service account validation
az policy assignment create \
  --name "gc-sql-service-account" \
  --display-name "SQL Server services should not use SA account" \
  --policy "/providers/Microsoft.Authorization/policyDefinitions/<definition-id>" \
  --scope "/subscriptions/<subscription-id>/resourceGroups/rg-arc-sqlserver-prod" \
  --mi-system-assigned \
  --location eastus2

# Guest Configuration policies require a managed identity on the assignment
# Grant the assignment's managed identity the necessary role
ASSIGNMENT_PRINCIPAL=$(az policy assignment show \
  --name "gc-sql-service-account" \
  --scope "/subscriptions/<subscription-id>/resourceGroups/rg-arc-sqlserver-prod" \
  --query identity.principalId -o tsv)

az role assignment create \
  --assignee $ASSIGNMENT_PRINCIPAL \
  --role "Guest Configuration Resource Contributor" \
  --scope "/subscriptions/<subscription-id>/resourceGroups/rg-arc-sqlserver-prod"
```

18.6 Arc as a Migration Bridge

Arc is frequently positioned as a permanent hybrid management solution, but its most underappreciated use case is as a **migration bridge** — a way to bring Azure governance and tooling to an on-premises instance before migrating it to Azure SQL Managed Instance, Azure SQL Database, or SQL Server on Azure VMs. This section addresses that phased migration pattern.

The migration bridge pattern works in three phases:

Phase 1: Onboard		Phase 2: Assess		Phase 3: Migrate
Install Arc agent	→	Run BPA assessments	→	Execute migration
Connect to ARM		Evaluate Defender alerts		Decommission Arc resource
Establish monitoring		Size MI/DB targets		Retain audit history
Configure alerts		Address blockers		in Log Analytics

Azure Arc as a Migration Bridge

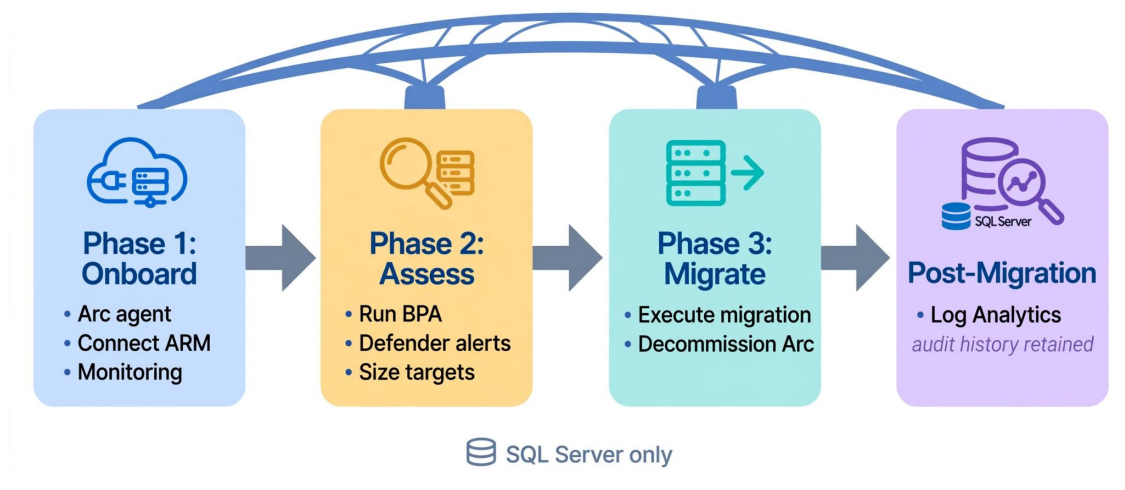


Figure 18.1 — Azure Arc as a Migration Bridge.

The bridge arc tells the story: Arc is a temporary crossing, not a destination. Phase 1 onboards the server with the Arc agent and ARM connection so monitoring flows; Phase 2 assesses it with Best Practices Analyzer runs, Defender alerts, and migration-target sizing; Phase 3 executes the migration and decommissions the Arc resource. Log Analytics keeps the audit history after the server itself is gone — the evidence outlives the bridge.

Arc used as a migration bridge: governance and assessment tooling attach to the on-premises instance before it is retired in favor of a native Azure SQL target.

During Phase 2, Arc provides two migration-critical capabilities: **Azure Migrate integration** and the **SQL Server assessment reports** that identify migration blockers.

```
# Create Azure Migrate project for hybrid SQL assessment
az migrate project create \
  --resource-group rg-arc-sqlserver-prod \
  --project-name migrate-sql-2024 \
  --location eastus2

# Arc-connected SQL Server instances appear automatically in Azure Migrate
# when the subscription and workspace are linked. View discovered instances:
az migrate machine list \
  --resource-group rg-arc-sqlserver-prod \
  --project-name migrate-sql-2024 \
  --output table

# The SQL Server migration extension provides target recommendation
az sql server-arc migration-assessment create \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --target-type ManagedInstance \
  --workspace-id $WORKSPACE_ID
```

The migration assessment identifies blockers such as linked servers (not supported in Azure SQL Database), cross-database queries (requires Managed Instance or Elastic Pools), CLR assemblies using unsafe or external access, distributed transactions, SQL Server Agent jobs using unsupported subsystems, and Windows Authentication without Azure AD federation configured.

Maintaining Arc connectivity during a rolling migration helps when you have multiple databases on a single instance and migrate them one at a time:

```

-- SQL Server: Track database migration status across hybrid fleet
-- Create a tracking table in a DBA management database (on-prem)
CREATE TABLE dba.MigrationTracker (
    InstanceName          NVARCHAR(128) NOT NULL,
    DatabaseName          NVARCHAR(128) NOT NULL,
    ArcResourceId          NVARCHAR(500) NULL,
    MigrationTarget        NVARCHAR(200) NULL, -- e.g., 'mi-prod.database.windows.net'
    MigrationStatus        NVARCHAR(50)  NOT NULL DEFAULT 'Pending',
        -- Pending, Assessing, InProgress, Cutover, Complete
    AssessmentDate         DATETIME2 NULL,
    MigrationStartDate     DATETIME2 NULL,
    CutoverDate            DATETIME2 NULL,
    BlockerCount           INT NULL,
    Notes                  NVARCHAR(MAX) NULL,
    ModifiedBy             NVARCHAR(128) NOT NULL DEFAULT SUSER_SNAME(),
    ModifiedDate           DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME(),
    CONSTRAINT PK_MigrationTracker PRIMARY KEY (InstanceName, DatabaseName)
);
GO

-- Populate with current databases
INSERT INTO dba.MigrationTracker (InstanceName, DatabaseName)
SELECT
    @@SERVERNAME,
    name
FROM sys.databases
WHERE name NOT IN ('master', 'model', 'msdb', 'tempdb')
AND state_desc = 'ONLINE';
GO

-- Query migration status
SELECT
    InstanceName,
    MigrationStatus,
    COUNT(*) AS DatabaseCount,
    MIN(AssessmentDate) AS EarliestAssessment,
    MAX(CutoverDate) AS LatestCutover
FROM dba.MigrationTracker
GROUP BY InstanceName, MigrationStatus
ORDER BY InstanceName, MigrationStatus;

```

After migration, clean up Arc resources for instances that have been fully migrated:

```
# Verify no remaining databases before disconnecting Arc
az sql db-arc list \
  --server-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --output table

# Disconnect the Arc SQL Server extension first
az connectedmachine extension delete \
  --machine-name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --name WindowsAgent.SqlServer \
  --yes

# Disconnect the machine from Arc
# (Run this on the host itself OR remotely via ARM)
az connectedmachine delete \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --yes

# Or from the host, run the agent disconnect command:
# azcmagent disconnect --service-principal-id <id> --service-principal-secret
# <secret>

# Retain Log Analytics data per your retention policy
# The workspace data persists independently of Arc resource deletion
```

Gotcha: Deleting the Arc connected machine resource in ARM does not uninstall the Arc agent from the host OS. The agent continues running but in a disconnected state. On decommissioned hosts this is harmless; on hosts that will be repurposed, uninstall the agent explicitly using `Add/Remove Programs` on Windows or the package manager on Linux before reprovisioning.

18.7 Azure AD Authentication for Arc SQL Server

One of the more transformative capabilities Arc enables for on-premises SQL Server is **Azure Active Directory (Azure AD) authentication**, which Microsoft now calls Microsoft Entra ID authentication. Traditionally, on-premises SQL Server relied on Windows Authentication against local Active Directory, or SQL authentication. Arc-enabled SQL Server 2022 can authenticate users directly against Azure AD without requiring Azure AD Connect or AD Federation Services — the Arc agent handles the token validation.

This matters for hybrid environments where:

- → Remote workers lack VPN access to on-premises AD but have Azure AD credentials

- → Azure-native applications need to connect to on-premises SQL Server using managed identities
- → You want unified audit trails across Azure SQL and on-premises SQL Server

Prerequisites for Azure AD auth on Arc SQL Server:

1. SQL Server 2022 (Azure AD auth is not available for earlier versions via Arc)
2. Arc Connected Machine Agent version 1.13 or later
3. Azure AD admin assigned to the SQL Server instance
4. Outbound connectivity to Azure AD endpoints

```
# Set Azure AD admin for Arc SQL Server instance
az sql server-arc ad-admin create \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod \
  --display-name "SQL Arc Admins" \
  --object-id <azure-ad-group-object-id> \
  --type Group

# Verify AD admin assignment
az sql server-arc ad-admin show \
  --name SQLPROD01 \
  --resource-group rg-arc-sqlserver-prod
```

Once configured, create Azure AD logins using T-SQL syntax:

```

-- SQL Server 2022: Create Azure AD logins on Arc-connected instance
-- Connect as sysadmin first

-- Create login for an Azure AD user
CREATE LOGIN [dbadmin@contoso.com]
FROM EXTERNAL PROVIDER;
GO

-- Create login for an Azure AD group
CREATE LOGIN [SQL-DBAs@contoso.com]
FROM EXTERNAL PROVIDER;
GO

-- Create login for an Azure managed identity
-- Useful when Azure Functions or App Service needs to connect to on-prem SQL via
Arc
CREATE LOGIN [my-azure-function-app]
FROM EXTERNAL PROVIDER;
GO

-- Grant server-level permissions
ALTER SERVER ROLE [sysadmin] ADD MEMBER [SQL-DBAs@contoso.com];
GO

-- Create database user mapped to the Azure AD login
USE [HRDB];
GO

CREATE USER [dbadmin@contoso.com] FROM LOGIN [dbadmin@contoso.com];
GO

ALTER ROLE [db_datareader] ADD MEMBER [dbadmin@contoso.com];
ALTER ROLE [db_datawriter] ADD MEMBER [dbadmin@contoso.com];
GO

-- Verify Azure AD logins
SELECT
    name,
    type_desc,
    authentication_type_desc,
    create_date,
    is_disabled
FROM sys.server_principals
WHERE type IN ('E', 'X') -- E = External User, X = External Group
ORDER BY create_date DESC;

```

Connecting to Arc SQL Server using Azure AD authentication:

```

# Python: Connect to Arc-enabled SQL Server using Azure AD token
# Uses azure-identity and pyodbc

import pyodbc
from azure.identity import DefaultAzureCredential
import struct

def get_azure_ad_token():
    """Acquire Azure AD token for SQL Server authentication."""
    credential = DefaultAzureCredential()
    # Resource ID for SQL Server
    token = credential.get_token("https://database.windows.net/.default")
    return token.token

def connect_arc_sql_server(server_name: str, database: str) -> pyodbc.Connection:
    """
    Connect to Arc-enabled SQL Server using Azure AD authentication.
    server_name: FQDN or IP of on-premises SQL Server
    database: Target database name
    """
    token = get_azure_ad_token()

    # Convert token to bytes for ODBC access token attribute
    token_bytes = token.encode('utf-16-le')
    token_struct = struct.pack(f'<I{len(token_bytes)}s',
                               len(token_bytes),
                               token_bytes)

    connection_string = (
        f"DRIVER={{ODBC Driver 18 for SQL Server}};"
        f"SERVER={server_name};"
        f"DATABASE={database};"
        f"Encrypt=yes;"
        f"TrustServerCertificate=no;"
    )

    conn = pyodbc.connect(
        connection_string,
        attrs_before={1256: token_struct} # SQL_COPT_SS_ACCESS_TOKEN
    )
    return conn

# Usage
conn = connect_arc_sql_server("sqlprod01.corp.contoso.com", "HRDB")
cursor = conn.cursor()
cursor.execute("SELECT SYSTEM_USER, USER_NAME(), @@SERVERNAME")
row = cursor.fetchone()
print(f"Logged in as: {row[0]}, User: {row[1]}, Server: {row[2]}")
conn.close()

```


Gotcha: Azure AD authentication on Arc SQL Server 2022 requires the SQL Server certificate to be trusted by clients. Self-signed certificates generated during SQL Server installation will cause connection failures with `TrustServerCertificate=no`. Either issue a certificate from an enterprise CA or update clients to trust the self-signed certificate during a transition period — but do not leave `TrustServerCertificate=yes` in production connection strings permanently.

18.8 Operational Runbooks for Arc-Managed SQL Server

Managing an Arc-connected SQL Server fleet requires operational procedures that account for both the Azure control plane and the on-premises SQL Server layer. This section documents runbook patterns for the most common operational scenarios.

Runbook: Arc agent health check and remediation

Arc agent connectivity issues are the most common operational problem. The agent can lose connectivity due to proxy changes, certificate expiration, service account password rotation, or network policy changes. Detect and remediate with this pattern:

```
#!/bin/bash
# arc-agent-health-check.sh
# Run from Azure Cloud Shell or from a jumpbox with Azure CLI access

RESOURCE_GROUP="rg-arc-sqlserver-prod"
SUBSCRIPTION_ID="<subscription-id>"

echo "=== Arc Agent Health Check: $(date) ==="

# List all Arc machines and their connectivity status
az connectedmachine list \
  --resource-group $RESOURCE_GROUP \
  --query "[].{Name:name, Status:properties.status,
LastHeartbeat:properties.lastStatusChange, AgentVersion:properties.agentVersion}" \
  --output table

# Find machines that haven't reported in over 30 minutes
STALE_THRESHOLD=$(date -u -d "30 minutes ago" '+%Y-%m-%dT%H:%M:%SZ')
echo ""
echo "=== Machines with stale heartbeat (> 30 min) ==="
az connectedmachine list \
  --resource-group $RESOURCE_GROUP \
  --query "[?properties.lastStatusChange < '$STALE_THRESHOLD'}.{Name:name,
LastHeartbeat:properties.lastStatusChange, Status:properties.status}" \
  --output table

# Check agent version - identify outdated agents
echo ""
echo "=== Agent versions in fleet ==="
az connectedmachine list \
  --resource-group $RESOURCE_GROUP \
  --query "[].{Name:name, AgentVersion:properties.agentVersion}" \
  --output table
```

On the host side, diagnose agent connectivity issues:

```
# Diagnose-ArcAgent.ps1
# Run on the Arc-connected host when connectivity issues are reported

# Check agent service status
Get-Service -Name "himds" | Select-Object Name, Status, StartType
Get-Service -Name "GCArcService" | Select-Object Name, Status, StartType

# Check agent connectivity
& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" check

# Common output codes:
# OK          - All endpoints reachable
# FAILED      - Cannot reach Azure endpoints (proxy/firewall issue)
# WARNING     - Partial connectivity

# View agent logs
Get-Content "C:\ProgramData\AzureConnectedMachineAgent\Log\himds.log" -Tail 100

# Test specific endpoint connectivity
$endpoints = @(
    "management.azure.com",
    "login.microsoftonline.com",
    "guestnotificationservice.azure.com"
)

foreach ($endpoint in $endpoints) {
    $result = Test-NetConnection -ComputerName $endpoint -Port 443 -WarningAction SilentlyContinue
    Write-Host "$endpoint : $(if($result.TcpTestSucceeded){'REACHABLE'} else{'UNREACHABLE'})"
}

# Restart agent if connectivity checks pass but agent shows disconnected
if ((& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" check) -match "OK") {
    Restart-Service -Name "himds" -Force
    Start-Sleep -Seconds 15
    Get-Service -Name "himds" | Select-Object Name, Status
}
```

Runbook: Bulk license type audit and correction

In large estates, license types accumulate errors over time as servers are onboarded by different teams. This runbook identifies and corrects inconsistencies:

```
#!/bin/bash
# audit-arc-license-types.sh

RESOURCE_GROUP="rg-arc-sqlserver-prod"

echo "=== Arc SQL License Type Audit ==="

# Export all instances with their current license types
az sql server-arc list \
  --resource-group $RESOURCE_GROUP \
  --query "[].{Name:name, LicenseType:properties.licenseType, Edition:properties.edition, Version:properties.version, vCores:properties.vCores}" \
  --output table

# Count by license type
echo ""
echo "=== License Type Summary ==="
az sql server-arc list \
  --resource-group $RESOURCE_GROUP \
  --query "[].properties.licenseType" \
  --output tsv | sort | uniq -c | sort -rn

# Identify PAYG instances that should be Azure Hybrid Benefit
# (instances with 'Production' tag but PAYG license type - common misconfiguration)
echo ""
echo "=== Potential AHB Candidates (Production tagged, PAYG licensed) ==="
az sql server-arc list \
  --resource-group $RESOURCE_GROUP \
  --query "[?tags.Environment=='Production' && properties.licenseType=='PAYG']. {Name:name, LicenseType:properties.licenseType}" \
  --output table

# Correct license type for specific instances (requires approval workflow in production)
# Example: Update a single instance to LicenseIncluded (ESU scenario)
# az sql server-arc update \
#   --name SQLLEGACY01 \
#   --resource-group $RESOURCE_GROUP \
#   --license-type LicenseIncluded
```

Runbook: Forced Best Practices Assessment and report export

Before a major SQL Server upgrade or migration, trigger an on-demand BPA run and export results:

```
# Trigger immediate BPA assessment on all Arc SQL instances in resource group
for instance in $(az sql server-arc list \
  --resource-group rg-arc-sqlserver-prod \
  --query "[].name" --output tsv); do

  echo "Triggering assessment for $instance..."
  az sql server-arc assessment create \
    --name $instance \
    --resource-group rg-arc-sqlserver-prod \
    --workspace-id $WORKSPACE_ID

  echo "Assessment triggered for $instance"
done
```

Then extract results from Log Analytics:

```
// KQL: Export full BPA findings for pre-migration report
SqlAssessment_CL
| where TimeGenerated > ago(24h)
| where IsRuleInWarningState_b == true
| project
  AssessmentTimestamp = TimeGenerated,
  ServerName = Computer,
  Category = Category_s,
  Severity = Severity_s,
  RuleId = RuleId_s,
  RuleName = RuleName_s,
  Description = Description_s,
  Recommendation = Recommendation_s,
  AffectedObjects = AffectedObjects_s
| order by Severity asc, Category asc, ServerName asc
```

18.9 Limitations and When Not to Use Arc

Arc solves real problems, but it is not the right tool in every scenario. Understanding the limitations prevents over-engineering.

Arc does not provide real-time query performance insights equivalent to Query Store UI in SSMS. The data that flows to Log Analytics has ingestion latency — typically two to five minutes, sometimes longer under high telemetry load. For active performance troubleshooting of an in-flight incident, DBAs still connect directly to the SQL Server instance using SSMS, Azure Data Studio, or direct T-SQL queries against DMVs. Arc is a fleet management and governance layer, not a low-latency diagnostic tool.

Arc agent overhead is minimal but not zero. The Connected Machine Agent and SQL Server extension consume approximately 200-400 MB of memory on the host and generate periodic CPU spikes during telemetry collection windows. On memory-constrained legacy servers (32 GB or less) running large SQL workloads, this overhead should be evaluated before mass rollout.

Connectivity dependency is real. If an Arc-connected instance loses outbound internet access, it continues running SQL Server normally — Arc has no operational dependency on Azure connectivity. However, policy evaluations stop updating, Defender alerts stop flowing, monitoring gaps appear in Log Analytics, and BPA assessments fail silently. Design monitoring for Arc agent connectivity as a separate health signal.

Arc is not a replication or high availability solution. It provides no data-layer capabilities. DBAs who encounter Arc for the first time sometimes conflate it with Azure SQL Managed Instance Link (which does provide replication). Arc manages metadata; Managed Instance Link synchronizes data. These are complementary, not competing.

Sovereign cloud and regulatory constraints affect Arc deployments. Arc endpoints route through Azure's commercial cloud by default. Organizations subject to data sovereignty regulations in jurisdictions where Azure Government, Azure China, or Azure Germany regions apply must configure Arc to use the appropriate sovereign cloud endpoints — and some highly regulated industries prohibit any management-plane data leaving specific geographic boundaries, which Arc's current architecture does not accommodate.

```
# Configure Arc for Azure Government cloud
# (Must be done before connecting the agent)
& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" config set cloud
"AzureUSGovernment"

# Or for Azure China
& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" config set cloud
"AzureChinaCloud"

# Verify cloud configuration
& "C:\Program Files\AzureConnectedMachineAgent\azcmagent.exe" config list
```

Key Takeaways

- → **Arc projects on-premises SQL Server into Azure Resource Manager as a first-class resource**, enabling unified RBAC, tagging, policy enforcement, and monitoring through the same Azure portal and API surface used for Azure SQL services — without moving any data or changing SQL Server's operational behavior.

- → **Onboarding uses an outbound-only HTTPS connection** via the Connected Machine Agent; no inbound firewall rules are required. At-scale onboarding through configuration management tools (Ansible, SCCM, GPO) is the production approach, and license type configuration at onboarding time prevents billing discrepancies.
- → **Azure Monitor, Log Analytics KQL queries, Defender for SQL, and Best Practices Assessment** all extend to Arc-connected instances under the same frameworks used for Azure SQL, eliminating parallel monitoring and security toolchains for hybrid fleets — though Arc telemetry has ingestion latency unsuitable for real-time incident diagnostics.
- → **Arc serves as a migration bridge**, not just a permanent hybrid solution: its integration with Azure Migrate and SQL Server assessment reporting surfaces migration blockers, tracks per-database migration status, and preserves historical audit data in Log Analytics that persists after the Arc resource is decommissioned.
- → **Azure AD authentication for Arc SQL Server 2022 enables token-based authentication** from Azure-native workloads and remote users without AD Federation Services or Azure AD Connect, but requires properly trusted certificates on the SQL Server host — self-signed certificates must be addressed before enabling this capability in production.

Chapter 19: Running SQL Server on Google Cloud: Cloud SQL for SQL Server and GCE

AWS and Azure dominate the SQL Server cloud conversation, but Google Cloud is a legitimate third option — especially for organizations already committed to GCP for analytics, Kubernetes, or data engineering. SQL Server on Google Cloud comes in two flavors: **Cloud SQL for SQL Server** (the managed service) and **SQL Server on Compute Engine** (self-managed VMs, the EC2/Azure-VM analog). This chapter covers both, the licensing mechanics unique to GCP, and when GCP is the right cloud for a SQL Server workload at all.

19.1 Cloud SQL for SQL Server: The Managed Option

Cloud SQL for SQL Server is GCP's analog to RDS: automated backups, point-in-time recovery, patching, and high availability via regional persistent disks with automatic failover. The supported versions track the current SQL Server releases (2019, 2022, and newer as Google adds them), in Web, Standard, and Enterprise editions. Key characteristics SQL Server DBAs should know:

- → **HA via regional disks, not AGs.** Like RDS Multi-AZ, Cloud SQL HA keeps a standby in another zone with synchronous disk-level replication and automatic failover — you do not get readable secondaries or AG-level control. If you need readable secondaries on GCP, you need Compute Engine.
- → **No SQL Server Agent on some tiers.** Agent availability and capability on Cloud SQL is limited compared to self-managed — check the current feature matrix for job scheduling, and plan on Cloud Scheduler + Cloud Functions or Workflows for orchestration (the Chapter 12 patterns, GCP-flavored).
- → **Connectivity via private IP and the Auth Proxy.** The Cloud SQL Auth Proxy provides IAM-based, encrypted connectivity without exposing the instance publicly — the GCP analog to RDS Proxy's connection management and Azure Private Link. Private IP on your VPC is the production standard.
- → **Storage autoscaling.** Cloud SQL can grow storage automatically as data grows (up to configured limits), which removes the manual storage-scaling runbook — at the cost of storage that only grows, never shrinks.

19.2 Licensing on GCP

Cloud SQL for SQL Server is **license-included**: the per-hour price bundles the SQL Server license, with edition (Web/Standard/Enterprise) as a pricing dimension. For Compute Engine, GCP supports both license-included VM images and **bring-your-own-license** via sole-tenant nodes (dedicated hardware, which satisfies license-mobility hardware-dedication requirements). The cost math mirrors Chapter 8's EC2 analysis: steady-state production favors BYOL on sole-tenant nodes or committed-use discounts; variable workloads favor license-included. Get the licensing position documented — GCP's sole-tenant mechanics differ from AWS dedicated hosts, and the compliance details matter.

19.3 SQL Server on Compute Engine: Self-Managed on GCP

Compute Engine VMs with SQL Server preinstalled images are the GCP path to full engine control: Always On availability groups across zones, readable secondaries, SSIS/SSAS/SSRS, and the exact SQL Server version and CU you need. The implementation notes:

- → **AG listeners and GCP load balancing.** Like Azure's internal load balancer pattern (Chapter 13), cross-subnet AG listeners on GCP sit behind an internal TCP/UDP load balancer with a health check that only the primary passes. Test listener failover explicitly.
- → **Persistent Disk for data and log.** Separate PD volumes for data, log, and tempdb; regional PD for synchronous cross-zone replication of the volumes themselves. PD performance scales with size and type (balanced vs SSD) — provision for throughput, not just capacity.
- → **Backup to Cloud Storage.** Native SQL Server backups to Cloud Storage buckets (via the backup-to-URL pattern with GCP's storage), with lifecycle policies for retention and cross-region copies for DR.
- → **Live migration.** GCP's transparent maintenance (live migration of VMs during host maintenance) reduces — but does not eliminate — the patching burden. SQL Server patching itself remains yours.

19.4 When GCP Is the Right Cloud for SQL Server

Choose GCP for SQL Server when the organization is already GCP-committed: BigQuery analytics consuming SQL Server data (Chapter 22's offload pattern is native here), GKE-hosted applications needing a nearby database, or data engineering pipelines built on GCP services. The integration advantages are real: private connectivity between Cloud SQL and BigQuery, unified IAM, and one bill. Do not choose GCP for SQL Server in isolation when the estate is otherwise AWS/Azure — the third cloud adds operational surface for no

compensating advantage. And be candid about the ecosystem gap: the SQL Server community, tooling, and third-party integrations are thinnest on GCP, so budget extra validation time for anything beyond the mainstream path.

19.5 Migrating SQL Server to GCP

The migration paths from Chapters 33–34 apply with GCP tooling. **Database Migration Service (DMS)** supports SQL Server as a source for homogeneous migrations into Cloud SQL for SQL Server, handling the initial load and continuous replication with minimal downtime. For Compute Engine targets, the universal method works: native backup to Cloud Storage, restore on the GCE VM, then transactional replication or log shipping to catch up before cutover. For large databases, seed the target with a backup taken as close to cutover as possible — the initial DMS load over the network is the long pole, and a backup-seeded start shortens it dramatically.

Watch the conversion gotchas: SQL Server Agent jobs must be rebuilt (Cloud SQL's Agent limitations may force the Cloud Scheduler pattern from Chapter 12); linked servers to on-premises systems need Private Service Connect or VPN connectivity validated *before* cutover; and CLR assemblies, if you use them, must be re-certified on the target version. Run the migrated workload in shadow or read-only mode for a full business cycle before cutover — the migration that "worked" in testing but never saw month-end close is the one that fails on the 31st.

19.6 Cost: Committed Use and Pricing Gotchas

GCP's pricing levers for SQL Server: **committed use discounts** (1- or 3-year commitments on Cloud SQL and Compute Engine, up to roughly half off on-demand) for steady-state workloads, and **sustained use discounts** that apply automatically to Compute Engine VMs running most of the month. The gotchas: Cloud SQL for SQL Server Enterprise edition pricing multiplies quickly — verify the edition is actually needed (most workloads run fine on Standard); storage autoscaling only grows, so a one-time data load permanently raises the storage bill unless you provision a fresh instance; and cross-region egress for DR copies or BigQuery exports is metered — model it for chatty replication topologies. As with every cloud, the cheapest SQL Server on GCP is the one sized from measurements (Chapter 39) with commitments on the stable baseline and on-demand only for the variable margin.

19.7 Cross-Cloud Consistency

If you run SQL Server on multiple clouds, GCP participates in the same patterns as the rest of the book: native backups to object storage as the portable artifact, transactional replication or AGs for cross-cloud data movement where latency allows, and the monitoring/backup/security baselines from Chapters 29–32 applied uniformly. The goal is one operational model with cloud-specific implementations — not three different ways of running SQL Server.

Key Takeaways

- → **GCP offers both managed and self-managed SQL Server.** Cloud SQL for SQL Server for the RDS-like experience (HA via regional disks, no readable secondaries); Compute Engine for full control with AGs across zones.
- → **Licensing: license-included on Cloud SQL, BYOL via sole-tenant nodes on GCE.** Document the compliance position — GCP's dedication mechanics differ from AWS and Azure.
- → **AGs on GCE need the load-balancer listener pattern.** Same architecture as Azure's ILB approach: health check that only the primary passes, explicit listener failover testing.
- → **Choose GCP for SQL Server when the organization is GCP-committed.** BigQuery integration, GKE proximity, and unified IAM are the reasons — not the SQL Server experience in isolation.
- → **One operational model, cloud-specific implementations.** Backups to object storage, standard monitoring and security baselines — applied uniformly across every cloud you run.

Chapter 20: SQL Server on Kubernetes in the Cloud

Databases on Kubernetes used to be a contrarian idea. It is now a mainstream pattern for specific workloads: dev/test environments that spin up in seconds, CI pipelines that need disposable databases, edge and multi-cloud deployments where Kubernetes is the common substrate, and organizations standardizing every workload on one orchestrator. SQL Server runs well in containers — and Kubernetes adds scheduling, self-healing, and declarative configuration. This chapter covers when SQL Server on Kubernetes makes sense, the storage and availability architecture, and the operational realities.

20.1 When Kubernetes Fits (and When It Does Not)

Kubernetes is the right home for SQL Server when: environments are ephemeral (dev, test, CI — a database per pull request, torn down after merge); you need identical database environments across clouds and on-premises (the container image is the contract); or your platform team already operates Kubernetes and absorbing the database into the same GitOps pipeline reduces organizational friction. It is the *wrong* home when: you need the highest availability with the least operational effort (RDS, Azure SQL, and Cloud SQL all beat a hand-built Kubernetes HA setup); your team does not already operate Kubernetes (learning Kubernetes *and* stateful SQL Server simultaneously is a recipe for incidents); or the workload needs every last IOPS (the container and storage layers add overhead versus bare VMs).

20.2 The Architecture: StatefulSets, Storage, and Availability

SQL Server on Kubernetes runs as a **StatefulSet** — the Kubernetes primitive for stateful workloads with stable identities and persistent storage. Each SQL Server pod gets a **PersistentVolumeClaim** backed by the cloud's block storage (EBS via the EBS CSI driver, Azure Disk, GCP Persistent Disk). The critical settings:

- → **One SQL Server per pod, one pod per node (anti-affinity).** For availability groups across pods, spread replicas across nodes and zones with pod anti-affinity rules. Co-located replicas on one node share fate — the opposite of availability.
- → **Storage class matters.** Use the cloud's SSD-backed storage class (gp3/io2 on AWS, Premium SSD/Ultra on Azure) with appropriate IOPS provisioning. The database files live on the persistent volume; the container image holds only the binaries.

- → **AGs in Kubernetes are real but manual.** Always On availability groups across SQL Server pods work — the AG listener becomes a Kubernetes Service — but failover orchestration, quorum, and health probing are your responsibility. For most teams, a single SQL Server pod with fast storage-level recovery (pod reschedules, volume reattaches) plus solid backups is the pragmatic architecture; multi-pod AGs are for teams with Kubernetes depth.

```
# SQL Server as a StatefulSet (dev/test pattern)
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mssql-dev
spec:
  serviceName: mssql-dev
  replicas: 1
  selector:
    matchLabels: { app: mssql-dev }
  template:
    metadata:
      labels: { app: mssql-dev }
    spec:
      containers:
        - name: mssql
          image: mcr.microsoft.com/mssql/server:2022-latest
          env:
            - name: ACCEPT_EULA
              value: "Y"
            - name: MSSQL_SA_PASSWORD
              valueFrom:
                secretKeyRef: { name: mssql-secrets, key: sa-password }
          ports:
            - containerPort: 1433
          volumeMounts:
            - name: mssql-data
              mountPath: /var/opt/mssql
      volumeClaimTemplates:
        - metadata:
            name: mssql-data
          spec:
            accessModes: ["ReadWriteOnce"]
            storageClassName: fast-ssd
            resources:
              requests: { storage: 200Gi }
```

20.3 Configuration, Secrets, and GitOps

The Kubernetes-native way to manage SQL Server configuration is declarative: environment variables for server settings (`MSSQL_` prefixed variables cover the common ones), **Secrets** for passwords and certificates (backed by the cloud KMS via external-secrets or the CSI secrets driver — never plaintext in manifests), and **ConfigMaps** for initialization scripts. Manage the

manifests in Git with a GitOps tool (Argo CD, Flux): the database deployment becomes a pull request like any other. Initialization (creating databases, logins, Agent jobs) runs via init containers or post-start hooks executing versioned T-SQL scripts — the same scripts Chapter 12's automation would run, now triggered by the orchestrator.

20.4 Backup, Monitoring, and Day-2 Operations

Kubernetes does not change the fundamentals — it changes the plumbing:

- → **Backups:** SQL Server Agent inside the container (or a sidecar/CronJob) takes native backups to object storage. Volume snapshots of the persistent volume give you crash-consistent copies — fast for dev/test refresh, not a substitute for real backups in production.
- → **Monitoring:** expose SQL Server metrics via a Prometheus exporter sidecar or the built-in telemetry, scraped into the cluster's monitoring stack. Logs go to stdout and into the cluster log aggregator — no more RDP-to-check-the-error-log.
- → **Upgrades:** change the image tag, roll the StatefulSet. Test the upgrade against a copy of production data first — the container makes rollback trivial (previous image tag), which is a genuine operational advantage.
- → **Resource limits:** set CPU and memory requests/limits deliberately. SQL Server's memory model (max server memory) must fit inside the container's memory limit with headroom for the OS — an OOM-killed SQL Server pod is an avoidable outage.

20.5 Networking: Exposing SQL Server from Kubernetes

Clients reach SQL Server through a Kubernetes **Service**. For in-cluster clients, a ClusterIP service is enough. For external clients, a cloud load balancer service (type LoadBalancer) provisions a private IP on your VPC — keep it internal; a SQL Server exposed to the internet via a public load balancer is an incident waiting to happen. For AG topologies, the availability group listener becomes a Service whose endpoints track the primary replica — either via the operator managing endpoint updates or via an external-dns style mechanism. Whichever pattern you choose, verify connection behavior during pod rescheduling: clients must reconnect cleanly when the Service endpoint moves, which means the retry logic from Chapter 9 applies inside Kubernetes too.

20.6 Operators and Managed Kubernetes Database Solutions

Hand-rolled StatefulSets are the starting point, not the ceiling. **Kubernetes operators** for SQL Server encode the operational knowledge — automated failover, backup scheduling, certificate management — into a controller that watches your custom resources. Third-party operators and storage platforms (Portworx-style replicated storage, commercial SQL Server Kubernetes offerings) add synchronous storage replication and simpler multi-pod HA.

Evaluate operators the way you evaluate any managed service: who handles the 3 AM failover, what the backup story is, and whether the abstraction leaks at the worst moment. An operator you do not understand is just a more complicated StatefulSet.

20.7 Production Readiness Checklist

Before running production SQL Server on Kubernetes: pod disruption budgets (so node maintenance does not evict your database unexpectedly); anti-affinity across zones for multi-pod topologies; tested backup restores from object storage (not just volume snapshots); secrets rotation that does not require pod restarts mid-transaction; resource quotas so a noisy neighbor pod cannot starve the database; and a runbook for the scenario every Kubernetes DBA eventually faces — the persistent volume that will not reattach. If that list sounds like significant engineering, it is: production databases on Kubernetes are a platform investment, not a shortcut.

Key Takeaways

- → **Kubernetes fits ephemeral and standardized workloads.** Dev/test per pull request, CI databases, and multi-cloud consistency are the wins. It is not the low-effort HA path — managed services win that comparison.
- → **StatefulSet + persistent volumes + anti-affinity.** One SQL Server per pod, SSD-backed storage classes, replicas spread across nodes and zones. AGs across pods work but are operationally demanding.
- → **Manage it declaratively.** Secrets in KMS-backed secret stores, configuration in Git, GitOps deployment, versioned T-SQL init scripts. The database becomes a pull request.
- → **Day-2 operations still apply.** Real backups to object storage (snapshots are not backups), Prometheus-sidecar monitoring, image-tag upgrades with tested rollback, and memory limits that respect max server memory.
- → **Production on Kubernetes is a platform investment.** Pod disruption budgets, zone anti-affinity, quota isolation, and a volume-reattach runbook — budget the engineering or stay on managed services.

Chapter 21: Distributed SQL Server: Replication and Scale-Out Topologies

Not every scaling problem is solved by a bigger VM. When the workload outgrows a single primary — reporting queries starving OLTP, geographically distributed users needing local reads, or data that must flow between systems — SQL Server's replication and scale-out topologies come into play. This chapter maps the options (readable AG secondaries, transactional replication, distributed AGs, and sharding), their cloud-specific considerations, and how to choose without building a distributed system you cannot operate.

21.1 Readable Secondaries: The First Scale-Out Step

The simplest scale-out is routing read traffic to asynchronous AG secondaries. On EC2, Azure VMs, and GCE you control the full topology: add async replicas in the same region (or cross-region for geographic reads), mark them readable, and split connection strings by workload — `ApplicationIntent=ReadOnly` sends reporting traffic to secondaries while OLTP stays on the primary. On Azure SQL, the Business Critical tier and Hyperscale provide readable secondaries as a platform feature; on RDS, SQL Server has no read replicas, so this topology requires EC2.

The operational rules: monitor replication lag per secondary (a reporting query against a 10-minute-stale replica is a correctness bug, not a performance feature); size secondaries for the read workload, not as clones of the primary; and remember that readable secondaries still need maintenance (statistics, index maintenance) — they are databases, not free copies. When lag exceeds the application's tolerance, the answer is usually a bigger secondary or less write throughput, not a different topology.

21.2 Transactional Replication: Data Flow Between Systems

Transactional Replication Flow

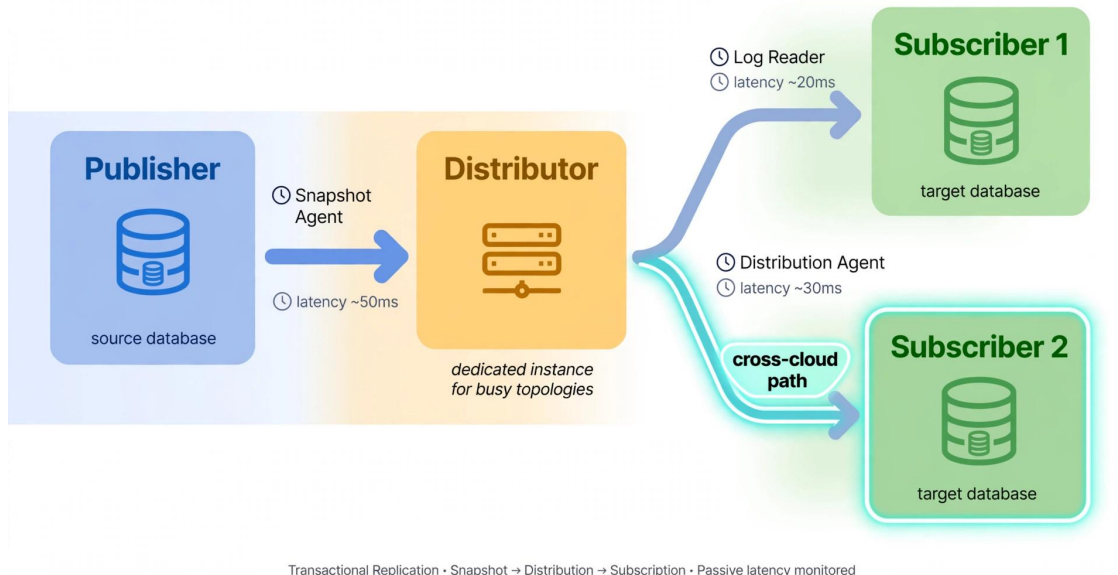


Figure 21.1 — Transactional Replication Data Flow. The agents are the topology: each arrow is a SQL Agent job with a latency budget.

Follow the arrows: the Snapshot Agent seeds the subscriber, then the Log Reader feeds the distributor and the Distribution Agent pushes changes to subscribers — each hop annotated with its latency contribution, because in replication the agents *are* the topology. The distributor gets its own box for a reason: on busy topologies it lives on a dedicated instance, never co-located with a struggling publisher. And the highlighted cross-cloud path is the lowest-drama way to move SQL Server data between clouds — publish on AWS, subscribe on Azure, and the heterogeneous topology just works.

Transactional replication publishes committed transactions from a publisher to subscribers with low latency — the classic pattern for feeding reporting databases, data warehouses, or downstream systems without touching the primary's read capacity. In the cloud, replication works on EC2/Azure VMs/GCE (full control of distributor, agents, and topology) and is limited or unavailable on managed services (RDS supports it with restrictions; Azure SQL Database supports it as a subscriber in some configurations — verify current support before designing around it).

Cloud-specific considerations: the distributor belongs on a dedicated instance for busy topologies (never co-locate the distributor with a struggling publisher); agent jobs become the Chapter 12 automation problem on managed platforms; network latency between publisher and subscriber directly bounds throughput — keep them in the same region unless

the use case tolerates lag; and monitor undistributed commands and replication latency as first-class alerts. Replication is also the lowest-drama cross-cloud data movement for SQL Server: publish on AWS, subscribe on Azure, and the heterogeneous topology just works.

21.3 Distributed Availability Groups: Cross-Site and Cross-Cloud

Distributed AGs (Chapter 16's DR mechanism) are also a scale-out mechanism: each participating AG has its own primary, and applications can write to the local primary in each site. This is genuine multi-site write scaling — with the fundamental constraint that the same data cannot be written in two sites simultaneously without conflicts. Use distributed AGs for: DR with a runnable secondary site (the primary use case), geographic read locality (each site serves local reads from its primary), and migrations (the old and new environments as the two AGs, with a controlled cutover). Do not use them for sharding or for write-scaling a single logical dataset — that way lies conflict resolution, and SQL Server has no built-in conflict resolver worth relying on.

21.4 Sharding: The Last Resort

When a single database genuinely cannot hold the workload — tens of terabytes with write throughput beyond one primary — sharding splits the data across multiple databases by a shard key (customer ID, region, tenant). SQL Server has no built-in auto-sharding; you build it with application-level routing, elastic database tools (Azure's shard map manager), or a proxy layer. The costs are permanent: cross-shard queries become application problems, transactions cannot span shards (design for shard-local transactions or sagas), schema changes roll out across every shard (elastic jobs at fleet scale, Chapter 14's discipline), and rebalancing a hot shard is a data migration project. Shard only when you have proven — with measurements, not projections — that vertical scaling and read scale-out are exhausted. Most teams that think they need sharding need a bigger primary and readable secondaries.

21.5 The Physics: Consistency, Latency, and Distance

Every distributed topology trades consistency against latency and availability — the physics does not negotiate. **Synchronous replication** (sync AG replicas) guarantees zero data loss on failover but makes every commit wait for the network round trip: within a region or AZ this costs single-digit milliseconds; across regions it costs tens to hundreds of milliseconds per commit, which throttles write throughput directly. **Asynchronous replication** keeps the primary fast but admits lag — seconds under normal conditions, minutes or more when the network or secondary falls behind.

The practical rules: keep synchronous replicas within one region (latency-bounded); use async for anything cross-region; define the application's maximum tolerable staleness in writing and alert on it; and never let a sync secondary sit across a WAN link "temporarily" — temporary becomes permanent, and the latency tax becomes the new normal. When someone proposes synchronous cross-region replication for zero RPO, make them price the commit-latency impact on the workload first. Most businesses choose async with eyes open once they see the numbers.

21.6 Monitoring a Distributed Topology

A distributed topology multiplies the things that can silently degrade. The monitoring baseline: replication/AG lag per secondary (alert on thresholds tied to application tolerance, not round numbers); undistributed commands and replication agent job health; distributor database size and throughput; listener/DNS endpoint resolution from each client region; and per-site resource utilization so a secondary does not become the bottleneck that back-pressures the primary. Build one dashboard per topology showing data flow end to end — publisher to distributor to subscriber, primary to each secondary — because the failure you will debug at 3 AM is always "which link in the chain is behind," and the dashboard should answer it in one glance.

21.7 Choosing: A Decision Framework

Need	Topology	Cloud notes
Offload reporting reads	Readable AG secondaries	Needs EC2/VMs/GCE; RDS SQL Server has no read replicas
Feed warehouse/downstream	Transactional replication	Full control on VMs; limited on managed services
Multi-site writes/DR	Distributed AGs	EC2/VMs/GCE only; no cross-site write conflicts allowed
Beyond one primary's capacity	Sharding	Application-built; last resort after measured proof

Key Takeaways

- → **Scale out in order of complexity.** Readable secondaries first, transactional replication for data flow, distributed AGs for multi-site, sharding last. Each step up multiplies operational cost permanently.

- → **Readable secondaries need self-managed infrastructure on AWS/GCP.** RDS for SQL Server has no read replicas; the readable-secondary topology requires EC2, Azure VMs, or GCE (or Azure SQL's built-in secondaries).
- → **Replication lag is a correctness parameter.** Monitor it per subscriber/secondary, define the application's tolerance, and size the receiving side for its workload — secondaries are databases, not free copies.
- → **Distributed AGs are for sites, not shards.** Multi-site DR, geographic reads, migrations — yes. Write-scaling one dataset across sites — no; SQL Server will not resolve your conflicts.
- → **Shard only on measured proof.** Vertical scaling and read scale-out cover nearly every workload. Sharding's costs (cross-shard queries, no cross-shard transactions, fleet schema changes) are permanent — demand evidence first.

Chapter 22: SQL Server Analytics: Columnstore and Workload Offload

Analytical queries — aggregations over millions of rows, year-over-year comparisons, dashboard rollups — are the workload that kills OLTP performance. The rowstore engine that excels at single-row lookups reads entire rows to aggregate one column, and a heavy report can saturate the I/O and CPU that transactions need. This chapter covers SQL Server's in-engine answer (columnstore indexes), the architectural answer (offloading analytics to a separate system), and how to decide between them in the cloud.

22.1 Columnstore: Analytics Inside SQL Server

Columnstore indexes store data column-by-column instead of row-by-row, with heavy compression and batch-mode execution. The result: aggregation queries that scanned gigabytes in rowstore scan megabytes in columnstore — 10x to 100x speedups are routine for the right queries. Two flavors:

- → **Clustered columnstore:** the table *is* the columnstore. Best for large fact tables that are primarily scanned and rarely updated row-by-row. Compression ratios of 5–10x are typical, which directly reduces cloud storage and I/O costs.
- → **Nonclustered columnstore:** a secondary index on a rowstore table. The OLTP workload keeps its rowstore performance while analytical queries use the columnstore copy. This is the HTAP pattern — one database serving both workloads.

The operational notes: columnstore segments have a minimum size for efficiency — trickle-loaded tables (constant small inserts) create small, inefficient rowgroups unless you manage them (tuple mover handles this, but monitor rowgroup quality on trickle feeds); updates and deletes in clustered columnstore are expensive (they go through a delete bitmap and the segment is rebuilt on reorganization — design for bulk loads, not row-by-row mutation); and columnstore loves memory (batch mode wants generous memory grants — watch for the memory pressure that analytical queries add to a shared instance).

```
-- Nonclustered columnstore: HTAP on the OLTP table
CREATE NONCLUSTERED COLUMNSTORE INDEX ncci_orders
ON dbo.Orders (OrderDate, CustomerId, ProductId, Quantity, UnitPrice);

-- Clustered columnstore for the fact table (analytics-first)
CREATE CLUSTERED COLUMNSTORE INDEX cci_sales_fact
ON dbo.SalesFact;

-- Check rowgroup health (aim for ~1M rows per compressed rowgroup)
SELECT object_name(object_id) AS tbl,
       state_description,
       total_rows,
       deleted_rows
FROM sys.column_store_row_groups
WHERE object_id = object_id('dbo.SalesFact');
```

22.2 When In-Engine Is Not Enough: Offload Architectures

Columnstore handles a great deal, but some analytical workloads belong on a separate system: when the analytics need dedicated elastic scale (burst to hundreds of cores for an hour, then to zero); when the data volume dwarfs the operational database (years of history versus months of hot data); or when the analytics team needs a different engine's capabilities. The offload patterns:

- → **Readable secondary as analytics replica.** Route analytical queries to an async AG secondary (Chapter 21). Zero data movement, near-real-time data — but the secondary shares the primary's schema and sizing, and heavy analytics can still lag replication.
- → **Cloud data warehouse offload.** Replicate or ETL SQL Server data into the cloud's warehouse (BigQuery on GCP, Synapse on Azure, Redshift on AWS). The warehouse scales independently and the analytics team gets its own platform. Cost: data movement latency (minutes to hours), duplicate storage, and a pipeline to operate.
- → **Lakehouse export.** Export to Parquet in object storage (S3/ADLS/GCS) for serverless query engines. Cheapest storage, maximum flexibility, highest latency — the right answer for cold historical analytics.

22.3 The Decision: In-Engine vs Offload

Default to columnstore in-engine: it keeps one system, one security model, and real-time data. Offload when the analytics workload's resource profile (bursty, massive, or team-separate) conflicts with the OLTP database's stability. The hybrid is common: recent hot data analyzed in-engine via columnstore, historical data in the warehouse/lake. And whatever you

choose, protect the OLTP workload first — resource governance (workload groups limiting analytics memory/CPU) is not optional when both workloads share an engine. An unbounded dashboard query taking down order entry is the failure this entire chapter exists to prevent.

22.4 Feeding the Pipeline: CDC and Change Tracking

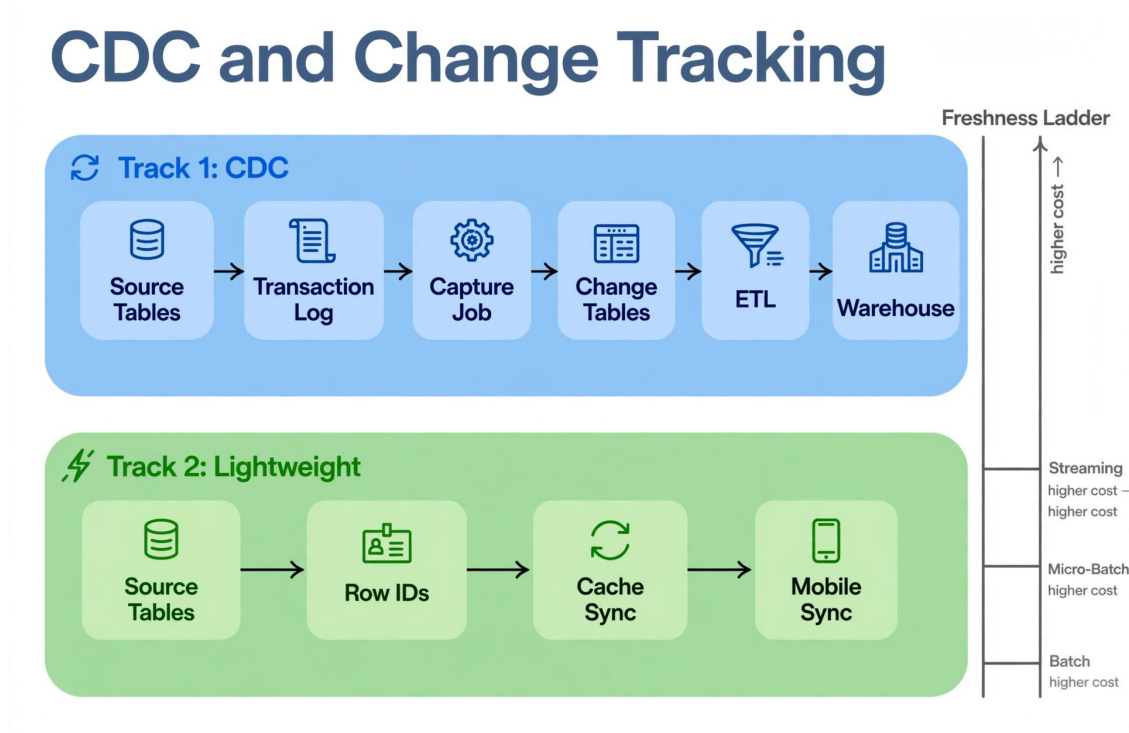


Figure 22.1 — CDC and Change Tracking: Two Tracks. Full change images for the warehouse, lightweight row IDs for caches — pick the track that matches the consumer.

Two tracks, one decision. Track 1 (CDC) reads the transaction log through the capture job into change tables, then ETL into the warehouse — full before/after images, a real analytics feed, and real write overhead on the source. Track 2 (Change Tracking) is the lightweight sibling: just the changed row IDs, enough to invalidate a cache or sync a mobile client. The freshness ladder on the right is the cost conversation: batch is cheap, streaming is dear, and most "real-time dashboard" requests are satisfied by 5-minute micro-batches at a tenth of the complexity.

Every offload pipeline needs to know what changed. SQL Server offers two built-in mechanisms: **Change Data Capture (CDC)**, which asynchronously captures inserted, updated, and deleted rows with full before/after images — ideal for ETL into a warehouse; and **Change Tracking**, a lightweight synchronous mechanism that records only *which* rows changed (not the data), ideal for syncing caches or mobile clients. CDC is the analytics feeder: enable it on the source tables, and the ETL reads the change tables incrementally instead of scanning for deltas.

The cloud caveats: CDC adds write overhead on the source (the capture job reads the transaction log) — measure it on your workload before enabling it on every table; on RDS, CDC is supported but the capture/schedule behavior differs from self-managed (verify current support and job management); and the change tables themselves need retention management, or they grow until someone notices. For near-real-time analytics (seconds of freshness), CDC plus a streaming pipeline beats batch ETL — but near-real-time is a requirement to be proven, not a default to assume. Most "real-time dashboard" requests are satisfied by 5-minute micro-batches at a tenth of the complexity.

22.5 Cost: In-Engine vs Warehouse

The cost comparison is workload-shaped. In-engine columnstore costs the incremental compute/memory on the existing database plus nothing else — cheapest when the analytics fit within the current instance's headroom. A cloud warehouse costs the warehouse compute (often the dominant line), duplicate storage, and data transfer — but scales to zero or near-zero when idle, which can make it cheaper than upsizing the operational database for a workload that runs two hours a day. Model both: the warehouse wins for bursty, team-separate analytics; in-engine wins for continuous, operationally-coupled analytics. And revisit annually — warehouse pricing models change frequently, and last year's expensive answer may be this year's cheap one.

22.6 Data Freshness and Pipeline Discipline

Every offload architecture is a data pipeline, and pipelines fail. Define the freshness SLA per consumer (real-time for operational dashboards, hourly for management reporting, daily for finance), monitor pipeline lag against it, and alert before the business notices stale numbers. Validate row counts and checksums across the pipeline boundary — the ETL that silently drops 0.1% of rows produces analytics that are confidently wrong. And version the pipeline with the schema: when the source table changes, the pipeline, the warehouse schema, and the downstream dashboards change together, tested as a unit.

Key Takeaways

- → **Columnstore is the in-engine answer.** Clustered for analytics-first fact tables, nonclustered for HTAP on OLTP tables. 10–100x aggregation speedups with 5–10x compression — monitor rowgroup quality on trickle-loaded tables.
- → **Offload when the resource profiles conflict.** Bursty massive analytics, years of cold history, or a separate analytics team point to a warehouse or lakehouse. Default to in-engine; offload on evidence.
- → **Protect OLTP first.** Resource governance caps on analytics workloads sharing the engine. The dashboard must never take down order entry.

- → **Pipelines are production systems.** Freshness SLAs, lag monitoring, checksum validation across the boundary, and schema changes tested end to end. Silently stale analytics are worse than no analytics.
- → **The hybrid pattern is normal.** Hot data in-engine via columnstore, cold history in the warehouse/lake. Tier by data temperature, not by habit.

Chapter 23: Database DevOps: CI/CD for SQL Server in the Cloud

Application code ships through pipelines: versioned, reviewed, tested, deployed automatically. Database changes too often ship through a different channel — a DBA running scripts by hand, a shared folder of migration files, a deployment night. The result is the familiar failure: the application deploys successfully and breaks because the database change ran in the wrong order, or twice, or not at all. Database DevOps applies CI/CD discipline to SQL Server schema and programmable objects: every change versioned, reviewed, tested, and deployed through a pipeline. In the cloud — where environments are cheap and automation is the native language — there is no excuse left for manual database deployments.

23.1 State-Based vs Migration-Based Deployments

Two philosophies compete. **State-based** (declarative): you version the desired end state of the schema (a database project — SSDT/.sqlproj), and the tooling computes the diff and generates the deployment script. **Migration-based** (imperative): you version an ordered sequence of change scripts (V001, V002, ...), and the tooling applies each exactly once, in order, tracking what has run. State-based is simpler for developers (edit the model, deploy); migration-based gives DBAs explicit control over exactly what runs in production (critical for large tables where the generated diff might choose a table rebuild you would never approve).

The pragmatic answer for most estates: migration-based for production, where control matters; state-based for development environments, where speed matters. Whatever you choose, the non-negotiable rules are identical: every change in version control, every change reviewed, every deployment repeatable and logged.

23.2 The Pipeline: Build, Test, Deploy

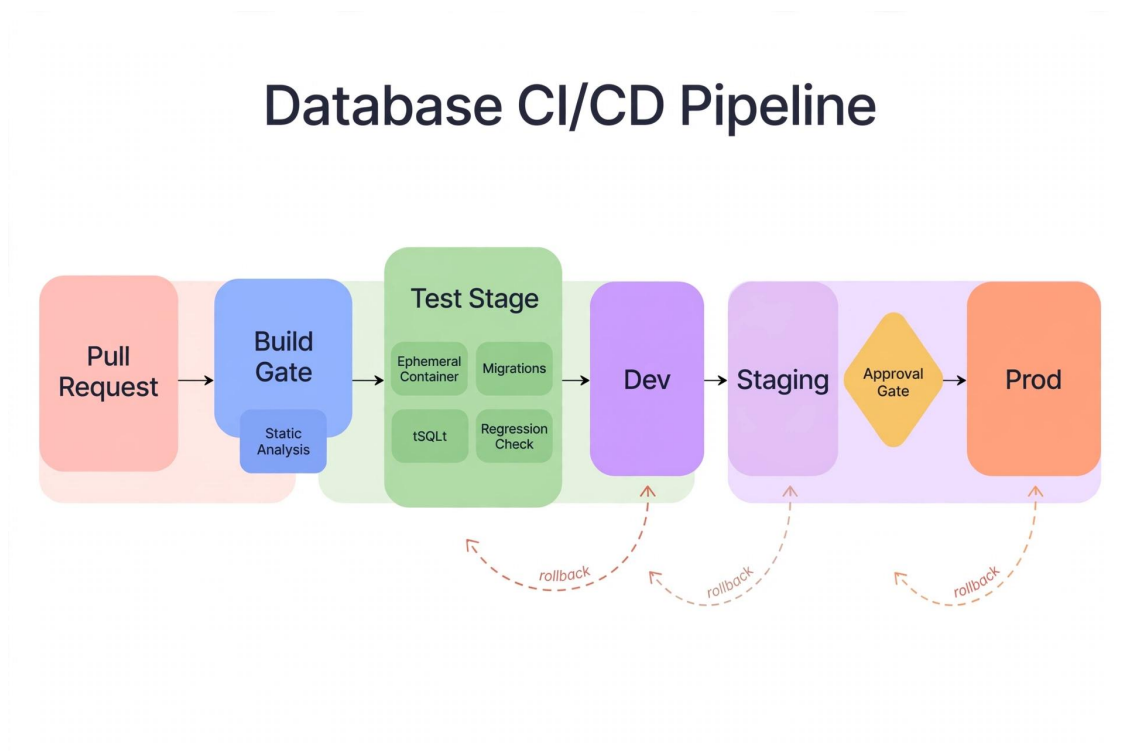


Figure 23.1 — The Database CI/CD Pipeline. Build, test, deploy — with a rollback path at every stage, because a pipeline that only goes forward ships outages.

Read it left to right: the pull request enters the build gate (static analysis fails fast on the errors a human would catch in review), then the test stage spins up an ephemeral container, runs the migrations, and executes tSQLt unit tests, data-migration validation, and performance regression checks. Promotion flows dev → staging → prod with an approval gate before production — and the dashed rollback arrows are the point: every stage must be able to step back to the previous one. A pipeline that can only move forward is a pipeline that ships outages.

A SQL Server CI/CD pipeline has three stages:

- → **Build:** compile the database project or validate the migration scripts (syntax check, static analysis for known anti-patterns — missing WHERE clauses on UPDATE/DELETE, non-SARGable patterns, deprecated features). Fail fast on errors a human would catch in review.
- → **Test:** deploy to an ephemeral database (a container per pull request — Chapter 20's pattern — or a serverless copy), run the migrations, then execute the test suite: tSQLt unit tests for programmable objects, data migration validation (row counts before/after), and performance regression checks on the critical queries (Query Store comparisons catch the migration that doubles a key query's runtime).

- → **Deploy:** promote through environments (dev → test → staging → production) with approvals at the production gate. Production deployment runs the migrations transactionally where possible, logs every step, and verifies post-deployment health checks before declaring success.

```
# GitHub Actions: deploy database migrations to Azure SQL
jobs:
  deploy-db:
    runs-on: ubuntu-latest
    environment: production # approval gate lives here
    steps:
      - uses: actions/checkout@v4
      - name: Deploy migrations
        uses: azure/sql-action@v2
        with:
          connection-string: ${ secrets.AZURE_SQL_CONNECTION }
          path: ./migrations/*.sql
      - name: Post-deploy health check
        run: |
          sqlcmd -S $SQL_SERVER -d appdb -Q \
            "EXEC dbo.usp_PostDeployHealthCheck"
```

23.3 Zero-Downtime Schema Changes

The pipeline enables the deployment patterns that keep the business running:

- → **Expand/contract (parallel change):** for breaking changes, deploy in phases — first add the new column/table alongside the old (expand), migrate the application to use it, then remove the old (contract). The database is never in a state incompatible with the running application.
- → **Online index operations:** `ONLINE = ON` for index builds and rebuilds on Enterprise (and increasingly on Standard in recent versions) — the deployment pipeline should default to online operations.
- → **Backward-compatible migrations only.** The pipeline rule: every migration must be deployable while the previous application version is still running. If a migration requires simultaneous app+database deployment, it is two migrations (expand, then contract).

23.4 Rollback: The Honest Conversation

Database rollback is harder than application rollback — you cannot un-migrate data. The honest strategies: **roll forward** (fix the problem with a new migration — the default for most issues); **restore** (point-in-time restore for data-corrupting failures — accept the data loss since the restore point); and **expand/contract discipline** (if every change is backward-compatible, the application can roll back independently while the database moves forward).

What does not work: assuming down-migrations will save you — they are rarely tested and often wrong. Test the rollback path for the migrations that need it, and prefer designs that do not need rollback at all.

23.5 Environment Strategy: Realistic, Cheap, and Safe

A pipeline is only as good as the environments it deploys to. The cloud makes the ideal cheap: **ephemeral review environments** (a containerized SQL Server per pull request, seeded with a masked subset of production data — Chapter 20's pattern); **staging as a production mirror** (same tier, same configuration, restored from a recent production backup — the only environment where performance testing means anything); and **production-like data without production data** — non-production environments must never hold real customer data. Use data masking (dynamic data masking for live queries, static masking or synthetic generation for restored copies) so a compromised dev environment is an embarrassment, not a breach notification.

Seed strategy matters: an empty database tests nothing. Maintain a versioned seed dataset (representative volume, skewed like production — the query plan that works on 1,000 uniform rows and fails on 10 million skewed rows is the classic CI blind spot). Refresh staging from production backups on a schedule so it never drifts into irrelevance. The environments cost pennies compared to the production incidents they prevent — this is the cheapest reliability investment in the chapter.

23.6 Secrets, Drift, and Compliance

Three pipeline hygiene items: **secrets** — the pipeline connects with short-lived credentials (managed identities, OIDC federation to the cloud) — never a DBA's personal login baked into a variable; **drift detection** — someone will change production by hand eventually; a scheduled job that compares production schema to version control and alerts on drift keeps the pipeline honest; and **compliance** — the pipeline's logs (who approved, what ran, when) are the audit trail for schema changes, which satisfies the change-management evidence auditors ask for far better than tickets and screenshots.

Start small: version-control the next change, add a review step, automate one deployment. The pipeline grows organically from there — but the cultural shift (database changes are code, reviewed and tested like code) has to come first, and it has to include the DBAs, not route around them.

Key Takeaways

- → **Version every database change.** Migration-based for production control, state-based for dev speed. No change reaches production except through the pipeline.

- → **Build, test, deploy — with a production approval gate.** Ephemeral databases per pull request, tSQLt and performance regression tests, staged promotion with logged, verifiable production deployments.
- → **Deploy backward-compatible changes only.** Expand/contract for breaking changes, online index operations by default. The database must never be incompatible with the running application.
- → **Plan for roll-forward, not rollback.** Down-migrations are rarely tested and often wrong. Restore for data corruption; forward fixes for everything else.
- → **Pipeline hygiene is production hygiene.** Short-lived credentials, drift detection against version control, and deployment logs as the audit trail.

Chapter 24: Multi-Cloud Database Strategy: Benefits and Traps

Running SQL Server on more than one cloud is rarely anyone's first choice — it is usually the result of an acquisition, a business unit that standardized elsewhere, or a deliberate resilience strategy. Whatever the cause, the DBA inherits the reality: SQL Server estates on AWS and Azure (and sometimes GCP), each with different managed services, different networking, different identity systems, and one team expected to operate all of it coherently. This chapter is the strategy layer: when multi-cloud is justified, the architectures that work, and the traps that turn two clouds into twice the incidents.

24.1 Why Organizations Go Multi-Cloud (And Why Some Shouldn't)

The legitimate reasons: **resilience** (a second cloud as the ultimate blast-radius boundary — no shared control plane, no shared fate); **mergers and acquisitions** (the acquired company's SQL Server estate lives on the other cloud, and migration is a multi-year project); **best-of-breed services** (BigQuery analytics on GCP consuming SQL Server data from Azure, for example); and **commercial leverage** (the credible ability to move spend disciplines pricing negotiations). The illegitimate reasons: resume-driven architecture, vague "avoiding lock-in" without a threat model (Chapter 28), and the belief that two clouds are inherently more reliable — they are not, unless you engineer the independence deliberately. A multi-cloud architecture where both clouds depend on the same identity provider, the same deployment pipeline, and the same on-call team has one throat to choke, not two.

24.2 Architecture Patterns That Actually Work in Production

Three patterns survive contact with production:

- → **Active-passive with portable artifacts.** Production SQL Server runs on one cloud; the other cloud holds restorable backups (native backup files in object storage, replicated cross-cloud) and the infrastructure-as-code to rebuild. RTO is measured in restore time, not seconds — but the operational cost is minimal and the independence is real.
- → **Workload placement.** Different workloads on different clouds — OLTP on Azure SQL near the application, analytics replicas on AWS — with data flowing via transactional replication or ETL. Each workload uses its cloud's strengths; no workload spans clouds synchronously.
- → **Active-active with conflict avoidance.** Distributed AGs or transactional replication with strict write partitioning (each site owns its data subset). Genuine multi-site operation — and genuine operational complexity. Only for the resilience requirements that justify it.

What does not work: synchronous replication across clouds (the latency makes it a performance disaster), and "we'll figure out failover later" (failover across clouds is a business process involving DNS, identity, and application configuration — Chapter 16's client redirection problem, squared).

24.3 Cross-Cloud Networking: The Layer That Breaks Everything

Connect the clouds with private connectivity: AWS Direct Connect to Azure ExpressRoute via a colocation exchange, or site-to-site VPN for lower-throughput needs. The database traffic must never traverse the public internet — not for performance, but because the security model collapses. Key details: **overlapping CIDRs** are the most common blocker (two VPCs/VNets both using 10.0.0.0/16 cannot peer or route — plan address space before the first deployment); **egress costs** are metered on every cloud, and cross-cloud replication is chatty — model the data transfer bill before committing to continuous replication; and **DNS** must resolve across clouds (private DNS forwarding), because connection strings with IP addresses are a maintenance nightmare.

24.4 Data Consistency and the CAP Theorem in Practice

Cross-cloud replication is asynchronous — accept it and design for it. The consistency model for SQL Server across clouds: **single-writer** (one cloud accepts writes; the other is a replica or restore target — no conflicts possible, lag is the only inconsistency); or **partitioned writers** (each site owns disjoint data — conflicts are impossible by design, but the application must route correctly). Bidirectional replication with conflict resolution exists (merge replication, custom CDC conflict handlers) and should be treated as a last resort: conflicts detected hours after the fact, in business logic, at 3 AM. The CAP theorem is not academic here — during a cross-cloud network partition, you choose availability (keep serving, accept divergence) or consistency (stop writes, stay correct). Decide in advance, document it, and make sure the business signed off.

24.5 Identity, Access, and the Security Boundary Problem

Each cloud has its own identity system (IAM, Entra ID, Google IAM), and SQL Server authentication must bridge them. The pattern: **federate, don't duplicate** — one identity provider (usually Entra ID for SQL Server estates) federated to each cloud, so a departing DBA loses access everywhere at once. For SQL Server itself, contained database users and Azure AD / Entra authentication on Azure, IAM-integrated auth where the managed service supports it, and certificate-based service accounts for cross-cloud replication agents — with secrets in each cloud's native secret store, replicated by your secrets management process,

never by email. Audit who can access production SQL Server on *each* cloud quarterly; access reviews that cover one cloud and forget the other are how ex-employees keep production access.

24.6 Schema Management Across Cloud Boundaries

The DevOps pipeline from Chapter 23 becomes multi-cloud: one migration sequence, deployed to every cloud's SQL Server in order, with per-cloud deployment jobs. The rules: migrations run everywhere (no cloud-specific schema forks — a fork is the beginning of two different databases); deployment order respects replication topology (publishers before subscribers for replication-affecting changes); and drift detection runs per cloud (Chapter 23's discipline, multiplied). Test the full multi-cloud deployment in staging that mirrors the topology — the migration that works on Azure SQL and fails on RDS (a platform limitation neither developer knew about) is found in staging or in production.

24.7 Observability Across Cloud Boundaries

One pane of glass or it didn't happen. SQL Server metrics from each cloud (CloudWatch, Azure Monitor, Cloud Monitoring) flow into a single observability platform with unified dashboards and alerting: replication lag across the cloud boundary as a first-class metric, per-cloud resource utilization, and cross-cloud network throughput/loss. The alerting rule that matters most: **alert on replication lag, not just on outages** — a silently growing lag is the multi-cloud failure mode, and by the time it becomes an outage the data loss is already decided. Standardize the SQL Server metric set (the DMVs from Chapter 6) so a DBA can read any cloud's dashboard without translation.

24.8 Failover Runbooks and the Discipline of Practice

The multi-cloud failover runbook is a superset of Chapter 16's regional drill: promote or restore on the second cloud, redirect clients (DNS with low TTL, or application configuration), re-point replication, verify identity and secrets work in the second cloud, and run the business's critical transactions. Drill it at least annually — and drill the *failback*, which is always harder (data written on cloud B must return to cloud A without loss). Every drill will find something: the second cloud's SQL Server version drifted, the KMS keys don't exist there, the runbook references decommissioned endpoints. That is the drill working as intended.

24.9 When to Walk Away from Multi-Cloud

Multi-cloud has a carrying cost: duplicated expertise, duplicated tooling, cross-cloud networking bills, and the cognitive load of two platforms' failure modes. Walk away (consolidate to one cloud) when: the original justification expired (the acquisition is fully

integrated, the resilience requirement was met another way); the second cloud hosts a trivial workload that one region of the primary cloud could absorb; or the team cannot staff expertise in both clouds and is operating one of them on hope. Consolidation is not failure — it is the rational response to costs exceeding benefits. The portable artifacts (native backups, IaC, migration-based deployments) that made multi-cloud viable are exactly what make consolidation safe.

Key Takeaways

- → **Justify multi-cloud explicitly.** Resilience, M&A reality, best-of-breed services, commercial leverage — or walk away. "Avoiding lock-in" without a threat model is not a justification.
- → **Three patterns work: portable- artifact passive, workload placement, partitioned active-active.** Synchronous cross-cloud replication does not work. Failover is a business process, not a database feature.
- → **Networking and identity are the hard parts.** Private connectivity, non-overlapping CIDRs, modeled egress costs, federated identity, per-cloud access reviews.
- → **One pipeline, one schema, one dashboard.** Migrations deploy everywhere in order; drift detection runs per cloud; replication lag is the metric that matters most.
- → **Drill failover and failback annually.** The drill finds the drifted versions, missing keys, and stale endpoints — that is its purpose.

Chapter 25: Cross-Cloud Replication Architectures

Chapter 24 set the strategy; this chapter is the engineering. Moving SQL Server data between clouds — continuously, reliably, and observably — using the mechanisms SQL Server actually provides: transactional replication, distributed availability groups, CDC-based pipelines, and backup-based seeding. Each mechanism's consistency model, latency profile, failure modes, and cost, so the choice is an engineering decision instead of a guess.

25.1 Understanding the Cross-Cloud Replication Landscape

Every cross-cloud replication design answers four questions: **granularity** (whole database or selected tables?), **latency** (seconds, minutes, or hours behind?), **direction** (one-way or bidirectional?), and **conflict policy** (what happens when both sides change the same row?). SQL Server's mechanisms span the space: backup/restore (whole database, hours, one-way, no conflicts); transactional replication (tables, seconds, one-way, no conflicts); distributed AGs (whole database, seconds, one-way per AG, no conflicts); merge replication (tables, minutes, bidirectional, resolvable conflicts — with all the caveats of Chapter 24). Match the mechanism to the answers, not the other way around.

25.2 Transactional Replication Across Clouds

Transactional replication is the workhorse for cross-cloud SQL Server data movement: the publisher (on cloud A) streams committed transactions to the distributor, which delivers them to subscribers (on cloud B) with seconds of latency. The cross-cloud specifics:

- → **Distributor placement.** Put the distributor on cloud A near the publisher (local latency for the high-volume publisher-to-distributor leg); the distributor-to-subscriber leg crosses the cloud boundary once. Never stretch the publisher-to-distributor link across clouds.
- → **Initialization over the WAN.** Snapshot initialization across clouds is slow and expensive (egress on every byte). Seed the subscriber from a native backup restored on cloud B, then initialize replication from the backup (replication supports initialization from backup) — the snapshot crosses the boundary as a compressed backup file, once.
- → **Encryption and authentication.** Replication traffic crosses the private interconnect (Chapter 24), encrypted in transit; the replication agents authenticate with certificate-based logins whose secrets live in each cloud's secret store.

- → **Managed-service limits.** RDS for SQL Server supports replication with restrictions (verify current publisher/subscriber support); Azure SQL Database can serve as a subscriber in supported configurations. Full topology control needs EC2/Azure VMs/GCE on at least the publisher side.

25.3 CDC-Based Pipelines for Cross-Cloud Data Movement

When the target is not SQL Server (a warehouse, a lakehouse, a different engine) or when you need transformation in flight, **CDC-based pipelines** replace replication: SQL Server CDC (or the transaction log via a CDC tool) captures changes, a pipeline (Debezium/Kafka, cloud-native CDC services, or ETL) moves and transforms them, and the target applies them. This is the Chapter 22 offload pattern extended across clouds. The engineering concerns: exactly-once vs at-least-once delivery semantics (design the target apply to be idempotent — the pipeline *will* redeliver); schema change propagation (the pipeline must handle DDL or fail loudly, never silently); and lag monitoring end to end (source capture lag + pipeline lag + target apply lag = the business's actual data freshness).

25.4 Distributed AGs Across Clouds

A distributed availability group with one AG on each cloud gives whole-database replication with seconds of lag and automatic page-repair semantics — the closest SQL Server gets to a cross-cloud standby. Requirements: EC2/VMs/GCE on both sides (managed services cannot participate), private cross-cloud connectivity with latency you have measured (async mode tolerates it; the lag equals roughly the network round trip plus apply time), and independent quorum in each site (cloud witnesses in each cloud's storage — never one witness for both sites). Use it for: DR across clouds (the Chapter 24 active-passive pattern with seconds of RPO), and geographic read locality. Do not use it for bidirectional writes — distributed AGs are one-way per AG, and "we'll coordinate writes" is the conflict problem wearing a different hat.

25.5 Handling Schema Changes and DDL Propagation

Schema changes are where cross-cloud replication goes to die. The discipline: **DDL goes through the same pipeline as DML** — the migration-based deployment from Chapter 23 runs on every replica's database in dependency order (publisher first, then subscribers; primary AG first, then secondaries). Transactional replication propagates *some* schema changes automatically (configurable per article) — but relying on automatic DDL propagation across a cloud boundary is asking for a 3 AM surprise; explicit deployment is auditable, automatic propagation is not. For CDC pipelines, every DDL change must update the pipeline's schema mapping — make it a checklist item in the migration review, enforced by the pipeline failing closed on unknown schema.

25.6 Failover and Switchover in Cross-Cloud Architectures

Failover across clouds is never just a database operation. The sequence: stop writes on cloud A (or confirm it is down), let replication drain (verify zero lag — failing over with 5 minutes of undelivered transactions is data loss you chose), promote on cloud B (restore with recovery, or AG failover, or replication role reversal), redirect clients (DNS/endpoint flip, Chapter 16), and verify the business's critical transactions. **Planned switchovers** (for maintenance or drills) follow the same sequence without the time pressure — which is why you drill: the planned switchover is the rehearsal for the unplanned failover. And plan the failback *before* the failover: data written on cloud B must return to cloud A, which means reverse replication configured and tested, not improvised.

25.7 Conflict Resolution in Bidirectional Replication

If you truly need both clouds accepting writes to the same data, you need conflict detection and resolution — and you should first re-read Chapter 24's advice to avoid it. When there is no alternative: **partition by design** (each site owns disjoint key ranges — conflicts impossible, routing mandatory); **last-writer-wins with vector clocks or timestamps** (simple, lossy — someone's write disappears, and you'd better be able to explain whose and why); or **application-level merge** (the application understands the domain and can merge meaningfully — the only correct option for data where loss is unacceptable, and the most expensive to build). Log every conflict with both versions preserved; "resolved automatically" without an audit trail is how you discover data loss during the quarterly review.

25.8 Cost Modeling for Cross-Cloud Replication

Cross-cloud replication has four cost centers: **egress** (every replicated byte is metered leaving the source cloud — for chatty OLTP this is often the largest line; compress where possible and model from measured transaction log volume, not guesses); **the second cloud's infrastructure** (the subscriber/secondary is a real database with real compute and storage); **the interconnect** (Direct Connect/ExpressRoute ports and hours, or VPN); and **operations** (the DBA hours spent on the topology — usually underestimated by half). Build the model before committing, re-measure quarterly, and set a billing alert on the egress line specifically — it is the one that surprises.

Key Takeaways

- → **Answer the four questions first: granularity, latency, direction, conflicts.** Then match the mechanism — backup/restore, transactional replication, distributed AGs, or CDC pipelines — to the answers.

- → **Seed across the WAN with backups, not snapshots.** Initialize replication from a restored backup; keep the high-volume replication legs local to each cloud.
- → **DDL goes through the pipeline, everywhere, in order.** Automatic DDL propagation across cloud boundaries is a 3 AM surprise. Explicit, auditable, reviewed.
- → **Failover is a sequence; failback is the hard part.** Drain replication to zero lag, promote, redirect clients, verify business transactions — and have reverse replication ready before you need it.
- → **Model all four cost centers, especially egress.** Measure transaction log volume, set billing alerts on egress, and re-measure quarterly.

Chapter 26: Data Sovereignty and Residency Compliance

Data sovereignty — the principle that data is subject to the laws of the jurisdiction where it resides — has moved from legal theory to engineering constraint. GDPR, Schrems II, sectoral regulations, and an expanding set of national data-localization laws mean the DBA must now answer questions that used to belong to counsel: where does this database's data physically live, where do its backups live, who holds the encryption keys, and can you prove it? This chapter turns those questions into SQL Server engineering: region selection, replication boundaries, key custody, audit evidence, and the right-to-erasure problem.

26.1 Understanding the Regulatory Landscape for Cloud DBAs

You do not need a law degree, but you need the working vocabulary: **data residency** (data stored in a specific jurisdiction), **data sovereignty** (data subject to a jurisdiction's laws — residency is one mechanism, not the whole concept), **data localization** (laws requiring certain data to stay within borders), and **transfer mechanisms** (the legal basis for moving data across borders where allowed). The DBA's job is not to interpret the law — it is to implement the controls the business's interpretation requires, and to produce the evidence: region configurations, replication topologies, backup locations, key custody records, and access logs. When counsel asks "prove EU customer data never leaves the EU," the answer is architecture plus logs, not assurances.

26.2 AWS: Region, Replication, and Key Custody

On AWS, sovereignty is built from: **region pinning** (RDS instances, EC2, and S3 buckets in the required region — with SCPs/permission boundaries preventing deployment elsewhere); **replication boundaries** (no cross-region snapshot copies, no S3 cross-region replication to non-compliant regions — audit the automation, because a well-meaning DR setup is the most common sovereignty violation); and **key custody** (KMS keys in the same region, with key policies restricting use; for the strictest requirements, dedicated key stores or CloudHSM-backed keys where the customer holds exclusive key control). Document the backup chain end to end: automated RDS snapshots, manual snapshots, and S3 native backups each have their own region — verify all three.

26.3 Azure SQL: Geo-Replication, Paired Regions, and Data Boundary

Azure's sovereignty tooling is the most explicit: **paired regions** (each Azure region has a designated pair, usually in the same geography — failover groups default within the pair, which keeps DR inside the sovereignty boundary); the **EU Data Boundary** (Microsoft's commitment that EU customer data stays in the EU — understand exactly which services and data flows it covers); and **customer-managed keys in Key Vault** (preferably Managed HSM with customer-held key material for the strictest regimes). The traps: **support telemetry and diagnostics** — verify what leaves the region for monitoring and support (Azure Monitor, diagnostic settings destinations); and **long-term retention backups** — LTR stores in the paired region by default, which is usually compliant, but verify the pairing for your specific region rather than assuming.

26.4 Google Cloud: Residency Controls for SQL Server

On GCP: **organization policies** constraining resource locations (allowed regions for Cloud SQL, Compute Engine, and Cloud Storage — the GCP analog of AWS SCPs); **Cloud KMS** keys pinned to the compliant region with customer-managed encryption keys (CMEK) on Cloud SQL instances; and **Access Transparency / Assured Workloads** for the regimes that require personnel-access controls, not just data-location controls. The GCP-specific check: BigQuery and analytics exports — the sovereignty boundary most often breached is not the database but the pipeline that copies its data into an analytics project in another region (Chapter 22's offload pattern needs a residency review).

26.5 Audit Logging for Residency Compliance

Evidence is the product. The audit baseline: **control-plane logs** (CloudTrail, Azure Activity Log, GCP Audit Logs — proving *where* resources were created and *what* replication was configured); **SQL Server audit** (logins, data access to classified tables, DDL — proving *who* touched the data); and **backup/restore logs** (proving where copies went). Ship all three to immutable, retention-locked storage in the compliant region — audit logs that can be modified are not evidence. Run the "prove it" exercise annually: pick a regulated dataset and trace every copy (primary, replicas, backups, exports, analytics derivatives) to a compliant location, with logs. The gaps this finds are always surprising.

26.6 Data Classification and Residency-Aware Schema Design

Not all data carries the same obligations. **Classify** (public, internal, confidential, regulated/personal) using SQL Server's data classification labels or your governance tooling, then design residency around the classification: regulated columns isolated into dedicated tables or databases that can be pinned to compliant regions, while non-regulated data lives

wherever is cheapest. The schema patterns: **vertical partitioning by classification** (personal data in its own tables with restricted access — also simplifies erasure, section 26.7); and **tokenization/pseudonymization** at the boundary (analytics and non-production environments get tokens, not real personal data — the sovereignty problem disappears when the data isn't personal). Classification is a living process: new columns get classified at migration-review time (Chapter 23's pipeline), not discovered during the audit.

26.7 Right to Erasure and the Database Engineering Challenge

The right to erasure ("delete my data") is a database engineering problem wearing a legal costume. The challenges: **finding the data** (personal data scattered across tables, replicas, backups, exports, and analytics derivatives — the classification from 26.6 is what makes this tractable); **backups** (you cannot surgically delete from a backup chain — the accepted practice is erasure from live systems plus backup expiration on the normal retention cycle, documented and defensible); and **referential integrity** (erasure must not corrupt the database — anonymize foreign-key-referenced rows rather than deleting them where history must be preserved). Build the erasure runbook *before* the first request arrives: the identified queries, the verification steps, and the evidence package. The 30-day clock starts when the request arrives, not when you're ready.

26.8 Operationalizing Sovereignty: Runbooks and Governance

Sovereignty fails operationally, not architecturally. The governance baseline: **region allow-lists enforced by policy** (not by convention — SCPs, Azure Policy, org policies that *deny* non-compliant deployments); **the new-service review** (every new cloud service or feature gets a residency check before production use — yesterday's compliant architecture plus today's new feature is tomorrow's finding); **quarterly copy-mapping** (the "prove it" exercise from 26.5, on a schedule); and **incident runbooks** for sovereignty breaches (a misconfigured replication caught by audit is an incident: stop the flow, assess exposure, notify per the breach plan). Assign ownership — sovereignty without an owner is a shared assumption, and shared assumptions are how data ends up in the wrong region.

Key Takeaways

- → **Sovereignty is architecture plus evidence.** Region pinning, replication boundaries, and key custody — proven by control-plane logs, SQL Server audit, and backup records in immutable storage.
- → **Audit the automation, not just the architecture.** DR copies, analytics exports, and new features are how data leaves the compliant region. Policy-enforced allow-lists beat conventions.

- → **Classify data, then design residency around classification.** Isolate regulated data, tokenize at boundaries, classify new columns at migration-review time.
- → **Build the erasure runbook before the first request.** Find-every-copy via classification, erasure from live systems, defensible backup expiration, anonymization where history must survive.
- → **Assign ownership and run the "prove it" exercise annually.** Trace a regulated dataset to every copy with logs. The gaps are always surprising — that is the point.

Chapter 27: Hybrid Cloud: On-Premises and Cloud Together

Most SQL Server estates will be hybrid for years: production partially in the cloud, partially on-premises, with data flowing between them. Hybrid is not a waystation you pass through on the way to "all cloud" — for regulated industries, latency-sensitive workloads, and sunk data-center investments, it is the steady state. This chapter covers the hybrid architecture for SQL Server: the network, the data synchronization, hybrid DR, identity, and the operational discipline of running one estate across two worlds.

27.1 Network Architecture: Connecting Your Data Center to the Cloud

The foundation is private, reliable connectivity: **dedicated circuits** (Direct Connect, ExpressRoute, Cloud Interconnect) for production database traffic — consistent latency, no internet traversal, SLA-backed; **VPN** as backup path or for lower-throughput needs. Design for redundancy from day one: two circuits (or circuit plus VPN failover), because the hybrid architecture's single point of failure is always the link. Size for the replication and backup traffic, not just the application traffic — the initial seeding of a multi-terabyte database across the link is what saturates it. And resolve the **address space** question early: the data center and cloud networks must not overlap, and DNS must resolve both directions (conditional forwarders or private DNS peering) so connection strings work identically everywhere.

27.2 Data Synchronization Patterns Across Boundaries

The synchronization mechanism follows the use case:

- → **Migration seeding**: native backup to cloud object storage, restored on the cloud target — the universal method (Chapters 33–34). For the initial multi-terabyte seed, physical media shipment (AWS Snowball, Azure Data Box) beats the network when the math favors it.
- → **Ongoing replication**: transactional replication (on-premises publisher, cloud subscriber — or the reverse) for table-level flows; distributed AGs with one replica on-premises and one in the cloud for whole-database DR with seconds of lag. Both need the distributor/AG topology designed around the link latency (Chapter 25's rules apply within hybrid too).
- → **Azure Arc (Chapter 18)**: for Azure-hybrid estates, Arc extends management, policy, and (with the right configuration) data services to on-premises SQL Server — one control plane for the hybrid estate.
- → **ETL/ELT pipelines**: for analytics offload (on-premises SQL Server to cloud warehouse), CDC-based pipelines on a schedule — with the freshness SLA defined per consumer (Chapter 22).

27.3 Hybrid Failover and Disaster Recovery

Hybrid DR runs in both directions, and you must design both: **on-premises to cloud** (data center failure — fail over to the cloud replica; the cloud's elasticity is an advantage here, since the DR target can scale up on demand rather than sitting idle); and **cloud to on-premises** (cloud region failure or cloud exit — the reverse path, which is always harder because on-premises capacity is fixed). The distributed AG spanning data center and cloud is the cleanest mechanism for the first direction; backup-based restore covers both but with hours of RTO. The critical hybrid-DR question is **capacity**: the DR side must actually absorb the production workload — an on-premises DR target sized three years ago for a smaller database, or a cloud DR target with no reserved capacity, fails the drill exactly when it matters.

27.4 Hybrid Identity and Access Management

One identity system across both worlds — for SQL Server estates this almost always means **Active Directory / Entra ID spanning on-premises and cloud**: domain-joined cloud VMs (via AD replication to the cloud or Entra Domain Services), Entra authentication for Azure SQL, and Kerberos working across the link for AGs and linked servers (SPNs registered, constrained delegation configured — the classic on-premises Kerberos discipline, now extended across the boundary). The failure mode is always the same: authentication that works inside each world and breaks across the boundary, discovered during the failover drill. Test cross-boundary authentication explicitly and continuously — a scheduled probe that authenticates to every SQL Server from every network segment catches the drift before the incident does.

27.5 Cross-Boundary Query Patterns and Their Performance Implications

Queries that span the boundary — linked servers from on-premises to cloud, elastic queries, ETL pulls — pay the latency tax on every round trip. The rules: **never run chatty cross-boundary queries** (a linked-server query that pulls a million rows row-by-row across the link is the classic hybrid performance disaster — rewrite as a single bulk operation or move the processing to the data); **keep joins local** (join across the boundary only after filtering each side to the minimum row set); and **cache reference data** (replicate slow-changing dimensions across the boundary on a schedule rather than querying them live). Monitor cross-boundary query performance as its own category — when the link degrades, these queries are the first symptom and the DBA needs to recognize the pattern instantly.

27.6 Monitoring a Hybrid Database Environment

The hybrid monitoring stack has three layers: **unified metrics** (the same SQL Server DMV-based metric set from on-premises agents and cloud monitoring, in one platform — Chapter 24's one-pane-of-glass rule); **link health** (circuit/VPN throughput, latency, and loss as first-class metrics — the link is infrastructure your databases depend on, so it gets SLAs and alerts like any other); and **synchronization health** (replication lag, AG sync state, and backup-copy completion across the boundary). The on-call rotation must cover both worlds with runbooks that assume either side can be the problem — "the cloud is down" and "the data center is down" are both in the playbook.

27.7 Managing Schema Changes Across Hybrid Environments

Chapter 23's pipeline extends across the boundary: one migration sequence deployed to on-premises and cloud in dependency order, with the replication topology respected (publishers before subscribers). The hybrid-specific risks: **version skew** (on-premises SQL Server on an older version than the cloud target — every migration must be compatible with the lowest version in the estate, enforced by the pipeline's compatibility check); and **deployment windows** (on-premises maintenance windows and cloud deployment automation must be coordinated — the migration that deploys to the cloud at 2 AM but waits for the on-premises window creates a half-migrated estate). Drift detection runs on both sides; the estate is one logical database fleet regardless of where the servers physically sit.

27.8 Cost Engineering in Hybrid Architectures

Hybrid cost engineering is placement optimization: run each workload where it is cheapest *over its lifetime*, counting the fully-loaded costs. On-premises looks cheap per-server until you count power, cooling, hardware refresh, and the DBA hours; cloud looks expensive per-hour until you count elasticity and managed services. The honest model: steady-state predictable workloads often favor on-premises (sunk hardware, AHB-licensed VMs); variable, bursty, or new workloads favor cloud; and the **link and egress costs** are the hybrid-specific lines that decide close calls (Chapter 25's egress warning applies within hybrid too). Revisit placement annually — hardware refresh cycles and cloud pricing changes move the answer regularly, and "we've always run it on-premises" is not a cost model.

Key Takeaways

- → **Hybrid is a steady state, not a waystation.** Design the network (redundant private links, non-overlapping address space, bidirectional DNS) as permanent infrastructure.

- → **Match the sync mechanism to the use case.** Backup-seed for migrations, transactional replication or distributed AGs for ongoing sync, Arc for Azure-hybrid management, CDC pipelines for analytics.
- → **Design DR in both directions and verify capacity.** The DR side must absorb production — undersized DR targets fail exactly when needed.
- → **One identity, one pipeline, one dashboard.** Federated identity across the boundary, one migration sequence everywhere, unified monitoring with link health as a first-class metric.
- → **Keep cross-boundary queries bulk and local.** No chatty linked-server queries across the link; cache reference data; monitor cross-boundary query performance as its own category.

Chapter 28: Avoiding Vendor Lock-In Without Over-Engineering

"Avoiding vendor lock-in" is the most expensive phrase in cloud architecture when it is used without a threat model. Teams build abstraction layers, avoid every managed feature, and run everything on VMs — paying the operational cost of lock-in avoidance every day to insure against a migration that never happens. This chapter gives lock-in the honest treatment: what it actually costs, which of it is worth avoiding, and how to stay portable where it matters without over-engineering where it doesn't.

28.1 Understanding What Lock-In Actually Means in Practice

Lock-in is not binary — it is a spectrum of **switching costs**: data gravity (multi-terabyte databases take weeks to move — the dominant cost, and it exists with every platform including on-premises); proprietary features (the deeper you go into platform-specific capabilities, the more re-engineering a move requires); operational knowledge (your team's expertise is in one cloud's failure modes); and commercial terms (committed-use discounts and enterprise agreements that penalize exit). Note what is *not* on the list: using a managed database service at all. The lock-in conversation that matters is about *which* dependencies you take on, not whether you take any — a team running SQL Server on EC2 to "avoid lock-in" while storing backups only in S3 with no tested restore elsewhere has the same data gravity as everyone else, plus the operational cost of self-management.

28.2 The Abstraction Layer Trap — and Where Abstractions Actually Help

The classic over-engineering: a database abstraction layer that hides which cloud the SQL Server runs on — custom deployment tooling, cloud-agnostic monitoring, in-house connection management. The trap: the abstraction must be maintained forever, it always lags behind each cloud's native capabilities, and it is tested far less than the native services it replaces. The abstractions that *do* earn their keep are the ones you'd want anyway: **infrastructure as code** (Terraform/Bicep modules per cloud — the module boundary *is* the abstraction, and it's useful on day one); **migration-based deployments** (Chapter 23's pipeline works on any SQL Server anywhere — portability as a side effect of good practice); and **standard SQL Server features** (prefer engine features over platform features where the engine feature is adequate — not for portability theater, but because engine knowledge transfers between jobs and platforms).

28.3 Choosing Which Proprietary Features Are Worth the Lock-In

Some proprietary features are worth it. The decision framework — three questions:

- → **What does it replace?** Azure SQL's built-in HA, RDS automated backups, Hyperscale's architecture — each replaces engineering you would otherwise build and operate. Price the build alternative honestly (including on-call burden) before declining the feature for portability.
- → **How deep does it reach into the application?** A proprietary backup mechanism is shallow lock-in (backups are replaceable). A proprietary query dialect, a platform-specific security model woven through the application, or a data warehouse the entire analytics stack depends on — deep lock-in. Prefer shallow, contain the deep.
- → **What is the actual exit scenario?** "We might move clouds someday" is not a scenario. "Our enterprise agreement renews in 18 months and we need negotiating leverage" is. "Regulators may require a second provider" is. Design portability for the scenarios with names and dates; accept lock-in everywhere else.

28.4 Portable Infrastructure Patterns That Earn Their Complexity

The portability patterns worth their cost — because they pay off even if you never migrate:

- → **Native backups as the portable artifact.** Every SQL Server everywhere can restore a native backup. Keep backup chains in standard formats in object storage you control — this single practice is 80% of practical portability.
- → **Containerized applications, standard SQL Server.** The application in containers (Chapter 20) with plain T-SQL against standard engine features migrates by changing a connection string and restoring a backup.
- → **Secrets and configuration externalized.** No cloud-specific SDK calls in the data path for secrets or config — the application reads from its environment, and each cloud injects differently.
- → **Documented data flows.** The map of where data lives, how it moves, and what depends on it (Chapter 26's copy-mapping) is the migration plan's first page — and it's useful operationally every week.

28.5 Realistic Migration Paths — Planning the Exit You May Never Take

The exit plan is a document, not an architecture. For each production SQL Server, the plan states: the portable artifacts (backup location and format, IaC modules, migration history), the estimated migration approach (backup/restore for most, replication-seeded for the large ones — Chapters 33–34), the measured data-transfer time for the full size (not estimated — measured, from a test), and the dependencies that would need re-engineering (the deep

lock-in from 28.3, listed explicitly). Review it annually; update it when the architecture changes. The plan's value is mostly in its existence: it forces the discipline (portable backups, IaC, documented flows) that makes the plan feasible, and that discipline is valuable whether or not the exit ever happens.

28.6 When Multi-Cloud Database Architectures Are Actually Justified

Revisiting Chapter 24 with the lock-in lens: multi-cloud as a *lock-in avoidance strategy* is almost never justified on its own — the cost of operating two clouds dwarfs the expected value of the option to leave one, unless the exit scenario from 28.3 is concrete. Multi-cloud is justified by resilience requirements, M&A reality, and best-of-breed needs (Chapter 24.1) — with portability as a welcome side effect, not the purpose. The team that runs active-active across clouds "to avoid lock-in" has purchased the most expensive insurance policy in the industry against a risk they never quantified.

28.7 Practical Portability Audit: What to Document Right Now

The audit you can run this week: for each production SQL Server, list every cloud-proprietary dependency (managed features, platform-specific backup mechanisms, cloud IAM integration points, proprietary extensions), rate each as shallow or deep (28.3), and confirm the portable alternative exists for the deep ones (native backups restorable, IaC rebuild tested, migration path documented). The output is a one-page-per-database portability sheet — which doubles as the DR documentation (Chapter 16) and the migration plan input (Chapters 33–34). Portability documentation that serves three purposes gets maintained; documentation that serves only a hypothetical migration gets stale.

Key Takeaways

- → **Lock-in is switching cost, not a moral failing.** Data gravity dominates; proprietary features vary from shallow to deep. Quantify before insuring.
- → **Abstractions must earn their keep.** IaC, migration-based deployments, and standard engine features are portability you get for free with good practice. Custom abstraction layers are a forever tax.
- → **Take the proprietary features that replace real engineering.** Judge by replacement cost, depth of application reach, and whether the exit scenario has a name and a date.
- → **Native backups are 80% of practical portability.** Standard formats, storage you control, tested restores — plus externalized secrets and documented data flows.
- → **The exit plan is a document, reviewed annually.** Portable artifacts, measured transfer times, explicit deep dependencies. Its discipline is valuable whether the exit happens or not.

Chapter 29: Backup and Restore in the Cloud: Automated, Native, and Tested

Backups are the one operational discipline where the cloud changes the mechanics but not the accountability. The platform can automate snapshots, copy them across regions, and enforce retention — but when the restore fails at 3 AM, it is still the DBA explaining. This chapter covers the full SQL Server backup picture in the cloud: platform-automated backups, native backups you control, point-in-time recovery on each platform, cross-region strategies, and the restore testing that makes the whole thing real.

29.1 How Platform-Automated Backups Work

Each managed platform automates the backup chain differently: **RDS** takes automated snapshots with transaction log backups enabling point-in-time recovery within the retention window (up to 35 days); **Azure SQL** provides fully automated full, differential, and log backups with PITR (7–35 days configurable) plus long-term retention policies stretching to years; **Cloud SQL** offers automated daily backups with configurable retention and PITR via log archiving. The common trait: the platform owns the schedule and the storage, and you own the verification. Know exactly what each platform's "automated backup" includes — RDS automated backups do not include the transaction logs you need for sub-hour RPO unless log backup behavior is understood; Azure's PITR granularity depends on log backup frequency, which the platform manages but you should confirm.

29.2 Native Backups: The Backups You Control

Platform backups are convenient; native SQL Server backups are portable. On EC2, Azure VMs, and GCE you run the same `BACKUP DATABASE ... TO URL / TO DISK` commands as on-premises, writing to S3, Blob storage, or Cloud Storage. On RDS, the `rds_backup_database` stored procedure takes native backups to S3 — the escape hatch that makes RDS restores portable. The native-backup discipline:

- → **Full + differential + log cadence.** Weekly full, daily differential, log backups every 15 minutes is the classic starting point — tune to the RPO. The log backup frequency *is* the RPO for native chains.
- → **Compression and checksums.** `WITH COMPRESSION`, `CHECKSUM` on every backup — compression reduces storage and transfer costs; checksums catch corruption at backup time rather than restore time.

- → **Verify with RESTORE VERIFYONLY — and real restores.** VERIFYONLY checks readability, not recoverability. Only a real restore proves the chain.

29.3 Backup Windows, Retention, and the RPO Contract

Set the backup window outside peak hours — but understand what the window actually constrains (on RDS it constrains snapshot timing; log backups continue regardless).

Retention is the RPO contract made concrete: 35 days of RDS automated backups means you can reach any point in the last 35 days — and not one day more. For compliance horizons beyond platform maximums, native backups to object storage with lifecycle policies carry the depth (Chapter 16's long-retention discussion). Document the retention per database tier: production 35 days PITR + 1 year native; non-production 7 days. When legal asks "can we restore March?", the answer is in this document, not in someone's memory.

29.4 Point-in-Time Recovery: Execution Under Pressure

PITR is the restore you will run during an incident, so rehearse the mechanics: on RDS, restore to a new instance to a specified time (restoring "in place" is not how RDS works — plan for the new-instance workflow including endpoint updates); on Azure SQL, point-in-time restore creates a new database (same server or another); on EC2/VMs, the classic

`RESTORE ... WITH STOPAT` log sequence. The incident-time details that matter: **know the exact target time before starting** (when did the bad data commit? — find it in the audit/fn_dblog forensics first, because every wrong guess costs another full restore cycle); **restore aside, not over** (keep the damaged database available for forensics until the recovery is verified); and **measure restore throughput now** (a 4 TB restore at 200 MB/s is 5.5 hours — that number belongs in the RTO document, not discovered during the outage).

29.5 Cross-Region and Cross-Account Backup Copies

Backups in one region protect against database failure; backups in two regions protect against regional failure; backups in a second account protect against compromised credentials (Chapter 16). The implementations: RDS cross-region snapshot copy (automated on a schedule); S3 cross-region replication for native backups; Azure geo-redundant storage for automated backups plus LTR in the paired region; and **separate-account backup vaults** with write-only access from production (production can write backups, cannot delete them). Automate the copy verification — alert when a scheduled copy does not complete, because a DR copy that silently stopped is the classic audit finding.

29.6 Restore Testing: The Framework That Makes Backups Real

The testing cadence: **automated weekly restores** of a sample of databases (restore to an isolated instance, run DBCC CHECKDB, run a smoke-test query suite, report success/failure); **quarterly PITR drills** (pick a random point in time, restore to it, verify the data matches expectations — this exercises the log chain, not just the full backup); and **annual full DR restores** (the Chapter 16 regional drill). Automate the weekly restores completely — a manual restore test gets skipped the week it matters. Track restore success rate as a KPI; anything below 100% is an incident, because a backup that does not restore is not a backup.

29.7 Backup Encryption, Access Control, and Compliance

Encrypt everything: platform backups inherit the instance's encryption (RDS encryption at rest via KMS, Azure TDE + storage encryption); native backups get `WITH ENCRYPTION` using a certificate, with the certificate backed up separately (an encrypted backup without its certificate is a random number collection). Access control: backup storage buckets/accounts allow writes from the database's role and reads only from the restore automation's role — no human needs routine read access to backup storage. For compliance: immutability (S3 Object Lock, immutable blob storage) for the retention period, and the access logs proving who touched the backups. The ransomware scenario from Chapter 16 is defeated here or nowhere.

29.8 Backup Cost Management

Backups cost real money at scale: snapshot storage per GB-month, cross-region transfer, long-term retention accumulation. The controls: **lifecycle policies** (hot storage for recent backups, cooler tiers for aging ones, expiration at the retention boundary — automate it, because manual cleanup never happens); **differential discipline** (differentials reduce full-backup frequency but grow with change rate — monitor differential size and take fulls before they approach full size); **compression everywhere** (smaller backups are cheaper to store, faster to restore, and cheaper to replicate); and **per-database retention tiers** (production keeps 35 days; dev keeps 7 — the backup bill should reflect the data's value). Review backup spend quarterly; it grows silently and is easy to trim once visible.

Key Takeaways

- → **The platform automates; you verify.** Know exactly what each platform's automated backup includes, and own the verification regardless of who owns the schedule.
- → **Native backups are the portable layer.** Full + differential + 15-minute logs, compression and checksums always, stored in object storage you control. Portability is 80% of DR.

- → **Restore aside, never over — and know the target time first.** Forensics before restore; measured restore throughput in the RTO document.
- → **Two regions, two accounts, immutable.** Cross-region copies, separate-account vaults with write-only production access, immutability for the retention period.
- → **Automated weekly restores are non-negotiable.** CHECKDB + smoke tests on every restore, PITR drills quarterly, restore success rate as a KPI. A backup that doesn't restore isn't a backup.

Chapter 30: Monitoring Cloud Databases: CloudWatch, Azure Monitor, and Beyond

In the cloud you lose the server room — no walking up to the rack, no local PerfMon console — and gain the platform's telemetry firehose. The monitoring challenge for SQL Server in the cloud is not data scarcity but signal: every platform emits hundreds of metrics, and the DBA's job is to select the thirty that predict incidents, alert on the ten that demand action, and dashboard the rest for diagnosis. This chapter builds the SQL Server monitoring stack per cloud, the alerting discipline, and the integration with your existing observability.

30.1 What You Lost and What You Gained

Lost: direct OS access on managed services (no PerfMon, no Task Manager — you get what the platform exposes), and the single-pane view of a homogeneous estate. Gained: every metric with history by default (no more "we weren't collecting that"), platform-native anomaly detection, log aggregation without agents to maintain, and metrics that correlate database and infrastructure automatically. The philosophy shift: monitor **outcomes and leading indicators** (query latency, replication lag, connection failures, backup age) rather than raw counters. A CPU graph is trivia; *p95 query latency by workload* is the business.

30.2 CloudWatch: Monitoring RDS and EC2 SQL Server

For RDS: the essential metric set is `CPUUtilization`, `FreeableMemory`, `ReadIOPS/WriteIOPS` and `ReadLatency/WriteLatency` (latency matters more than IOPS), `DatabaseConnections`, `FreeStorageSpace`, `ReplicaLag` (where applicable), and `TransactionLogsDiskUsage` (the log-disk-full outage is the most preventable RDS incident). Enable **Enhanced Monitoring** for OS-level granularity (process-level CPU/memory on 1-second resolution) and **Performance Insights** for the query-level view (top SQL by load, wait events — the managed replacement for digging through DMVs). For EC2 SQL Server, add the CloudWatch agent for PerfMon counters (the full SQL Server counter set: buffer manager, memory manager, SQL statistics, waits) — self-managed means self-instrumented.

30.3 Azure Monitor: Metrics, Diagnostics, and Log Analytics

Azure Monitor covers Azure SQL (Database, Managed Instance, VMs) with platform metrics plus **diagnostic settings** streaming to Log Analytics: `cpu_percent`, `physical_data_read_percent`, `log_write_percent`, `dtu_consumption_percent` (DTU model), connection counts, and deadlock/timeout counters. The killer feature is **Query Store +**

Intelligent Insights: the platform detects query regressions, timeouts, and resource exhaustion automatically and reports root causes in plain language. Route diagnostics to Log Analytics for KQL-powered investigation (top queries by duration across the fleet, error log aggregation, audit trails) — the queries you write once become the fleet's shared diagnostics.

```
-- Top resource-consuming queries across the fleet (Log Analytics / KQL pattern)
// AzureDiagnostics | where Category == "QueryStoreRuntimeStatistics"
// | summarize avg(duration) by query_id | top 20 by avg_duration_

// Or in T-SQL via Query Store on any single database:
SELECT TOP 20 q.query_id,
              SUM(rs.avg_duration * rs.count_executions) / 1000 AS total_sec,
              SUM(rs.count_executions) AS executions
FROM sys.query_store_query q
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
GROUP BY q.query_id
ORDER BY total_sec DESC;
```

30.4 Google Cloud Monitoring for Cloud SQL and GCE

Cloud Monitoring covers Cloud SQL for SQL Server with database/cpu/utilization, disk bytes/ops, and connections; Cloud Logging aggregates the SQL Server error log without agent maintenance. For GCE-hosted SQL Server, the Ops Agent collects the Windows/SQL Server metrics into the same dashboards. The GCP strength is **uptime checks + alerting policies** integrated with the same monitoring as the application tier — the database and the app that depends on it alert from one system. Query-level insight on Cloud SQL comes from Query Insights (the Performance Insights analog) — enable it; the default metrics alone will not find the bad query.

30.5 Building Effective Dashboards

Three dashboards per database tier: **the health dashboard** (one screen: availability, p95 latency, error rate, replication lag, backup age — the "is it healthy" answer in five seconds); **the performance dashboard** (top queries by total duration, wait statistics breakdown, CPU/memory/I/O trends, plan regression alerts — the "what's slow" workspace); and **the capacity dashboard** (storage growth trajectory, connection trends, CPU headroom — the "when do we act" forecast). Standardize the layout across clouds so a DBA reads any cloud's dashboard without translation (Chapter 24's rule). Every dashboard gets a documented owner and a quarterly review — dashboards nobody looks at are worse than no dashboards, because they create the illusion of observability.

30.6 Alerting Strategy: Signal Without Fatigue

The alerting hierarchy: **page** (availability down, failover occurred, backup failed, log disk critical, replication lag beyond tolerance — wake someone up); **ticket** (CPU sustained above threshold, storage trending to full in 30 days, query regression detected, certificate expiring — work it this week); **log** (everything else — searchable, not notifying). The rules that prevent fatigue: alert on **symptoms and leading indicators**, not raw counters (alert "p95 latency above SLO" not "CPU above 80%"); require **duration** (5 minutes above threshold, not one spike); give every alert a **runbook link** (an alert without a runbook is a notification of confusion); and review alert history monthly — the alert that fires weekly and is always ignored is either miscalibrated or measuring something nobody owns. Kill it or fix it.

30.7 Common Instrumentation Gaps

The gaps that bite: **no query-level visibility** (Performance Insights/Query Insights/Query Store not enabled — the platform metrics show *that* it's slow, never *what* is slow); **no backup/restore monitoring** (backup age and restore-test results are metrics — alert on them like any other); **no client-side metrics** (connection failures and timeouts as seen by the application — the database can look healthy while the app can't reach it); **no cross-boundary visibility** (replication lag across clouds/regions unmonitored — Chapter 24's rule); and **log retention shorter than incident cycles** (keep error logs and audit trails long enough to investigate last month's incident, not just yesterday's).

30.8 Integrating with Existing Observability Stacks

The cloud platforms are not the whole stack — export to yours. Ship CloudWatch metrics/logs to your central system (Datadog, Grafana, Splunk, ELK) via the platform's export mechanisms; keep the cloud-native dashboards as the deep-drill tool and the central platform as the single pane. Standardize **tagging/labels** (environment, tier, owner, application) on every database resource — untagged resources are unmonitorable at fleet scale and unallocatable at billing time. And include the database in the **SLO framework**: availability and latency SLOs per tier, error budgets, and the incident review process — the database is a service with customers (the applications), and it deserves the same reliability engineering as any other.

Key Takeaways

- → **Monitor outcomes, not counters.** p95 latency, error rate, replication lag, backup age — the business's experience of the database, with raw counters for diagnosis.
- → **Enable the query-level tools everywhere.** Performance Insights, Query Store, Query Insights — platform metrics show *that*; query telemetry shows *what*.

- → **Three dashboards per tier, one layout across clouds.** Health, performance, capacity — standardized so any DBA reads any cloud.
- → **Alert on symptoms with duration, link every alert to a runbook.** Page/ticket/log hierarchy; monthly alert review; kill or fix the chronic false alarms.
- → **Close the classic gaps.** Backup age, client-side failures, cross-boundary lag, and log retention — the incidents come from the unmonitored corners.

Chapter 31: SQL Server Authentication in the Cloud: Entra ID, Secrets, and Kerberos

Authentication is where cloud SQL Server diverges most from the on-premises world you know. On-premises, it was Windows authentication against Active Directory — simple, integrated, done. In the cloud, each platform offers a different identity story, SQL logins still exist everywhere, and secrets management becomes a first-class operational concern. This chapter maps the authentication options per platform, the secrets lifecycle, and the hardening that keeps the front door locked. One honest note up front: unlike some other engines, **RDS for SQL Server does not support IAM database authentication** — there is no token-based login for SQL Server on RDS. Plan accordingly.

31.1 The Authentication Landscape per Platform

Platform	Integrated identity	SQL logins	Notes
RDS for SQL Server	Windows Authentication via Managed AD	Yes	No IAM DB auth; Secrets Manager integration for rotation
Azure SQL / MI	Microsoft Entra ID (native)	Yes (contained users)	Entra-only authentication mode available
EC2 / Azure VMs / GCE	Active Directory / Kerberos (domain-joined)	Yes	Full on-premises parity when domain-joined
Cloud SQL for SQL Server	Cloud IAM via Auth Proxy (connectivity)	Yes	Auth Proxy handles IAM-based access control

31.2 Entra ID Authentication for Azure SQL

Azure SQL's native identity is **Microsoft Entra ID**: users and managed identities authenticate without passwords in connection strings. The production pattern: applications use **managed identities** (no stored credentials at all — the platform injects the identity), humans use their Entra accounts with MFA, and SQL logins exist only where Entra cannot reach (legacy integrations). Enable **Entra-only authentication** on servers that have completed the migration — it disables SQL authentication entirely, eliminating the password-spray attack surface. For Managed Instance, Entra authentication works alongside the traditional Windows-auth patterns your team already knows.

31.3 Secrets Management: Rotation Without Outages

Where SQL logins remain (RDS, Cloud SQL, legacy integrations), the credentials live in the platform's secret store — **AWS Secrets Manager**, **Azure Key Vault**, **GCP Secret Manager** — never in code, config files, or (heaven forbid) the deployment pipeline's variables. The rotation discipline: **automatic rotation** on a schedule (Secrets Manager natively rotates RDS credentials, including through RDS Proxy — Chapter 10); **dual-password rotation** where supported (the old password stays valid during the transition so in-flight connections survive); and **application integration** (applications read the secret at startup and handle rotation signals — or better, use the platform's integrated auth and skip passwords entirely). Test rotation in staging first: the rotation that breaks the application's connection pool is discovered at 2 AM on rotation day unless you rehearse it.

31.4 Kerberos and Active Directory in the Cloud

Self-managed SQL Server on EC2, Azure VMs, and GCE can join Active Directory — via **AWS Managed Microsoft AD**, **Entra Domain Services**, or an extended on-premises domain over the hybrid link (Chapter 27) — giving full Windows authentication parity: Kerberos for AGs and linked servers, Windows logins, group-based access. The operational requirements: **SPNs registered correctly** (the AG listener and each replica need proper Service Principal Names — the classic Kerberos failure, now in the cloud); **time synchronization** (Kerberos tolerates 5 minutes of skew — the cloud's NTP usually handles this, but verify on domain controllers); and **constrained delegation** configured for double-hop scenarios (application → SQL Server → linked server). RDS supports Windows Authentication via Managed AD join — the one integrated-auth path on RDS, and worth the setup for estates with Windows-auth applications.

31.5 Contained Users and Least Privilege

Regardless of platform, the authorization model should be **contained database users** where the platform supports them (Azure SQL natively; partially contained on other editions): users authenticated at the database level, portable with the database across failover and migration, with no server-level login to synchronize. The least-privilege discipline: application accounts get exactly the permissions the application needs (execute on specific procedures, not db_owner); human access goes through groups, never individual grants; and the `sa` / admin credentials are vaulted, never used by applications, with every use alerted. Review effective permissions quarterly — privilege creep is silent and permanent unless audited.

31.6 Auditing Authentication: Who Connected, From Where

Authentication without auditing is trust without verification. The baseline: **SQL Server audit** capturing successful and failed logins (failed logins are the brute-force signal — alert on bursts); **platform audit trails** (CloudTrail for Secrets Manager access, Azure Activity Log for Key Vault, Cloud Audit Logs) proving who touched the credentials; and **connection source tracking** (client IP/hostname in the audit — the login from an unexpected network is the compromised-credential signal). Ship audit records to immutable storage with retention matching the compliance requirement (Chapter 26). And audit the auditors' access — the team that can read the audit logs can also, in principle, modify them; separation of duties applies to observability too.

31.7 Troubleshooting Authentication Failures

The failure taxonomy, in order of frequency: **expired/rotated secrets** (the application holds the old password — check rotation timing vs application restart); **network path** (authentication fails because the client cannot reach the domain controller / Entra endpoint / secret store — always verify connectivity before debugging credentials); **Kerberos/SPN issues** (falling back to NTLM or failing outright — check SPNs, delegation, and time skew); **Entra token problems** (expired tokens, wrong tenant, managed identity misconfiguration — the Azure-side equivalent of Kerberos issues); and **permission changes** (the login works but the database denies access — someone changed a role membership). The diagnostic order never changes: network, then credential validity, then identity provider, then authorization. Skipping steps is how a 10-minute fix becomes a 3-hour bridge call.

31.8 Hardening Beyond Authentication

Authentication is one layer. The hardening checklist: **TLS everywhere** (Chapter 32 — encrypted connections enforced, not optional); **no public endpoints** on production databases (private networking only); **MFA for human access** (via Entra / the identity provider — the DBA's account is the highest-value target); **break-glass accounts** (vaulted, monitored, tested — for the day the identity provider is the problem); and **regular access reviews** (quarterly: who has access, at what level, on every platform — Chapter 24's per-cloud review rule). Security is a process with a checklist, not a feature with a checkbox.

Key Takeaways

- → **Know each platform's identity story — including the gaps.** Entra ID natively on Azure SQL; Managed AD join for RDS Windows auth; domain-joined VMs for full Kerberos parity. RDS SQL Server has no IAM database authentication — plan around it.

- → **Passwords live in secret stores with automatic rotation.** Never in code or config. Rehearse rotation in staging; prefer managed identities and Entra-only mode where available.
- → **Contained users and least privilege, everywhere.** Portable authentication, application accounts with minimal grants, vaulted admin credentials, quarterly permission reviews.
- → **Audit logins and credential access.** Failed-login alerting, client source tracking, immutable audit storage. Audit the auditors too.
- → **Troubleshoot in order: network, credential, identity provider, authorization.** And harden beyond auth: TLS, no public endpoints, MFA, break-glass accounts, quarterly reviews.

Chapter 32: VPC, Private Endpoints, and Network Security

The network is the security perimeter for cloud databases — there is no data-center door to lock, so the virtual network, the firewall rules, and the encryption in transit are the locks. This chapter builds the network security architecture for SQL Server on each cloud: isolation, private connectivity, least-privilege firewalling, TLS enforcement, and the misconfigurations to hunt down.

32.1 Virtual Networks: The Foundation of Isolation

Every production SQL Server lives in a **private subnet** — no public IP, no internet gateway route — on AWS (VPC), Azure (VNet), and GCP (VPC). The standard topology: public subnets for load balancers and bastion only; private application subnets; **isolated database subnets** with no route to the internet at all (not even via NAT — patches come through VPC endpoints, backups go through gateway endpoints). Separate subnets per tier (production vs non-production in different subnets or different VPCs/VNets entirely — the blast-radius boundary). And plan the **address space** once, globally: non-overlapping CIDRs across VPCs, regions, clouds, and the on-premises network (Chapters 24 and 27's rule — renumbering a production VPC is the project nobody wants).

32.2 Private Endpoints: Eliminating Public Exposure

Managed databases get private connectivity through each cloud's private-endpoint mechanism: **AWS PrivateLink** (interface VPC endpoints for RDS — traffic never leaves the AWS network), **Azure Private Link / Private Endpoints** (Azure SQL and Managed Instance accessible only via private IPs in your VNet — combined with denying public network access at the server level), and **GCP Private Service Connect / private IP** for Cloud SQL. The configuration that matters: **disable public access explicitly** (on Azure SQL, set public network access to Deny — the default-allow posture is the misconfiguration; on RDS, the publicly-accessible flag must be false and verified by audit, not assumed). Private DNS zones must resolve the database hostnames to the private IPs — the application connects to the same hostname everywhere, and DNS decides the path.

32.3 Security Groups, Firewall Rules, and Least Privilege

The firewall discipline, per cloud (security groups / NSGs / firewall rules — same concept): **default deny, explicit allow**; database ports (1433) reachable only from the application subnets' security groups — never from 0.0.0.0/0, never from "the whole VPC"; **reference**

security groups, not IP addresses (allow from the app tier's security group so autoscaling doesn't break the rules); separate rules per environment; and **egress control** on the database subnets (the database should not initiate outbound internet connections — if it does, something is wrong or something is exfiltrating). For Azure SQL's server-level firewall: prefer VNet rules and private endpoints over IP rules; IP rules are the exception path for break-glass admin access, time-boxed and audited.

32.4 Cross-VPC Connectivity: Peering, Transit, and Shared Services

When the application and database live in different VPCs/VNets (or different accounts/subscriptions — the landing-zone pattern), connect them with **VPC peering** (simple, non-transitive — fine for hub-and-spoke pairs), **Transit Gateway / Virtual WAN** (the enterprise pattern — transitive routing with centralized firewall inspection), or **PrivateLink/Private Service Connect** (expose the database as a service to consumer VPCs without peering at all — the cleanest isolation). The rules: peering is non-transitive ($A \leftrightarrow B$ and $B \leftrightarrow C$ does not give $A \leftrightarrow C$ — design accordingly); route tables must be updated on both sides (the forgotten return route is the classic peering failure); and DNS must resolve across the peering (private hosted zones associated with all participating VPCs).

32.5 Defense in Depth: Combining Network and Database Security

Network controls and database authentication are independent layers — a failure in one must not mean total compromise: the network restricts *who can reach* the database (private subnets, firewall rules); authentication restricts *who can log in* (Chapter 31); authorization restricts *what they can do* (least privilege); and **auditing** records what happened (Chapters 26 and 31). Add **threat detection** where the platform offers it (Azure Defender for SQL's vulnerability assessment and threat detection, GuardDuty for RDS-adjacent anomaly detection) — anomalous login patterns and SQL injection signatures caught by the platform, not by hope. No single layer is trusted alone; the combination is the security posture.

32.6 TLS Enforcement and Certificate Management

Encrypt in transit, enforced: **require encrypted connections** on every SQL Server (RDS: the `rds.force_ssl` parameter; Azure SQL: encryption is enforced by default with the option to verify; self-managed: Force Encryption in SQL Server Configuration Manager with a proper certificate). The details that matter: **certificate validation** — the client must validate the server certificate (`TrustServerCertificate=false` in production connection strings — the application that skips validation is vulnerable to MITM on its own network); **TLS version floor** (disable TLS 1.0/1.1 — compliance requires it, and the handshake failures from legacy clients

are found in testing, not production); and **certificate rotation** — platform-managed certificates rotate automatically (verify the rotation doesn't break clients that pinned the old cert); self-managed certificates need a rotation runbook with client trust-store updates.

32.7 Auditing Network Access and Detecting Anomalies

The network audit baseline: **flow logs** (VPC Flow Logs, NSG flow logs, GCP firewall logging — who connected to the database subnets, when, how much data moved); **firewall change auditing** (every security group / NSG / firewall rule change logged and alerted — an opened database port is an incident until proven otherwise); and **anomaly detection** (connections from new networks, unusual data volumes, off-hours access patterns — the compromised-credential signal from Chapter 31, seen at the network layer). Retain flow logs long enough to investigate (30–90 days minimum); the breach discovered in week six needs week-one logs.

32.8 Network Security as Code

Every firewall rule, private endpoint, and route table in version-controlled infrastructure code (Chapter 28's IaC rule) — with **policy enforcement** (AWS Config rules, Azure Policy, GCP org policies) that *deny* the dangerous configurations: no public database endpoints, no 0.0.0.0/0 ingress to database ports, mandatory TLS, required tags. Manual console changes to network security are drift (Chapter 23's discipline) — detect and revert automatically. The network security posture is a codebase with tests, not a collection of console clicks someone remembers making.

32.9 Common Misconfigurations and How to Find Them

The hunt list, run quarterly: publicly accessible database flags set true; 0.0.0.0/0 in database security groups; Azure SQL servers with public network access allowed; forgotten IP firewall rules from break-glass sessions; cross-account peering to unknown accounts; expired or self-signed certificates in production; DNS resolving database hostnames to public IPs; and flow logs disabled on database subnets. Automate the hunt — a scheduled audit script across all three clouds (or the CSPM tool you already pay for) — because manual quarterly reviews get skipped exactly when the misconfiguration arrives.

Key Takeaways

- → **Private subnets, private endpoints, explicit public-access denial.** No production database with a public IP. Private DNS resolving to private IPs.
- → **Firewall least privilege: 1433 from app-tier security groups only.** Reference groups, not IPs. Egress-controlled database subnets. IP rules as time-boxed exceptions.

- → **TLS enforced with certificate validation.** Force encryption, validate server certs (TrustServerCertificate=false), TLS 1.2+ floor, rotation runbooks.
- → **Defense in depth, audited.** Network + auth + authorization + auditing + threat detection. Flow logs, firewall-change alerts, anomaly detection.
- → **Network security as code with policy enforcement.** Deny the dangerous configurations by policy; hunt the misconfiguration list quarterly, automated.

Chapter 33: Database Migration to Cloud: The Practical Runbook

Migration is the project where cloud database theory meets the production estate: hundreds of databases, decades of accumulated quirks, linked servers, Agent jobs, CLR assemblies, and a business that expects zero surprises. This chapter is the practical runbook — assessment, method selection, the migration wave plan, cutover, rollback, and the post-migration optimization that finishes the job. Chapter 34 covers the zero-downtime techniques in depth; this chapter covers running the whole program.

33.1 Migration Assessment: Knowing What You're Moving

Inventory before anything moves. For each SQL Server: version and edition, database sizes and growth rates, compatibility level, and the feature inventory that determines the target — Agent jobs (Chapter 12), linked servers, replication topologies, CLR assemblies, Service Broker, FILESTREAM/FileTable, Always On AGs, SSIS/SSAS/SSRS dependencies, cross-database queries and three-part naming. Run the platform assessment tools (**Azure Data Migration Assistant / Azure Migrate**, **AWS DMS Fleet Advisor / Schema Conversion Tool**) — they flag breaking changes and unsupported features mechanically, which beats discovering them during cutover. Classify each database: **rehost** (lift-and-shift to EC2/VMs — minimal change), **replatform** (to RDS/Azure SQL/MI — some adaptation), or **retain/replace** (stays on-premises, or the application is being replaced anyway). The classification drives the wave plan.

33.2 Choosing the Migration Method per Database

Method	Downtime	Best for
Native backup/restore	Hours (restore time)	Small-medium DBs with maintenance windows; universal fallback
Log shipping / differential catch-up	Minutes	Large DBs where the seed restores early and logs catch up
Transactional replication	Seconds–minutes	Near-zero downtime; publisher stays live until cutover
Distributed AG	Seconds	EC2/VM targets; automatic failover semantics for cutover
	Minutes	

Method	Downtime	Best for
DMS / cloud migration service		Managed-service targets; homogeneous SQL Server supported

The default for most databases: backup/restore for the small ones (simple, well-understood), log-shipping-style catch-up for the large ones, replication or DMS where downtime is constrained. Always have the backup/restore path available as the fallback — the exotic method that fails at 2 AM needs a boring alternative.

33.3 The Wave Plan: Order, Grouping, and Dependencies

Migrate in waves, not all at once: **wave zero** (non-production, low-risk databases — proves the pipeline, the tooling, and the runbook); **early waves** (independent production databases with tolerant downtime windows); **late waves** (the interconnected systems — linked servers, replication chains, shared Agent jobs — migrated as dependency groups, in dependency order). Map the dependencies explicitly (which database's linked server points where, which Agent job touches which databases, which application connects to which) — the migration that breaks an unmapped dependency is the classic program failure. Each wave gets a go/no-go review: the previous wave's lessons incorporated, the next wave's runbook updated.

33.4 The Target Environment: Built Before the First Migration

The cloud landing zone must be production-ready before wave zero: networking (private connectivity, DNS, firewall rules — Chapter 32), identity (Chapter 31), monitoring (Chapter 30), backup (Chapter 29), and the target SQL Server instances sized from the assessment (Chapter 39's right-sizing, applied prospectively). Deploy the **automation first**: the Agent job equivalents (Chapter 12), the maintenance routines, the alerting — so the first migrated database lands in an operated environment, not an empty cloud account. And load-test the target: restore the largest production database and run the peak workload against it before the business depends on it.

33.5 Cutover Runbook: The Final 60 Minutes

Every cutover follows the same script (customize per database, rehearse per wave): stop application writes (or put the application in read-only/maintenance mode); let replication/logs drain to zero lag (verify — do not assume); run the final differential/log backup and restore; update statistics and check integrity on the target (`DBCC CHECKDB` — the restore that passes CHECKDB is the restore you trust); redirect connection strings (DNS flip, config deployment, or listener update — Chapter 16's redirection discipline); start the application and run the smoke-test suite (the business's critical transactions, not just "can we connect");

and monitor the new production intensely for the first hours (the query plan regressions from a new environment show up fast). Assign every step an owner and a timebox; the runbook with "someone" as the owner is a wish, not a plan.

33.6 Rollback Plan

Define the rollback trigger *before* cutover (which failures roll back vs roll forward — Chapter 23's discipline applied to migration): keep the source database intact and replication-capable until the migration is declared successful (the rollback window — typically 24–72 hours); for replication-based migrations, reverse replication can return writes to the source; for backup/restore migrations, rollback means redirecting clients back and accepting the writes made on the target (reconcile or lose them — decide in advance which). The rollback that was never tested is a hope; test it on a non-production wave.

33.7 Post-Migration Optimization

Migration is not finished at cutover. The post-migration checklist: **update statistics and rebuild fragmented indexes** (the restore process fragments; the new storage has different characteristics); **re-baseline performance** (Query Store captures the new normal — compare against pre-migration baselines and investigate regressions immediately); **right-size the target** (the assessed size was a guess — Chapter 39 with real cloud metrics, usually downward); **enable the cloud-native features** (automated backups verified, monitoring dashboards live, alerting active — the target is not "done" until it's operated); and **decommission the source** (on a schedule, with a final backup retained — the source server that lingers "just in case" becomes a security and licensing liability).

33.8 Licensing and Commercial Readiness

Migration has a commercial track running parallel to the technical one: confirm **license entitlements** for the target (Azure Hybrid Benefit eligibility with Software Assurance, License Mobility verification for AWS, sole-tenant requirements for GCP BYOL — Chapter 7/8/13/19's rules, resolved before the first workload moves, not during); true-up the **cloud spend forecast** against the business case (the first invoice after migration is the moment the program gets judged — make sure it matches the model); and align **support contracts** (the on-premises support you're retiring, the cloud support plan you're buying — ensure there's no coverage gap during the transition). The migration that succeeds technically but violates licensing or blows the budget is not a success — commercial readiness is part of done.

Key Takeaways

- → **Assess mechanically, classify explicitly.** Feature inventory per database, platform assessment tools, rehost/replatform/retain classification driving the wave plan.
- → **Match the method to the downtime budget.** Backup/restore as the universal fallback; log catch-up, replication, or DMS for constrained windows. Boring beats exotic at 2 AM.
- → **Waves with dependency mapping and go/no-go reviews.** Wave zero proves the pipeline; dependency groups migrate together, in order.
- → **Cutover is a scripted 60 minutes.** Drain to zero lag, CHECKDB the target, redirect clients, smoke-test the business transactions — every step owned and timeboxed.
- → **Migration ends with optimization and decommissioning.** Re-baseline performance, right-size on real metrics, operate the target fully, retire the source on schedule.

Chapter 34: Zero-Downtime Cloud Migrations

Some databases cannot take a maintenance window — the order system, the trading platform, the always-on SaaS. For these, migration must happen with the application running: continuous replication from source to target, a cutover measured in seconds, and a rollback path that works just as fast. This chapter covers the zero-downtime techniques for SQL Server: the replication architectures, the cutover choreography, and the validation that proves the migration before the business notices it.

34.1 What "Zero Downtime" Actually Requires

Zero downtime does not mean zero risk — it means the risk moves from the cutover window to the replication design. The prerequisites: **a replication mechanism with seconds of lag** (transactional replication, distributed AGs, or DMS CDC — Chapter 33's table); **schema parity guaranteed** (the target schema deployed via the Chapter 23 pipeline, with drift detection proving source and target match before cutover); **a cutover that is a client redirect, not a data move** (by cutover time, the data is already there — the only operation is pointing clients at the target); and **rehearsal** (the full cutover practiced on a staging pair until it is boring). If any prerequisite is missing, you do not have a zero-downtime migration — you have a hope with replication.

34.2 Transactional Replication for Near-Zero Cutover

The classic approach: the on-premises (or source-cloud) SQL Server publishes, the cloud target subscribes, and replication runs continuously for days or weeks before cutover. The engineering details: **seed from backup** (initialize the subscription from a native backup, not a snapshot — Chapter 25's WAN rule); **replicate everything the application needs** (tables, of course — but also verify: identity ranges managed so the subscriber doesn't collide, sequences, and any lookup data the application expects); **monitor lag as the migration's vital sign** (the lag graph should be flat and small for the entire pre-cutover period — a growing lag means the cutover will fail); and **handle the non-replicated objects** (Agent jobs rebuilt per Chapter 12, logins migrated with SIDs preserved, linked servers recreated — the migration checklist from Chapter 33, executed during the replication window rather than during downtime).

34.3 Distributed AGs as the Migration Vehicle

Where the target is EC2/VMs/GCE, a **distributed availability group** spanning source and target is the most elegant vehicle: the source AG and the target AG replicate continuously, and cutover is an AG failover — seconds, with the database engine handling the consistency. The setup: seed the target AG from backup, join it to the distributed AG as the secondary, and let it synchronize. The cutover choreography: quiesce writes briefly (or accept the async lag window explicitly), fail over the distributed AG, redirect clients to the target listener. The advantage over replication: whole-database fidelity (everything — including the objects replication forgets — moves as one unit) and engine-managed consistency. The constraint: both sides need AG-capable infrastructure (no managed-service targets).

34.4 DMS and Cloud-Native CDC Paths

For managed-service targets (RDS, Azure SQL, Cloud SQL), the cloud migration services provide the CDC path: **AWS DMS** (full load plus ongoing CDC from SQL Server sources — verify the source version support and the LOB handling for your schema); **Azure Database Migration Service** (online migrations to Azure SQL targets with minimal downtime); **GCP Database Migration Service** (homogeneous SQL Server to Cloud SQL). The universal caveats: **test LOB and large-row handling** (CDC tools have opinions about varchar(max), varbinary(max), and wide rows — find the limits with your data, not the documentation's); **validate data continuously** (row counts and checksums during the replication window, not just at cutover — DMS table statistics lie by omission); and **understand the cutover model** (each service has a specific cutover procedure — rehearse it exactly, because improvising the vendor's procedure under pressure is how migrations fail).

34.5 Cutover Orchestration: The Seconds That Matter

The cutover sequence, orchestrated and timed: **pre-cutover checks** (lag at zero, schema parity confirmed, target health verified, rollback path armed); **the quiet second** (drain application connections or briefly pause writes — even "zero downtime" migrations usually have a seconds-long write pause; the honest ones admit it); **the redirect** (DNS flip with pre-lowered TTL, connection string deployment, or listener update — Chapter 16's discipline); **verification** (smoke tests against the target within 60 seconds — the business's critical writes succeeding on the new primary); and **the point of no return** (declared explicitly — after this, rollback means reverse migration, not a quick flip). Automate the sequence; the human's job is go/no-go decisions, not typing commands.

34.6 Post-Migration Validation: Proving It Worked

Validation is a checklist, not a feeling: **data validation** (row counts per table, checksums on critical tables, spot-checks of recent transactions — the data that arrived in the last replication seconds is the data most likely to be wrong); **application validation** (the full smoke suite plus the business's day-one critical paths); **performance validation** (Query Store baselines compared — the new environment's plans will differ; investigate regressions in the first hours, not the first week); and **operational validation** (backups running, monitoring live, alerting active, Agent jobs executing — Chapter 33's "operated environment" confirmed). Keep the source in rollback-ready state until every validation passes plus a safety margin.

34.7 Rollback at Speed and Post-Migration Cleanup

The zero-downtime rollback: **reverse the redirect** (clients back to the source — fast, because the source was kept warm); **reconcile the writes** (the writes that landed on the target during the failed cutover must be replayed to the source or explicitly discarded — this is the hard part, and the reason the verification in 34.5 must be ruthless); **reverse replication where available** (DMS and distributed AGs can often reverse direction — test this before cutover, not during rollback). After a successful migration: decommission replication (remove publishers, drop the distributed AG, stop DMS tasks — leftover replication is a failure waiting to happen), retire the source on schedule, and hold the post-mortem while memories are fresh — every migration teaches something the runbook should capture.

34.8 Application Compatibility: The Silent Migration Killer

The data migrates cleanly; the application breaks. The compatibility checklist for zero-downtime migrations: **connection behavior** (connection string changes, TLS requirements, and retry logic — the application must handle the brief redirect at cutover without cascading failures); **version-specific behavior** (a newer SQL Server version on the target means new cardinality estimator behavior, deprecated features removed, and compatibility-level differences — test the application against the target version under load, not just functionally); **case sensitivity and collation** (verify the target's collation matches — a collation mismatch surfaces as bizarre, intermittent query failures); and **distributed transactions** (MSDTC across the migration boundary rarely survives — identify and redesign them before cutover). Run the application's full integration suite against the replicated target during the pre-cutover window; the migration rehearsal that skips application testing is rehearsing half the migration.

Key Takeaways

- → **Zero downtime moves risk to replication design.** Seconds of lag, guaranteed schema parity, client-redirect cutover, rehearsed until boring — or it's not zero-downtime.
- → **Seed from backup, monitor lag as the vital sign.** Transactional replication, distributed AGs, or DMS CDC — the mechanism matters less than the discipline around it.
- → **Cutover is an orchestrated sequence with a declared point of no return.** Automate the steps; humans make go/no-go decisions.
- → **Validate data, application, performance, and operations.** Checksums on recent transactions, Query Store baselines, backup/monitoring/Agent jobs confirmed live.
- → **Rollback is reverse-redirect plus write reconciliation.** Test reverse replication before cutover; decommission migration infrastructure after success.

Chapter 35: Query Performance Tuning on Managed Services

The cloud does not change the query optimizer — a bad plan is a bad plan on RDS, Azure SQL, or bare metal. What changes is the tooling around tuning: which DMVs you can reach, how execution plans are captured, what index operations are allowed, and how you validate improvements without the on-premises lab. This chapter is SQL Server performance tuning adapted to managed services: finding the slow queries, reading the plans, fixing indexes, analyzing waits, and proving the improvement.

35.1 Capturing and Identifying Slow Queries

The source of truth is **Query Store** — enabled by default on Azure SQL, available on RDS and self-managed: it captures query text, plans, and runtime statistics persistently, surviving restarts and failovers. The identification query never changes:

```
-- Top queries by total duration (the tuning backlog, ordered by impact)
SELECT TOP 20
    q.query_id,
    SUM(rs.count_executions) AS executions,
    SUM(rs.avg_duration * rs.count_executions) / 1000000.0 AS total_seconds,
    SUM(rs.avg_cpu_time * rs.count_executions) / 1000000.0 AS total_cpu_seconds,
    SUM(rs.avg_logical_io_reads * rs.count_executions) / 1024.0 AS total_logical_mb
FROM sys.query_store_query q
JOIN sys.query_store_plan p ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats rs ON p.plan_id = rs.plan_id
GROUP BY q.query_id
ORDER BY total_seconds DESC;
```

On RDS, supplement with **Performance Insights** (top SQL by load with wait-event breakdown — the fastest path from "it's slow" to the query); on Azure, **Intelligent Insights** flags regressions automatically. Tune in order of total impact (total_seconds), not average duration — the query running 100,000 times a day at 50ms matters more than the nightly report at 5 minutes.

35.2 Reading Execution Plans on Managed Services

Execution plans work identically on managed services — capture actual plans via Query Store (forced plans visible per query_id) or `SET STATISTICS PROFILE` / Extended Events where the platform permits. The reading discipline: **find the most expensive operator by actual rows vs estimated rows** (the cardinality estimation error is the root of most bad plans); check for

implicit conversions (the plan warning that turns a seek into a scan — fix the data type, not the query); look for **spills** (sort/hash warnings — memory grant issues, section 35.4); and verify **parameter sensitivity** (the plan that's fast for one parameter value and catastrophic for another — Query Store's plan forcing or `OPTION (RECOMPILE)` / optimize-for hints, applied surgically).

35.3 Index Tuning in the Cloud: What Works and What Doesn't

Index mechanics are unchanged; the cloud constraints are operational: **online index operations** (`ONLINE = ON`) for every production build/rebuild — the managed service won't forgive a blocking offline rebuild any more than on-premises would; **missing-index DMVs** (`sys.dm_db_missing_index_details`) as the suggestion feed, filtered by impact and validated against the workload (the DMV suggests per-query indexes — the DBA designs the index set); **unused-index removal** (`sys.dm_db_index_usage_stats` — every unused index costs write IOPS and storage, and on provisioned-IOPS cloud storage that cost is metered); and **intelligent maintenance** (threshold-based rebuild/reorganize, not blanket nightly rebuilds — Chapter 12's rule, because the IOPS are billed).

35.4 Wait Statistics: Finding the Real Bottleneck

When the query is "slow" but the plan looks fine, the answer is in `sys.dm_os_wait_stats`: the waits tell you what the queries are waiting for. The cloud-relevant wait types:

PAGEIOLATCH_* (storage latency — the cloud storage tier is too slow or throttled; Chapter 36); **CXPACKET/CXCONSUMER** (parallelism — often excessive parallelism on cloud VMs with many vCPUs; tune `MAXDOP` and cost threshold); **LCK_*** (blocking — application transaction design, not a cloud problem, but the cloud's latency makes it more visible);

RESOURCE_SEMAPHORE (memory grant waits — the instance is under-memory for the workload); **ASYNC_NETWORK_IO** (the application isn't consuming results — a client problem masquerading as a database problem). Sample waits differentially (snapshot, wait, snapshot) — cumulative counters since restart lie about the present.

35.5 Configuration Tuning Within Managed Limits

Managed services restrict some knobs and expose others. The high-value settings available on RDS (parameter groups), Azure SQL (database scoped configurations), and self-managed: **MAXDOP and cost threshold for parallelism** (the two most impactful instance-level settings — start `MAXDOP` at the core count up to 8, raise the cost threshold from the ancient default of 5); **max server memory** (self-managed and RDS — leave headroom for the OS; on managed services the platform manages it, but verify); **database scoped configurations** (`LEGACY_CARDINALITY_ESTIMATION`, `PARAMETER_SNIFFING`, `QUERY_OPTIMIZER_HOTFIXES` — per-database, online, and revertible — the safe way to test optimizer behavior changes);

and **intelligent query processing** (batch mode on rowstore, adaptive joins, memory grant feedback — enabled by compatibility level; keep databases at current compat levels to receive them).

35.6 Query Rewrites and Application-Level Optimizations

When indexes and configuration are exhausted, the query itself changes: **eliminate the patterns** (SELECT *, functions on indexed columns, OR-chains that defeat seeks, cursors doing set work); **fix the data types** (implicit conversions resolved at the schema or parameter level); **batch the chatter** (the application doing 1,000 round trips does not get faster with a bigger database — table-valued parameters and set-based APIs); and **cache deliberately** (the repeated identical query is a caching problem, not a database problem). Every rewrite gets a before/after measurement from Query Store — the tuning that isn't measured is a story, not an improvement.

35.7 Monitoring Performance Over Time

Tuning decays without monitoring: **Query Store regression detection** (alert on plan changes that increase duration — automatic on Azure, queryable everywhere); **the weekly top-queries review** (the tuning backlog from 35.1, revisited — new queries enter the top 20 as the workload evolves); **wait-trend dashboards** (the bottleneck migrates as you fix things — today's PAGEIOLATCH is tomorrow's CXPACKET); and **post-deployment verification** (every application release gets a Query Store comparison — the release that doubles a key query's runtime is caught in hours, not discovered in the monthly review). Performance is a process with a cadence, not a project with an end date.

35.8 Plan Forcing and Hints: The Last Resort, Used Properly

When the optimizer persistently chooses the wrong plan and rewrites aren't feasible, **Query Store plan forcing** pins the known-good plan: identify the good plan_id from Query Store history, force it with `sp_query_store_force_plan`, and monitor — forced plans are verified at each execution, and the force fails safe (the optimizer falls back to normal optimization) if the plan becomes invalid. The discipline: force plans, not hints in code (hints embedded in application SQL are forever; Query Store forces are visible, manageable, and removable); document every forced plan with the reason and a review date; and re-test forces after statistics updates, index changes, and version upgrades — the forced plan that was optimal in March can be the bottleneck in September. Plan forcing is a splint, not a cure: the underlying statistics or schema issue still deserves fixing.

Key Takeaways

- → **Query Store is the source of truth on every platform.** Tune by total impact (total_seconds), not average duration. Performance Insights/Intelligent Insights accelerate the hunt.
- → **Read plans for cardinality errors, conversions, spills, and parameter sensitivity.** The optimizer is the same in the cloud — the diagnostic discipline is what transfers.
- → **Index operations are online-only in production.** Missing-index DMVs as suggestions, usage stats for removal, threshold-based maintenance — metered IOPS make every rebuild a cost decision.
- → **Waits identify the true bottleneck.** Sample differentially; map PAGEIOLATCH to storage, CXPACKET to parallelism, RESOURCE_SEMAPHORE to memory, ASYNC_NETWORK_IO to the client.
- → **Tune MAXDOP/cost threshold first, use database scoped configurations for the rest.** Keep compat levels current for intelligent query processing. Measure every change in Query Store.

Chapter 36: Storage Tiers: gp2 vs gp3 vs io1 vs io2

Storage decisions that took minutes on-premises — "provision a 2TB LUN from the SAN, set stripe size to 64KB, done" — become architecture decisions in AWS RDS with lasting cost and performance consequences. The volume type you select at instance creation determines not just your baseline throughput and IOPS ceiling, but also how you scale, what you pay, and whether you can migrate away without downtime. This chapter breaks down the four RDS storage types — gp2, gp3, io1, and io2 — from the perspective of a DBA who already understands disk I/O mechanics and just needs to know how AWS maps those mechanics onto managed storage. You will learn when to use each tier, how to configure them correctly via CLI and Terraform, how to avoid the gp2 burst credit trap that degrades production performance silently, and how to make the right call for OLTP, analytics, and mixed workloads.

Understanding the Storage Landscape: What Replaced Your SAN

On-premises, you controlled every layer: disk spindle count, RAID level, controller cache, queue depth, stripe alignment. RDS abstracts all of that, but the underlying physics still apply. What AWS gives you is a set of pre-tuned abstractions backed by EBS (Elastic Block Store), and your job is to pick the right abstraction for your workload pattern.

The four volume types map loosely to familiar concepts:

Volume Type	On-Premises Analog	Best For
gp2	Mid-range SAN with burst tier	Dev, test, small OLTP
gp3	Performance SAN with flat pricing	Most production workloads
io1	All-flash array, dedicated IOPS	I/O-intensive OLTP
io2	All-flash array, enterprise durability	Mission-critical, financial systems

Key distinction before anything else: gp2 and gp3 are "general purpose" SSDs. io1 and io2 are "provisioned IOPS" SSDs. The naming matters because it controls what guarantees AWS makes about latency and IOPS consistency. General purpose volumes can throttle. Provisioned IOPS volumes guarantee sub-millisecond latency at the configured IOPS level.

The one fact that changes everything: gp2 ties IOPS to storage size. gp3 decouples them. This single architectural difference makes gp3 almost always the better default for new provisioning in 2024.

gp2: The Legacy Default You Should Stop Using

gp2 allocates 3 IOPS per GB of storage, with a minimum of 100 IOPS and a maximum of 16,000 IOPS. That ceiling hits at 5,334 GB (approximately 5.3 TB). Beyond that, you get no additional IOPS regardless of how much storage you add.

The burst mechanism works exactly like EC2 T-series CPU burst credits. Volumes under 1 TB accumulate burst credits allowing 3,000 IOPS for short periods. When credits are exhausted, performance drops to the baseline 3 IOPS/GB. A 100 GB gp2 volume has a baseline of 300 IOPS. During credit burst it can hit 3,000 IOPS. When the credits run out — which happens after roughly 30 minutes of sustained 3,000 IOPS on a 100 GB volume — performance collapses to 300 IOPS. Applications do not receive warnings. CloudWatch metrics will show `BurstBalance` declining, but unless you have alarms configured, you will discover the problem when your query times triple at 2 AM.

The burst credit math:

```
Credit balance: starts at 5,400,000 (enough for 30 minutes at 3,000 IOPS)
Credit earn rate: baseline_IOPS * 1 credit/second
Credit consumption: (actual_IOPS - baseline_IOPS) * 1 credit/second
```

```
For a 200 GB gp2 volume (600 IOPS baseline):
- Burst IOPS available: 3,000
- Consumption rate at full burst: 2,400 credits/second
- Depletion time: 5,400,000 / 2,400 = 37.5 minutes
```

For volumes over 1 TB (baseline > 3,000 IOPS), the burst mechanism no longer applies because the baseline already meets or exceeds 3,000 IOPS. This is why many teams accidentally run fine on gp2 for years — their storage grew large enough to push past the burst ceiling before they noticed.

gp3: The Right Default for Most Production Workloads

gp3 decouples IOPS from storage size entirely. Every gp3 volume starts with 3,000 IOPS and 125 MB/s throughput included in the base price, regardless of volume size. You can provision up to 16,000 IOPS and 1,000 MB/s independently, paying only for what you configure above the baseline.

This matters practically: a 100 GB gp3 volume gets the same guaranteed 3,000 IOPS as a 10 TB gp3 volume at base pricing. No burst credits, no performance collapse, no minimum storage floor to hit an IOPS target.

Pricing structure (verify exact current rates for your region in the AWS Pricing Calculator before budgeting — EBS pricing has changed over time and varies by region):

Volume Type	Storage Cost	IOPS Cost	Throughput Cost
-------------	--------------	-----------	-----------------

---	---	---	---
-----	-----	-----	-----

gp2	Per GB-month, IOPS bundled into the rate	Included (tied to size)	Included
-----	--	-------------------------	----------

gp3	Per GB-month, lower than gp2's rate	3,000 IOPS included free, then billed per additional provisioned IOPS	125 MB/s included free, then billed per additional MB/s
-----	-------------------------------------	---	---

io1	Per GB-month, higher than gp2/gp3	Billed per provisioned IOPS	Included
-----	-----------------------------------	-----------------------------	----------

io2	Same per-GB rate as io1	Billed per provisioned IOPS (tiered — lower per-unit rate at very high IOPS)	Included
-----	-------------------------	--	----------

The storage cost alone makes gp3 cheaper per GB than gp2. The bigger driver of savings, though, is structural: on a volume that needs IOPS well above the 3-IOPS-per-GB gp2 ratio, gp2 forces you to over-provision storage just to reach the IOPS target, while gp3 lets you buy the IOPS and the storage independently. Run the actual numbers for your target IOPS and storage size against current pricing before committing — the relative savings depend on how far your IOPS requirement outpaces what gp2's ratio would force you to provision, not on a fixed percentage.

io1 vs io2: When You Need Guaranteed IOPS

Provisioned IOPS volumes exist for workloads where gp3's 16,000 IOPS ceiling is insufficient, or where you need single-digit millisecond latency guarantees that you are willing to pay for explicitly.

io1: The Workhorse Provisioned Tier

io1 allows up to 64,000 IOPS per volume (on Nitro-based instances) and up to 1,000 MB/s throughput. The IOPS-to-storage ratio constraint is critical: you can provision a maximum of 50 IOPS per GB of storage. A volume needing 64,000 IOPS requires at minimum 1,280 GB of storage, even if your actual data fits in 200 GB. This is the io1 "storage tax" — you pay for storage you do not need just to satisfy the ratio constraint.

io1 durability is 99.8–99.9% (an annual failure rate of between 0.1% and 0.2% per volume). This sounds low, but io2 improves it dramatically.

io2: io1's Successor with Better Economics

io2 offers 99.999% durability (100x more durable than io1), the same IOPS ceiling of 64,000 per volume, and a more favorable IOPS-to-storage ratio of 500 IOPS per GB (10x better than io1). The 500:1 ratio means that same 64,000 IOPS volume only requires 128 GB of storage rather than 1,280 GB, eliminating most of the storage tax.

Per-IOPS pricing between io1 and io2 is the same, making io2 strictly better than io1 for new provisioning — you get the same IOPS cost with a far better IOPS-to-storage ratio and materially higher durability. AWS has not officially deprecated io1, but the industry direction is clear: io2 is the recommended provisioned tier.

io2 Block Express (available on R5b, X2idn, X2iedn, and newer instance families) extends the ceiling further: 256,000 IOPS, 4,000 MB/s throughput, and sub-millisecond P99 latency. For extreme OLTP workloads — financial transaction systems, high-frequency trading audit logs, real-time fraud detection — this is the tier to evaluate.

When provisioned IOPS actually makes sense:

- → You need more than 16,000 IOPS (gp3 ceiling)
- → Your workload has consistent I/O patterns and latency sensitivity that gp3's shared infrastructure cannot guarantee
- → You have a mixed read/write ratio with concurrent I/O from dozens of connections
- → Regulatory requirements mandate documented performance guarantees (some compliance frameworks require it)

When provisioned IOPS is overkill:

- → Workloads with high average IOPS but acceptable burst latency
- → Development and staging environments
- → Read-heavy workloads with caching layers (ElastiCache, readable AG secondaries) absorbing the majority of reads
- → Any workload under 16,000 sustained IOPS

Configuring Storage: CLI, Terraform, and Modification Patterns

Creating Instances with Specific Storage Types

```

# Create a production SQL Server RDS instance with gp3 storage
# Note: --iops and --storage-throughput only apply to gp3/io1/io2
aws rds create-db-instance \
  --db-instance-identifier prod-sqlserver-01 \
  --db-instance-class db.r6i.2xlarge \
  --engine sqlserver-se \
  --engine-version 15.00.4236.7.v1 \
  --master-username dbadmin \
  --master-user-password "$(aws secretsmanager get-secret-value \
    --secret-id prod/rds/master-password \
    --query SecretString \
    --output text)" \
  --allocated-storage 500 \
  --storage-type gp3 \
  --iops 6000 \
  --storage-throughput 500 \
  --storage-encrypted \
  --kms-key-id arn:aws:kms:us-east-1:123456789012:key/mrk-abc123 \
  --vpc-security-group-ids sg-0abc123def456789 \
  --db-subnet-group-name prod-db-subnet-group \
  --backup-retention-period 7 \
  --preferred-backup-window "03:00-04:00" \
  --preferred-maintenance-window "sun:05:00-sun:06:00" \
  --deletion-protection \
  --region us-east-1

# Create an io2 instance for high-IOPS OLTP workload
aws rds create-db-instance \
  --db-instance-identifier prod-sqlserver-oltp \
  --db-instance-class db.r6i.4xlarge \
  --engine sqlserver-ee \
  --engine-version 15.00.4312.2.v1 \
  --master-username sa \
  --master-user-password "${SA_PASSWORD}" \
  --allocated-storage 2000 \
  --storage-type io2 \
  --iops 64000 \
  --storage-encrypted \
  --license-model license-included \
  --region us-east-1

```

```

# Modify an existing gp2 instance to gp3
# This is the most common migration task – no downtime required
aws rds modify-db-instance \
  --db-instance-identifier legacy-sqlserver-gp2 \
  --storage-type gp3 \
  --iops 6000 \
  --storage-throughput 500 \
  --apply-immediately

# Monitor the modification status
aws rds describe-db-instances \
  --db-instance-identifier legacy-sqlserver-gp2 \
  --query 'DBInstances[0].{
    Status:DBInstanceStatus,
    StorageType:StorageType,
    IOPS:Iops,
    Throughput:StorageThroughput,
    PendingModified:PendingModifiedValues
  }' \
  --output table

# Check the pending modified values before apply-immediately
aws rds describe-db-instances \
  --db-instance-identifier legacy-sqlserver-gp2 \
  --query 'DBInstances[0].PendingModifiedValues'

```

```
# Monitor gp2 burst balance – set this alarm before any production gp2 usage
aws cloudwatch put-metric-alarm \
  --alarm-name "RDS-BurstBalance-Low-legacy-sqlserver" \
  --alarm-description "gp2 burst balance below 30% – performance degradation
imminent" \
  --metric-name BurstBalance \
  --namespace AWS/RDS \
  --statistic Average \
  --dimensions Name=DBInstanceIdentifier,Value=legacy-sqlserver-gp2 \
  --period 300 \
  --evaluation-periods 2 \
  --threshold 30 \
  --comparison-operator LessThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:rds-alerts \
  --ok-actions arn:aws:sns:us-east-1:123456789012:rds-alerts \
  --treat-missing-data missing

# Also alarm on IOPS hitting the provisioned limit
aws cloudwatch put-metric-alarm \
  --alarm-name "RDS-IOPSNearLimit-prod-sqlserver" \
  --alarm-description "IOPS exceeding 90% of provisioned capacity" \
  --metric-name ReadIOPS \
  --namespace AWS/RDS \
  --statistic Average \
  --dimensions Name=DBInstanceIdentifier,Value=prod-sqlserver-01 \
  --period 60 \
  --evaluation-periods 5 \
  --threshold 5400 \
  --comparison-operator GreaterThanThreshold \
  --alarm-actions arn:aws:sns:us-east-1:123456789012:rds-alerts
```

Terraform Configuration for Storage Tiers

```

# terraform/modules/rds/main.tf
# Production-grade RDS module with parameterized storage configuration

variable "storage_config" {
  description = "Storage configuration for the RDS instance"
  type = object({
    type           = string
    allocated_gb   = number
    iops           = optional(number)
    throughput_mbps = optional(number)
    max_allocated_gb = optional(number) # enables autoscaling
  })

  validation {
    condition = contains(["gp2", "gp3", "io1", "io2"], var.storage_config.type)
    error_message = "storage_config.type must be one of: gp2, gp3, io1, io2"
  }

  validation {
    condition = !(var.storage_config.type == "gp2" && var.storage_config.iops != null)
    error_message = "gp2 does not support explicit IOPS configuration"
  }

  validation {
    condition = !(
      contains(["io1", "io2"], var.storage_config.type) &&
      var.storage_config.iops == null
    )
    error_message = "io1 and io2 require explicit IOPS configuration"
  }
}

locals {
  # io1 max ratio: 50 IOPS/GB; io2 max ratio: 500 IOPS/GB
  max_iops_ratio = {
    io1 = 50
    io2 = 500
    gp3 = 500 # gp3 allows up to 500 IOPS/GB above 3000 baseline
    gp2 = 3   # implicit, not configurable
  }

  # Validate io1/io2 IOPS-to-storage ratio
  iops_to_storage_ratio = (
    var.storage_config.iops != null ?
    var.storage_config.iops / var.storage_config.allocated_gb :
    0
  )
}

resource "aws_db_instance" "this" {
  identifier      = var.db_identifier
  engine          = var.engine

```

```

engine_version = var.engine_version
instance_class = var.instance_class

# Storage configuration
allocated_storage      = var.storage_config.allocated_gb
max_allocated_storage = var.storage_config.max_allocated_gb # null disables
autoscaling
storage_type           = var.storage_config.type
iops                   = var.storage_config.iops
storage_throughput     = var.storage_config.throughput_mbps # gp3 only

storage_encrypted = true
kms_key_id        = var.kms_key_arn

# Prevent accidental deletion
deletion_protection      = var.environment == "prod"
skip_final_snapshot      = var.environment != "prod"
final_snapshot_identifier = var.environment == "prod" ? "${var.db_identifier}-
final" : null

tags = merge(var.common_tags, {
  StorageTier      = var.storage_config.type
  ProvisionedIOPS = toString(coalesce(var.storage_config.iops, 0))
})
}

# Outputs for downstream monitoring configuration
output "storage_details" {
  value = {
    type           = aws_db_instance.this.storage_type
    allocated_gb   = aws_db_instance.this.allocated_storage
    iops           = aws_db_instance.this.iops
    throughput     = aws_db_instance.this.storage_throughput
    arn            = aws_db_instance.this.arn
  }
}

```

```

# terraform/environments/prod/rds_instances.tf
# Example invocations for different workload profiles

# Standard OLTP – gp3 with boosted IOPS
module "app_database" {
  source          = "../../modules/rds"
  db_identifier   = "app-prod-01"
  engine          = "sqlserver-se"
  engine_version = "16.00.4125.1.v1"
  instance_class  = "db.r6i.2xlarge"
  environment     = "prod"

  storage_config = {
    type           = "gp3"
    allocated_gb   = 500
    iops           = 9000
    throughput_mbps = 500
    max_allocated_gb = 2000 # autoscale up to 2TB
  }
}

# High-frequency transactional – io2
module "payments_database" {
  source          = "../../modules/rds"
  db_identifier   = "payments-prod-01"
  engine          = "sqlserver-se"
  engine_version = "16.00.4125.1.v1"
  instance_class  = "db.r6i.8xlarge"
  environment     = "prod"

  storage_config = {
    type           = "io2"
    allocated_gb   = 2000
    iops           = 64000
    # throughput not configurable for io2, included
  }
}

# Development – gp3 minimum configuration
module "dev_database" {
  source          = "../../modules/rds"
  db_identifier   = "app-dev-01"
  engine          = "sqlserver-se"
  engine_version = "16.00.4125.1.v1"
  instance_class  = "db.t3.medium"
  environment     = "dev"

  storage_config = {
    type           = "gp3"
    allocated_gb   = 100
    # No IOPS override – baseline 3,000 IOPS is sufficient
  }
}

```

```
}  
}
```

Storage Autoscaling, Modification Limits, and the Gotchas Nobody Tells You

Storage Can Only Grow

This is the most important operational constraint in RDS storage management: **you cannot shrink allocated storage**. Once you provision 2 TB, you are paying for 2 TB forever on that instance. The only escape is a snapshot restore to a new instance with smaller storage, which requires downtime or a blue-green deployment.

The practical consequence: start conservatively and use autoscaling.

```
# Enable storage autoscaling on an existing instance  
# max-allocated-storage must be at least 10% larger than allocated-storage  
aws rds modify-db-instance \  
  --db-instance-identifier prod-sqlserver-01 \  
  --max-allocated-storage 2000 \  
  --apply-immediately  
  
# Disable autoscaling by setting max equal to current allocated (not below)  
aws rds modify-db-instance \  
  --db-instance-identifier prod-sqlserver-01 \  
  --max-allocated-storage 500 \  
  --apply-immediately
```

RDS autoscaling triggers when: free storage falls below 10% of allocated storage AND remains there for 5 minutes, AND at least 6 hours have passed since the last autoscale event. It increases storage in increments of either 10 GB or 10% of current allocation, whichever is larger. It will not scale above `max-allocated-storage`.

The 6-hour cooldown is a real operational risk. If your database has a runaway load that consumes storage at 1 GB/minute, autoscaling at the 5-minute threshold fires once, adds 50 GB, and then is locked for 6 hours while storage continues filling. The right pattern is autoscaling plus a CloudWatch alarm on free storage at the 15% threshold so you have warning before autoscaling even activates.

The Modification Window Problem

Storage type changes (e.g., gp2 to gp3) use `--apply-immediately`, but storage size increases do not always complete immediately. A storage modification on a running instance triggers an optimization phase that AWS calls "storage optimization" — this is essentially an online resize. Depending on volume size, this can take minutes to hours. During optimization, the instance remains available, but:

- → You cannot trigger another storage modification
- → The instance shows status `storage-optimization` in `describe-db-instances`
- → Failover times for Multi-AZ may increase slightly during optimization

```
# Check if a storage modification is still in progress before attempting another
aws rds describe-db-instances \
  --db-instance-identifier prod-sqlserver-01 \
  --query 'DBInstances[0].{
    Status:DBInstanceStatus,
    PendingValues:PendingModifiedValues,
    StorageType:StorageType,
    AllocatedStorage:AllocatedStorage,
    IOPS:Iops
  }'

# Example output when optimization is in progress:
# {
#   "Status": "storage-optimization",
#   "PendingValues": {},
#   "StorageType": "gp3",
#   "AllocatedStorage": 1000,
#   "IOPS": 6000
# }
```

Multi-AZ and Storage Modifications

For Multi-AZ instances, storage modifications are applied to the standby first, then the primary. This means:

1. AWS modifies the standby EBS volume
2. A failover occurs (brief interruption, typically under 60 seconds)
3. The former standby (now primary) runs on the new storage
4. The former primary (now standby) gets its storage updated

Applications connected with proper retry logic will reconnect automatically. Connection poolers like RDS Proxy absorb the failover without the application seeing it. If you are connecting directly to the RDS endpoint without retry logic, that 60-second window causes errors.

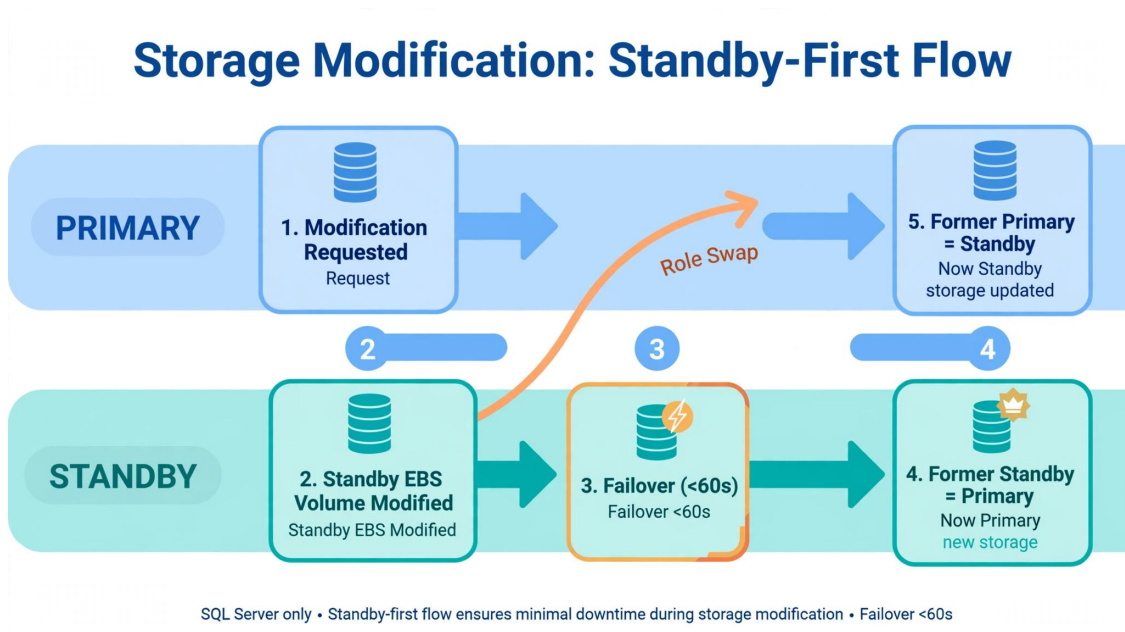


Figure 36.1 — Storage Modification: Standby-First Flow.

Read the numbered steps across the two lanes: the modification is requested against the primary, but AWS applies it to the standby's EBS volume first (step 2), keeping production untouched. A failover — typically under 60 seconds — swaps the roles (step 3), so the former standby becomes the primary already running on the new storage (step 4). The former primary, now the standby, gets its storage updated last (step 5): the design guarantees the new storage is proven under production traffic before the old volume is ever touched.

Multi-AZ storage changes go through a failover — apps without retry logic will see a brief connection error during step 3.

This also means `--apply-immediately` does not mean "instantly" for Multi-AZ instances — it means "during this maintenance event" rather than "at next maintenance window," but the actual modification still requires the failover cycle.

Benchmarking Your Storage Tier Before Migrating Production

Before migrating a workload from on-premises or from one storage tier to another, establish a baseline. This requires understanding what I/O profile your database actually has — sequential vs. random, read vs. write, queue depth.

```
#!/usr/bin/env python3
"""
rds_storage_profiler.py
Profiles I/O patterns on an RDS instance using CloudWatch metrics to recommend
the appropriate storage tier.

Requires: boto3, configured AWS credentials with cloudwatch:GetMetricStatistics
and rds:DescribeDBInstances permissions.
"""

import boto3
import json
import sys
from datetime import datetime, timedelta, timezone
from dataclasses import dataclass, asdict
from typing import Optional

@dataclass
class StorageProfile:
    db_identifier: str
    storage_type: str
    allocated_gb: int
    provisioned_iops: Optional[int]
    avg_read_iops: float
    avg_write_iops: float
    peak_read_iops: float
    peak_write_iops: float
    avg_read_latency_ms: float
    avg_write_latency_ms: float
    avg_queue_depth: float
    avg_throughput_mbps: float
    burst_balance_min: Optional[float]
    recommended_tier: str
    recommendation_reason: str

def get_metric_stats(cw_client, db_id: str, metric: str, stat: str,
                    start: datetime, end: datetime, period: int = 300) -> list:
    response = cw_client.get_metric_statistics(
        Namespace='AWS/RDS',
        MetricName=metric,
        Dimensions=[{'Name': 'DBInstanceIdentifier', 'Value': db_id}],
        StartTime=start,
        EndTime=end,
        Period=period,
        Statistics=[stat]
    )
    return sorted(response['Datapoints'], key=lambda x: x['Timestamp'])

def profile_instance(db_identifier: str, lookback_days: int = 14) -> StorageProfile:
    rds = boto3.client('rds')
    cw = boto3.client('cloudwatch')
```

```

end = datetime.now(timezone.utc)
start = end - timedelta(days=lookback_days)

# Get instance metadata
desc = rds.describe_db_instances(DBInstanceIdentifier=db_identifier)
inst = desc['DBInstances'][0]
storage_type = inst['StorageType']
allocated_gb = inst['AllocatedStorage']
provisioned_iops = inst.get('Iops')

def avg_metric(metric, stat='Average'):
    points = get_metric_stats(cw, db_identifier, metric, stat, start, end)
    if not points:
        return 0.0
    return sum(p[stat] for p in points) / len(points)

def max_metric(metric):
    points = get_metric_stats(cw, db_identifier, metric, 'Maximum', start, end)
    if not points:
        return 0.0
    return max(p['Maximum'] for p in points)

avg_read_iops = avg_metric('ReadIOPS')
avg_write_iops = avg_metric('WriteIOPS')
peak_read_iops = max_metric('ReadIOPS')
peak_write_iops = max_metric('WriteIOPS')

# Latency in seconds from CloudWatch, convert to ms
avg_read_latency_ms = avg_metric('ReadLatency') * 1000
avg_write_latency_ms = avg_metric('WriteLatency') * 1000

avg_queue_depth = avg_metric('DiskQueueDepth')

# Throughput: ReadThroughput + WriteThroughput (bytes/sec → MB/s)
avg_throughput_mbps = (
    avg_metric('ReadThroughput') + avg_metric('WriteThroughput')
) / (1024 * 1024)

# BurstBalance only relevant for gp2
burst_balance_min = None
if storage_type == 'gp2':
    points = get_metric_stats(cw, db_identifier, 'BurstBalance', 'Minimum',
start, end)
    if points:
        burst_balance_min = min(p['Minimum'] for p in points)

peak_total_iops = peak_read_iops + peak_write_iops
avg_total_iops = avg_read_iops + avg_write_iops

# Recommendation logic
recommended_tier = 'gp3'
recommendation_reason = ''

```

```

if peak_total_iops > 64000:
    recommended_tier = 'io2'
    recommendation_reason = (
        f'Peak IOPS ({peak_total_iops:.0f}) exceeds gp3 ceiling of 16,000 '
        f'and io2/io1 ceiling of 64,000 – evaluate io2 Block Express on '
        f'supported instance families.'
    )
elif peak_total_iops > 16000:
    recommended_tier = 'io2'
    recommendation_reason = (
        f'Peak IOPS ({peak_total_iops:.0f}) exceeds gp3 ceiling of 16,000. '
        f'io2 recommended over io1 for identical cost and higher durability.'
    )
elif avg_write_latency_ms > 5.0 or avg_read_latency_ms > 5.0:
    recommended_tier = 'io2'
    recommendation_reason = (
        f'Latency exceeds 5ms average (read: {avg_read_latency_ms:.2f}ms, '
        f'write: {avg_write_latency_ms:.2f}ms). Provisioned IOPS provides '
        f'more consistent sub-millisecond latency guarantees.'
    )
elif avg_queue_depth > 10:
    recommended_tier = 'io2'
    recommendation_reason = (
        f'High average queue depth ({avg_queue_depth:.1f}) indicates I/O '
        f'saturation. Provisioned IOPS will reduce queuing latency.'
    )
elif storage_type == 'gp2':
    if burst_balance_min is not None and burst_balance_min < 50:
        recommended_tier = 'gp3'
        recommendation_reason = (
            f'Currently on gp2 with minimum burst balance of '
            f'{burst_balance_min:.1f}%. Performance is credit-constrained. '
            f'Migrating to gp3 eliminates burst risk at lower cost per GB.'
        )
    else:
        recommended_tier = 'gp3'
        recommendation_reason = (
            'Currently on gp2. gp3 is cheaper per GB and provides '
            'guaranteed 3,000 IOPS baseline without burst credits. '
            'Migrate to gp3 for equivalent or better performance at '
            'reduced cost.'
        )
elif peak_total_iops < 8000 and avg_total_iops < 3000:
    recommended_tier = 'gp3'
    recommendation_reason = (
        f'Workload is within gp3 baseline (avg: {avg_total_iops:.0f} IOPS, '
        f'peak: {peak_total_iops:.0f} IOPS). gp3 at base configuration is '
        f'sufficient and most cost-effective.'
    )
else:
    recommended_tier = 'gp3'
    recommendation_reason = (

```

```

        f'Peak IOPS ({peak_total_iops:.0f}) within gp3 range. Configure '
        f'gp3 with {int(peak_total_iops * 1.2):.0f} IOPS to provide '
        f'20% headroom above observed peak.'
    )

    return StorageProfile(
        db_identifier=db_identifier,
        storage_type=storage_type,
        allocated_gb=allocated_gb,
        provisioned_iops=provisioned_iops,
        avg_read_iops=round(avg_read_iops, 1),
        avg_write_iops=round(avg_write_iops, 1),
        peak_read_iops=round(peak_read_iops, 1),
        peak_write_iops=round(peak_write_iops, 1),
        avg_read_latency_ms=round(avg_read_latency_ms, 3),
        avg_write_latency_ms=round(avg_write_latency_ms, 3),
        avg_queue_depth=round(avg_queue_depth, 2),
        avg_throughput_mbps=round(avg_throughput_mbps, 2),
        burst_balance_min=round(burst_balance_min, 1) if burst_balance_min is not
None else None,
        recommended_tier=recommended_tier,
        recommendation_reason=recommendation_reason,
    )

if __name__ == '__main__':
    if len(sys.argv) < 2:
        print("Usage: rds_storage_profiler.py <db-instance-identifier> [lookback-
days]")
        sys.exit(1)

    db_id = sys.argv[1]
    lookback = int(sys.argv[2]) if len(sys.argv) > 2 else 14

    print(f"Profiling {db_id} over the last {lookback} days...")
    profile = profile_instance(db_id, lookback)
    print(json.dumps(asdict(profile), indent=2))

```

Run this profiler against your existing RDS instances before making storage tier decisions. The 14-day lookback captures weekly patterns including business-day peaks and any scheduled batch jobs. For databases with monthly cycles (end-of-month reporting, billing runs), extend to 35 days.

Workload-Specific Recommendations

OLTP (Online Transaction Processing)

OLTP workloads are characterized by many small, random I/Os with low latency requirements and a high write percentage. Typical patterns:

- → Queue depth: 1–20
- → Average I/O size: 4–16 KB
- → Read/write ratio: 60/40 to 40/60
- → Latency sensitivity: high (users feel > 10ms query times)

For OLTP under 16,000 IOPS: gp3 with IOPS configured to 120% of your measured peak. Set throughput to 500 MB/s unless you can measure a specific throughput requirement. Use autoscaling with a max 2–3x your starting allocation.

For OLTP between 16,000 and 64,000 IOPS: io2. The IOPS guarantee matters here — a gp3 volume sharing underlying infrastructure with other tenants can experience latency variability during neighbor-noisy periods. io2's dedicated provisioning removes that variable.

SQL Server-specific: Monitor `WRITELOG` and `PAGEIOLATCH_*` waits to understand write pressure. Excessive log-write waits can appear as a storage problem when log throughput tuning (right-sized log file, faster log volume) would solve it more cheaply than a tier upgrade.

```
-- SQL Server: check write/log pressure vs buffer pressure
-- Run this to evaluate whether storage pressure is
-- workload-driven or configuration-driven

SELECT wait_type,
       waiting_tasks_count,
       wait_time_ms / 1000.0 AS wait_seconds,
       100.0 * wait_time_ms / SUM(wait_time_ms) OVER() AS pct_of_waits
FROM sys.dm_os_wait_stats
WHERE wait_type IN ('WRITELOG', 'PAGEIOLATCH_SH', 'PAGEIOLATCH_EX',
                   'PAGEIOLATCH_UP', 'LOGBUFFER', 'LOGMGR_RESERVE_APPEND')
ORDER BY wait_time_ms DESC;

-- High WRITELOG relative to PAGEIOLATCH: the log volume is the
-- bottleneck -- move it to faster storage before upgrading the tier.
-- High PAGEIOLATCH_SH with low Page Life Expectancy: the buffer pool
-- is undersized or the workload is genuinely I/O-bound.
```

```
-- SQL Server: check I/O stall statistics to identify storage pressure
-- vs. locking or other wait types
-- Run on RDS SQL Server via SSMS or sqlcmd

SELECT
    DB_NAME(vfs.database_id)          AS database_name,
    mf.physical_name,
    mf.type_desc,
    vfs.io_stall_read_ms,
    vfs.num_of_reads,
    CASE WHEN vfs.num_of_reads = 0 THEN 0
         ELSE vfs.io_stall_read_ms / vfs.num_of_reads
    END                               AS avg_read_stall_ms,
    vfs.io_stall_write_ms,
    vfs.num_of_writes,
    CASE WHEN vfs.num_of_writes = 0 THEN 0
         ELSE vfs.io_stall_write_ms / vfs.num_of_writes
    END                               AS avg_write_stall_ms,
    vfs.io_stall,
    vfs.num_of_bytes_read / 1048576.0 AS mb_read,
    vfs.num_of_bytes_written / 1048576.0 AS mb_written
FROM sys.dm_io_virtual_file_stats(NULL, NULL) vfs
JOIN sys.master_files mf
    ON vfs.database_id = mf.database_id
    AND vfs.file_id = mf.file_id
WHERE DB_NAME(vfs.database_id) NOT IN ('master', 'model', 'msdb', 'tempdb')
ORDER BY vfs.io_stall DESC;

-- Decision rule:
-- avg_read_stall_ms or avg_write_stall_ms consistently > 20ms:
--   Storage tier upgrade likely needed
-- avg stall < 5ms but high IOPS: you're at gp3 ceiling, evaluate io2
-- avg stall > 20ms but low IOPS: check for lock contention,
--   inefficient queries before blaming storage
```

Analytics and Data Warehousing

Analytics workloads are the inverse of OLTP: large sequential scans, high throughput requirements, lower IOPS sensitivity per query (but many concurrent queries compound this), and predominantly read-heavy with periodic bulk loads.

For Redshift, BigQuery, or Athena you are not picking EBS storage types directly — those services manage their own storage. But for SQL Server used for mixed OLTP/reporting:

- → **Throughput matters more than IOPS.** A 500 MB/s sequential read is one IOPS at 500 MB transfer size, or 125,000 IOPS at 4 KB. CloudWatch `ReadThroughput` is more meaningful than `ReadIOPS` for these workloads.

- → **gp3 at maximum throughput (1,000 MB/s) is often sufficient.** Analytics queries scanning hundreds of GBs benefit from high sequential throughput that gp3 can provide without the cost of provisioned IOPS.
- → **Readable secondaries change the calculus.** If reporting queries run against a readable AG secondary rather than the primary, the primary's storage only needs to handle write I/O. The secondary can be on a different storage tier than the primary.

```
# Reporting offload for SQL Server: RDS has no read replicas, so route
# reporting reads to a readable secondary. On Azure, provision a Hyperscale
# named replica on its own storage configuration; on self-managed EC2/VMs,
# add an Always On AG readable secondary whose disks are sized for the
# reporting workload independently of the primary.
az sql db replica create \
  --name prod-sqlserver-reporting \
  --server prod-sql-server \
  --resource-group prod-rg \
  --secondary-type Named \
  --compute-model Provisioned \
  --service-objective HS_Gen5_4

# Note: the readable secondary's storage and compute are sized for the
# reporting workload – the primary's storage only needs to handle write I/O.
```

Mixed Workloads: OLTP + Reporting on the Same Instance

The most common real-world RDS deployment is not pure OLTP or pure analytics — it is a production OLTP database that also serves some reporting queries, either during business hours or in scheduled overnight batch windows. This creates competing I/O patterns:

- → Daytime: random 4–16 KB reads/writes, low queue depth, latency-sensitive
- → Overnight: large sequential scans for reporting, high throughput demand

Storage tier selection for mixed workloads should optimize for the latency-sensitive daytime OLTP pattern, because:

1. Report queries can be made to wait; user-facing OLTP cannot
2. Query plans for reports can often be rewritten to use indexes, reducing scan I/O
3. Read replicas or scheduled reports against S3 exports are architectural alternatives

For a mixed workload, configure gp3 at the IOPS level needed for daytime OLTP peaks, with throughput at 500–1,000 MB/s. If overnight reports are degrading the primary, the right fix is a read replica or a scheduled export to S3 followed by Athena queries — not a storage tier upgrade.

Decision Framework: Picking the Right Tier

Use this decision tree before provisioning any RDS storage:

```
START
|
├─ Is this dev/test/staging?
|   └─ YES → gp3, base configuration (no IOPS override needed)
|       Rationale: 3,000 IOPS baseline sufficient for non-production
|
├─ Do you need > 64,000 sustained IOPS?
|   └─ YES → io2 Block Express on supported instance types
|       If not available, consider sharding or readable secondaries
|
├─ Do you need > 16,000 sustained IOPS?
|   └─ YES → io2 (identical price to io1, better durability, better ratio)
|       Size storage to satisfy 500 IOPS/GB constraint
|
├─ Is your workload latency-sensitive AND did benchmarking show
|   >5ms average latency on gp3?
|   └─ YES → Evaluate io2. But first: check whether the latency comes
|       from storage vs. lock contention (see SQL Server/PG queries above)
|
├─ Is your workload currently on gp2?
|   └─ YES → Migrate to gp3. Always. No exceptions.
|       Even if gp2 is "working," gp3 is cheaper and more predictable.
|
└─ Everything else → gp3
    Configure IOPS to 120% of measured peak.
    Configure throughput to 500 MB/s unless measurement shows you need more.
    Enable autoscaling with 2x headroom on max_allocated_storage.
```

The "migrate gp2 to gp3" branch deserves emphasis. There is no scenario where gp2 is preferable to gp3 for a new provisioning decision. The only reason gp2 instances exist in 2024 is organizational inertia — they were created before gp3 was available (gp3 launched in late 2020) or before teams updated their provisioning playbooks.

Monitoring Storage Health in Production

Beyond the burst balance alarm covered earlier, a complete storage monitoring setup for RDS should include:

```
#!/usr/bin/env python3
"""
rds_storage_monitor.py
Creates a comprehensive CloudWatch dashboard and alarm set for RDS storage health.
Covers gp2 burst, IOPS saturation, latency thresholds, and free storage.
"""

import boto3
import json
from typing import Any

def create_storage_alarms(
    db_identifier: str,
    storage_type: str,
    provisioned_iops: int | None,
    sns_topic_arn: str,
    region: str = 'us-east-1'
) -> list[str]:
    """
    Creates appropriate CloudWatch alarms based on storage tier.
    Returns list of created alarm names.
    """
    cw = boto3.client('cloudwatch', region_name=region)
    created_alarms = []

    common_dims = [{'Name': 'DBInstanceIdentifier', 'Value': db_identifier}]

    def put_alarm(name: str, metric: str, stat: str, threshold: float,
                  operator: str, period: int, eval_periods: int,
                  description: str, unit: str | None = None) -> None:
        kwargs: dict[str, Any] = dict(
            AlarmName=f'RDS-{db_identifier}-{name}',
            AlarmDescription=description,
            MetricName=metric,
            Namespace='AWS/RDS',
            Statistic=stat,
            Dimensions=common_dims,
            Period=period,
            EvaluationPeriods=eval_periods,
            Threshold=threshold,
            ComparisonOperator=operator,
            AlarmActions=[sns_topic_arn],
            OKActions=[sns_topic_arn],
            TreatMissingData='missing',
        )
        if unit:
            kwargs['Unit'] = unit
        cw.put_metric_alarm(**kwargs)
        created_alarms.append(kwargs['AlarmName'])

    # Free storage alarm – 15% threshold gives time to act before autoscaling
    # Uses absolute bytes: 15% of typical starting sizes
    # For production, calculate 15% of allocated storage in bytes
```

```

put_alarm(
    name='FreeStorageLow',
    metric='FreeStorageSpace',
    stat='Average',
    threshold=10 * 1024 ** 3, # 10 GB floor as a starting point
    operator='LessThanThreshold',
    period=300,
    eval_periods=3,
    description=(
        f'Free storage on {db_identifier} below 10 GB. '
        'Review growth rate and autoscaling configuration.'
    ),
    unit='Bytes',
)

# Write latency – threshold tuned per tier
write_latency_threshold = 0.005 # 5ms for gp3
if storage_type in ('io1', 'io2'):
    write_latency_threshold = 0.002 # 2ms for provisioned

put_alarm(
    name='WriteLatencyHigh',
    metric='WriteLatency',
    stat='Average',
    threshold=write_latency_threshold,
    operator='GreaterThanThreshold',
    period=60,
    eval_periods=5,
    description=(
        f'Write latency on {db_identifier} exceeded '
        f'{write_latency_threshold * 1000:.0f}ms average for 5 minutes.'
    ),
    unit='Seconds',
)

put_alarm(
    name='ReadLatencyHigh',
    metric='ReadLatency',
    stat='Average',
    threshold=write_latency_threshold,
    operator='GreaterThanThreshold',
    period=60,
    eval_periods=5,
    description=(
        f'Read latency on {db_identifier} exceeded '
        f'{write_latency_threshold * 1000:.0f}ms average for 5 minutes.'
    ),
    unit='Seconds',
)

# Queue depth – sustained > 64 indicates serious I/O saturation
put_alarm(
    name='DiskQueueDepthHigh',

```

```

        metric='DiskQueueDepth',
        stat='Average',
        threshold=64,
        operator='GreaterThanThreshold',
        period=300,
        eval_periods=3,
        description=(
            f'Disk queue depth on {db_identifier} sustained above 64. '
            'I/O is saturated – consider storage tier upgrade or query
optimization.'
        ),
    )
)

# IOPS saturation – 90% of provisioned or gp3 maximum
if provisioned_iops:
    iops_ceiling = provisioned_iops
elif storage_type == 'gp3':
    iops_ceiling = 16000
else: # gp2
    iops_ceiling = 16000 # theoretical max; burst balance alarm is more useful

put_alarm(
    name='IOPSNearCeiling',
    metric='ReadIOPS',
    stat='Average',
    threshold=iops_ceiling * 0.9,
    operator='GreaterThanThreshold',
    period=300,
    eval_periods=3,
    description=(
        f'Read IOPS on {db_identifier} at 90% of '
        f'provisioned ceiling ({iops_ceiling}). '
        'Review workload or increase IOPS provisioning.'
    ),
)

# gp2-specific: burst balance
if storage_type == 'gp2':
    put_alarm(
        name='BurstBalanceLow',
        metric='BurstBalance',
        stat='Minimum',
        threshold=30,
        operator='LessThanThreshold',
        period=300,
        eval_periods=2,
        description=(
            f'gp2 burst balance on {db_identifier} below 30%. '
            'Performance degradation likely. Migrate to gp3 immediately.'
        ),
    )

return created_alarms

```

```

if __name__ == '__main__':
    import sys

    if len(sys.argv) < 4:
        print("Usage: rds_storage_monitor.py <db-id> <storage-type> "
              "<sns-topic-arn> [provisioned-iops]")
        sys.exit(1)

    db_id      = sys.argv[1]
    stype      = sys.argv[2]
    sns_arn    = sys.argv[3]
    prov_iops  = int(sys.argv[4]) if len(sys.argv) > 4 else None

    alarms = create_storage_alarms(db_id, stype, prov_iops, sns_arn)
    print(f"Created {len(alarms)} alarms:")
    for a in alarms:
        print(f"  {a}")

```

Migration Playbook: gp2 to gp3 at Scale

For environments with many RDS instances, a systematic gp2-to-gp3 migration requires inventory, planning, and staged execution. Here is the operational procedure:

```

#!/bin/bash
# gp2_to_gp3_migration.sh
# Identifies all gp2 RDS instances in an account/region and
# generates a migration plan with cost comparison.
# Execute the actual migrations separately after review.

set -euo pipefail

REGION="${AWS_DEFAULT_REGION:-us-east-1}"
OUTPUT_DIR="./gp2_migration_$(date +%Y%m%d_%H%M%S)"
mkdir -p "$OUTPUT_DIR"

echo "Scanning for gp2 RDS instances in $REGION..."

# Get all gp2 instances
aws rds describe-db-instances \
  --region "$REGION" \
  --query 'DBInstances[?StorageType==`gp2`].{
    ID:DBInstanceIdentifier,
    Class:DBInstanceClass,
    Engine:Engine,
    Status:DBInstanceStatus,
    StorageGB:AllocatedStorage,
    MultiAZ:MultiAZ,
    Tags:TagList
  }' \
  --output json > "$OUTPUT_DIR/gp2_instances.json"

INSTANCE_COUNT=$(jq length "$OUTPUT_DIR/gp2_instances.json")
echo "Found $INSTANCE_COUNT gp2 instances."

# Generate migration commands with cost comparison
jq -r '.[ ] | .ID + " " + (.StorageGB | tostring)' \
  "$OUTPUT_DIR/gp2_instances.json" | \
while IFS=' ' read -r DB_ID STORAGE_GB; do
  # gp2 IOPS (3 per GB, min 100, max 16000)
  GP2_IOPS=$(( STORAGE_GB * 3 ))
  if (( GP2_IOPS < 100 )); then GP2_IOPS=100; fi
  if (( GP2_IOPS > 16000 )); then GP2_IOPS=16000; fi

  # Target gp3 IOPS: max(3000, gp2_equivalent)
  # Add 20% headroom
  TARGET_IOPS=$(( GP2_IOPS > 3000 ? GP2_IOPS * 12 / 10 : 3000 ))
  if (( TARGET_IOPS > 16000 )); then TARGET_IOPS=16000; fi

  cat >> "$OUTPUT_DIR/migration_commands.sh" <<EOF
# Migrate $DB_ID: ${STORAGE_GB}GB gp2 → gp3 (${TARGET_IOPS} IOPS)
aws rds modify-db-instance \
  --db-instance-identifier "$DB_ID" \
  --storage-type gp3 \
  --iops $TARGET_IOPS \
  --storage-throughput 500 \
  --region "$REGION"
EOF
done

```

```
# Remove --apply-immediately for maintenance window scheduling
# Add --apply-immediately to execute within minutes

EOF
done

echo "Migration plan written to $OUTPUT_DIR/"
echo "Review $OUTPUT_DIR/migration_commands.sh before executing."
echo ""
echo "IMPORTANT CHECKLIST before running migrations:"
echo "  1. Verify BurstBalance CloudWatch alarms exist for each gp2 instance"
echo "  2. Confirm Multi-AZ instances have retry logic or RDS Proxy in place"
echo "  3. Schedule migrations during low-traffic windows for gp2 volumes > 1TB"
echo "  4. Monitor storage-optimization status after each modification"
echo "  5. Validate performance metrics 24-48 hours post-migration"
```

The migration from gp2 to gp3 is online — no maintenance window required, no downtime for the database engine. The underlying EBS optimization process runs while the instance serves traffic. For Multi-AZ, the failover cycle described earlier applies, but modern applications with connection retry should handle it transparently.

Beyond AWS: Azure and GCP Storage for SQL Server

The gp2/gp3/io1/io2 decision is AWS-specific; the underlying question — how much IOPS and throughput does the SQL Server workload need, at what latency, at what price — is universal. On **Azure**: Premium SSD v2 and Ultra Disk are the SQL Server workhorses (Ultra Disk for sub-millisecond log latency on demanding workloads), with the caching policies from Chapter 13 (data ReadOnly, log None) as important as the disk type. On **GCP**: Persistent Disk SSD (balanced vs performance tiers) with throughput scaling by disk size. The cross-cloud principles from this chapter transfer directly: separate data and log onto appropriate tiers, provision IOPS/throughput for the measured workload (not the hoped-for one), monitor latency (not just IOPS) as the primary signal, and re-evaluate when the workload changes. Storage is the one cloud component where the physics — latency, IOPS, throughput — is identical everywhere; only the product names change.

Key Takeaways

- → **gp3 is the right default for virtually every new RDS provisioning decision.** It decouples IOPS from storage size, costs less per GB than gp2, and provides a guaranteed 3,000 IOPS baseline with no burst credit mechanics. The only reasons to choose otherwise are specific IOPS or latency requirements that gp3 cannot satisfy.

- → **gp2's burst credit system is a silent performance trap.** Volumes under 1 TB will eventually deplete burst credits under sustained load, dropping to as few as 300 IOPS with no application-level warning. If you have gp2 instances in production, instrument `BurstBalance` in CloudWatch and migrate to gp3 before the next burst event.
- → **io2 is strictly superior to io1 for provisioned IOPS workloads.** Same price, 100x higher durability rating, and a 500 IOPS/GB ratio versus io1's 50 IOPS/GB — meaning less "storage tax" to meet ratio constraints. Use io2 whenever your workload genuinely exceeds gp3's 16,000 IOPS ceiling or requires single-digit millisecond latency guarantees under sustained concurrent load.
- → **RDS storage can only grow, never shrink.** Plan initial allocations conservatively and use `max_allocated_storage` autoscaling for organic growth. Reducing storage requires a snapshot restore to a new instance, which demands downtime or a blue-green deployment. The 6-hour autoscaling cooldown is a real operational risk for rapidly-growing databases — pair autoscaling with a 15% free-storage CloudWatch alarm.
- → **Storage tier decisions should follow measured workload data, not estimates.** Use CloudWatch metrics — `ReadIOPS`, `WriteIOPS`, `ReadLatency`, `WriteLatency`, `DiskQueueDepth` — over a 14-day minimum lookback to understand actual peak and average I/O patterns before provisioning. Upgrading from gp3 to io2 without measurement often wastes money; staying on gp2 or under-provisioning gp3 without measurement silently degrades production performance. The `rds_storage_profiler.py` script in this chapter automates that analysis and produces a tier recommendation you can act on directly.

Chapter 37: Secondary and Scale-Out Economics: When Extra Copies Pay Off

Every secondary copy of a SQL Server database — a readable AG replica, a Business Critical secondary, a Hyperscale replica, a DR standby — costs real money: compute, storage, and (cross-region) data transfer. The question this chapter answers is economic, not architectural: when does the second (or third) copy pay for itself, and when is it an expensive insurance policy that never pays out? Note the SQL Server-specific framing: **RDS for SQL Server has no read replicas**, so "read replica economics" as other engines know it doesn't apply — the scale-out copies available to you are AG secondaries, platform replicas, and DR standbys, each with its own cost model.

37.1 The Copies Available to SQL Server (and What Each Costs)

Copy type	Cost model	Serves reads?
AG secondary (EC2/VMs/GCE)	Full VM + storage + license	Yes (if readable)
Azure BC / MI BC secondary	Included in tier price	Yes (one free readable)
Hyperscale replicas	Per-replica compute	Yes (up to limit)
RDS Multi-AZ standby	Included (standby)	No
DR standby (any platform)	Full copy cost	Sometimes

37.2 The Real Cost Model: What You're Actually Paying For

A secondary's total cost has four components: **compute** (the replica's vCores/VM — often the largest line, and frequently oversized because secondaries get cloned from the primary's spec); **storage** (a full copy of the data — plus the IOPS tier, which is often provisioned to primary levels out of habit); **licensing** (self-managed secondaries need licenses unless covered by Software Assurance secondary rights — verify, because this is the line that surprises); and **transfer** (cross-AZ traffic is cheap or free; cross-region replication is metered — the distributed topologies from Chapters 16 and 25 pay this continuously). Model all four before adding a copy; the "free" readable secondary in Business Critical is free only because you already paid the tier premium.

37.3 When Secondaries Actually Pay Off

A readable secondary pays for itself when: **reporting/analytical queries have a measurable cost on the primary** (quantify it — the primary's CPU/IOPS consumed by reporting, translated to the instance size you'd need without offload); **the secondary absorbs a workload that would otherwise force a primary upsizing** (the classic win — a secondary at half the primary's size is cheaper than doubling the primary); or **geographic read locality** reduces latency enough to matter commercially. A secondary does *not* pay off when: the primary is idle (there's nothing to offload); the "reporting" is three queries a day (a scheduled extract serves them); or the secondary is sized identically to the primary "for safety" while serving 5% of its load — right-size the secondary to its actual workload (Chapter 39).

37.4 Provisioning and Routing: Getting the Architecture Right

The economics depend on correct routing: **ApplicationIntent=ReadOnly** connection strings (or the platform's read-scale endpoint) sending eligible traffic to secondaries automatically; **workload classification** (which queries are safe on a lagging secondary — the application must know its own staleness tolerance, Chapter 21); and **lag-aware routing** (if the secondary lags beyond tolerance, traffic returns to the primary — a secondary serving stale data to an intolerant application is a correctness incident, not a cost saving). The unrouted secondary — provisioned, paid for, receiving no traffic — is the most common waste in this chapter. Verify traffic split in monitoring, not in architecture diagrams.

37.5 Replica Lag: The Hidden Cost of Staleness

Lag has a price beyond correctness: **application complexity** (every read-after-write path needs primary routing — the code that forgets is the bug); **support burden** ("the dashboard shows yesterday's numbers" tickets); and **the lag spike** (secondaries lag most when the primary is busiest — exactly when offload matters most — so size secondaries for peak redo/apply throughput, not average). Monitor lag per secondary against the application's documented tolerance (Chapter 30); alert when tolerance is breached; and include lag behavior in the failover tests (Chapter 9) — the secondary you fail over to is the secondary your lag monitoring described.

37.6 The DR Standby: Insurance Economics

A DR standby that never serves traffic is pure insurance — price it as such: **the standby's cost vs the cost of the outage it prevents** (probability × impact — the business's numbers, not the DBA's); **cheaper standby patterns** (backup-based DR instead of a running standby where the RTO allows — Chapter 16's table; a stopped VM with attached storage; a smaller

standby that scales up on failover); and **the dual-use standby** (a DR secondary that also serves reads converts insurance into infrastructure — the readable DR secondary is the best economics in this chapter when the RPO/RTO requirements allow async). The standby nobody can justify with numbers is the first thing cut in the next budget review — have the numbers ready before the review.

37.7 Cost Optimization for Secondary Fleets

The optimization checklist: **right-size each secondary to its workload** (not to the primary's spec); **use the platform's included secondaries first** (the BC readable secondary, Hyperscale replicas — already paid for); **schedule what can be scheduled** (a reporting secondary needed 8 hours a day doesn't run 24 — stop/start where the platform allows); **commit to the stable secondaries** (reserved/commitment pricing applies to secondary compute too — Chapter 38); and **audit quarterly** (secondaries accumulate — the "temporary" reporting replica from 18 months ago is still billing). Every secondary gets an owner, a purpose, and a review date; undocumented secondaries get decommissioned.

37.8 Choosing the Right Scale-Out Tool per Cloud

Per cloud, the economic ranking for read scale-out: **AWS** — readable AG secondaries on EC2 (full control, full cost) vs RDS (no read replicas — the platform gap that pushes read-heavy SQL Server workloads toward EC2 or toward Azure); **Azure** — the included BC secondary first, Hyperscale replicas for elastic scale, MI/VM AG secondaries for full control; **GCP** — GCE AG secondaries (Cloud SQL for SQL Server's replica story is thinner — verify current capabilities before committing). The meta-lesson: the cheapest read scale-out is the one your platform includes — and platform choice (Chapter 7) should have considered this workload from the start.

37.9 Showback: Making Secondary Costs Visible

Secondaries get cut in budget reviews because their cost is visible and their value is not. Fix the visibility: **tag every secondary** with its purpose (reporting-offload, DR, geographic-reads) and owning team; **attribute the avoided cost** (the primary upsizing that didn't happen, the reporting SLA that's met — the business case from 37.3, kept current); and **report per-consumer** (which team's queries run on the secondary — chargeback or showback makes the beneficiary aware of the cost). The secondary with documented consumers, measured offload, and a named owner survives budget season; the anonymous replica does not. Economics is communication as much as arithmetic.

Key Takeaways

- → **Model all four cost components: compute, storage, licensing, transfer.** Secondaries cloned from primary specs are usually oversized; licensing surprises are the most expensive.
- → **Secondaries pay off by avoiding primary upsizing.** Quantify the offloaded load; verify traffic actually routes to the secondary; right-size to the read workload.
- → **Lag is a correctness and cost parameter.** Document staleness tolerance, route lag-aware, size for peak apply throughput.
- → **Price DR standbys as insurance — and prefer dual-use.** Backup-based DR where RTO allows; readable DR secondaries converting insurance into infrastructure.
- → **Every secondary gets an owner, a purpose, and a review date.** Quarterly audits; use included platform secondaries first; commit to the stable ones.

Chapter 38: Reserved Instances vs On-Demand: The Math

Cloud compute pricing is one of the most consequential decisions a cloud DBA makes, yet it's also one of the most frequently delegated to finance teams who lack the technical context to optimize it properly. The result is organizations either overpaying for on-demand flexibility they don't need, or committing to Reserved Instances at the wrong tier and locking in costs that hurt when the architecture changes. This chapter walks through the actual pricing mechanics across AWS RDS, Azure SQL Database, and Google Cloud SQL—with the arithmetic laid out plainly—so you can make purchasing recommendations backed by data rather than guesswork. The patterns here apply regardless of which cloud you're standardizing on, and the tooling examples show how to audit your existing spend before signing anything.

38.1 How Cloud Database Pricing Works Under the Hood

Before running any numbers, you need a mental model of what you're actually buying. On-premises, you paid for hardware, licenses, and labor separately. In the cloud, those costs are bundled into an hourly or per-second rate that changes based on your commitment level.

AWS RDS Pricing Layers

RDS charges have four components that stack on top of each other:

1. **Instance hours** — the compute cost, billed per second with a one-minute minimum
2. **Storage** — billed per GB-month, with the rate depending on volume type (GP3, io1, io2); provisioned-IOPS tiers add a separate per-IOPS charge
3. **I/O requests** — GP2 and GP3 include free I/O up to a baseline; io1/io2 charges per million requests
4. **Data transfer** — inbound is generally free; outbound to the internet and cross-AZ replication traffic are both billed per GB, at different rates

The instance hour rate is where Reserved Instances make their impact, and this is the number you must pull fresh for your own analysis — do not use a remembered or published figure from any point-in-time source, including this book, because RDS on-demand and reserved pricing varies by instance class, region, engine, and changes over time. The mechanics that matter regardless of the specific rate: a 1-year commitment (No Upfront, Partial Upfront, or All Upfront) buys a materially lower effective hourly rate than on-demand,

and a 3-year commitment buys a lower rate still. Get the current numbers for your instance class and region from the AWS Pricing Calculator or the Price List API (shown below) before running any break-even analysis — never plug in a percentage discount you recall from elsewhere.

Azure SQL Database Pricing Layers

Azure SQL Database pricing depends on the service tier you're running. The General Purpose tier bills per vCore-hour. The Business Critical tier carries a substantially higher multiplier for the same vCore count due to the in-memory OLTP and local SSD backing — pull the current multiplier for your region rather than assuming a fixed ratio, since Microsoft has adjusted relative pricing between tiers over time.

Reserved Capacity for Azure SQL follows the same shape as AWS RI mechanics: a 1-year reservation buys a lower effective hourly rate than pay-as-you-go, and a 3-year reservation buys a lower rate still. Pull current rates for your vCore count, tier, and region from the Azure Pricing Calculator before modeling savings.

Azure's Reserved Instance discounts apply to the compute portion only. Storage is charged separately, per GB-month, and is not discountable via reservations.

Google Cloud SQL Pricing Layers

Cloud SQL charges per vCPU-hour and per GB RAM-hour separately, plus a separate storage charge. This is different from AWS and Azure, which bundle CPU and RAM into instance family pricing. Google calls its commitment discounts Committed Use Discounts (CUDs) rather than Reserved Instances, but the economic model is the same: commit to a resource quantity for 1 or 3 years in exchange for a lower effective hourly rate than on-demand. Pull current per-vCPU-hour and per-GB-hour on-demand and CUD rates from the GCP Pricing Calculator for your region and instance shape — the discount percentage for 1-year versus 3-year commitments has been published by Google at different points, so verify it rather than citing a remembered figure.

38.2 Running the Break-Even Analysis

The decision to commit always comes down to one question: how long will this workload run at this scale? If the answer is uncertain, on-demand is the defensible choice regardless of the percentage discount. If the answer is "at least 12 months at this capacity," the math almost always favors reserving.

The break-even formula is straightforward:

$$\text{Break-Even Hours} = \text{Upfront Cost} / (\text{On-Demand Hourly Rate} - \text{Reserved Hourly Rate})$$

For example, for an AWS RDS instance on a 1-year Partial Upfront term, plug your own current on-demand hourly rate, reserved upfront payment, and reserved hourly rate (all pulled fresh from the AWS Pricing Calculator or Price List API for your specific instance class and region) into the formula:

$$\begin{aligned}\text{Break-Even Hours} &= \text{Upfront Payment} / (\text{On-Demand Hourly} - \text{Reserved Hourly}) \\ \text{Break-Even Days} &= \text{Break-Even Hours} / 24\end{aligned}$$

If the instance runs for longer than the break-even point, the Partial Upfront option has already paid for itself compared to pure on-demand. To project the total annual savings once you have current numbers:

$$\begin{aligned}\text{Annual Savings} &= (\text{On-Demand Hourly} \times 8760) - (\text{Upfront Payment} + \text{Reserved Hourly} \times 8760) \\ \text{Savings \%} &= \text{Annual Savings} / (\text{On-Demand Hourly} \times 8760) \times 100\end{aligned}$$

Run this with your own instance's real pricing rather than a figure from any published example — the break-even point and savings percentage both shift with instance class, region, term length, and payment option, and change whenever the provider updates pricing. Multiply the per-instance savings across every instance you're evaluating to see the aggregate impact of a commitment decision on a multi-instance production cluster.

The following Python script automates this analysis across multiple instance types by pulling current pricing from the AWS Price List API:


```

import boto3
import json
from decimal import Decimal

def get_rds_ondemand_price(region_code, instance_class, engine, deployment_option):
    """
    Retrieves on-demand hourly price for an RDS instance.
    Note: AWS Price List API is only available in us-east-1.
    """
    client = boto3.client('pricing', region_name='us-east-1')

    filters = [
        {'Type': 'TERM_MATCH', 'Field': 'instanceType', 'Value':
instance_class},
        {'Type': 'TERM_MATCH', 'Field': 'databaseEngine', 'Value': engine},
        {'Type': 'TERM_MATCH', 'Field': 'deploymentOption', 'Value':
deployment_option},
        {'Type': 'TERM_MATCH', 'Field': 'location', 'Value':
region_code},
        {'Type': 'TERM_MATCH', 'Field': 'termType', 'Value': 'OnDemand'},
    ]

    response = client.get_products(
        ServiceCode='AmazonRDS',
        Filters=filters,
        MaxResults=1
    )

    if not response['PriceList']:
        return None

    price_item = json.loads(response['PriceList'][0])
    terms = price_item['terms']['OnDemand']

    for term_key in terms:
        for price_dimension_key in terms[term_key]['priceDimensions']:
            pd = terms[term_key]['priceDimensions'][price_dimension_key]
            if pd['unit'] == 'Hrs':
                return Decimal(pd['pricePerUnit']['USD'])

    return None

def calculate_breakeven(ondemand_hourly, reserved_hourly, upfront_cost):
    """
    Returns break-even in hours and days.
    """
    hourly_savings = ondemand_hourly - reserved_hourly
    if hourly_savings <= 0:
        return None, None

    breakeven_hours = float(upfront_cost) / float(hourly_savings)
    breakeven_days = breakeven_hours / 24
    return breakeven_hours, breakeven_days

```

```

def project_annual_savings(ondemand_hourly, reserved_hourly, upfront_cost,
                           utilization_hours=8760):
    """
    Projects total cost and savings over one year at a given utilization.
    Assumes utilization_hours = 8760 for 24x7 workloads.
    """
    ondemand_annual = float(ondemand_hourly) * utilization_hours
    reserved_annual = float(upfront_cost) + (float(reserved_hourly) *
utilization_hours)
    savings = ondemand_annual - reserved_annual
    savings_pct = (savings / ondemand_annual) * 100

    return {
        'on_demand_annual': round(ondemand_annual, 2),
        'reserved_annual': round(reserved_annual, 2),
        'savings': round(savings, 2),
        'savings_pct': round(savings_pct, 1)
    }

# Example: db.r6i.2xlarge, SQL Server SE, Multi-AZ, us-east-1
# These rates are illustrative; pull live rates from AWS Price List or Savings Plans
API
instances_to_evaluate = [
    {
        'instance_class': 'db.r6i.2xlarge',
        'engine': 'sqlserver-se',
        'deployment': 'Multi-AZ',
        'ondemand_hourly': 0.960, # Multi-AZ doubles compute cost
        'reserved_upfront': 2200,
        'reserved_hourly': 0.352,
        'term': '1yr Partial Upfront'
    },
    {
        'instance_class': 'db.r6i.4xlarge',
        'engine': 'sqlserver-se',
        'deployment': 'Multi-AZ',
        'ondemand_hourly': 1.920,
        'reserved_upfront': 4400,
        'reserved_hourly': 0.704,
        'term': '1yr Partial Upfront'
    }
]

for inst in instances_to_evaluate:
    beh, bed = calculate_breakeven(
        inst['ondemand_hourly'],
        inst['reserved_hourly'],
        inst['reserved_upfront']
    )
    projection = project_annual_savings(
        inst['ondemand_hourly'],
        inst['reserved_hourly'],

```

```

        inst['reserved_upfront']
    )

    print(f"\n{'='*60}")
    print(f"Instance:      {inst['instance_class']} ({inst['deployment']})")
    print(f"Term:          {inst['term']}")
    print(f"Break-even:      {beh:.0f} hours ({bed:.0f} days)")
    print(f"On-Demand/yr:    ${projection['on_demand_annual']:, .2f}")
    print(f"Reserved/yr:     ${projection['reserved_annual']:, .2f}")
    print(f"Annual savings:  ${projection['savings']:, .2f}
    ({projection['savings_pct']}%)")

```

Running this against your actual instance inventory gives you a defensible business case for RI purchases without relying on vendor-provided calculators that tend to assume 100% utilization.

38.3 Reserved Instance Mechanics: What the Contracts Actually Say

Understanding the contract structure prevents the most common RI mistakes—buying the wrong scope, overcommitting on instance family, or missing the difference between Convertible and Standard RIs.

AWS RI Types and Flexibility Rules

AWS RDS offers two RI types:

- → **Standard Reserved Instances:** Maximum discount, but instance family is fixed. You cannot modify a `db.r6i` Standard RI to cover a `db.r7g` instance.
- → **Convertible Reserved Instances:** A somewhat smaller discount than Standard RIs at the same term and payment option — check the current gap for your instance class, since AWS has adjusted relative pricing between the two types over time — but you can exchange them for different instance types, regions, or database engines at any time during the term.

The flexibility argument for Convertible RIs is real when you're within 18–24 months of a likely architecture change—say, a SQL Server 2019 cluster that will migrate to SQL Server 2022 (or to Azure SQL Managed Instance) within the RI term. Standard RIs would strand that commitment; Convertible RIs let you exchange them.

AWS also separates RI scope into **Regional** and **Zonal**:

- → Regional RIs apply to any instance of the matching type in any AZ within that region and get Availability Zone flexibility

- → Zonal RIs apply to a specific AZ only, but guarantee capacity reservation in that AZ—useful if you're running in capacity-constrained regions

For production databases, Regional scope is almost always correct unless you have a specific capacity reservation requirement.

Checking your current RDS RI inventory and utilization:

```
# List all current RDS Reserved Instances
aws rds describe-reserved-db-instances \
  --region us-east-1 \
  --output table \
  --query 'ReservedDBInstances[*].{
    ID:ReservedDBInstanceId,
    Class:DBInstanceClass,
    Engine:ProductDescription,
    MultiAZ:MultiAZ,
    State:State,
    StartTime:StartTime,
    Duration:Duration,
    FixedPrice:FixedPrice,
    UsagePrice:UsagePrice,
    Count:DBInstanceCount
  }'

# List RI offerings available for purchase (useful for comparing options)
aws rds describe-reserved-db-instances-offerings \
  --region us-east-1 \
  --db-instance-class db.r6i.2xlarge \
  --product-description "sql server standard" \
  --multi-az \
  --offering-type "Partial Upfront" \
  --output table \
  --query 'ReservedDBInstancesOfferings[*].{
    OfferingID:ReservedDBInstancesOfferingId,
    Class:DBInstanceClass,
    Duration:Duration,
    FixedPrice:FixedPrice,
    UsagePrice:UsagePrice,
    MultiAZ:MultiAZ
  }'

# Purchase a 1-year Partial Upfront RI (replace offering-id with actual ID from
above)
aws rds purchase-reserved-db-instances-offering \
  --reserved-db-instances-offering-id "offering-id-from-previous-command" \
  --reserved-db-instance-id "prod-sqlserver-primary-ri-2024" \
  --db-instance-count 1 \
  --region us-east-1
```

Azure SQL Reserved Capacity

Azure sells reserved capacity for SQL Database and SQL Managed Instance at the vCore level. A reservation for 8 vCores of General Purpose covers any General Purpose database in the selected region that together consume 8 vCores—whether that's one 8-vCore database or two 4-vCore databases. This pooling behavior is more flexible than AWS's per-instance model.

```
# View Azure SQL reserved capacity offerings
az reservations catalog show \
  --service-name "sql" \
  --location eastus \
  --output table

# Purchase a 1-year reservation for 8 vCores General Purpose SQL Database
# Note: az reservations requires the reservations extension
az extension add --name reservations

az reservations reservation-order purchase \
  --reservation-order-id "${uuidgen}" \
  --sku "SQL_Database_General_Purpose_Compute_8_vCores" \
  --location eastus \
  --reserved-resource-type "SqlDatabases" \
  --billing-scope "/subscriptions/your-subscription-id" \
  --term P1Y \
  --billing-plan Upfront \
  --quantity 1 \
  --display-name "prod-sqlldb-gp-8vcores-1yr"
```

Google Cloud Committed Use Discounts

GCP's CUD model commits to a resource quantity (vCPUs and memory) rather than a specific instance type. This means a 3-year CUD for 8 vCPUs and 32 GB of memory in `us-central1` can cover any Cloud SQL instance in that region as long as the aggregate resource consumption matches the committed amount.

```
# Create a 1-year Committed Use Discount for Cloud SQL compute
# CUDs are created at the project level via the Compute Engine API
gcloud compute commitments create prod-cloudsql-commitment \
  --plan 12-month \
  --region us-central1 \
  --resources vcpu=8,memory=32GB \
  --type GENERAL_PURPOSE \
  --project your-project-id

# View existing commitments and their utilization
gcloud compute commitments list \
  --region us-central1 \
  --project your-project-id \
  --format="table(name, plan, status, startTimestamp, endTimestamp)"
```

38.4 Sizing Commitments Against Real Workload Data

The most expensive RI mistake is buying capacity based on current provisioning rather than actual utilization. If you're running a `db.r6i.4xlarge` at 15% average CPU utilization because the instance was provisioned for peak load three years ago, committing to that instance class locks in overspend.

Before any RI purchase, pull at least 90 days of utilization data. For AWS RDS, that data lives in CloudWatch.

```

import boto3
from datetime import datetime, timedelta
import statistics

def get_rds_utilization_stats(db_instance_id, region, days=90):
    """
    Retrieves CPU, FreeableMemory, DatabaseConnections, and ReadIOPS/WriteIOPS
    for the last N days to inform RI sizing decisions.
    """
    cloudwatch = boto3.client('cloudwatch', region_name=region)

    end_time = datetime.utcnow()
    start_time = end_time - timedelta(days=days)

    metrics = {
        'CPUUtilization': {'stat': 'Average', 'unit': 'Percent'},
        'FreeableMemory': {'stat': 'Average', 'unit': 'Bytes'},
        'DatabaseConnections': {'stat': 'Maximum', 'unit': 'Count'},
        'ReadIOPS': {'stat': 'Average', 'unit': 'Count/Second'},
        'WriteIOPS': {'stat': 'Average', 'unit': 'Count/Second'},
    }

    results = {}

    for metric_name, config in metrics.items():
        response = cloudwatch.get_metric_statistics(
            Namespace='AWS/RDS',
            MetricName=metric_name,
            Dimensions=[{'Name': 'DBInstanceIdentifier', 'Value': db_instance_id}],
            StartTime=start_time,
            EndTime=end_time,
            Period=3600, # 1-hour granularity
            Statistics=[config['stat']]
        )

        datapoints = sorted(response['Datapoints'], key=lambda x: x['Timestamp'])
        values = [dp[config['stat']] for dp in datapoints]

        if values:
            results[metric_name] = {
                'mean': round(statistics.mean(values), 2),
                'median': round(statistics.median(values), 2),
                'p95': round(sorted(values)[int(len(values) * 0.95)], 2),
                'p99': round(sorted(values)[int(len(values) * 0.99)], 2),
                'max': round(max(values), 2),
                'samples': len(values)
            }

    return results

def recommend_instance_class(stats, current_instance_class):
    """
    Provides a rough right-sizing recommendation based on CPU utilization.

```

```

In practice, combine with memory pressure and connection count analysis.
"""

cpu_p95 = stats.get('CPUUtilization', {}).get('p95', 100)
cpu_mean = stats.get('CPUUtilization', {}).get('mean', 100)

recommendation = current_instance_class
notes = []

if cpu_p95 < 20:
    notes.append(f"P95 CPU {cpu_p95}% – strong candidate for downsizing one
instance class")
elif cpu_p95 < 40:
    notes.append(f"P95 CPU {cpu_p95}% – current sizing is adequate; commit at
this level")
elif cpu_p95 < 70:
    notes.append(f"P95 CPU {cpu_p95}% – healthy utilization; commit at this
level")
else:
    notes.append(f"P95 CPU {cpu_p95}% – consider upsizing before committing")

return recommendation, notes

# Example usage
db_instance_id = 'prod-sqlserver-primary'
region = 'us-east-1'
current_class = 'db.r6i.4xlarge'

print(f"Analyzing {db_instance_id} over 90 days...")
stats = get_rds_utilization_stats(db_instance_id, region, days=90)

for metric, values in stats.items():
    print(f"\n{metric}:")
    print(f"  Mean: {values['mean']} | Median: {values['median']}")
    print(f"  P95: {values['p95']} | P99: {values['p99']} | Max:
{values['max']}")

recommended_class, notes = recommend_instance_class(stats, current_class)
print(f"\nRight-sizing analysis for {current_class}:")
for note in notes:
    print(f"  → {note}")

```

For Azure SQL, the equivalent utilization data comes from Azure Monitor. The `sys.dm_db_resource_stats` DMV provides per-database resource consumption at 15-second intervals for the last hour; `sys.resource_stats` in the `master` database covers the last 14 days at 1-minute granularity.


```

-- Azure SQL Database: 14-day resource utilization summary
-- Run this against the master database of your Azure SQL logical server
SELECT
    database_name,
    CAST(start_time AS DATE)                                AS sample_date,
    ROUND(AVG(avg_cpu_percent), 2)                         AS avg_cpu_pct,
    ROUND(MAX(avg_cpu_percent), 2)                         AS max_cpu_pct,
    ROUND(AVG(avg_memory_usage_percent), 2)               AS avg_mem_pct,
    ROUND(MAX(avg_memory_usage_percent), 2)               AS max_mem_pct,
    ROUND(AVG(avg_data_io_percent), 2)                    AS avg_data_io_pct,
    ROUND(MAX(avg_data_io_percent), 2)                    AS max_data_io_pct,
    MAX(max_session_count)                                AS peak_sessions,
    COUNT(*)                                                AS sample_count
FROM
    sys.resource_stats
WHERE
    start_time >= DATEADD(DAY, -14, GETUTCDATE()) -- sys.resource_stats retains 14
days
GROUP BY
    database_name,
    CAST(start_time AS DATE)
ORDER BY
    database_name,
    sample_date DESC;

-- Identify databases consistently below 20% CPU – right-sizing candidates
-- PERCENTILE_CONT is a per-row windowed value, so compute it in a CTE
-- before aggregating – combining it directly with GROUP BY would collapse
-- each database to a single row before the percentile has anything to work over
WITH per_row_percentiles AS (
    SELECT
        database_name,
        avg_cpu_percent,
        PERCENTILE_CONT(0.95) WITHIN GROUP
            (ORDER BY avg_cpu_percent) OVER
            (PARTITION BY database_name) AS p95_cpu_pct
    FROM sys.resource_stats
    WHERE start_time >= DATEADD(DAY, -14, GETUTCDATE()) -- sys.resource_stats
retains 14 days
)
SELECT
    database_name,
    ROUND(AVG(avg_cpu_percent), 2) AS avg_cpu_pct,
    ROUND(MAX(p95_cpu_pct), 2)    AS p95_cpu_pct,
    ROUND(MAX(avg_cpu_percent), 2) AS max_cpu_pct
FROM per_row_percentiles
GROUP BY database_name
HAVING
    AVG(avg_cpu_percent) < 20
    AND MAX(avg_cpu_percent) < 40
ORDER BY
    avg_cpu_pct ASC;

```

For Google Cloud SQL, utilization data is available through Cloud Monitoring (formerly Stackdriver). The `gcloud` CLI can pull metric time series, but for bulk analysis the Python client is more practical:

```

from google.cloud import monitoring_v3
from datetime import datetime, timedelta
import time

def get_cloudsql_utilization(project_id, instance_id, region, days=90):
    """
    Retrieves CPU utilization and memory utilization for a Cloud SQL instance
    over the specified window.
    """
    client = monitoring_v3.MetricServiceClient()
    project_name = f"projects/{project_id}"

    end_time = datetime.utcnow()
    start_time = end_time - timedelta(days=days)

    interval = monitoring_v3.TimeInterval(
        {
            "end_time": {"seconds": int(end_time.timestamp())},
            "start_time": {"seconds": int(start_time.timestamp())},
        }
    )

    metrics_to_pull = {
        "cloudsql.googleapis.com/database/cpu/utilization": "CPU",
        "cloudsql.googleapis.com/database/memory/utilization": "Memory",
    }

    results = {}

    for metric_type, label in metrics_to_pull.items():
        aggregation = monitoring_v3.Aggregation(
            {
                "alignment_period": {"seconds": 3600},
                "per_series_aligner": monitoring_v3.Aggregation.Aligner.ALIGN_MEAN,
            }
        )

        filter_str = (
            f'metric.type="{metric_type}" '
            f'AND resource.labels.database_id="{project_id}:{instance_id}"'
        )

        results_iter = client.list_time_series(
            request={
                "name": project_name,
                "filter": filter_str,
                "interval": interval,
                "view":
monitoring_v3.ListTimeSeriesRequest.TimeSeriesView.FULL,
                "aggregation": aggregation,
            }
        )

```

```

    values = []
    for series in results_iter:
        for point in series.points:
            values.append(point.value.double_value * 100) # convert to
percentage

    if values:
        sorted_vals = sorted(values)
        results[label] = {
            "mean": round(sum(values) / len(values), 2),
            "p95": round(sorted_vals[int(len(sorted_vals) * 0.95)], 2),
            "p99": round(sorted_vals[int(len(sorted_vals) * 0.99)], 2),
            "max": round(max(values), 2),
            "samples": len(values),
        }

    return results

# Example usage
project_id = "your-gcp-project"
instance_id = "prod-sqlserver-primary"
region = "us-central1"

stats = get_cloudsql_utilization(project_id, instance_id, region, days=90)
for resource, values in stats.items():
    print(f"\n{resource} Utilization (90-day):")
    print(f" Mean: {values['mean']}% | P95: {values['p95']}% | P99:
{values['p99']}% | Max: {values['max']}%")

```

The utilization analysis should drive the commitment quantity, not the provisioned instance size. A general rule used in practice: commit at the P75 utilization level for stable workloads, leaving headroom for P95 spikes to absorb against on-demand pricing. For databases with highly seasonal traffic—a retail OLTP database with holiday peaks, for example—consider committing at base load and letting the seasonal burst run on-demand.

38.5 Cross-Cloud Comparison: Where Each Provider's Model Has Edges

No single cloud's commitment model is strictly superior. The right choice depends on workload characteristics, architectural stability, and financial flexibility requirements.

AWS RDS Advantages

- → Convertible RIs give genuine architectural flexibility without sacrificing the full discount in one step
- → Per-instance granularity means you can commit selectively: reserve your three stable production primaries, leave dev/test and read replicas on-demand

- → Multi-AZ RIs are purchased as a single SKU; you don't separately reserve primary and standby
- → Savings Plans (compute-level, not database-level) can cover EC2-based database workloads if you're running databases on EC2 rather than RDS

Azure SQL Advantages

- → vCore pooling means a single reservation can float across multiple databases as they scale up and down, reducing the risk of stranded reservation capacity
- → The Azure Hybrid Benefit can stack on top of reservations: if you have existing SQL Server licenses with Software Assurance, you can apply them to reduce the effective rate further by eliminating the license portion of the rate — check current Azure Hybrid Benefit terms and your license entitlement to quantify the exact reduction for your configuration
- → Reservation exchanges (swapping to a different SKU) are supported in Azure without the Convertible RI equivalent overhead

Google Cloud SQL Advantages

- → CUDs apply at the resource level (vCPUs + memory) rather than the instance level, giving the most flexibility for workloads that fluctuate in instance shape
- → Unlike Compute Engine VMs, Cloud SQL does not receive automatic Sustained Use Discounts — the CUD is the only discount lever for Cloud SQL compute, so there is no passive safety net for teams that fail to commit; verify current GCP documentation for your specific service before assuming any automatic discount applies

When On-Demand Is the Correct Answer

The math strongly favors reservations for stable production workloads, but on-demand is genuinely correct in several scenarios:

1. **Workloads under 6 months:** Even with a 1-year commitment, if there's meaningful probability the workload ends or changes significantly within 6–9 months, on-demand is lower total risk.
2. **Active migration projects:** A database in the middle of a cloud-to-cloud or on-prem-to-cloud migration is not a stable commitment candidate. Buy the RI 30–60 days after the migration stabilizes.
3. **Development and testing environments:** Unless your dev environment is truly persistent and large enough to make the math work, on-demand keeps options open for scaling down or pausing instances.

4. **Pilot workloads on a new engine:** If you're evaluating a new service tier or platform, you don't yet know the steady-state instance class. Running on-demand through the evaluation period is the correct approach.

38.6 Auditing Existing Commitments for Waste

Just as important as making the right purchase decision is monitoring whether existing commitments are being utilized. Stranded Reserved Instances—reservations covering instances that were resized, migrated, or decommissioned—are a common source of invisible waste.

AWS: Checking RI Utilization via Cost Explorer

AWS Cost Explorer provides RI utilization reports through both the console and the API. The API is useful for scheduled reporting.

```

import boto3
from datetime import datetime, timedelta

def get_ri_utilization_report(region, lookback_days=30):
    """
    Pulls RI utilization from AWS Cost Explorer.
    Flags any RI with utilization below 80% as potentially stranded.
    """
    ce_client = boto3.client('ce', region_name='us-east-1') # CE API is global,
    endpoint is us-east-1

    end_date = datetime.utcnow().strftime('%Y-%m-%d')
    start_date = (datetime.utcnow() - timedelta(days=lookback_days)).strftime('%Y-
    %m-%d')

    response = ce_client.get_reservation_utilization(
        TimePeriod={
            'Start': start_date,
            'End': end_date,
        },
        GroupBy=[
            {'Type': 'DIMENSION', 'Key': 'SUBSCRIPTION_ID'},
        ],
        Filter={
            'Dimensions': {
                'Key': 'SERVICE',
                'Values': ['Amazon Relational Database Service'],
            }
        }
    )

    print(f"\nRDS RI Utilization Report ({start_date} to {end_date})")
    print("=" * 70)

    total_purchased_hours = 0
    total_utilized_hours = 0
    stranded_reservations = []

    for group in response.get('UtilizationsByTime', []):
        for item in group.get('Groups', []):
            sub_id = item['Keys'][0]
            util = item['Utilization']

            purchased_hours = float(util.get('PurchasedHours', 0))
            utilized_hours = float(util.get('TotalActualHours', 0))
            util_pct = float(util.get('UtilizationPercentage', 0))
            unused_cost =
            float(util.get('UnusedAmortizedUpfrontFeeForBillingPeriod', 0)) \
                + float(util.get('UnusedRecurringFee', 0))

            total_purchased_hours += purchased_hours
            total_utilized_hours += utilized_hours

```

```

        status = "✓ OK" if util_pct >= 80 else "⚠ REVIEW"
        print(f"{status} Subscription: {sub_id[:20]:<22} "
              f"Utilization: {util_pct:5.1f}% "
              f"Unused cost: ${unused_cost:,.2f}")

    if util_pct < 80:
        stranded_reservations.append({
            'subscription_id': sub_id,
            'utilization_pct': util_pct,
            'unused_cost': unused_cost
        })

    overall_util = (total_utilized_hours / total_purchased_hours * 100) \
        if total_purchased_hours > 0 else 0
    print(f"\nOverall RDS RI Utilization: {overall_util:.1f}%")

    if stranded_reservations:
        total_stranded_cost = sum(r['unused_cost'] for r in stranded_reservations)
        print(f"\n⚠ {len(stranded_reservations)} reservation(s) below 80%
utilization")
        print(f"    Estimated wasted spend this period: ${total_stranded_cost:,.2f}")
        print("    Consider modifying, exchanging (Convertible only), or ")
        print("    listing on the AWS Reserved Instance Marketplace.")

    return stranded_reservations

stranded = get_ri_utilization_report('us-east-1', lookback_days=30)

```

Azure: Reservation Utilization via Azure Monitor

Azure surfaces reservation utilization through the Cost Management API. A reservation utilization below 100% means you're paying for vCore-hours you're not consuming.

```

# Get reservation utilization summary for the past 30 days
az consumption reservation summary list \
  --reservation-order-id "your-reservation-order-id" \
  --reservation-id "your-reservation-id" \
  --grain monthly \
  --start-date "$(date -d '30 days ago' +%Y-%m-%d)" \
  --end-date "$(date +%Y-%m-%d)" \
  --output table \
  --query '[].{
    Period:usageDate,
    PurchasedQty:minUtilizationPercentage,
    AvgUtil:avgUtilizationPercentage,
    MaxUtil:maxUtilizationPercentage
  }'

```


For Azure SQL specifically, if a reservation is consistently below utilization, the correct remediation is to exchange it for a smaller vCore count reservation—Azure allows this without penalty during the reservation term.

Google Cloud: Commitment Utilization

GCP commitment utilization is visible in the Cloud Billing reports and via the `gcloud` CLI:

```
# View commitment utilization breakdown
gcloud compute commitments describe prod-cloudsql-commitment \
  --region us-central1 \
  --project your-project-id \
  --format="yaml(name, plan, status, resources, startTimestamp, endTimestamp)"

# For programmatic access to billing data, use the Cloud Billing Budget API
# or export billing data to BigQuery and query it directly
bq query --use_legacy_sql=false \
'SELECT
  service.description                AS service,
  sku.description                    AS sku,
  SUM(usage.amount)                  AS total_usage,
  SUM(cost)                          AS total_cost,
  SUM(credits.amount)                AS total_credits
FROM
  `your-billing-project.billing_export.gcp_billing_export_v1_XXXXXX`
WHERE
  DATE(usage_start_time) >= DATE_SUB(CURRENT_DATE(), INTERVAL 30 DAY)
  AND service.description LIKE "%SQL%"
GROUP BY
  1, 2
ORDER BY
  total_cost DESC
LIMIT 20'
```

38.7 The RI Purchase Workflow for Production Teams

The mechanics of the analysis matter less than having a repeatable process that prevents both panic-buying and perpetual deferral. The following workflow can be run quarterly and produces a prioritized recommendation list ready for finance approval.

Step 1: Inventory all running database instances

Pull a complete list of running database instances across all accounts and regions, including their instance class, engine, Multi-AZ status, and whether they're currently covered by an RI.

```
# AWS: inventory all RDS instances across regions
for region in $(aws ec2 describe-regions --query 'Regions[].RegionName' --output
text); do
  echo "=== $region ==="
  aws rds describe-db-instances \
    --region "$region" \
    --query 'DBInstances[*].{
      ID:DBInstanceIdentifier,
      Class:DBInstanceClass,
      Engine:Engine,
      EngineVersion:EngineVersion,
      MultiAZ:MultiAZ,
      Status:DBInstanceStatus,
      Storage:AllocatedStorage
    }' \
    --output table 2>/dev/null
done

# Cross-reference against existing RIs to find uncovered instances
aws rds describe-reserved-db-instances \
  --region us-east-1 \
  --query 'ReservedDBInstances[?State==`active`].{
    Class:DBInstanceClass,
    Engine:ProductDescription,
    MultiAZ:MultiAZ,
    Count:DBInstanceCount,
    Expiry:StartTime
  }' \
  --output table
```

Step 2: Score each instance on commitment suitability

Not all instances deserve the same commitment strategy. Score each candidate on three axes:

| Axis | Low Suitability (0) | High Suitability (2) |

|-----|-----|-----|

| Workload stability | Active migration in progress | Running unchanged 12+ months |

| Utilization efficiency | P95 CPU < 20% (likely to be right-sized) | P95 CPU 40–70% |

| Time horizon | Architectural change expected < 9 months | No changes planned for 18+ months |

Instances with a total score of 4 or higher are strong RI candidates. Instances with scores of 2 or below should run on-demand until conditions improve.

Step 3: Calculate the purchase recommendation

For each high-scoring instance, run the break-even and annual savings calculation from Section 38.2. Produce a ranked list sorted by total annual savings, not by savings percentage—a 38% discount on a large instance produces more absolute dollars saved than a 40% discount on a small one.



Figure 38.1 — Reserved Instance Purchase Workflow.

Work the flow left to right: inventory every running instance, then score each on stability, utilization, and time horizon. The diamond is the gate — anything scoring 4 or above moves forward; anything below stays on-demand and gets reassessed next quarter. For qualifiers, calculate break-even and annual savings, stage the purchase starting with your highest-confidence instances, and set 90/60/30-day expiration alerts so a reservation never silently lapses back into on-demand pricing.

A repeatable quarterly process beats a one-time bulk purchase decision — staging limits exposure if workload assumptions turn out wrong.

Step 4: Stage the purchase

Don't commit the entire inventory in one transaction. A staged approach reduces risk:

- → **Week 1:** Purchase RIs for instances with scores of 6 (maximum) that have been running stable for 18+ months
- → **Month 1:** Add instances with scores of 4–5 after confirming no architectural changes are imminent
- → **Quarter 2:** Reassess remaining on-demand instances for newly earned stability

For AWS, if you're uncertain between Standard and Convertible RIs, run the first purchase as Convertible. The discount gap is modest, and the exchange flexibility is genuinely valuable during initial portfolio optimization.

Step 5: Set expiration alerts

RI expirations are a known risk: a reservation that expires unnoticed flips the instance back to on-demand rates silently. Set calendar alerts 90, 60, and 30 days before each RI expiration date. For AWS, you can automate this with EventBridge:

```
# Create a CloudWatch alarm that fires when an RI is expiring within 60 days
# This uses the AWS Health API to surface RI expiration events
aws events put-rule \
  --name "RDSReservedInstanceExpirationAlert" \
  --event-pattern '{
    "source": ["aws.health"],
    "detail-type": ["AWS Health Event"],
    "detail": {
      "service": ["RDS"],
      "eventTypeCategory": ["scheduledChange"],
      "eventTypeCode": ["AWS_RDS_RESERVED_INSTANCE_EXPIRATION_SCHEDULED"]
    }
  }' \
  --state ENABLED \
  --region us-east-1

# Attach an SNS topic to the rule for notification
aws events put-targets \
  --rule "RDSReservedInstanceExpirationAlert" \
  --targets '[{
    "Id": "1",
    "Arn": "arn:aws:sns:us-east-1:123456789012:db-ops-alerts"
  }]' \
  --region us-east-1
```

Key Takeaways

- → **The discount is real and substantial, but so is the commitment risk:** across all three major clouds, 1-year commitments deliver a meaningfully lower effective hourly rate than on-demand, and 3-year commitments deliver a lower rate still — but the exact percentage varies by provider, instance class, region, and payment option, and changes as providers update pricing. Pull current numbers before committing, and only after the workload runs as expected. Analyze workload stability before size, and size before committing.

- → **Utilization data must precede the purchase:** Committing to an instance class based on provisioned size rather than 90-day utilization history locks in overspend for the entire term. Pull P95 CPU and memory data, right-size first, then commit.
- → **Each cloud's commitment model has distinct flexibility characteristics:** AWS Convertible RIs allow mid-term exchanges at the cost of a modest discount reduction; Azure vCore pooling floats coverage across multiple databases automatically; GCP CUDs apply at the resource level and survive instance type changes. Match the commitment type to your architectural confidence level.
- → **Stranded reservations are as costly as missing reservations:** A quarterly audit of RI utilization using Cost Explorer (AWS), Azure Cost Management, or GCP Billing export will surface underutilized commitments early enough to exchange or sell them before the waste compounds.
- → **Build a repeatable purchase workflow, not a one-time event:** Inventory all instances, score them on stability and utilization efficiency, calculate ranked savings, stage purchases over weeks rather than days, and set automated expiration alerts. Organizations that treat RI management as an ongoing operational process consistently outperform those that approach it as an annual finance exercise.

Chapter 39: Right-Sizing Cloud Database Instances

Right-sizing is the highest-ROI activity in cloud database operations: the workload needs what it needs, and everything above that is waste billed by the hour. On-premises, oversizing was rational (hardware is bought once, and running out hurts more than overprovisioning). In the cloud, oversizing is a subscription to waste — and unlike hardware, it can be fixed this afternoon. This chapter builds the right-sizing practice for SQL Server: the measurement methodology, the resizing mechanics per platform, storage right-sizing, and the automation that keeps sizes honest as workloads evolve.

39.1 The Measurement Methodology: What "Right" Means

"Right" is defined by the workload's actual consumption with headroom for peaks — not by the on-premises server it replaced, not by the vendor's recommendation, and not by anyone's comfort. The methodology: collect **4–8 weeks of metrics** (CPU utilization, memory pressure signals, IOPS and throughput, connection counts — covering a full business cycle including month/quarter-end); compute the **p95 and peak** for each dimension (averages lie — size for the peaks you must absorb, not the average you usually see); and define the **headroom policy** (typically 30–50% above p95 for production — the buffer that absorbs growth and spikes without paging anyone). The output is a sizing envelope per database: "needs 6–8 vCores at peak, 32 GB memory working set, 3,000 IOPS p95" — a specification the instance selection answers, not a guess.

39.2 Identifying Sizing Mismatches from Cloud Metrics

The mismatch signatures: **chronically low CPU** (p95 below 30% — the instance is at least one size too large); **memory pressure without CPU pressure** (low page life expectancy, RESOURCE_SEMAPHORE waits — needs a memory-optimized shape, not a bigger general-purpose one); **IOPS throttling with idle CPU** (storage-bound — fix the storage tier per Chapter 36, not the instance size); **CPU-saturated with memory to spare** (compute-optimized shape or scale-out, not a memory upgrade). SQL Server-specific signals: `sys.dm_os_wait_stats` dominated by CXPACKET (too many schedulers for the workload — smaller or MAXDOP tuning beats upsizing); persistent SOS_SCHEDULER_YIELD (genuinely CPU-bound — upsize or scale out). Run the mismatch scan fleet-wide monthly — it takes an hour and routinely finds 20–30% waste.

39.3 Vertical Scaling: Resizing Mechanics per Platform

The resize operation per platform: **RDS** — modify the instance class (minutes of downtime for Single-AZ; Multi-AZ resizes with a failover — plan the timing); **Azure SQL Database/MI** — change vCores or service tier online (the platform handles it — the closest to "free" resizing in the industry); **EC2/Azure VMs/GCE** — stop, resize the VM, start (downtime measured in minutes — schedule it); **Cloud SQL** — update machine type (brief downtime). The practice: resize during the maintenance window, verify the workload's key queries after (Query Store comparison — a smaller instance changes plan choices), and keep the previous size documented for fast rollback. Downsize in steps (one size at a time, observe a full business cycle) — the aggressive two-size downsize that causes a Monday-morning incident erases a year of savings goodwill.

39.4 Storage Right-Sizing: IOPS, Throughput, and Growth

Storage is the forgotten half of right-sizing: **provisioned IOPS** sized to measured p95 (not to the instance default — io2 volumes provisioned "to be safe" are a major waste line); **throughput** matched to the workload's actual MB/s (gp3's independent throughput provisioning exists precisely for this); **allocated size** with growth runway (storage only grows on most platforms — the one-time data load that permanently raised the bill from Chapter 19's warning); and **tier placement** (data on the performance tier, backups and cold data on cheaper tiers — Chapter 36's discipline). Monitor free space trends with alerts at 70% and 85% — the storage-full outage is the most embarrassing preventable incident, and autoscaling (where available) is the safety net, not the strategy.

39.5 Horizontal Scaling and Secondary Sizing

When vertical scaling hits limits (or economics), scale out per Chapter 37: readable secondaries sized to the *read* workload (not cloned from the primary), elastic pools for the multi-tenant fleets (Chapter 14), and Hyperscale for the workloads that outgrow provisioned tiers. The right-sizing lens on scale-out: every secondary gets the same measurement methodology as the primary (39.1) — secondaries sized by hope are the waste this chapter hunts. And revisit the scale-out decision itself annually: the workload that needed a secondary last year may fit on a current-generation primary this year (hardware improves; the architecture should be re-justified, not grandfathered).

39.6 Scheduled Scaling for Predictable Patterns

Not all workloads are flat — the 9-to-5 OLTP system, the nightly batch window, the weekend-quiet SaaS. Where the platform supports it (Azure SQL elastic jobs + automation, EC2 scheduled scaling, RDS scheduled actions), **scale up for the known peak and down for the**

known trough: bigger during business hours, smaller overnight. The economics: a database at peak 12 hours a day on a smaller tier the other 12 saves roughly a quarter of compute — more than most reservation discounts, with zero commitment. The prerequisites: the scaling must be automated (manual scaling gets forgotten in exactly one direction — up), the application must tolerate the brief transitions, and the schedule must be reviewed when business patterns change (the "weekend quiet" that ended when the company went global).

39.7 Cost Allocation and Showback

Right-sizing sticks when the cost is visible to the owner: **tag everything** (application, team, environment, cost-center — the tagging discipline from Chapter 30); **showback reports** per team (your databases cost X, of which Y is identified waste with a recommended action); and **the waste backlog** (oversized instances as tracked work items with owners and dates — the same operational seriousness as the security backlog). The cultural point: waste is not blame — it's the default outcome of growth and caution. The showback report's job is to make the right size the easy choice, not to shame the team that provisioned generously during a launch.

39.8 Right-Sizing During Migration Waves

Migration is the best right-sizing opportunity you'll ever get (Chapter 33): the assessment phase produces the measurements, the target is being built fresh, and there's no "but it's always been that size" inertia. The practice: **size the target from measured source metrics** (not from the source server's spec — the 64-core on-premises box at 15% utilization becomes an 8-vCore cloud instance); **start conservative-safe, then downsize** (one size above the measurement for cutover safety, then the 39.3 downsize cadence on real cloud metrics); and **revisit at 30/60/90 days** post-migration (the migration-sized instance is a hypothesis — the 90-day review with production metrics is the verdict). Programs that skip this leave the migration's largest savings on the table.

Key Takeaways

- → **Size from 4–8 weeks of measured p95/peak, plus 30–50% headroom.** The sizing envelope is a specification, not a guess. Averages lie; peaks define.
- → **Read the mismatch signatures.** Low CPU = oversized; memory pressure = memory-optimized shape; IOPS throttling = storage tier; CXPACKET dominance = scheduler/parallelism tuning.
- → **Downsize in steps, verify with Query Store.** One size at a time, a full business cycle of observation, documented rollback. Azure's online tier changes are the easiest; plan downtime for the rest.

- → **Storage is half the bill — right-size it too.** Provisioned IOPS to measured p95, throughput matched, growth runway without bloat, tier placement by data temperature.
- → **Automate the cadence: monthly mismatch scans, scheduled scaling, showback.** Migration waves are the best right-sizing opportunity — size targets from measurements, revisit at 90 days.

Chapter 40: The Cloud DBA's Cost and Performance Dashboard

The final chapter ties the book together: one dashboard where cost and performance meet. Every preceding chapter made promises — right-sizing saves money (39), secondaries must earn their keep (37), commitments beat on-demand for stable workloads (38), storage tiers match the workload (36) — and the dashboard is where those promises are kept or broken, visibly, every week. This chapter builds the unified cost-performance view: the metrics, the attribution, the governance, and the operating cadence.

40.1 The Cost-Performance Relationship

Cost and performance are not opposing forces — they are the same system viewed two ways. The healthy relationship: **every dollar traced to a workload outcome** (this database costs \$X/month and delivers Y transactions at Z latency); **waste defined as cost without outcome** (the idle instance, the oversized secondary, the forgotten snapshot — Chapter 39's backlog); and **performance investment justified by outcome** (the upsizing that fixes the latency SLO is not waste — it's the business buying performance deliberately). The dashboard's job is to make both visible together: the team that sees cost and latency on one screen makes better trade-offs than the team that sees them in separate tools owned by separate departments.

40.2 Building the Unified View: Metrics That Matter

The dashboard's metric set, per database: **cost** (daily/monthly spend from the billing APIs — AWS Cost Explorer, Azure Cost Management, GCP Billing — allocated by tag to the database); **performance** (p95 query latency, throughput, error rate — Chapter 30's outcome metrics); **efficiency** (cost per million transactions, cost per GB-month, utilization vs provisioned — the ratios that reveal waste); and **commitment coverage** (what fraction of the stable baseline is covered by reservations/commitments — Chapter 38's KPI). The composite views: fleet cost trend with anomaly detection (the bill that jumps 20% week-over-week is an incident); top-10 most expensive databases with their efficiency ratios (the optimization backlog, ordered by dollars); and waste identified vs waste remediated (the program's scorecard).

40.3 Query-Level Cost Attribution

The advanced practice: attribute cost to queries, not just databases. Using Query Store's resource consumption per query (CPU time, logical reads) combined with the instance's hourly cost, estimate **cost per query pattern** — the nightly report that consumes 15% of a

\$4,000/month instance costs \$600/month, which reframes the tuning conversation entirely (Chapter 35's tuning backlog, now ordered by dollars). This doesn't need accounting precision — directional accuracy ("this query family costs ~\$500/month") is enough to prioritize. The queries that survive this analysis are either worth their cost (revenue-critical) or get tuned, cached, or moved to a cheaper tier (Chapter 22's offload decision, now with numbers).

40.4 Cost Governance: Budgets, Alerts, and Tagging

Governance makes the dashboard actionable: **budgets per team/application** with alerts at 80% and 100% (the team learns about the overrun from the alert, not the invoice); **anomaly alerts** on spend (the compromised credential mining crypto, the runaway analytics query on a pay-per-query warehouse — cost anomalies are security signals too); **mandatory tagging** enforced by policy (untagged resources get flagged, then restricted — Chapter 30's rule, with teeth); and **the monthly cost review** (engineering and finance together, walking the dashboard: waste remediated, new spend justified, commitments adjusted). The review is the heartbeat — without it, the dashboard is wallpaper.

40.5 Building It: Data Sources and Automation

The dashboard is a data pipeline. Sources: **billing APIs** (AWS Cost Explorer / CUR, Azure Cost Management exports, GCP BigQuery billing export — daily granularity minimum, resource-level tags for allocation); **performance telemetry** (the Chapter 30 monitoring stack — p95 latency, throughput, error rates per database); **Query Store aggregates** (per-query resource consumption for the cost attribution in 40.3); and **inventory** (the fleet list with instance types, tiers, commitment coverage, and secondary relationships — Chapters 37–39's inputs). Build it as code (the dashboard definition in version control, the ETL on a schedule), refresh daily, and keep history — the year-over-year trend is what makes the quarterly strategy review (40.6) meaningful. Start with the top-10 databases by spend; expand to the fleet once the pattern works.

40.6 Presenting to Leadership: The Executive View

Leadership needs a different dashboard than engineering: **total database spend trend** (month over month, with the business growth overlaid — spend growing slower than revenue is the headline); **unit economics** (cost per transaction, per customer, per GB — the numbers that survive the CFO's questions); **optimization program results** (waste identified vs remediated in dollars, commitment savings realized, the right-sizing program's cumulative impact); and **forward commitments** (upcoming renewals, planned tier changes, license true-ups — no surprises). The executive view is updated quarterly and owned jointly by

engineering and finance — it is the artifact that turns the DBA team's operational excellence into organizational credibility, and it is how the next round of infrastructure investment gets approved.

40.7 Dashboard Anti-Patterns

The mistakes that kill cost dashboards: **vanity metrics** (total spend without allocation — a number nobody can act on); **stale data** (the dashboard refreshed monthly is a report, not an operations tool — daily minimum); **no ownership** (the dashboard nobody owns rots within a quarter — metrics break, tags drift, and nobody fixes them); **cost without performance context** (the cheapest database that misses its SLO is not a win — the dashboard must show both, always); and **actionless insights** (every waste finding needs an owner and a date — the dashboard that identifies \$50k of waste with no remediation tracking is just expensive wallpaper). Review the dashboard itself quarterly: is it accurate, is it used, does it drive actions? The dashboard is a product with users — treat it like one.

40.8 The Operating Cadence: Weekly, Monthly, Quarterly

The dashboard drives a cadence: **weekly** (scan the waste backlog — new oversized instances, failed backup cost, anomaly alerts; 30 minutes); **monthly** (the cost review — trend analysis, commitment coverage check, right-sizing actions from Chapter 39's scans, secondary audits from Chapter 37); **quarterly** (strategic — commitment purchases/renewals per Chapter 38, tier and platform decisions, license true-ups, the showback report to leadership). Assign ownership: the dashboard has a named owner, the reviews have scheduled invites, and the action items have the same tracking as production incidents. Cost optimization without a cadence is a one-time project that decays; with a cadence, it's operations.

Key Takeaways

- → **Cost and performance are one system.** Every dollar traced to a workload outcome; waste defined as cost without outcome; performance investment justified deliberately.
- → **Dashboard the four metric families: cost, performance, efficiency, commitment coverage.** Fleet trends, top-10 by spend with efficiency ratios, waste identified vs remediated.
- → **Attribute cost to queries directionally.** Query Store resource consumption × instance cost = the tuning backlog ordered by dollars.
- → **Govern with budgets, anomaly alerts, and mandatory tagging.** Monthly cost review with engineering and finance — the heartbeat that keeps the dashboard from becoming wallpaper.

- → **Run the cadence: weekly scans, monthly reviews, quarterly strategy.** Named owner, scheduled reviews, action items tracked like incidents. This is the operating system for everything the book taught.